

The Backend Engineer's Library

Security, Authentication, and Cryptography

Volume 9

Contents

Applied Cryptography for Engineers

Hashing, MACs, KDFs, and Password Storage

Symmetric and Asymmetric Encryption in Practice

Certificates, PKI, and TLS Operations

Authentication: Sessions, Tokens, and JWTs

OAuth 2.0 and OpenID Connect

Authorization: RBAC, ABAC, and ReBAC (Zanzibar)

Secrets Management

Application Security: OWASP, Injection, SSRF, and Deserialization

Zero Trust and Service-to-Service Authentication: mTLS and SPIFFE/
SPIRE

Threat Modeling and Secure Design

Chapter 1 — Applied Cryptography for Engineers

What this chapter covers. This is the foundation for everything in Volume 9 and a prerequisite for Volume 7 (System Design) and Volume 8 (APIs). Cryptography is the mechanism that makes authentication, confidentiality, and integrity possible in a distributed system where every network hop is attacker-controlled and every storage layer is potentially exfiltrated. This chapter gives you the working mental model: what primitives exist, what guarantees each provides and — crucially — what it does *not* provide, how they compose into protocols, and how to use them without building a bespoke cryptosystem that silently fails. We cover random number generation, hash functions, message authentication codes, key derivation, authenticated encryption, key exchange, and digital signatures at the level a senior backend engineer needs: enough to choose correctly, use the right library call, review code for misuse, and reason about failure modes. We do not derive the mathematics of AES or elliptic curves — we treat them as well-specified black boxes with known properties and known misuse modes.

Learning goals — after this chapter you should be able to:

- Distinguish the six primitive families (RNG, hash, MAC, KDF, encryption, signature) by the security property each provides and the threat it assumes.
- Explain why "encrypt then MAC" and "encrypt and MAC" are broken compositions and why AEAD (RFC 5116) is the only safe default for symmetric encryption.
- Choose SHA-256 vs SHA-3 vs BLAKE3 for a given task and explain length-extension, collision, and preimage resistance in operational terms.
- Use `crypto/rand`, OpenSSL, and libsodium to generate keys, hash, and sign — and explain why `math/rand` and `/dev/urandom` misuse are the most common crypto bugs in backend services.
- Reason about when to use symmetric vs asymmetric primitives and why hybrid construction (asymmetric key exchange + symmetric bulk encryption) is universal at scale.

- Describe the distributed-systems consequences of crypto choices: key distribution, rotation, forward secrecy, and the blast radius of a compromised key.

Boundary notes. This chapter covers *primitives and their correct use*. The wire protocol that composes them into TLS 1.3 (RFC 8446) is covered in Vol 3, Ch 6 — read that for handshake mechanics, record framing, and 0-RTT. The *operational* side of that same protocol — certificate lifecycle, PKI chains, rotation, revocation, and mTLS deployment — is Vol 9, Ch 4. This chapter gives you the vocabulary; those chapters give you the system.

Why applied crypto is different from textbook crypto

Textbook cryptography proves theorems about ideal primitives. Applied cryptography keeps services from being breached when engineers glue those primitives together under deadline pressure. The gap between the two is where nearly every real-world crypto failure lives — not in a broken AES, but in a reused nonce, a timing side-channel, an unauthenticated ciphertext, or a key that was never rotated.

Three principles govern this entire volume:

1. Never invent primitives or protocols. If you are designing a new cipher mode, a custom key-derivation loop, or a novel hand-rolled signature scheme, you are almost certainly introducing a vulnerability. Use vetted, version-pinned constructions: AES-GCM or ChaCha20-Poly1305 for AEAD (RFC 5116 / RFC 8439), HKDF (RFC 5869), Argon2id (RFC 9106) for passwords, Ed25519 (RFC 8032) or ECDSA P-256 for signatures. The standard is the safety.

2. Misuse resistance matters more than primitive strength. AES-256 is not meaningfully safer than AES-128 against any realistic adversary — both are far beyond brute force — but AES-GCM with a reused nonce is catastrophically broken with *either* key length. Most audit findings are misuse, not primitive weakness.

3. Keys are the system. A cryptosystem is only as strong as its key management: generation, distribution, storage, rotation, and destruction. Chapters 4, 8, and 10 are entirely about this. This chapter notes the key lifecycle at every step because a primitive without a key-management story is incomplete.

Randomness: the root of everything

Every other primitive consumes random bytes: keys, nonces, IVs, salts. If the RNG is weak, everything built on it falls.

What "cryptographically secure" means

A Cryptographically Secure PRNG (CSPRNG) has two properties beyond statistical uniformity: *unpredictability* (given all past output, the next byte is still unpredictable) and *backtracking resistance* (given current state, past outputs remain unpredictable). The OS CSPRNG — `getrandom(2)` on Linux, `BCryptGenRandom` on Windows, `arc4random` on BSDs — is seeded from hardware entropy at boot and continuously reseeded from interrupt timing, RDRAND/RDSEED on x86, and other sources.

On Linux, `getrandom(2)` blocks until the entropy pool is initialized (since kernel 5.6, ChaCha20-based CRNG with 256-bit state), then never blocks again. `/dev/urandom` is equivalent on modern kernels. `/dev/random` blocking behavior is a historical artifact — do not use it to "get better randomness."

Generating keys correctly

```
# Generate a 256-bit (32-byte) symmetric key – hex-encoded for config
$ openssl rand -hex 32
a3f7c9e2d1b0485f6a2e9c7d03f8b1a4e5c6d7f8091a2b3c4d5e6f708192a3b4c5

# 32 random bytes as raw binary (for piping to a KMS or file)
$ openssl rand 32 | od -An -tx1
a3 f7 c9 e2 d1 b0 48 5f 6a 2e 9c 7d 03 f8 b1 a4
e5 c6 d7 f8 09 1a 2b 3c 4d 5e 6f 70 81 92 a3 b4

# libsodium (C) – the reference for correct-by-default crypto
# Generates a 32-byte key using the OS CSPRNG
unsigned char key[crypto_secretbox_KEYBYTES]; // 32
crypto_secretbox_keygen(key); // wraps randombytes_buf internally
```

In Go, the distinction is sharp and frequently confused in code review:

```

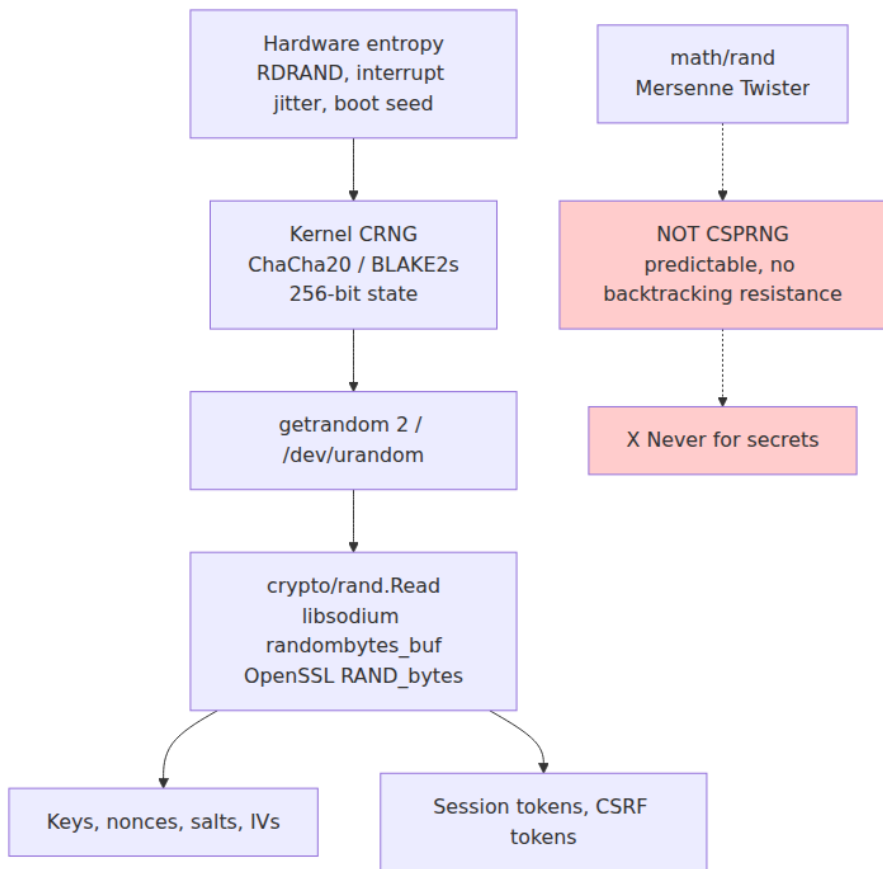
package main

import (
    "crypto/rand" // CSPRNG – backed by getrandom(2)
    mathrand "math/rand" // NOT cryptographically secure – Mersenne
    Twister
    "encoding/hex"
    "fmt"
    "log"
)

func GenerateKey() []byte {
    key := make([]byte, 32) // 256 bits
    if _, err := rand.Read(key); err != nil {
        log.Fatal(err) // getrandom failed – do not continue
    }
    return key
}

func main() {
    k := GenerateKey()
    fmt.Println(hex.EncodeToString(k))
    // Output: e.g. 7f3a9c... (64 hex chars = 32 bytes)
    // NEVER use math/rand for keys, nonces, or tokens:
    // mathrand.Read(key) – predictable after observing 624 outputs
}

```



Review checklist for RNG usage:

Pattern	Verdict
<code>crypto/rand.Read</code> (Go), <code>randombytes_buf</code> (libsodium), <code>RAND_bytes</code> (OpenSSL)	Correct
<code>math/rand</code> , <code>random.random()</code> (Python stdlib), <code>Math.random()</code> (JS)	Never for secrets
<code>uuid.New()</code> / <code>uuid.NewV4()</code> (Go <code>google/uuid</code> uses <code>crypto/rand</code>)	OK for random IDs
<code>uuid.New()</code> from <code>satori/go.uuid V1</code> (MAC + timestamp)	Not random — predictable

Pattern	Verdict
Seeding with <code>time.Now().UnixNano()</code>	Broken

Hash functions: integrity fingerprints

A cryptographic hash function $H: \{0,1\}^* \rightarrow \{0,1\}^n$ maps arbitrary input to a fixed-size digest. Three security properties matter, each resisting a different attack:

- **Preimage resistance:** given h , finding any m with $H(m) = h$ takes $\sim 2^n$ work. Protects password hashes and commitment schemes.
- **Second-preimage resistance:** given m_1 , finding $m_2 \neq m_1$ with $H(m_1) = H(m_2)$ takes $\sim 2^n$. Protects integrity checks where the original is known.
- **Collision resistance:** finding any pair $m_1 \neq m_2$ with $H(m_1) = H(m_2)$ takes $\sim 2^{(n/2)}$ (birthday bound). Protects signatures and deduplication where the attacker controls both inputs. This is the hardest property to maintain — collisions are quadratically easier than preimages.

Choosing a hash

Function	Output	Standard	Notes
SHA-256	256 bits	FIPS 180-4	Conservative default; ubiquitous, hardware-accelerated (SHA-NI)
SHA-512	512 bits	FIPS 180-4	Faster than SHA-256 on 64-bit CPUs; use SHA-512/256 for truncated variant
SHA-3-256 (Keccak)	256 bits	FIPS 202	Different construction (sponge); hedge against SHA-2 cryptanalysis
BLAKE3	256 bits	Informal (BLAKE3 spec)	Fastest in software; parallelizable; not yet NIST-standardized
SHA-1, MD5	160/128 bits	Broken	Collision attacks are practical (SHA-1 shattered, 2017). Never for security.

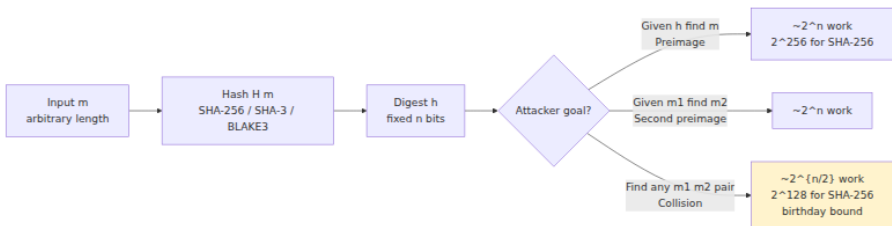

```
# Hashing with OpenSSL – note the hex digest is deterministic for the
same input
$ echo -n "hello" | openssl dgst -sha256
SHA2-256(stdin)= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e730433
62938b9824

$ echo -n "hello" | openssl dgst -sha3-256
SHA3-256(stdin)= 3338be694f50c5f33854368fea51034a4364dd52019b6de187a657
5026b8e9f4d

# Go – SHA-256
```

```
import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
)

func main() {
    h := sha256.Sum256([]byte("hello"))
    fmt.Println(hex.EncodeToString(h[:]))
    // Output:
    2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
}
```



Where hashes go wrong

Length-extension attacks affect Merkle-Damgård hashes (SHA-256, SHA-512) but not SHA-3 or BLAKE3. Given $H(\text{key} || \text{message})$ an attacker can compute $H(\text{key} || \text{message} || \text{extension})$ without knowing key . This is why $\text{HMAC-SHA256}(\text{key}, \text{message})$ exists rather than $\text{SHA256}(\text{key} || \text{message})$. If you must use SHA-256 in a MAC-like construction, use HMAC.

Do not use raw hashes for passwords. $\text{SHA-256}(\text{"password123"})$ is computed in nanoseconds on a GPU. Password storage requires a deliberately slow, salted KDF — see Chapter 2.

Message Authentication Codes

A hash tells you *what* was hashed. A MAC tells you *who* vouched for it. `MAC(K, m)` is a keyed primitive: only someone holding key `K` can produce or verify the tag.

HMAC (RFC 2104) is the standard construction: `HMAC(K, m) = H((K' xor opad) || H((K' xor ipad) || m))`. It is safe with any approved hash (HMAC-SHA256 is the default).

```
import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
)

func computeHMAC(key, message []byte) string {
    mac := hmac.New(sha256.New, key)
    mac.Write(message)
    return hex.EncodeToString(mac.Sum(nil))
}

// Example - webhook signature verification (Stripe, GitHub style)
func verifyWebhook(secret, payload, claimedTag []byte) bool {
    mac := hmac.New(sha256.New, secret)
    mac.Write(payload)
    expected := mac.Sum(nil)
    // hmac.Equal is constant-time - never use == for tags
    return hmac.Equal(expected, claimedTag)
}
```

The critical detail is in that last line. A naive `==` comparison short-circuits on the first differing byte, leaking timing information that lets an attacker forge a tag byte-by-byte. `hmac.Equal` (Go), `hmac.compare_digest` (Python), `crypto.timingSafeEqual` (Node) all run in constant time regardless of where the mismatch occurs.

KMAC / HMAC-SHA3 variants exist but HMAC-SHA256 remains the conservative choice. For new systems where you control both sides, consider using an AEAD instead — it authenticates and encrypts in one step.

Key Derivation Functions

A KDF stretches a shared secret, password, or keying material into one or more cryptographically strong keys.

KDF	Input	Standard	Use case
HKDF (RFC 5869)	High-entropy IKM (e.g., ECDH shared secret)	RFC 5869	Derive session keys from a handshake secret (TLS 1.3 key schedule)
PBKDF2 (RFC 8018)	Password	PKCS#5, NIST SP 800-132	Legacy password hashing; use Argon2 for new code
scrypt (RFC 7914)	Password	RFC 7914	Memory-hard; better than PBKDF2, superseded by Argon2
Argon2id (RFC 9106)	Password	RFC 9106	Current best practice for password storage (Ch 2)

```
import "golang.org/x/crypto/hkdf"
import "crypto/sha256"
import "io"

// Derive two independent keys from one ECDH shared secret
func deriveKeys(ikm, salt, info []byte) (encKey, macKey []byte) {
    // HKDF-Extract: salt + IKM -> PRK (pseudorandom key)
    // HKDF-Expand: PRK + info -> output keys
    r := hkdf.New(sha256.New, ikm, salt, []byte("enc-key"))
    encKey = make([]byte, 32)
    io.ReadFull(r, encKey)

    r2 := hkdf.New(sha256.New, ikm, salt, []byte("mac-key"))
    macKey = make([]byte, 32)
    io.ReadFull(r2, macKey)
    return
}
```

HKDF's two-step structure matters: *Extract* concentrates dispersed entropy into a uniform PRK, *Expand* stretches it to the desired length with domain separation via `info`. In TLS 1.3 (RFC 8446, Section 7.1), the entire key schedule is HKDF — handshake secrets, traffic keys, resumption secrets all derive through HKDF-Extract/Expand with distinct labels.

Authenticated encryption: the only symmetric encryption you should use

Unauthenticated encryption (AES-CBC, AES-CTR alone) is malleable: an attacker can flip bits in the ciphertext to produce predictable changes in the plaintext. Every classic padding-oracle and bit-flipping attack exploits this. The fix is **AEAD** — Authenticated Encryption with Associated Data (RFC 5116) — which encrypts and authenticates in a single operation and returns an error on any tampering.

Two AEADs dominate modern practice:

- **AES-GCM** — hardware-accelerated via AES-NI + CLMUL on x86/ARMv8. The default where hardware support is assumed (servers, cloud VMs). Nonce is 96 bits (12 bytes); tag is 128 bits.
- **ChaCha20-Poly1305** (RFC 8439) — fast in pure software, no timing side-channels from S-box lookups. Preferred on mobile, embedded, and where constant-time AES is unavailable.

```

package main

import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/rand"
    "fmt"
    "io"
    "log"

    "golang.org/x/crypto/chacha20poly1305"
)

// AES-GCM – server-side default (requires AES-NI for performance)
func encryptAESGCM(key, plaintext, aad []byte) (nonce, ciphertext []byte) {
    block, err := aes.NewCipher(key) // key must be 16, 24, or 32 bytes
    if err != nil { log.Fatal(err) }
    aead, err := cipher.NewGCM(block)
    if err != nil { log.Fatal(err) }

    nonce = make([]byte, aead.NonceSize()) // 12 bytes for GCM
    if _, err := io.ReadFull(rand.Reader, nonce); err != nil { log.Fatal(err) }

    // Seal appends ciphertext + 16-byte auth tag; aad is authenticated
    // but not encrypted
    ciphertext = aead.Seal(nil, nonce, plaintext, aad)
    return nonce, ciphertext
}

func decryptAESGCM(key, nonce, ciphertext, aad []byte) ([]byte, error) {
    block, _ := aes.NewCipher(key)
    aead, _ := cipher.NewGCM(block)
    return aead.Open(nil, nonce, ciphertext, aad) // error if tag
    invalid
}

// ChaCha20-Poly1305 – alternative with no hardware dependency
func encryptChaCha(key, plaintext, aad []byte) (nonce, ciphertext []byte) {
    aead, err := chacha20poly1305.New(key) // 32-byte key
    if err != nil { log.Fatal(err) }
    nonce = make([]byte, aead.NonceSize()) // 12 bytes (or 24 for X
    variant)
    io.ReadFull(rand.Reader, nonce)
    ciphertext = aead.Seal(nil, nonce, plaintext, aad)
    return
}

```

```
func main() {
    key := make([]byte, 32)
    rand.Read(key)

    nonce, ct := encryptAESGCM(key, []byte("payment: $100 to alice"), [
byte("header-v1")])
    fmt.Printf("nonce: %x\nct+tag: %x (%d bytes)\n", nonce, ct, len(ct))
}

// Output: nonce: 7f3a... (12 bytes)
//         ct+tag: 9c1e... (21 bytes plaintext + 16 byte tag = 37
bytes)
```

```
# OpenSSL AES-GCM round-trip (for ops debugging – prefer libsodium/Go
in services)
$ echo -n "payment: $100 to alice" | openssl enc -aes-256-gcm \
-K $(openssl rand -hex 32) -iv $(openssl rand -hex 12) -a
# Output: base64 ciphertext+tag (OpenSSL appends tag; format is tool-
specific – not interoperable wire format)
```

The `aad` (Additional Authenticated Data) parameter is the feature that makes AEAD more than "encryption that checks integrity." AAD is authenticated but sent in cleartext — use it for headers, sequence numbers, or tenant IDs that must not be tampered with even though they are not secret.

Nonce discipline

Both AES-GCM and ChaCha20-Poly1305 fail catastrophically on nonce reuse with the same key. For GCM, reusing a nonce leaks the authentication key and then the plaintext XOR. Rules:

- **Random 96-bit nonce:** safe if you never encrypt more than $\sim 2^{32}$ messages under one key (birthday bound at 2^{48} for 96 bits — 2^{32} is the conservative limit per NIST SP 800-38D).
- **Counter nonce:** safe if you have a reliable persistent counter (not safe across restarts or distributed replicas without coordination).
- **XChaCha20-Poly1305 (192-bit nonce):** safe to generate randomly for practically unlimited messages — prefer this for long-lived keys or distributed encryptors that cannot coordinate counters.

Distributed-systems note. Nonce coordination is a hidden distributed-systems problem. Two replicas sharing a key and generating random 96-

bit nonces will eventually collide. Either give each replica a distinct key (via HKDF with a replica ID as `info`), use XChaCha20-Poly1305 with 192-bit random nonces, or partition the nonce space (e.g., 32-bit replica ID + 64-bit counter). Shared-key nonce reuse across replicas has caused real outages.

Asymmetric primitives: key exchange and signatures

Symmetric crypto needs a shared secret. Asymmetric crypto solves the distribution problem at the cost of slower operations. Two operations matter:

Key exchange (KEM / ECDH): two parties derive a shared secret over an insecure channel. Modern practice is **X25519** (RFC 7748, ECDH over Curve25519) or **ECDH P-256**. The shared secret is then fed through HKDF to derive symmetric keys — this is the hybrid construction that TLS 1.3, Noise, and nearly every encrypted channel uses.

Signatures: one party proves authorship and integrity with a private key; anyone with the public key can verify. Standards: **Ed25519** (RFC 8032, deterministic, 64-byte signatures) and **ECDSA P-256** (FIPS 186-4, requires careful RNG — nonce reuse leaks the private key). RSA signatures (RSASSA-PSS, RFC 8017) remain common for X.509 but are larger and slower.

```

import (
    "crypto/ecdh"
    "crypto/ed25519"
    "crypto/rand"
    "fmt"
)

// X25519 key exchange – hybrid encryption pattern
func keyExchangeExample() {
    // Alice generates a keypair
    alicePriv, _ := ecdh.X25519().GenerateKey(rand.Reader)
    alicePub := alicePriv.PublicKey()

    // Bob generates a keypair
    bobPriv, _ := ecdh.X25519().GenerateKey(rand.Reader)
    bobPub := bobPriv.PublicKey()

    // Shared secret – both sides derive the same 32 bytes
    aliceShared, _ := alicePriv.ECDH(bobPub)
    bobShared, _ := bobPriv.ECDH(alicePub)
    fmt.Printf("shared equal: %v\n", string(aliceShared) == string(bobShared)) // true
    // Feed aliceShared through HKDF to get AEAD keys – never use raw
    // ECDH output as a key
}

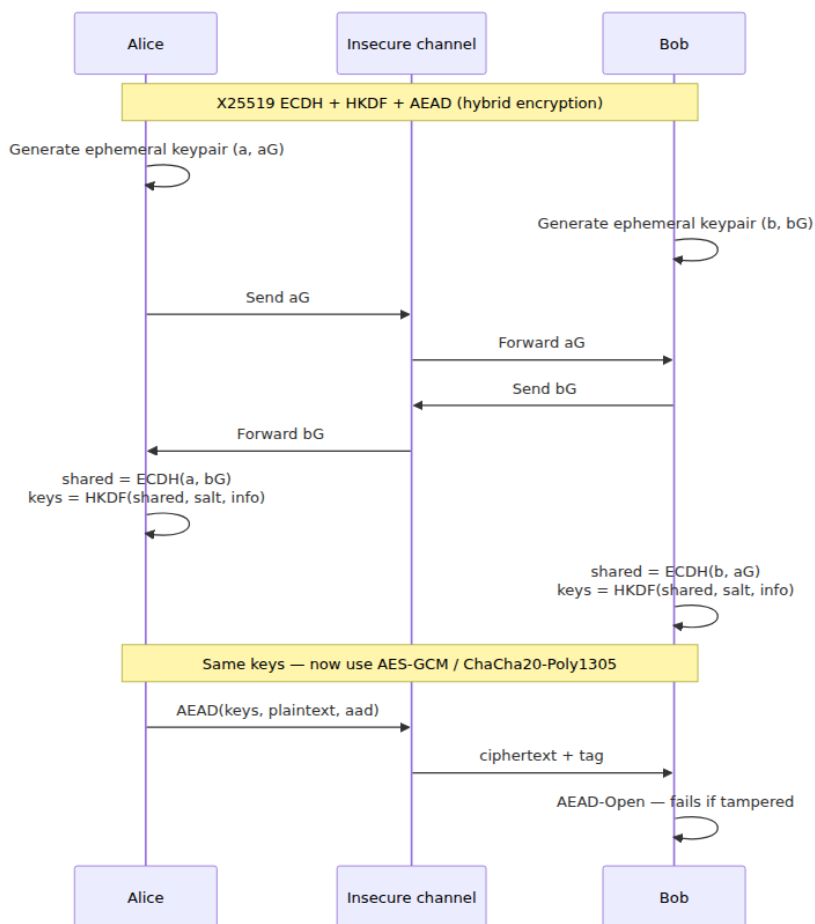
// Ed25519 signing – deterministic, no RNG needed at sign time
func signatureExample() {
    pub, priv, _ := ed25519.GenerateKey(rand.Reader)
    message := []byte("deploy artifact sha256:abc123...")
    sig := ed25519.Sign(priv, message)
    fmt.Printf("sig: %x (%d bytes)\n", sig, len(sig)) // 64 bytes
    fmt.Printf("valid: %v\n", ed25519.Verify(pub, message, sig)) //
    true
}

```

```

# libsodium – sealed box (X25519 + XSalsa20-Poly1305, anonymous sender)
# Generate keypair
$ openssl genpkey -algorithm X25519 -out x25519-priv.pem
$ openssl pkey -in x25519-priv.pem -pubout -out x25519-pub.pem
$ cat x25519-pub.pem
-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEA...
-----END PUBLIC KEY-----

```

When to use which

Goal	Primitive	Example
Bulk data encryption (at rest, in transit)	AEAD (AES-GCM / ChaCha20-Poly1305)	Encrypting a database field, a message queue payload
Key exchange over insecure channel	X25519 ECDH + HKDF	TLS 1.3 handshake, Noise protocol

Goal	Primitive	Example
Authentication / non-repudiation	Ed25519 / ECDSA signatures	Signing artifacts (Sigstore), JWTs (asymmetric), certificate chains
Password storage	Argon2id (Ch 2)	Never use plain hash or fast KDF

Forward secrecy. Ephemeral key exchange (ECDHE — the "E" is ephemeral) means each session uses a fresh keypair that is discarded afterward. An attacker who later steals the long-term identity key cannot decrypt recorded sessions. TLS 1.3 (RFC 8446) mandates (EC)DHE for this reason — it removed static RSA key transport entirely. For service-to-service encryption at scale, prefer ephemeral exchange even if it costs an extra round trip.

Composition and misuse patterns

Real systems compose primitives. The composition can be secure or broken independently of the primitives' strength.

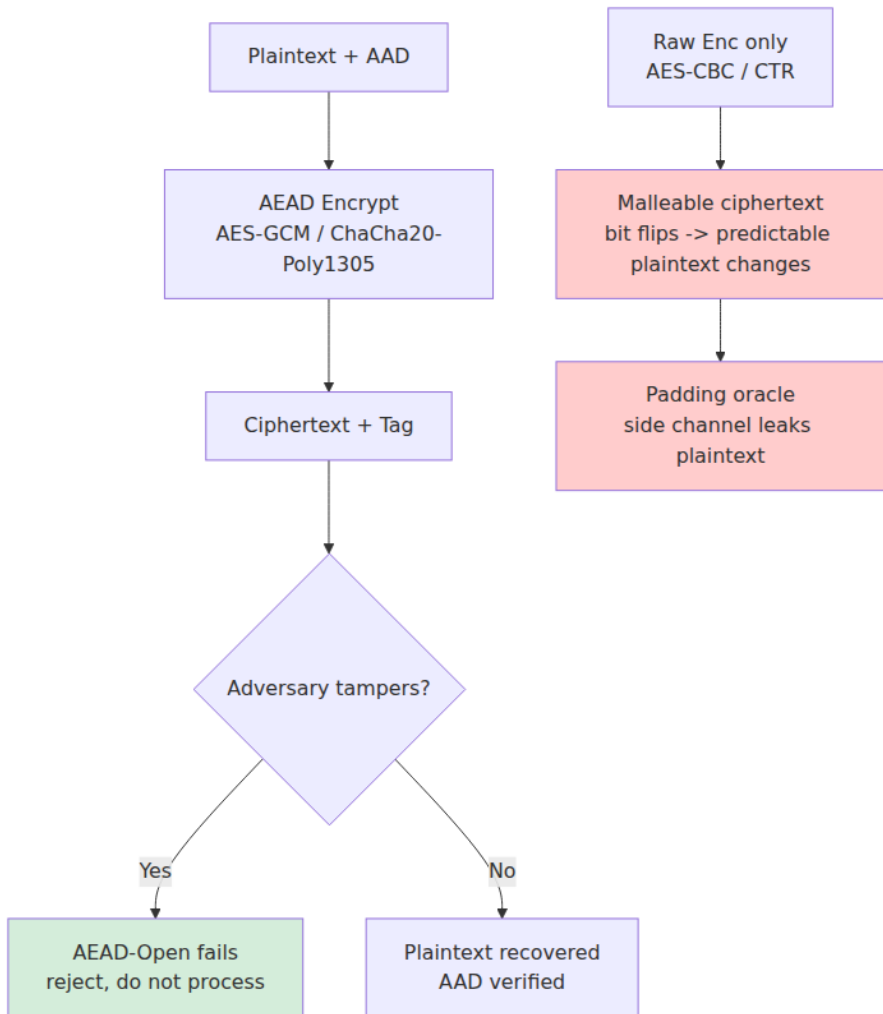
Authenticated encryption is the safe composition. Do not build "encrypt then MAC" yourself — use an AEAD. If you must compose manually (legacy protocol), the order matters and is easy to get wrong:

- *Encrypt-and-MAC* ($C = \text{Enc}(K1, m)$, $T = \text{MAC}(K2, m)$) — leaks plaintext equality through the MAC.
- *MAC-then-Encrypt* ($T = \text{MAC}(K1, m)$, $C = \text{Enc}(K2, m || T)$) — vulnerable to padding oracles (the TLS 1.0/1.1 CBC debacle).
- *Encrypt-then-MAC* ($C = \text{Enc}(K1, m)$, $T = \text{MAC}(K2, C)$) — the only manually safe order, but still requires two independent keys and constant-time verification. Just use an AEAD.

Hybrid encryption (asymmetric KEM + symmetric AEAD) is how you encrypt to a public key at scale: generate an ephemeral symmetric key, encrypt the payload with the AEAD, encrypt the symmetric key to the recipient's public key. Libraries like libsodium `crypto_box_seal` and Google Tink do this correctly — do not reimplement it.

Key separation: never use the same key for two purposes. Derive purpose-specific subkeys with HKDF and distinct `info` labels. A key used

for both encryption and MAC, or for two different protocols, can create cross-protocol attacks.



Signatures in the backend: supply chain, tokens, and logs

Signatures solve a different problem than encryption: they bind an identity to a statement so that anyone can verify it without being able to forge it. Three verification patterns dominate backend systems.

Artifact signing proves that a container image, binary, or SBOM was produced by a trusted builder. Sigstore (Cosign + Fulcio + Rekor) uses short-lived Ed25519/ECDSA keys bound to an OIDC identity, with the signature and certificate logged in a transparency log. Verification checks the signature, the certificate chain to Fulcio, and inclusion in Rekor. The key insight for this chapter is that the primitive — Ed25519 over SHA-512 — is only one link: the system also needs a trust anchor (Fulcio root), a distribution mechanism (OCI annotations or in-toto attestations), and a revocation story (short-lived certs, not CRLs). Companion Book 5, Chapters 3–5 covers this end-to-end; the primitive alone is insufficient.

Token signing proves that a JWT or PASETO token was issued by your auth service. Two modes exist: symmetric (HMAC-SHA256, HS256) where every verifier holds the shared key, and asymmetric (RS256, ES256, EdDSA) where verifiers hold only the public key. Symmetric mode is simpler but every verifier can forge tokens — a single compromised service can mint arbitrary JWTs. Asymmetric mode isolates the signing key to the auth service; verifiers fetch the public key via JWKS. For distributed fleets, prefer asymmetric (Ed25519 or ECDSA P-256) with a JWKS endpoint and `kid` rotation as described in Chapter 5. Never accept `alg: none` and never let the token dictate the algorithm — both have produced real forgeries.

Log and audit signing proves that a record existed at a point in time. Rekor and Certificate Transparency use Merkle-tree-backed transparency logs: each entry is signed, the tree head is signed, and inclusion/exclusion proofs are verifiable. The signature primitive is again Ed25519 or ECDSA, but the system property is append-only verifiability. If your audit pipeline signs each batch of logs with a per-batch Ed25519 signature and publishes the public key, an attacker who tampers with stored logs must also forge a signature or break the chain. This is covered in depth in Companion Book 5, Chapter 5; the takeaway here is that signing without a verifiable log is repudiation without evidence.

Signature verification has its own misuse class. ECDSA requires a fresh random nonce per signature — reusing the nonce leaks the private key (this has drained cryptocurrency wallets). Ed25519 is deterministic and avoids this failure mode, which is one reason it is preferred for new systems. RSA-PSS (RFC 8017) requires a proper salt length; PKCS#1 v1.5 signatures are malleable. Always verify that the library you call implements constant-time scalar multiplication and rejects small-

subgroup points for ECDH/ECDSA — Go `crypto/ecdh` and `crypto/ed25519` do; some older OpenSSL versions required explicit checks.

RNG failures: the bugs that break everything above

If the RNG fails, every key, nonce, and salt above is predictable. The failures are not hypothetical.

Debian OpenSSL (2008). A patch removed entropy sources from OpenSSL's RNG to silence a Valgrind warning, reducing the keyspace to 15 bits. Thousands of SSH and TLS keys were factorable. The fix was to restore the entropy sources — but the lesson is that RNG code is fragile and must not be patched without cryptographic review.

Android SecureRandom (2013). `SecureRandom` was seeded with insufficient entropy on some devices, producing repeated ECDSA nonces in Bitcoin wallets. Nonce reuse leaked private keys and funds were stolen. The fix was to seed from `/dev/urandom` and to move to deterministic ECDSA (RFC 6979) or Ed25519.

VM snapshot cloning. Cloning a VM or container without reseeding the RNG produces identical keys and nonces on every clone. Cloud images must reseed from host entropy on first boot; containers that fork many workers must call `getrandom(2)` per worker, not once before forking. Systemd's `random-seed` service and the kernel's CRNG reseeding on fork (Linux 5.6+) mitigate this, but application-level caching of keys across forks reintroduces the bug.

The operational response is threefold: never cache RNG output across forks, never seed your own PRNG from `time.Now()` or PID, and monitor for duplicate keys or nonces in production (a duplicate X25519 public key or AES-GCM nonce across hosts is an RNG failure until proven otherwise).

Crypto agility: versioning primitives without flag days

Primitives age. SHA-1 was standard, then broken. RSA-1024 was sufficient, then factored. TLS 1.2's CBC ciphersuites were recommended, then removed in TLS 1.3. Every cryptosystem you ship must be replaceable without a coordinated flag day across the fleet — a property called crypto agility.

Agility has three concrete mechanisms. First, every encrypted or signed value carries an algorithm identifier: the `alg` field in a JWT header, the `cipher_suite` in a TLS ClientHello, the `kek_version` in an envelope, or the `$argon2id$v=19$` prefix in a PHC string. Verifiers use the identifier to select the correct code path, which means old and new values coexist during migration. Second, negotiation is explicit and server-driven: the server advertises what it accepts and selects the strongest mutual option, rather than trusting a client-supplied preference unauthenticated. Third, deprecation is phased: add the new algorithm, dual-issue or dual-verify for a window, measure adoption, then remove the old algorithm only after the tail drops below a threshold. The TLS 1.3 design is the canonical example — it removed renegotiation, compression, and static RSA in one revision but did so by defining a new version number and requiring implementations to negotiate it, not by patching the old version in place.

For backend services, the practical implication is that no crypto choice should be a hardcoded constant. Key lengths, hash choices, AEAD selections, and KDF parameters belong in versioned configuration, not in string literals. A service that writes `aes.NewCipher(key)` with a fixed assumption of AES-256-GCM cannot migrate to ChaCha20-Poly1305 for a fleet of ARM edge nodes without a code change and redeploy. A service that writes `aeadForVersion(v).Seal(...)` with a registry of algorithms can migrate by changing config and doing a rolling deploy with dual-read. Build the registry on day one — retrofitting agility after a primitive is broken is an emergency migration under incident pressure, which is exactly when you will get it wrong.

Agility also governs failure response. When a primitive is weakened — as SHA-1 was after SHattered (2017) demonstrated a practical collision in $\sim 2^{63}$ work — the question is not whether to migrate but how fast. Teams with agility migrated by bumping a version identifier and rehashing affected objects in a background job. Teams without agility faced a choice between mass invalidation (downtime, user impact) and continued exposure. The same dynamics apply to post-quantum migration, which NIST has begun standardizing with ML-KEM (FIPS 203) and ML-DSA (FIPS 204) — even if you do not deploy them today, your agility layer is what will let you deploy them when you must.

Distributed-systems lens

Cryptography in a single process is a library call. Cryptography in a distributed system is a coordination problem.

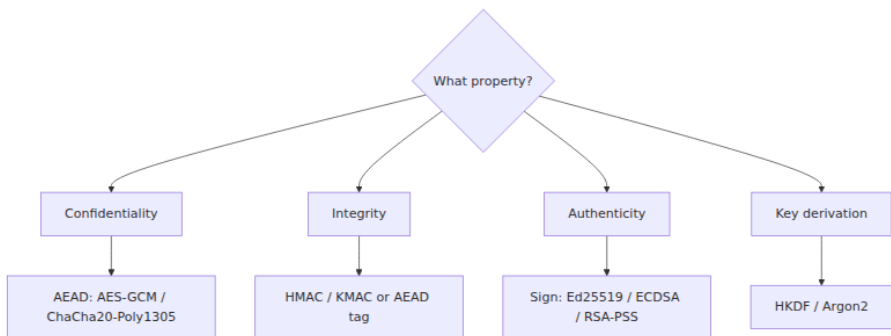
Key distribution is the central challenge. Symmetric keys must be shared among replicas that need to encrypt/decrypt, but every copy is additional blast radius. Prefer asymmetric patterns where possible: each service holds its own private key, public keys are distributed freely. Where symmetric keys are unavoidable (e.g., data-at-rest encryption), use envelope encryption — a data key encrypts the payload, a KMS-held key-encrypting key (KEK) encrypts the data key — so rotation and revocation happen at the KEK layer without re-encrypting all data. This is the pattern AWS KMS, GCP KMS, and Vault Transit all implement.

Rotation and versioning. Keys must rotate without downtime. Every encrypted value or token should carry a key identifier (`kid` in JWTs, key version in KMS ciphertext) so verifiers can select the correct key during the rotation window. Dual-read (try new key, fall back to old) during rotation is the standard technique — see Chapter 8 for full lifecycle.

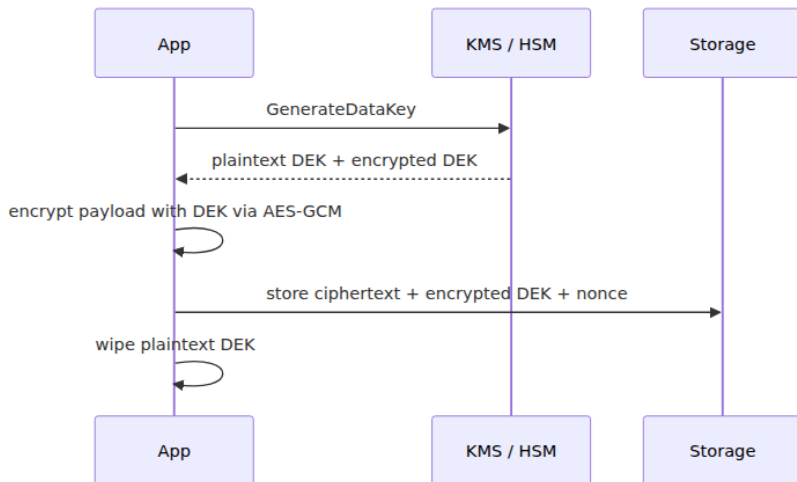
Consistency of trust stores. Certificate roots, public keys, and revocation lists must be consistent across the fleet. A rolling deploy that updates a trust store on half the fleet before the other half creates a window where inter-service authentication fails in one direction. Treat trust-store updates like schema migrations: backward-compatible, dual-trust windows, then cleanup.

Forward secrecy at scale means session keys are ephemeral and short-lived. For long-lived connections (gRPC streams, WebSocket), re-key periodically. For stored data, forward secrecy is approximated by prompt re-encryption after KEK rotation.

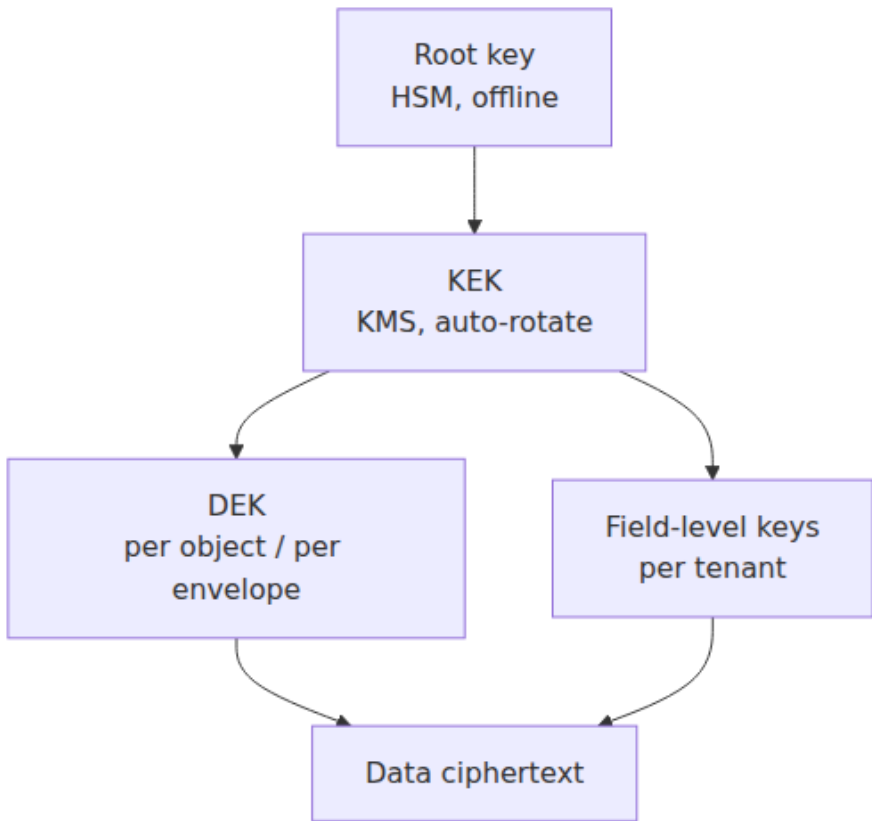
Crypto Primitive Selection



Envelope Encryption



Key Hierarchy



Key takeaways

- Use vetted, version-pinned primitives: AES-GCM or ChaCha20-Poly1305 for AEAD (RFC 5116/8439), HMAC-SHA256 for MACs, HKDF (RFC 5869) for key derivation, X25519 (RFC 7748) for key exchange, Ed25519 (RFC 8032) for signatures. Do not invent constructions.
- RNG failures are the most common crypto bug — always use the OS CSPRNG via `crypto/rand`, `randombytes_buf`, or `RAND_bytes`; never `math/rand`, `Math.random()`, or time-seeded PRNGs.
- Hash functions provide preimage, second-preimage, and collision resistance at different cost levels; SHA-256 and SHA-3-256 are

conservative defaults, BLAKE3 is the fast alternative. Never use MD5 or SHA-1 for security.

- AEAD is the only safe symmetric-encryption interface — it binds confidentiality and integrity in one operation with a single misuse mode (nonce reuse). Prefer XChaCha20-Poly1305 where nonce coordination across replicas is hard.
- Asymmetric crypto is for key exchange and signatures, not bulk encryption — hybrid construction (ECDH + HKDF + AEAD) is how real protocols compose them.
- Keys are the system: generation, distribution, rotation, and destruction dominate the security of any cryptosystem. Every key should have an identifier, a rotation schedule, and a defined blast radius.

Further reading

- **Standards:** RFC 5116 (AEAD), RFC 5869 (HKDF), RFC 7748 (X25519/X448), RFC 8032 (Ed25519/Ed448), RFC 8439 (ChaCha20-Poly1305), RFC 8446 (TLS 1.3), FIPS 180-4 (SHA-2), FIPS 202 (SHA-3), FIPS 186-4 (ECDSA), NIST SP 800-38D (GCM).
- D. J. Bernstein et al. — *The Security Impact of a New Cryptographic Library* (libsodium design rationale, 2012).
- Ferguson, Schneier, Kohno — *Cryptography Engineering* (Wiley, 2010). The practitioner counterpart to textbook crypto.
- Latacora — *Cryptographic Right Answers* (<https://latacora.micro.blog/2018/04/03/cryptographic-right-answers.html>) — opinionated, regularly updated primitive choices.
- libsodium documentation — <https://doc.libsodium.org/> — the best single reference for correct-by-default API usage.

- Go `crypto/*` package docs — <https://pkg.go.dev/crypto> — authoritative for Go service implementations.

Chapter 2 — Hashing, MACs, KDFs, and Password Storage

What this chapter covers. Hash functions are the most misused primitive in backend systems — not because the mathematics is subtle, but because engineers reach for the wrong tool for the job: SHA-256 where Argon2id is required, a fast hash where a slow one is needed, or no salt where uniqueness is essential. This chapter gives you the complete decision tree. We start with unkeyed cryptographic hashes (SHA-2, SHA-3, BLAKE3), their security properties, and their operational limits — length extension, collision bounds, and performance. We then add a key: message authentication codes (HMAC, KMAC, Poly1305) and their role in tamper detection. We then stretch and derive: key derivation functions from HKDF (RFC 5869) for high-entropy inputs to memory-hard password hashes — scrypt (RFC 7914) and Argon2 (RFC 9106) — for low-entropy human secrets. The chapter closes where most breaches begin: password storage done correctly per NIST SP 800-63B (2023 update), including salting, peppering, upgrade paths, and breach-response rotation.

Learning goals — after this chapter you should be able to:

- State the three hash security properties (preimage, second-preimage, collision resistance) with their quantitative bounds and explain why collision resistance fails first.
- Explain length-extension attacks on Merkle-Damgard hashes and why HMAC-SHA256 (RFC 2104) or SHA-3 is required for keyed use.
- Use HMAC correctly — including constant-time verification — and distinguish it from a plain hash and from an AEAD.
- Choose among HKDF, PBKDF2, scrypt, and Argon2id by input entropy: HKDF for key derivation from high-entropy secrets, Argon2id for password hashing, nothing else for new code.
- Implement Argon2id password storage in Go and with libsodium, with concrete parameters (RFC 9106) and guidance on tuning for your hardware.

- Apply NIST SP 800-63B rules for password handling: no composition rules, no periodic rotation, rate limiting, breach-corpus checks, and verifier requirements.
- Plan a password-hash migration (e.g., PBKDF2 or bcrypt to Argon2id) without forcing a mass password reset.

Boundary notes. The hash and MAC primitives introduced here underpin much of the rest of the volume. AEAD construction (AES-GCM, ChaCha20-Poly1305) and hybrid encryption are Ch 3. How hashes and signatures compose into certificate chains and the TLS 1.3 key schedule is Vol 3, Ch 6 (wire) and Vol 9, Ch 4 (operations). Token integrity via HMAC vs asymmetric signatures is Ch 5. Secrets management and KMS envelope encryption are Ch 8 — the operational home for the keys this chapter derives.

Hash functions in depth

Chapter 1 introduced hashes as fixed-output fingerprints. This section makes that precise and operational.

Properties and bounds

For a hash H with n -bit output:

Property	Attacker is given	Must find	Classical work
Preimage resistance	$h = H(m)$	any m' with $H(m') = h$	$\sim 2^n$
Second-preimage resistance	$m_1, h = H(m_1)$	$m_2 \neq m_1$ with $H(m_2) = h$	$\sim 2^n$
Collision resistance	—	any $m_1 \neq m_2$ with $H(m_1) = H(m_2)$	$\sim 2^{(n/2)}$

Collision resistance at $n/2$ bits is not a flaw — it is the birthday paradox, and it is why hash output lengths must be double the desired collision security. SHA-256 offers 128-bit collision security, which remains comfortably beyond feasible computation. MD5 (128-bit output, 64-bit collision security) and SHA-1 (160-bit output, ~ 63 -bit collision security

after the 2017 SHAttered attack) are below the threshold and must not be used for security decisions.

Two additional properties matter in practice:

- **Avalanche:** flipping one input bit flips roughly half the output bits. All approved hashes provide this.
- **Length extension (Merkle-Damgard):** given $H(m)$ with unknown m , an attacker can compute $H(m || \text{pad} || \text{extension})$ for SHA-256/SHA-512 without knowing m . SHA-3 (sponge) and BLAKE3 are immune.

Choosing a hash — version-pinned

Function	Output	Spec	Construction	Hardware accel	Notes
SHA-256	256	FIPS 180-4	Merkle-Damgard	SHA-NI (x86/ARM)	Conservative default; TLS 1.3 transcript hash
SHA-512/256	256	FIPS 180-4	Merkle-Damgard	64-bit faster than SHA-256	Truncated SHA-512; not vulnerable to length extension in HMAC use
SHA3-256	256	FIPS 202	Sponge (Keccak)	No	Hedge against SHA-2 breaks; no length extension
BLAKE3	256	BLAKE3 spec (2020)	Tree / parallel	SIMD (AVX2/NEON)	Fastest software hash; not NIST-standardized; excellent for content addressing

For content-addressable storage, deduplication, and Merkle trees (Vol 5, Ch 7; Vol 6, Ch 11), BLAKE3's parallel tree mode gives near-linear speedup on large inputs. For interoperability and compliance, SHA-256 remains the safe default — every KMS, HSM, and FIPS module implements it.

```
# OpenSSL – deterministic digests for the same input
$ echo -n "hello" | openssl dgst -sha256
SHA2-256(stdin)= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e730433
62938b9824

$ echo -n "hello" | openssl dgst -sha3-256
SHA3-256(stdin)= 3338be694f50c5f33854368fea51034a4364dd52019b6de187a657
5026b8e9f4d

# BLAKE3 via b3sum (https://github.com/BLAKE3-team/BLAKE3)
$ echo -n "hello" | b3sum
2f151418a81243880bedc5d1af6ab91f93c15380f5ac69ff5ed9e4bced5ff14f -
```

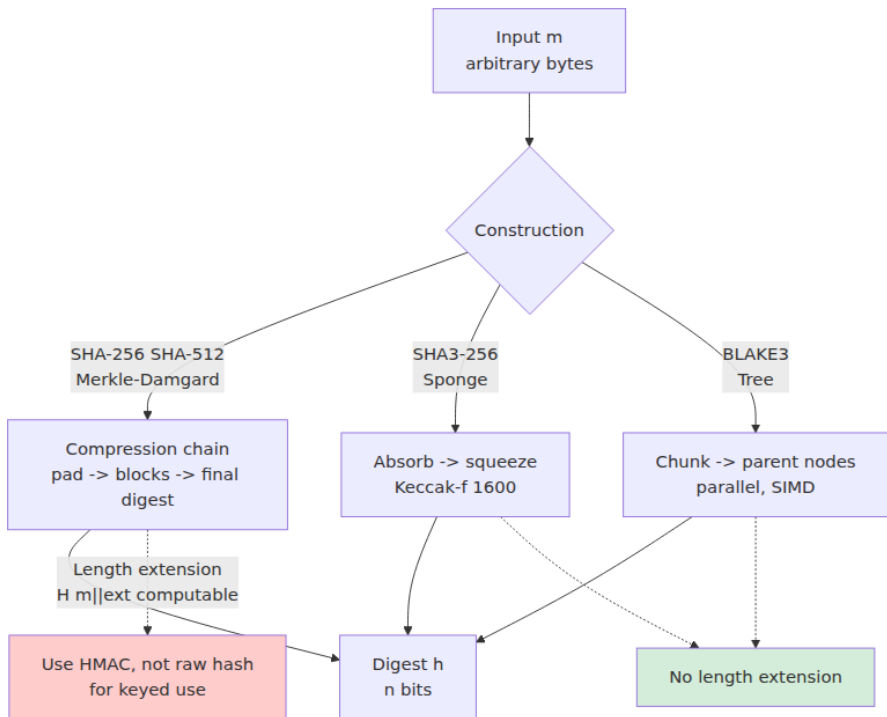
```
import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"

    "golang.org/x/crypto/
blake2b" // BLAKE2b is in x/crypto; BLAKE3 via lukechampine/blake3
)

func hashSHA256(data []byte) string {
    h := sha256.Sum256(data)
    return hex.EncodeToString(h[:])
}

func hashBLAKE2b(data []byte) string {
    h := blake2b.Sum256(data)
    return hex.EncodeToString(h[:])
}

func main() {
    fmt.Println(hashSHA256([]byte("hello")))
    // 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
    fmt.Println(hashBLAKE2b([]byte("hello")))
    //
    e4cfa39a3d37be31c59609e80797079307b24b114c59bcc19813160b5abe84c4e
}
```



Message authentication codes

A MAC answers a different question than a hash. A hash says "this is what was hashed." A MAC says "someone holding key K vouches for this message." Without a MAC (or AEAD), an attacker who can modify bytes in transit or at rest can forge valid-looking data.

HMAC (RFC 2104)

HMAC is the standard keyed-hash construction:

$$\text{HMAC}(K, m) = H((K' \text{ xor opad}) || H((K' \text{ xor ipad}) || m))$$

where K' is K padded/hashed to the block size, $\text{ipad} = 0 \times 36 \dots$, $\text{opad} = 0 \times 5c \dots$. The double-hash structure is what defeats length extension — even with a Merkle-Damgard H .

HMAC-SHA256 is the conservative default. HMAC-SHA512 is faster on 64-bit hardware and gives a longer tag if truncated appropriately. KMAC (Keccak MAC, NIST SP 800-185) is the SHA-3 counterpart but sees less deployment — prefer HMAC-SHA256 unless you have a SHA-3-only requirement.

```
import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
)

func signWebhook(secret, payload []byte) string {
    mac := hmac.New(sh256.New, secret)
    mac.Write(payload)
    return hex.EncodeToString(mac.Sum(nil))
}

func verifyWebhook(secret, payload []byte, claimedHex string) bool {
    claimed, _ := hex.DecodeString(claimedHex)
    mac := hmac.New(sh256.New, secret)
    mac.Write(payload)
    expected := mac.Sum(nil)
    // Constant-time comparison – never use bytes.Equal or == for tags
    return hmac.Equal(expected, claimed)
}

func main() {
    secret := []byte("whsec_very_secret_key_32_bytes!!")
    payload := []byte(`{"event": "payment.succeeded", "id": "evt_123"}`)
    tag := signWebhook(secret, payload)
    fmt.Println(tag)
    // e.g. 7f8a3b2c9e1d04f5... (64 hex chars = 32 bytes)
    fmt.Println(verifyWebhook(secret, payload, tag)) // true
}
```

The constant-time comparison is not optional. A naive `==` leaks timing: it returns faster when the first differing byte is earlier. An attacker can forge a tag byte-by-byte by measuring response time — this has been exploited against real webhook verifiers and token comparators. In Go use `hmac.Equal`, in Python `hmac.compare_digest`, in Node `crypto.timingSafeEqual`.

Flow. Sender computes `tag = HMAC-SHA256(K, message)` and sends `message || tag`. Receiver recomputes `expected = HMAC-SHA256(K,`

message') and checks `hmac.Equal(expected, tag')` in constant time — accept on match, reject without leaking which byte differed on mismatch.

When a MAC is not enough

A MAC authenticates but does not encrypt. If the payload is sensitive, you need both — which is exactly what AEAD (Ch 3) provides in one operation. Use HMAC when:

- You need to authenticate an unencrypted value (webhook signatures, cookie integrity, cache keys).
- You are deriving a subkey with domain separation (HKDF-Expand uses HMAC internally).
- You need a deterministic authenticator for a public value.

Do not use HMAC as a password hash — it is fast by design, which is the opposite of what password storage requires.

Key derivation functions

A KDF turns input keying material of varying quality into one or more uniform, cryptographically strong keys. The right KDF depends entirely on the input's entropy.

HKDF (RFC 5869) — high-entropy inputs

HKDF is for inputs that are already uniformly random or computationally hard to guess: ECDH shared secrets, master keys from a KMS, or session secrets from a handshake. It has two steps:

- **Extract:** `PRK = HMAC-SHA256(salt, IKM)` — concentrates dispersed entropy into a pseudorandom key. Salt can be zero bytes if IKM is already uniform, but a per-context salt is better.
- **Expand:** `OKM = HMAC-SHA256(PRK, info || counter)` — stretches PRK to the desired length with domain separation via `info`.

In TLS 1.3 (RFC 8446, Section 7.1), the entire key schedule is HKDF — handshake secrets, application traffic keys, exporter keys, and resumption PSKs all derive through HKDF with distinct labels. Getting HKDF wrong breaks forward secrecy.

```

import (
    "crypto/sha256"
    "io"

    "golang.org/x/crypto/hkdf"
)

// Derive independent subkeys from one ECDH shared secret
func deriveServiceKeys(ikm, salt []byte) (encKey, macKey []byte) {
    // Domain separation via info – never reuse the same (ikm, salt,
    // info) for two purposes
    r1 := hkdf.New(sh256.New, ikm, salt, []byte("service-a:enc:v1"))
    encKey = make([]byte, 32)
    io.ReadFull(r1, encKey)

    r2 := hkdf.New(sh256.New, ikm, salt, []byte("service-a:mac:v1"))
    macKey = make([]byte, 32)
    io.ReadFull(r2, macKey)
    return
}

```

```

# OpenSSL HKDF via KDF command (OpenSSL 3.x) – derive 32 bytes
$ openssl kdf -keylen 32 -kdfopt digest:SHA2-256 \
  -kdfopt key:"master secret hex" \
  -kdfopt salt:"random salt hex" \
  -kdfopt info:"service-a:enc:v1" HKDF 2>&1 | od -An -tx1
# Output: 32 derived bytes (hex)

```

PBKDF2, scrypt, Argon2 — low-entropy passwords

Human passwords have ~20–40 bits of entropy. An attacker can enumerate the likely space. The KDF's job is to make each guess expensive.

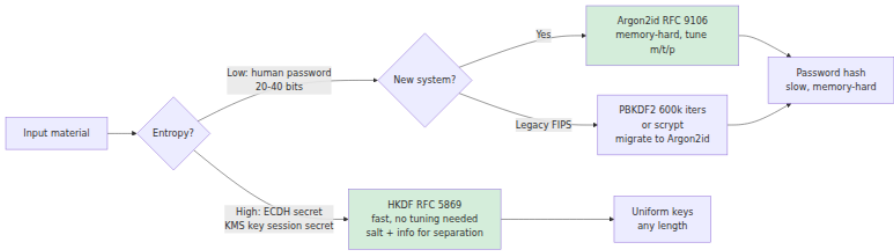
KDF	Spec	Hardness	Parallelism resistance	Recommendation
PBKDF2-HMAC-SHA256	RFC 8018 (PKCS#5)	CPU (iterations)	None — trivially GPU/ASIC-parallel	Legacy only; use for FIPS compliance when required
scrypt	RFC 7914	CPU + memory	Memory-hard; better	Good; superseded by Argon2

KDF	Spec	Hardness	Parallelism resistance	Recommendation
Argon2id	RFC 9106	CPU + memory	Memory-hard, side-channel resistant	Current best practice for password hashing

PBKDF2 with 600,000 iterations (OWASP 2023 guidance) is still seen in compliance-driven systems, but it is not memory-hard — a GPU can try billions of SHA-256 compressions per second with minimal memory. scrypt and Argon2 force each guess to consume significant RAM, raising the attacker's cost by orders of magnitude.

Argon2 variants (RFC 9106):

- **Argon2d** — data-dependent memory access, fastest but side-channel vulnerable.
- **Argon2i** — data-independent access, side-channel resistant but slightly weaker against TMTO.
- **Argon2id** — hybrid: first pass Argon2i, rest Argon2d. **Use Argon2id** unless you have a specific reason not to.



Password storage done correctly

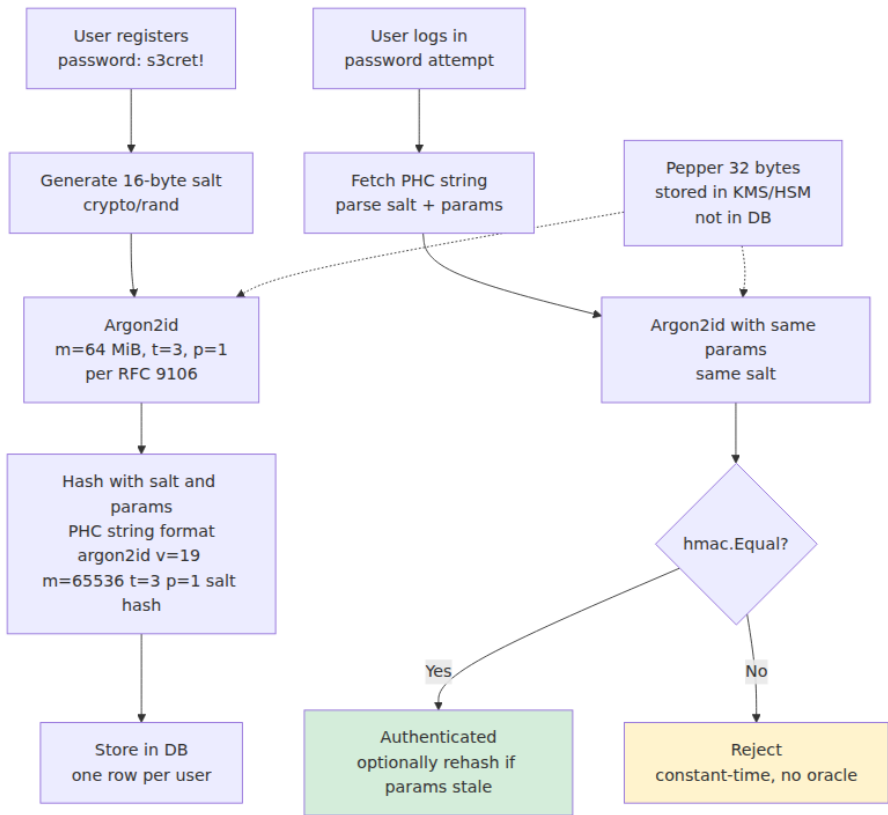
Password storage is not "hash the password." It is: salt, hash with a memory-hard function, store metadata for verification and upgrades, enforce verifier policy, and plan for breach.

NIST SP 800-63B (2023 update) — what changed

NIST SP 800-63B Digital Identity Guidelines (rev. 4 draft, 2023; rev. 3 is 2017) made several changes backend engineers must internalize:

- **No composition rules.** Do not require "one uppercase, one number, one symbol." They reduce entropy and frustrate users into `Password1!.`
- **No periodic rotation.** Do not force password changes on a schedule. Rotate only on compromise indication.
- **Minimum 8 characters, support at least 64.** Check against a breach corpus (e.g., Have I Been Pwned k-anonymity API) and reject known-compromised passwords.
- **Rate limiting and throttling** on the verifier, not just client-side. Slow, memory-hard hashing is part of the throttling.
- **Verifiers must use a suitable KDF** — specifically, a memory-hard function with appropriate cost. PBKDF2 is listed as acceptable, Argon2id and scrypt are preferred per the 2023 guidance and OWASP.

The full storage scheme



Parameters (RFC 9106, Section 4):

- **Memory `m`** : 64 MiB (65536 KiB) is a reasonable floor for interactive login in 2024–2026. Increase with available RAM — the goal is to make each guess consume enough memory that GPU/ASIC parallelism is uneconomical.
- **Iterations `t`** : 3 is the RFC 9106 minimum for Argon2id. Tune so hashing takes ~500 ms–1 s on your login hardware. Measure; do not guess.
- **Parallelism `p`** : 1 for single-threaded login handlers; increase if you have dedicated cores for hashing.
- **Salt**: 16 bytes (128 bits) from `crypto/rand` — unique per password, never reused, never derived from the username.
- **Output length**: 32 bytes (256 bits).

Go implementation — Argon2id with PHC format

```

package main

import (
    "crypto/rand"
    "crypto/subtle"
    "encoding/base64"
    "fmt"
    "log"
    "strings"

    "golang.org/x/crypto/argon2"
)

const (
    argonTime      = 3           // iterations
    argonMemory    = 64 * 1024 // 64 MiB in KiB
    argonThreads   = 1
    argonKeyLen    = 32
    argonSaltLen   = 16
)

// HashPassword returns a PHC-format string:
// $argon2id$v=19$m=65536,t=3,p=1$salt$hash
func HashPassword(password string) (string, error) {
    salt := make([]byte, argonSaltLen)
    if _, err := rand.Read(salt); err != nil {
        return "", err
    }
    hash := argon2.IDKey([]byte(password), salt, argonTime,
        argonMemory, argonThreads, argonKeyLen)

    b64Salt := base64.RawStdEncoding.EncodeToString(salt)
    b64Hash := base64.RawStdEncoding.EncodeToString(hash)
    phc := fmt.Sprintf("$argon2id$v=%d$m=%d,t=%d,p=%d%s%s",
        argon2.Version, argonMemory, argonTime, argonThreads, b64Salt,
        b64Hash)
    return phc, nil
}

// VerifyPassword parses PHC and compares in constant time
func VerifyPassword(phc, password string) (bool, error) {
    parts := strings.Split(phc, "$")
    // ["", "argon2id", "v=19", "m=65536,t=3,p=1", salt, hash]
    if len(parts) != 6 {
        return false, fmt.Errorf("invalid PHC format")
    }
    salt, _ := base64.RawStdEncoding.DecodeString(parts[4])
    expectedHash, _ := base64.RawStdEncoding.DecodeString(parts[5])

    computed := argon2.IDKey([]byte(password), salt, argonTime, argonMe

```

```

mory, argonThreads, argonKeyLen)
    // Constant-time comparison – same principle as HMAC verification
    if subtle.ConstantTimeCompare(expectedHash, computed) == 1 {
        return true, nil
    }
    return false, nil
}

func main() {
    phc, _ := HashPassword("s3cret! correct horse")
    fmt.Println(phc)
    // $argon2id$v=19$m=65536,t=3,p=1$Qj4f...$k9x2...

    ok, _ := VerifyPassword(phc, "s3cret! correct horse")
    fmt.Println("valid:", ok) // true
    ok, _ = VerifyPassword(phc, "wrong")
    fmt.Println("valid:", ok) // false

    // libsodium equivalent (C) – single call, same result:
    // char hash[crypto_pwhash_STRBYTES];
    // crypto_pwhash_str(hash, passwd, len, opslimit, memlimit);
    // // produces $argon2id$v=19$m=... format, verified by
    crypto_pwhash_str_verify
}

```

```

# libsodium CLI via Python binding (for ops verification)
$ python3 -c "
import nacl.pwhash, base64
passwd = b's3cret! correct horse'
h = nacl.pwhash.str(passwd) # Argon2id, opslimit/mmlimit = interactive
print(h.decode())
# \$argon2id\$v=19\$m=65536,t=2,p=1\$...
print(nacl.pwhash.verify(h, passwd)) # True
"

# Benchmark Argon2id cost on your hardware – target ~500ms
$ python3 -c "
import time, nacl.pwhash
passwd = b'benchmark'
start = time.time()
h = nacl.pwhash.str(passwd, opslimit=nacl.pwhash.OPSLIMIT_INTERACTIVE,
memlimit=nacl.pwhash.MEMLIMIT_INTERACTIVE)
print(f'hash time: {time.time()-start:.3f}s')
# hash time: 0.642s (tune opslimit/memlimit until this is ~0.5-1.0s on
your login hosts)
"

```


Peppering

A **pepper** is a 32-byte secret stored separately from the database — in a KMS, HSM, or environment-injected secret (Ch 8) — and mixed into the hash input: `Argon2id(password || pepper, salt)`. If the database is exfiltrated without the pepper, offline cracking is infeasible even for weak passwords. The pepper should be versioned (`pepper_v1` , `pepper_v2`) and rotated by rehashing on next login, same as KDF parameters.

Peppering is defense-in-depth, not a substitute for a strong KDF. It also creates an operational dependency: losing the pepper locks all users out. Store it in a durable, replicated KMS — not a single config file.

Migration without mass reset

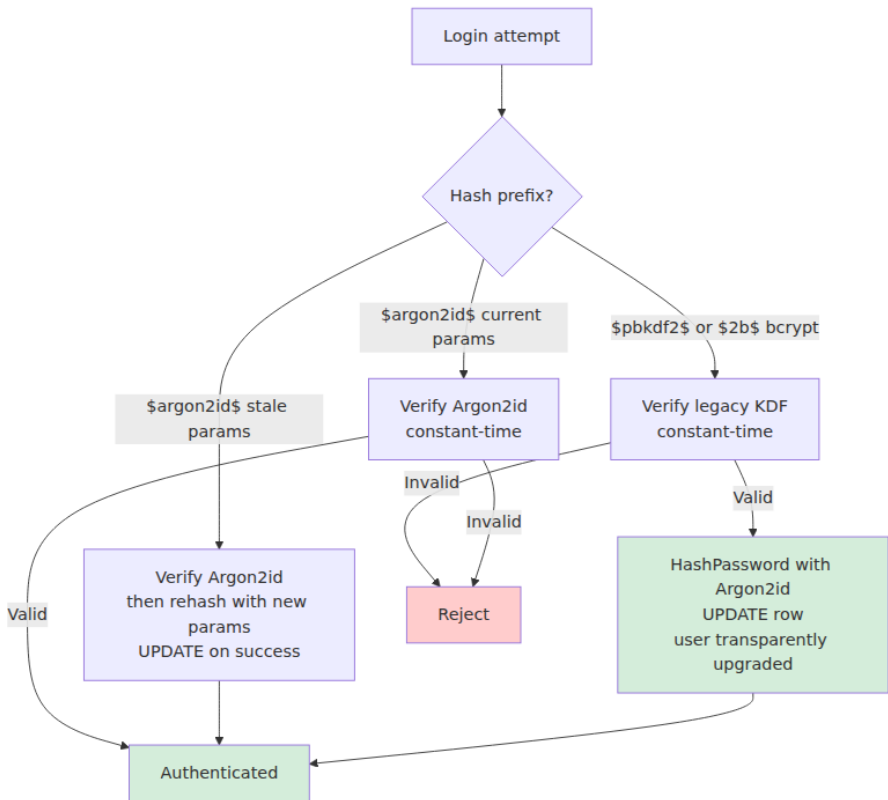
Most teams inherit a weaker scheme — SHA-256, PBKDF2 with low iterations, or bcrypt. Forcing every user to reset their password at once creates support load and drives users to weaker, reused passwords. Migrate incrementally:

```

// Upgrade path: verify with old hash, then rehash with Argon2id on
success
func VerifyAndUpgrade(phcOrLegacy, password string) (newPHC string, ok
bool) {
    // 1. Detect format by prefix
    if strings.HasPrefix(phcOrLegacy, "$argon2id$") {
        ok, _ := VerifyPassword(phcOrLegacy, password)
        if !ok {
            return "", false
        }
        // Optionally rehash if params are stale (e.g., memory
increased)
        if paramsNeedUpgrade(phcOrLegacy) {
            newPHC, _ := HashPassword(password)
            return newPHC,
true // caller should UPDATE users SET phc = newPHC
        }
        return "", true
    }
    // 2. Legacy path – e.g., PBKDF2 or bcrypt
    if verifyPBKDF2(phcOrLegacy, password) { // constant-time inside
        newPHC, _ := HashPassword(password) // upgrade to Argon2id now
that we have plaintext
        return newPHC, true
    }
    return "", false
}

func paramsNeedUpgrade(phc string) bool {
    // Parse m=,t=,p= and compare to current policy
    // If stored m < 65536 or t < 3, return true
    return false // simplified
}

```



Distributed-systems note. Password-hash upgrades are eventually consistent by nature — each user's hash upgrades on their next login, so the database contains a mix of schemes for weeks or months. Queries must handle both formats. Do not run a batch job that rehashes without the plaintext — you cannot upgrade a hash without the password. The `VerifyAndUpgrade` pattern above is the standard technique used by large identity providers to roll forward KDF parameters without downtime.

Timing side-channels and verifier hardening

Constant-time comparison is necessary but not sufficient. Verifiers leak timing in other places that attackers exploit when they can make many online guesses.

Early exit on user lookup. If your login handler returns faster for "user not found" than for "user found, password wrong" — because it skips the Argon2id computation — an attacker can enumerate valid usernames by measuring response time. The fix is to always execute a dummy hash when the user does not exist, or to return in constant time regardless of the lookup result. The same pattern applies to HMAC verification for webhooks: always compute the expected tag even when the request is otherwise malformed.

Variable-time KDF comparison. Some frameworks compare password hashes with `==` on hex strings or with `bytes.Equal` on raw digests. Both short-circuit. Use `subtle.ConstantTimeCompare` or `hmac.Equal` as shown above. For PHC strings, decode the hash bytes before comparing — comparing the base64 strings in variable time leaks length and prefix information.

Cache and branch predictors. On shared hardware, cache-timing and branch-predictor side channels can leak HMAC and KDF intermediate state. For HMAC verification on high-value paths (token verification, KMS request authentication), prefer implementations that use constant-time primitives end-to-end. For Argon2id, the `p` parameter controls parallelism but also affects side-channel surface; `p=1` is simplest for login handlers that do not need multi-core throughput.

Rate limiting as a timing control. Even with constant-time verification, online guessing must be bounded. NIST SP 800-63B requires the verifier to limit consecutive failed attempts, with exponential backoff per account and per IP, and to alert on credential-stuffing patterns. Argon2id's 500 ms cost is part of this budget: it bounds the verifier's throughput to roughly 2 guesses per second per core, which composes with network-level rate limiting to make online attacks infeasible while keeping offline attacks expensive through memory hardness.

Breach response and rotation playbook

Assume the credential database will be exfiltrated — plan for it. The playbook has four stages.

1. Detection. Monitor for anomalous bulk reads on the users table, unexpected replication lag, or access from unrecognized principals. Credential databases should emit audit logs for every bulk query, with alerting on thresholds. Have I Been Pwned and similar corpora provide a

post-breach check: after any incident, test whether new credentials appear in public dumps.

2. Containment. Rotate the pepper immediately in KMS — new hashes use the new pepper, old hashes remain verifiable during the dual-read window. Do not invalidate all sessions at once unless the session store is also compromised; mass invalidation creates a thundering herd on the login path that is itself a denial-of-service vector. Instead, mark sessions as requiring step-up authentication on next request.

3. Forced upgrades where needed. For users whose passwords were weak (those that would have been rejected by the breach-corpus check had it been in place), force a password reset with an email that does not contain a reset link — direct users to navigate to the site independently to avoid phishing. For users with strong, uncompromised passwords, transparent upgrade via `VerifyAndUpgrade` on next login is sufficient.

4. Parameter hardening. Use the incident as justification to increase Argon2id memory or time parameters. The upgrade is incremental and requires no mass reset — stale hashes rehash on next successful authentication. Document the new parameters as `argon2id:v2` so future audits can distinguish cohorts.

Hashing at scale: caching, sharding, and DoS resistance

Hash functions sit on hot paths — request deduplication, cache keys, sharding, and content addressing — where performance and adversarial robustness matter as much as collision resistance. Three operational concerns recur.

Hash flooding and DoS. If an attacker controls the keys you hash — for example, JSON object keys, HTTP headers, or hash-table bucket selection — a non-cryptographic hash like MurmurHash or FNV lets the attacker craft many keys that collide, turning $O(1)$ hash-table operations into $O(n)$ chains and causing CPU exhaustion. This was exploited against language runtimes in 2011–2012 (the `oCERT-2011-003` hash-flooding attacks against Python, Ruby, Java, and others). The fix is to use a keyed hash (SipHash, BLAKE3 keyed mode, or HMAC-SHA256 truncated) with a per-process random key for hash-table randomization, or to use a cryptographic hash where the attacker controls the input. Deterministic sharding hashes (e.g., consistent hashing for request routing) must also use a

cryptographic hash if the sharding key is externally supplied — otherwise an attacker can force all requests onto one shard.

Content addressing and deduplication. On the storage side, hashes are addresses. Container image layers (OCI digests), Git objects, and CAS blocks all use SHA-256 hex digests as names. Collision resistance is load-bearing: a collision means two distinct objects share an address, causing silent data loss or cache poisoning. For large objects, BLAKE3's tree mode gives near-linear parallel speedup by hashing chunks independently and combining — a 1 GiB file hashes in roughly half the time on a 4-core machine compared to SHA-256's serial Merkle-Damgard chain. For small keys (cache lookups, dedup at row level), SHA-256's hardware acceleration makes the difference negligible — choose based on interoperability rather than microbenchmarks.

HMAC for cache and queue integrity. Distributed caches (Redis, Memcached) and queues (SQS, Kafka) are often treated as trusted, but a compromised cache or a confused-deputy write can poison downstream consumers. Adding an HMAC tag to cached values — `value || HMAC-SHA256(cacheKey, value)` — lets readers detect tampering without encrypting the value. The cost is 32 bytes per entry and one HMAC on read/write, which is negligible compared to network latency. For queues where the producer and consumer share a key, the same pattern authenticates messages end-to-end even when the broker is untrusted. When the key cannot be shared — for example, an external webhook producer — the consumer verifies with the producer's public key via Ed25519 instead, which shifts key distribution from symmetric sharing to PKI.

Choosing parameters under concurrency constraints

Argon2id's memory parameter competes with the rest of the service for RAM. A login handler that hashes at 64 MiB per attempt and handles 50 concurrent logins holds 3.2 GiB in Argon2 working sets alone, plus heap, GC, and request buffers. On a 4 GiB container, this causes OOM kills under burst load — precisely when you can least afford them during a credential-stuffing attack.

Three strategies bound the memory. First, cap concurrent hashes with a semaphore: queue excess login attempts rather than hashing them in parallel, returning `429 Too Many Requests` with `Retry-After` when the

queue is full. Second, tune `p` (parallelism) to 1 for handlers that are already concurrent at the request level — intra-hash parallelism only helps when the handler is otherwise idle. Third, consider libsodium's `crypto_pwhash` `OPSLIMIT_INTERACTIVE` / `MEMLIMIT_INTERACTIVE` presets, which are calibrated for interactive login, or its `OPSLIMIT_SENSITIVE` / `MEMLIMIT_SENSITIVE` for high-value credentials where slower verification is acceptable. Measure peak RSS under simulated burst, not just mean hash latency, and set container memory limits to accommodate the worst case plus headroom for GC.

Pepper rotation and incident-triggered rehashing

Pepper rotation deserves its own runbook because it touches every stored hash and the coordination is easy to get wrong. Keep peppers versioned in KMS — `pepper_v1`, `pepper_v2` — and store the version that was used alongside each hash, either as a column (`pepper_version`) or embedded in the PHC string's associated data. Verification tries the current pepper first and falls back to the previous one during the rotation window, which is the same dual-read pattern used for Argon2 parameter upgrades and for KEK rotation in Chapter 3. Rehashing to the new pepper happens lazily on the next successful login, when the plaintext is available, so no bulk plaintext recovery is needed. For high-value accounts where waiting for the next login is unacceptable, trigger an out-of-band step-up flow that asks the user to re-authenticate explicitly, which gives you the plaintext for immediate rehashing. The old pepper remains readable until the last hash has migrated — track migration progress with a metric (`users.pepper_version == v1`) and alert if the tail stalls. Only after the metric hits zero should the old pepper be scheduled for deletion in KMS, with a 7 to 30 day recovery window in case of rollback.

Operational guidance

Benchmark on production-like hardware. Argon2's cost is hardware-dependent. A parameter set that takes 500 ms on your M-series laptop may take 2 s on a burstable cloud VM with noisy neighbors. Benchmark on the actual login fleet, under load, with memory limits that account for concurrent logins ($64 \text{ MiB} \times 100 \text{ concurrent hashes} = 6.4 \text{ GiB}$).

Do not cap password length below 64. Long passphrases are the strongest passwords. Argon2 handles arbitrary-length inputs. If you must cap for DoS protection, cap at 512-1024 bytes and document it —

truncating silently at 72 bytes (the bcrypt limit) has caused credential bugs where `very-long-password + "a"` and `very-long-password + "b"` both authenticate.

Rate limiting is mandatory. Even with Argon2id, online guessing must be throttled: exponential backoff per account, CAPTCHA after N failures, and alerting on credential-stuffing patterns. NIST SP 800-63B requires the verifier to limit consecutive failed attempts.

Breach-corpus checks. On registration and password change, check the candidate against a corpus of known-compromised passwords (HIBP Pwned Passwords k-anonymity API: send first 5 hex chars of SHA-1, receive suffix list, compare locally — the full password never leaves your service).

Distributed-systems lens

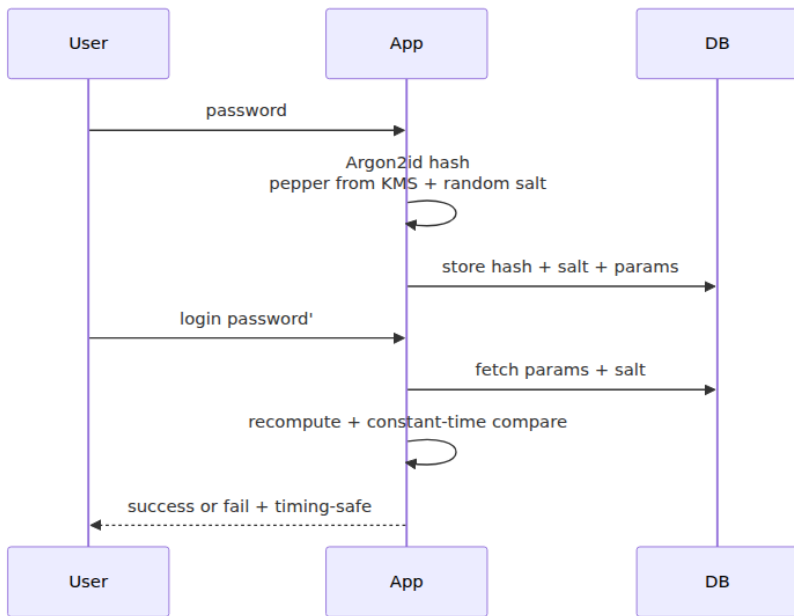
Hashing and KDF choices have fleet-wide consequences.

Consistency of pepper and parameters. If the Argon2 pepper or cost parameters differ across replicas, a password hashed on one replica may fail verification on another. Distribute pepper via a strongly consistent store (Ch 8 — Vault, KMS) and roll parameter changes as versioned config with dual-read windows, exactly like key rotation in Ch 1.

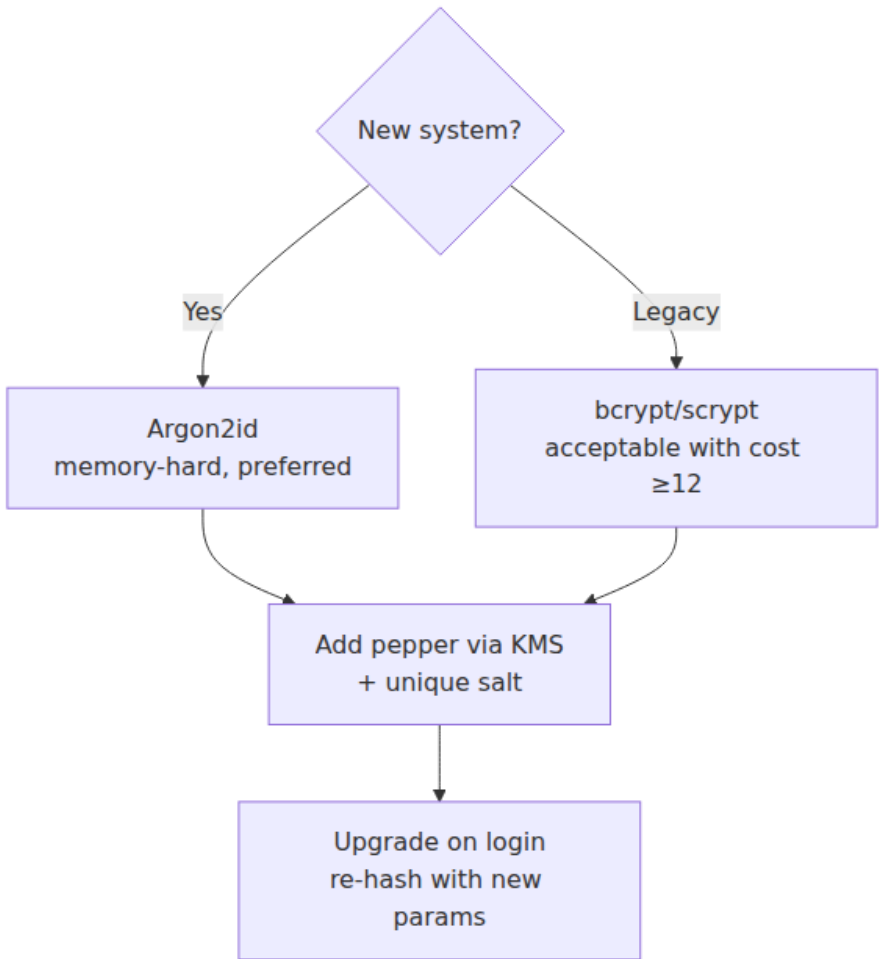
Hash agility. Every stored hash should be self-describing (PHC format) so verifiers can select the correct algorithm and parameters. This is the same principle as key identifiers (`kid`) in JWTs (Ch 5) and envelope-encryption key versions (Ch 3, Ch 8). Without agility, a fleet-wide KDF upgrade requires synchronized deploys.

Deduplication and content addressing. Where hashes are used for deduplication (CAS, Merkle trees, container layer digests in Vol 12), collision resistance is load-bearing — a collision means two distinct objects map to one address, causing silent data loss. Use at least 256-bit output and consider BLAKE3 for large objects where parallel hashing matters. Where hashes are used for sharding or load balancing, uniformity matters more than collision resistance — but still use a cryptographic hash if the input is attacker-controlled, to prevent hash-flooding DoS (Vol 14, Ch 2).

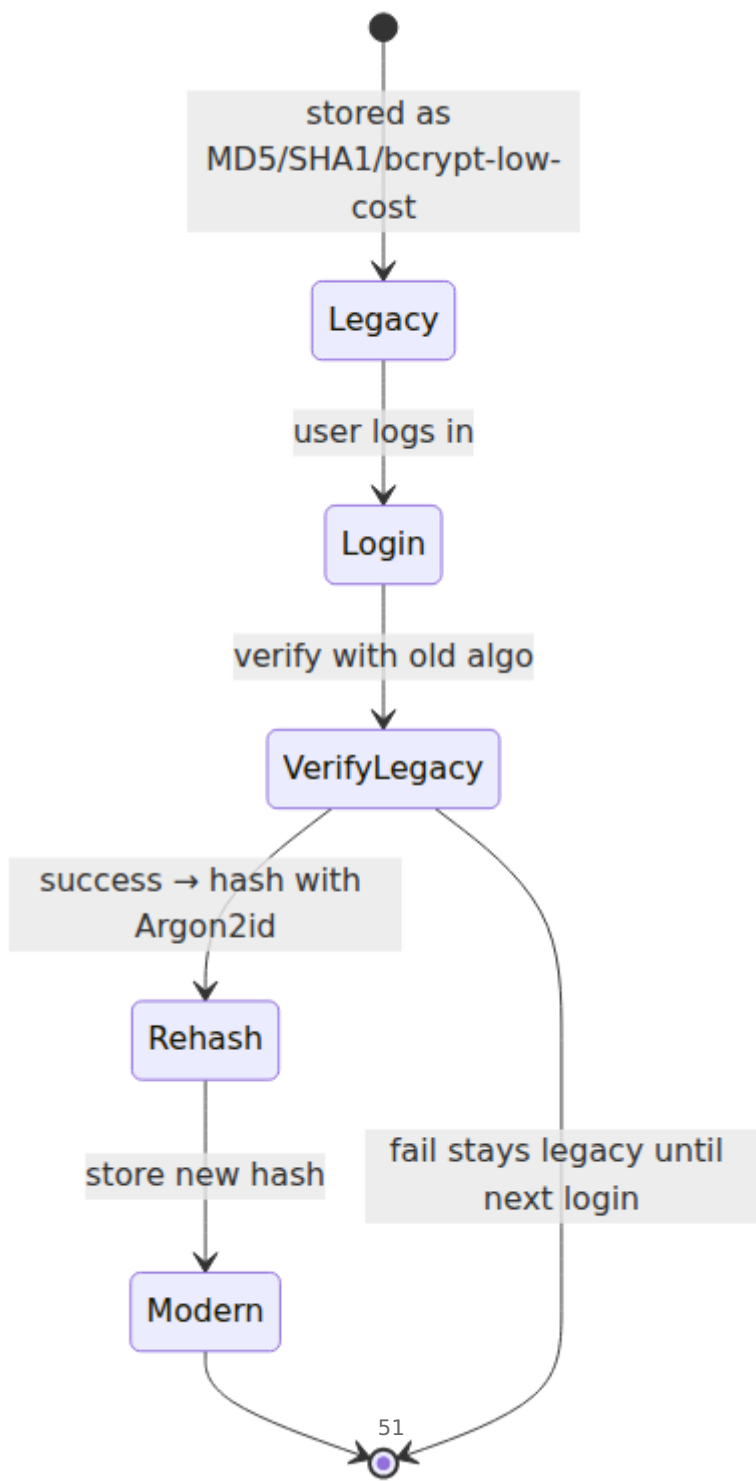
Password Hashing Flow



Hash Algorithm Choice



Weak Hash Migration



Key takeaways

- Hash functions provide preimage ($\sim 2^n$), second-preimage ($\sim 2^n$), and collision ($\sim 2^{(n/2)}$) resistance — collisions fail first, which is why MD5 and SHA-1 are broken and SHA-256/SHA-3-256/BLAKE3 are the current choices (FIPS 180-4, FIPS 202).
- Never use a raw hash as a MAC — length extension on SHA-256/SHA-512 lets an attacker forge `H(key || message || extension)`. Use HMAC-SHA256 (RFC 2104) or an AEAD, and always verify tags in constant time (`hmac.Equal`, `subtle.ConstantTimeCompare`).
- KDF selection is determined by input entropy: HKDF (RFC 5869) for high-entropy secrets (ECDH, KMS keys), Argon2id (RFC 9106) for low-entropy passwords. PBKDF2 (RFC 8018) and scrypt (RFC 7914) are legacy — migrate to Argon2id.
- Argon2id parameters must be tuned on production hardware: 64 MiB / $t=3$ / $p=1$ is a floor, target ~ 500 ms-1 s per hash, 16-byte random salt, 32-byte output, PHC string format for agility.
- Follow NIST SP 800-63B (2023): no composition rules, no periodic rotation, minimum 8 characters (support 64+), breach-corpus checks, rate limiting, and a memory-hard verifier.
- Peppering (KMS-held secret mixed into the hash) adds defense-in-depth against DB exfiltration but creates an operational dependency — version and replicate the pepper.
- Migrate hashes incrementally with `VerifyAndUpgrade` — verify with the old scheme, rehash with Argon2id on success, never force a mass reset and never rehash without the plaintext.

Further reading

- **Standards:** FIPS 180-4 (SHA-2), FIPS 202 (SHA-3), RFC 2104 (HMAC), RFC 5869 (HKDF), RFC 7914 (scrypt), RFC 8018 (PKCS#5 / PBKDF2), RFC 9106 (Argon2), NIST SP 800-63B (Digital Identity Guidelines, 2023 update), NIST SP 800-185 (KMAC).
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html — concrete Argon2/scrypt/PBKDF2 parameter guidance, updated 2023.

- Latacora — *How to Safely Store a Password* (2018, updated 2023) — <https://latacora.micro.blog/> — concise practitioner guidance aligned with this chapter.
- BLAKE3 specification — <https://github.com/BLAKE3-team/BLAKE3-specs> — performance and tree-mode details.
- libsodium `crypto_pwhash` docs — https://doc.libsodium.org/password_hashing — the reference implementation for Argon2id password hashing.

Chapter 3 — Symmetric and Asymmetric Encryption in Practice

What this chapter covers. Chapter 1 gave you the primitive taxonomy and Chapter 2 covered hashing, MACs, and KDFs. This chapter is about the operation that most engineers mean when they say "encryption": turning plaintext into ciphertext that is confidential and tamper-evident, storing or transmitting it through untrusted infrastructure, and recovering it only where the key is available. We treat symmetric encryption (one shared key) and asymmetric encryption (public key encrypts, private key decrypts) as distinct tools with different trust models, performance envelopes, and failure modes, and we show the hybrid construction that every real system uses to get the best of both. Along the way we cover the only safe interface for symmetric encryption — AEAD — and its two instantiations you will actually deploy (AES-GCM and ChaCha20-Poly1305), the nonce discipline that determines whether those AEADs remain safe when many replicas share a key, envelope encryption as the pattern that makes key rotation and KMS integration tractable at fleet scale, and the asymmetric options that remain viable in 2024–2026 (X25519-based ECIES and RSA-OAEP) versus the ones that do not (RSA PKCS#1 v1.5, unauthenticated RSA, textbook ElGamal). Every section includes real Go, OpenSSL, and libsodium code with realistic outputs.

Learning goals — after this chapter you should be able to:

- Use AES-GCM and ChaCha20-Poly1305 (RFC 5116, RFC 8439) correctly via Go `crypto/cipher`, OpenSSL, and libsodium — including nonce generation, AAD binding, and tag verification.

- Explain why unauthenticated modes (AES-CBC, AES-CTR alone) are broken for backend use and why AEAD is the only safe default.
- Reason about nonce reuse consequences quantitatively and choose among random 96-bit, counter, and XChaCha20-Poly1305 (192-bit) strategies for single-process vs distributed encryptors.
- Implement envelope encryption (DEK + KEK) with a KMS, reason about rotation without re-encrypting all data, and explain the blast-radius and latency trade-offs.
- Choose between RSA-OAEP (RFC 8017) and X25519-based hybrid encryption (RFC 7748 + AEAD) for public-key encryption, and explain why RSA PKCS#1 v1.5 must not be used.
- Trace the full lifecycle of an encryption key — generation, distribution, use, rotation, and destruction — in a multi-service, multi-region deployment.
- Identify the boundary between this chapter's at-rest / message-level encryption and Vol 3, Ch 6's transport encryption (TLS 1.3, RFC 8446) and Vol 9, Ch 4's PKI operations.

Boundary notes. This chapter covers *message-level and at-rest encryption* — encrypting fields, files, queue messages, and database rows with keys your services manage. *Transport encryption* — the TLS 1.3 handshake (RFC 8446), record layer, session resumption, and the Web PKI chain that authenticates the peer — is Vol 3, Ch 6. The *operational* side of that PKI — certificate issuance (ACME), rotation, revocation, CT logs, and mTLS deployment — is Vol 9, Ch 4. If you are deciding between "should I encrypt this payload myself" vs "is TLS enough," read this chapter's distributed-systems lens alongside Vol 3, Ch 6.

The one safe interface: AEAD

An AEAD (Authenticated Encryption with Associated Data, RFC 5116) does two things in one operation: it encrypts the plaintext so only key holders can read it, and it authenticates both the ciphertext and an additional cleartext header (AAD) so any tampering is detected. The API is:

```
(ciphertext, tag) = AEAD.Encrypt(key, nonce, plaintext, aad)
plaintext = AEAD.Decrypt(key, nonce, ciphertext, tag, aad) // or error
```

If any bit of `ciphertext`, `tag`, `nonce`, or `aad` is modified, `Decrypt` returns an error and no plaintext. There is no "decrypt but the tag was wrong, let me still look at the data" — that path has produced real vulnerabilities (padding oracles, Efail).

The two AEADs to use in new systems:

AEAD	Spec	Key	Nonce	Tag	Performance	Use when
AES-GCM	NIST SP 800-38D, RFC 5116	128/192/256 bits	96 bits (12 B)	128 bits	Fast with AES-NI + CLMUL (x86/ARMv8)	Servers, cloud VMs, anywhere hardware AES is available
ChaCha20-Poly1305	RFC 8439	256 bits	96 bits (12 B)	128 bits	Fast in pure software, constant-time	Mobile, embedded, Wasm, or where AES timing side-channels are a concern
XChaCha20-Poly1305	libsodium / draft-irtf-cfrg-xchacha	256 bits	192 bits (24 B)	128 bits	Same as ChaCha, larger nonce	Distributed encryptors that cannot coordinate nonces

AES-256-GCM and ChaCha20-Poly1305 provide equivalent security at the 128-bit level — the choice is about hardware and nonce management, not strength.

Why unauthenticated encryption is broken

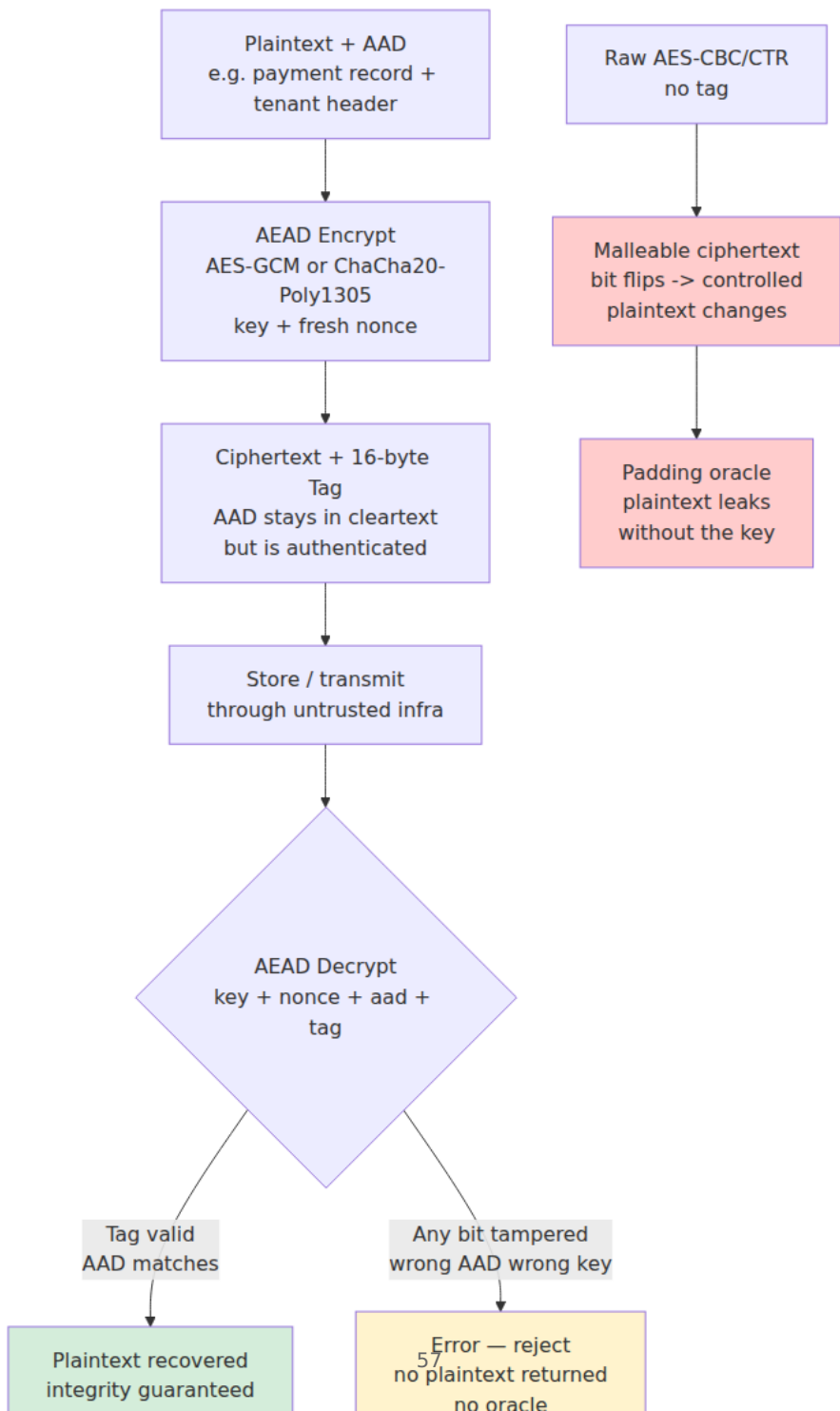
AES-CBC and AES-CTR alone provide confidentiality but not integrity. An attacker who can flip bits in the ciphertext produces predictable changes in the plaintext:

- **CBC bit-flipping:** flipping a bit in ciphertext block `Ci` flips the same bit in `Pi+1` after decryption. If the plaintext is JSON like

`{"role":"user"}` , flipping bits can change it to `{"role":"admin"}` without knowing the key.

- **CTR malleability:** CTR is a stream cipher — `ciphertext = plaintext xor keystream` . Flipping a ciphertext bit flips the corresponding plaintext bit directly.
- **Padding oracles (Vaudenay, 2002):** CBC with PKCS#7 padding leaks plaintext byte-by-byte when the server returns distinct errors for "bad padding" vs "bad MAC." This class broke TLS 1.0's CBC ciphersuites (Lucky13), XML Encryption, and countless custom protocols.

AEAD eliminates all three by authenticating the ciphertext. If you inherit a system using raw CBC/CTR, the migration is to wrap it in HMAC (encrypt-then-MAC with independent keys and constant-time verification) or — better — replace it with an AEAD.



AES-GCM and ChaCha20-Poly1305 in Go

```

package main

import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/rand"
    "encoding/hex"
    "fmt"
    "io"
    "log"

    "golang.org/x/crypto/chacha20poly1305"
)

// --- AES-GCM (server default with AES-NI) ---

func encryptAESGCM(key, plaintext, aad []byte) (nonce, ciphertext []byte) {
    block, err :=
aes.NewCipher(key) // 16, 24, or 32 bytes; use 32 for AES-256
    if err != nil {
        log.Fatal(err)
    }
    aead, err := cipher.NewGCM(block)
    if err != nil {
        log.Fatal(err)
    }
    nonce = make([]byte, aead.NonceSize()) // 12 bytes
    if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
        log.Fatal(err)
    }
    // Seal appends ciphertext || tag (16 bytes); aad is authenticated
    // but not encrypted
    ciphertext = aead.Seal(nil, nonce, plaintext, aad)
    return
}

func decryptAESGCM(key, nonce, ciphertext, aad []byte) ([]byte, error) {
    block, err := aes.NewCipher(key)
    if err != nil {
        return nil, err
    }
    aead, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }
    return aead.Open(nil, nonce, ciphertext, aad) // error if tag
    // invalid or aad mismatch
}

```

```
// --- ChaCha20-Poly1305 (software, constant-time) ---

func encryptChaCha(key, plaintext, aad []byte) (nonce, ciphertext []byte) {
    aead, err := chacha20poly1305.New(key) // 32-byte key
    if err != nil {
        log.Fatal(err)
    }
    nonce = make([]byte, aead.NonceSize()) // 12 bytes
    io.ReadFull(rand.Reader, nonce)
    ciphertext = aead.Seal(nil, nonce, plaintext, aad)
    return
}

// --- XChaCha20-Poly1305 (192-bit nonce – for distributed encryptors)
// ---

func encryptXChaCha(key, plaintext, aad []byte) (nonce, ciphertext []byte) {
    aead, err := chacha20poly1305.NewX(key) // 32-byte key, 24-byte nonce
    if err != nil {
        log.Fatal(err)
    }
    nonce = make([]byte, aead.NonceSize()) // 24 bytes – safe to generate randomly forever
    io.ReadFull(rand.Reader, nonce)
    ciphertext = aead.Seal(nil, nonce, plaintext, aad)
    return
}

func main() {
    key := make([]byte, 32)
    rand.Read(key)

    plaintext := []byte(`{"amount":100,"to":"alice","id":"txn_8841"}`)
    aad := []byte("tenant=acme:v1") // authenticated, not encrypted – e.g. routing header

    // AES-GCM round-trip
    nonce, ct := encryptAESGCM(key, plaintext, aad)
    fmt.Printf("AES-GCM nonce: %s (%d bytes)\n", hex.EncodeToString(nonce), len(nonce))
    fmt.Printf("AES-GCM ct+tag: %s (%d bytes, plaintext was %d + 16 tag)\n", hex.EncodeToString(ct[:16])+"...", len(ct), len(plaintext))

    recovered, err := decryptAESGCM(key, nonce, ct, aad)
    fmt.Printf("decrypt ok: %v plaintext: %s\n", err == nil, recovered)
}
```

```

// Tamper detection – flip one byte
ct[0] ^= 0x01
_, err = decryptAESGCM(key, nonce, ct, aad)
fmt.Printf("after tamper: %v\n", err) // cipher: message
authentication failed

// AAD mismatch – same ciphertext, wrong header
_, ct2 := encryptChaCha(key, plaintext, aad)
// (nonce2 distinct – not shown; decrypt with wrong AAD)
_ = ct2
}

```

Realistic output:

```

AES-GCM nonce: 7f3a9c1e4b2d08f6a1c5e9b3 (12 bytes)
AES-GCM ct+tag: 9cle4f8a03b2d671... (58 bytes, plaintext was 42 + 16 tag)
decrypt ok: true plaintext:
{"amount":100,"to":"alice","id":"txn_8841"}
after tamper: cipher: message authentication failed

```

```

# OpenSSL – AES-GCM for ops debugging (tool-specific format, not wire-
interop)
$ KEY=$(openssl rand -hex 32)
$ IV=$(openssl rand -hex 12)
$ echo -n '{"amount":100,"to":"alice"}' | openssl enc -aes-256-gcm \
-K "$KEY" -iv "$IV" -a
# Output: base64 blob (ciphertext + tag concatenated – OpenSSL CLI
appends tag opaquely)
# Prefer Go/libsodium in services; OpenSSL CLI GCM is for manual
inspection, not production wire format.

# libsodium sealed box – X25519 + XSalsa20-Poly1305 (C)
# unsigned char pk[crypto_box_PUBLICKEYBYTES],
sk[crypto_box_SECRETKEYBYTES];
# crypto_box_keypair(pk, sk);
# crypto_box_seal(ciphertext, message, mlen, pk); // anonymous
sender, no nonce management

```

Nonce discipline — the one rule that breaks everything

Both AES-GCM and ChaCha20-Poly1305 are broken on nonce reuse with the same key. For GCM, reusing a nonce leaks the authentication key H and then the XOR of the two plaintexts. For ChaCha20-Poly1305, it leaks the plaintext XOR directly. There is no "mostly safe" — one reuse is enough.

Strategy	Nonce size	Safe messages per key	Coordination needed	When to use
Random 96-bit (GCM/ChaCha)	96 bits	$\sim 2^{32}$ (birthday bound at 2^{48} , NIST SP 800-38D limit)	None	Single-process encryptor, low volume
Counter 96-bit	96 bits	2^{96} (if counter never repeats)	Persistent, crash-safe counter	Single writer with durable counter
XChaCha20-Poly1305 random 192-bit	192 bits	$\sim 2^{96}$ (effectively unlimited)	None	Distributed encryptors sharing a key
Partitioned: 32-bit replica ID + 64-bit counter	96 bits	2^{64} per replica	Replica ID assignment	Fleet sharing one key with coordination

Distributed-systems pitfall. Two replicas sharing an AES-GCM key and generating random 96-bit nonces independently will eventually collide — the birthday bound is $\sim 2^{48}$ messages total across all replicas, which a busy fleet reaches faster than expected. Three safe patterns: (a) give each replica a distinct key via HKDF with replica ID as `info` (Ch 1), (b) use XChaCha20-Poly1305 with 192-bit random nonces, or (c) partition the nonce space. Pattern (b) is simplest for new systems.

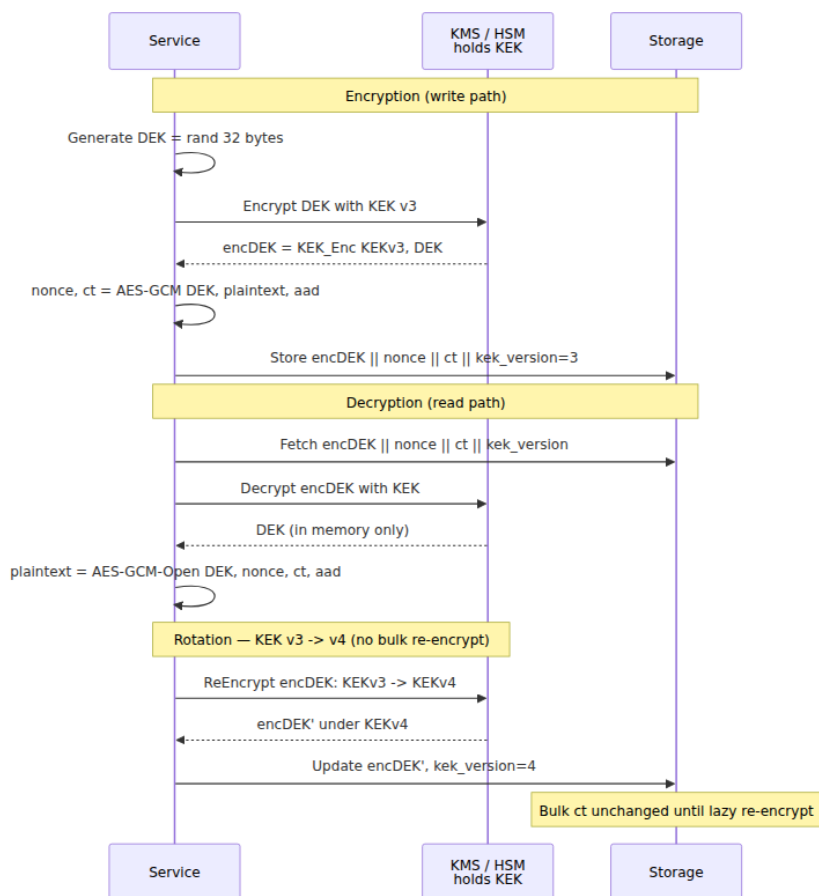
Nonce strategy for shared keys. Where replicas cannot coordinate, XChaCha20-Poly1305 with a 192-bit random nonce is safe without coordination. Where per-replica keys are acceptable, derive subkeys via HKDF with the replica ID as `info`. Where a shared key is required, partition the 96-bit nonce space (4-byte replica ID + 8-byte counter). Shared-key random 96-bit nonces without coordination collide near 2^{48} messages and leak the authentication key.

Envelope encryption: the pattern that makes key management tractable

Encrypting every row or file directly with a KMS key is slow (one KMS call per encrypt/decrypt), couples availability to the KMS, and makes rotation expensive (re-encrypt everything). Envelope encryption solves all three:

- **Data Encryption Key (DEK):** a 256-bit random key that encrypts one item (or a small batch). Generated locally via `crypto/rand`.
- **Key-Encrypting Key (KEK):** a long-lived key held in a KMS/HSM (AWS KMS, GCP KMS, Vault Transit, CloudHSM) that encrypts DEKs. Never leaves the KMS.
- **Envelope:** `KEK_Encrypt(DEK) || AEAD_Encrypt(DEK, plaintext)`. Store the encrypted DEK alongside the ciphertext.

Decryption fetches the KEK-encrypted DEK, calls KMS `Decrypt` to recover the DEK, then decrypts locally. Rotation re-encrypts only the DEKs under the new KEK — the bulk ciphertext is untouched until you choose to re-encrypt it.




```
// Envelope encryption with a KMS-like interface
package envelope

import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/rand"
    "encoding/base64"
    "io"
)

// KMS abstracts AWS KMS, GCP KMS, or Vault Transit
type KMS interface {
    Encrypt(keyID string, plaintext []byte) ([]byte, error)
    Decrypt(ciphertext []byte) ([]byte, error)
}

type Envelope struct {
    EncDEK      string `json:"enc_dek"`      // base64 KMS ciphertext
    Nonce       string `json:"nonce"`       // base64 12-byte nonce
    Ciphertext  string `json:"ct"`           // base64 AEAD output (ct +
tag)
    KEKVersion  string `json:"kek_version"` // for rotation tracking
    AAD         string `json:"aad"`         // authenticated header
    bound at decrypt
}

func Encrypt(kms KMS, kekID string, plaintext, aad []byte) (*Envelope,
error) {
    // 1. Generate DEK locally – never reuse
    dek := make([]byte, 32)
    if _, err := io.ReadFull(rand.Reader, dek); err != nil {
        return nil, err
    }
    // 2. Wrap DEK with KEK in KMS
    encDEK, err := kms.Encrypt(kekID, dek)
    if err != nil {
        return nil, err
    }
    // 3. Encrypt payload with DEK via AES-GCM
    block, _ := aes.NewCipher(dek)
    aead, _ := cipher.NewGCM(block)
    nonce := make([]byte, aead.NonceSize())
    io.ReadFull(rand.Reader, nonce)
    ct := aead.Seal(nil, nonce, plaintext, aad)

    return &Envelope{
        EncDEK:      base64.StdEncoding.EncodeToString(encDEK),
        Nonce:       base64.StdEncoding.EncodeToString(nonce),
        Ciphertext:  base64.StdEncoding.EncodeToString(ct),
    }, nil
}
```

```

        KEKVersion: kekID,
        AAD:         string(aad),
    }, nil
}

func Decrypt(kms KMS, env *Envelope) ([]byte, error) {
    encDEK, _ := base64.StdEncoding.DecodeString(env.EncDEK)
    nonce, _ := base64.StdEncoding.DecodeString(env.Nonce)
    ct, _ := base64.StdEncoding.DecodeString(env.Ciphertext)

    dek, err := kms.Decrypt(encDEK)
    if err != nil {
        return nil, err
    }
    block, _ := aes.NewCipher(dek)
    aead, _ := cipher.NewGCM(block)
    return aead.Open(nil, nonce, ct, []byte(env.AAD))
}

```

```

# AWS KMS envelope – what the API calls look like from the service
$ aws kms generate-data-key --key-id alias/app-kek --key-spec AES_256 \
  --query '{CiphertextBlob: CiphertextBlob, Plaintext: Plaintext}' --
output json | head
# Returns: CiphertextBlob (encDEK for storage) + Plaintext (DEK for
immediate use, zero after)

$ aws kms decrypt --ciphertext-blob fileb://encDEK.bin --query Plaintext
--output text | base64 -d | od -An -tx1
# Recover DEK for decryption (DEK never persisted in cleartext outside
process memory)

# GCP KMS equivalent
$ gcloud kms encrypt --key app-kek --keyring app --location us-
central1 \
  --plaintext-file dek.bin --ciphertext-file encDEK.bin

```

Rotation without downtime uses the `KEKVersion` field. During a rotation window, readers try the current KEK and fall back to the previous one — the same dual-read pattern as hash agility in Ch 2 and `kid` selection in JWTs (Ch 5). Re-encryption of DEKs is a background job; bulk ciphertext re-encryption is lazy (on next write or via a scrubber). This bounds the blast radius of a compromised KEK to the window before rotation completes.

DEK caching. Calling KMS on every row decrypt adds latency (typically 5–20 ms) and cost. Cache unwrapped DEKs in process memory with a TTL (e.g., 5 minutes) and bound the cache size. For high-throughput

paths, use a single DEK per batch or per partition rather than per row — the trade-off is larger blast radius per DEK vs fewer KMS calls.

Envelope encryption in depth: caching, batching, and audit

The envelope pattern's performance hinges on how often you call KMS. Three refinements make it viable at high throughput.

DEK caching with bounded lifetime. Unwrap a DEK once and cache it in process memory for a TTL — typically 5 minutes, or until the KEK version changes. The cache key is the `encDEK` ciphertext, not the plaintext DEK, so cache lookups do not expose key material in keys. Bound the cache by entry count (e.g., 1,000 DEKs) and total memory (e.g., 32 KiB of key material) to prevent unbounded growth. On eviction, zero the key bytes explicitly. A background goroutine that refreshes near-expiry DEKs avoids latency spikes on the read path when many entries expire together.

Batch and partition scoping. Per-row DEKs maximize blast-radius isolation but multiply KMS calls and envelope metadata. For bulk data — analytics partitions, log segments, backup chunks — a single DEK per partition (e.g., per Kafka partition, per daily Parquet file) amortizes the KMS cost while still limiting exposure to one partition. The envelope is stored once per partition header, not per row, which also reduces storage overhead from 16-byte tags and base64 encoding. Choose per-row DEKs for high-sensitivity fields (PII, payment data) and per-partition DEKs for bulk data where re-encrypting the partition is already the operational unit.

Audit and compliance. Every KMS `Encrypt` and `Decrypt` call should emit an audit event with the KEK identifier, caller identity (service account, pod, IAM role), and whether the call succeeded — but not the DEK or plaintext. Cloud KMS providers log this automatically to CloudTrail / Cloud Audit Logs. Alert on anomalous patterns: a sudden spike in `Decrypt` calls suggests bulk exfiltration, while `Decrypt` failures with `AccessDenied` suggest a compromised or misconfigured principal probing keys. For regulated workloads, envelope encryption also satisfies the requirement that data be encrypted with a customer-managed key (CMEK) where the cloud provider never sees the DEK in cleartext outside

the KMS boundary — the application holds the DEK only in process memory, and the KMS audit log proves who unwrapped it and when.

Asymmetric encryption: when the encryptor doesn't hold the secret

Symmetric encryption requires the encryptor and decryptor to share a key. Asymmetric (public-key) encryption removes that requirement: anyone with the public key can encrypt, only the private-key holder can decrypt. This is the right tool when the encryptor is untrusted, external, or otherwise should not hold decryption capability — client-side field encryption, encrypted push to a recipient's inbox, or encrypting backups to a key held offline.

What to use

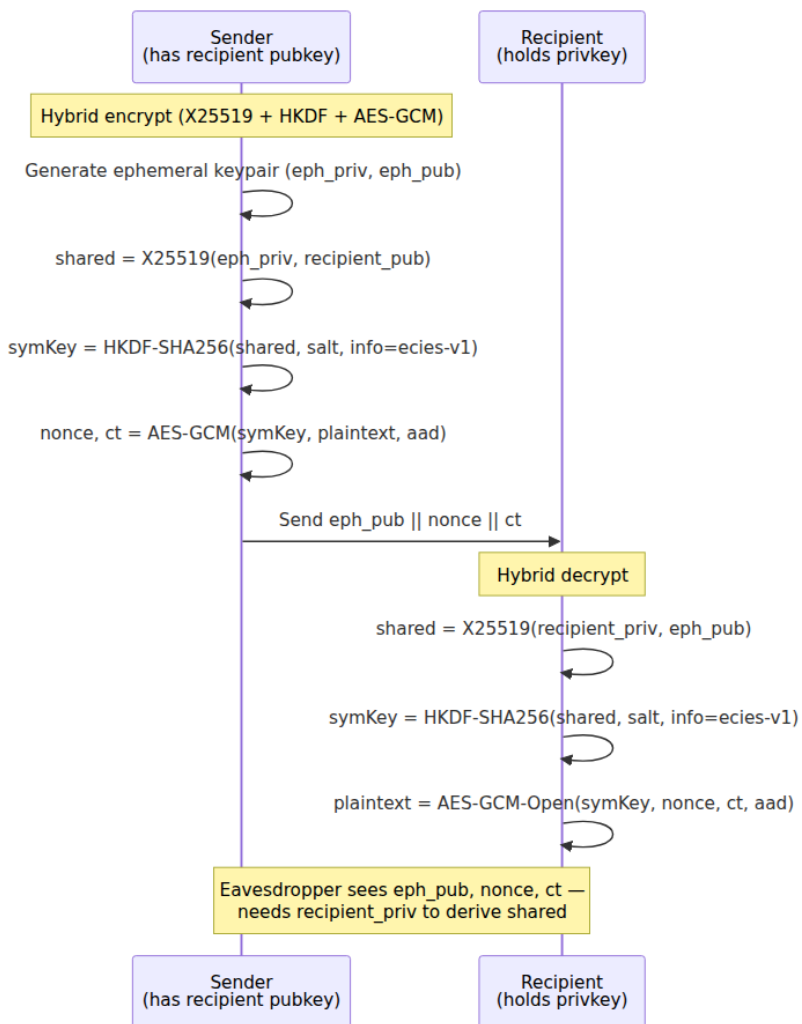
Scheme	Spec	Key size	Ciphertext overhead	Recommendation
X25519 + HKDF + AEAD (ECIES)	RFC 7748 + RFC 5869 + RFC 5116	32-byte pubkey	32-byte ephemeral pubkey + 16-byte tag	Preferred — small, fast, misuse-resistant
libsodium crypto_box_seal	libsodium	32-byte pubkey	48 bytes overhead	Easiest API — anonymous sender, no nonce mgmt
RSA-OAEP (SHA-256)	RFC 8017	2048-4096 bits	~42 bytes (OAEP padding)	Use when RSA is required for interop; 2048-bit minimum, 4096 preferred
RSA PKCS#1 v1.5	RFC 8017 (legacy)	—	—	Do not use — Bleichenbacher padding oracle, no integrity

RSA PKCS#1 v1.5 has been broken since Bleichenbacher (1998) and remains exploitable via side-channels (ROBOT, 2017) against real TLS

stacks. OAEP fixes the padding oracle but RSA remains large and slow. For new systems, ECIES over X25519 is smaller, faster, and has no padding oracle class.

Hybrid encryption — the construction

Asymmetric primitives can only encrypt short messages (key-size minus padding). The standard construction is hybrid: generate an ephemeral symmetric key, encrypt the payload with an AEAD, encrypt the symmetric key to the recipient's public key. This is what TLS 1.3 (RFC 8446), PGP, age, and libsodium all do internally.



```

import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/ecdh"
    "crypto/rand"
    "crypto/sha256"
    "io"

    "golang.org/x/crypto/hkdf"
)

// HybridEncrypt encrypts to a recipient's X25519 public key.
// Returns ephemeralPub || nonce || ciphertext+tag.
func HybridEncrypt(recipientPub *ecdh.PublicKey, plaintext, aad
[]byte) ([]byte, error) {
    // 1. Ephemeral keypair
    ephPriv, err := ecdh.X25519().GenerateKey(rand.Reader)
    if err != nil {
        return nil, err
    }
    ephPub := ephPriv.PublicKey()

    // 2. ECDH shared secret
    shared, err := ephPriv.ECDH(recipientPub)
    if err != nil {
        return nil, err
    }
    // 3. HKDF to derive AEAD key – never use raw ECDH output as a key
    hkdfReader := hkdf.New(sh256.New, shared, nil, []byte("vol09-
ch03:ecies:v1"))
    symKey := make([]byte, 32)
    io.ReadFull(hkdfReader, symKey)

    // 4. AEAD
    block, _ := aes.NewCipher(symKey)
    aead, _ := cipher.NewGCM(block)
    nonce := make([]byte, aead.NonceSize())
    io.ReadFull(rand.Reader, nonce)
    ct := aead.Seal(nil, nonce, plaintext, aad)

    // Wire format: ephPub(32) || nonce(12) || ct
    out := make([]byte, 0, 32+len(nonce)+len(ct))
    out = append(out, ephPub.Bytes()...)
    out = append(out, nonce...)
    out = append(out, ct...)
    return out, nil
}

func HybridDecrypt(recipientPriv *ecdh.PrivateKey, aad, wire []byte) (
[]byte, error) {

```

```

    if len(wire) < 32+12+16 {
        return nil, io.ErrUnexpectedEOF
    }
    ephPubBytes := wire[:32]
    nonce := wire[32:44]
    ct := wire[44:]

    ephPub, err := ecdh.X25519().NewPublicKey(ephPubBytes)
    if err != nil {
        return nil, err
    }
    shared, err := recipientPriv.ECDH(ephPub)
    if err != nil {
        return nil, err
    }
    hkdfReader := hkdf.New(sha256.New, shared, nil, []byte("vol09-
ch03:ecies:v1"))
    symKey := make([]byte, 32)
    io.ReadFull(hkdfReader, symKey)

    block, _ := aes.NewCipher(symKey)
    aead, _ := cipher.NewGCM(block)
    return aead.Open(nil, nonce, ct, aad)
}

```

```

# age – simple hybrid file encryption (X25519 + ChaCha20-Poly1305 +
HKDF)
# https://github.com/FiloSottile/age – the modern replacement for PGP
for files
$ age-keygen -o age-key.txt
# Public key:
age1ql3z7hj5gt80vryuj8phw4t3jhktnlhg20v6e7d0p84htzhqz9vqs2dg4pp
$ age -r age1ql3z7hj5gt80vryuj8phw4t3jhktnlhg20v6e7d0p84htzhqz9vqs2dg4p
p secrets.env > secrets.env.age
$ age --decrypt -i age-key.txt secrets.env.age > secrets.env
# Wire: X25519 ephemeral pubkey + ChaCha20-Poly1305 ciphertext – same
hybrid pattern as above

# libsodium sealed box (anonymous sender, simplest API)
# crypto_box_seal(ciphertext, message, mlen, recipient_pk);
# crypto_box_seal_open(plaintext, ciphertext, clen, recipient_pk,
recipient_sk);

```

RSA-OAEP when you must

Some systems — legacy PKI, HSMs without EC support, or compliance mandates — require RSA. Use OAEP with SHA-256 and MGF1-SHA-256, never PKCS#1 v1.5. In Go:


```

import (
    "crypto/rand"
    "crypto/rsa"
    "crypto/sha256"
)

func rsaOAPEncrypt(pub *rsa.PublicKey, plaintext []byte) ([]byte, error) {
    return rsa.EncryptOAEP(sh256.New(), rand.Reader, pub, plaintext, nil)
    // label is nil – include application context if cross-protocol use
    // is possible
}

func rsaOAEPDecrypt(priv *rsa.PrivateKey, ciphertext []byte) ([]byte, error) {
    return rsa.DecryptOAEP(sh256.New(), rand.Reader, priv, ciphertext, nil)
}

```

```

# OpenSSL – RSA-OAEP (note: CLI defaults to PKCS#1 v1.5 unless -pkeyopt
# is set)
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out rsa
-priv.pem
$ openssl pkey -in rsa-priv.pem -pubout -out rsa-pub.pem

# OAEP with SHA-256 (OpenSSL 3.x)
$ echo -n "DEK: 32 random bytes hex" | openssl pkeyutl -encrypt \
    -inkey rsa-pub.pem -pubin \
    -pkeyopt rsa_padding_mode:oaep \
    -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256 | od -An -t
x1 | head
# Output: 384 bytes (3072-bit RSA ciphertext)

# Decrypt
$ openssl pkeyutl -decrypt -inkey rsa-priv.pem \
    -pkeyopt rsa_padding_mode:oaep \
    -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256 \
    -in ciphertext.bin

```

Key sizes: 2048-bit RSA is the current minimum (112-bit security), 3072-bit for 128-bit security per NIST SP 800-57. An X25519 public key at 32 bytes provides the same 128-bit security as 3072-bit RSA — this size difference matters for JWEs, certificate chains, and anywhere keys are transmitted frequently.

Deterministic, searchable, and format-preserving encryption

AEAD with a random nonce is the default, but it has a property that sometimes hurts: the same plaintext encrypts to a different ciphertext each time, so equality checks, indexing, and sorting on the ciphertext are impossible. Three weaker constructions trade some security for functionality — use them only when the product requirement forces it and document the leakage.

Deterministic AEAD (SIV mode, RFC 5297). AES-GCM-SIV and AES-SIV derive the nonce deterministically from the plaintext and key (`nonce = PRF(key, plaintext)`), so equal plaintexts produce equal ciphertexts. This enables equality queries and deduplication without decrypting, but it leaks whether two rows share the same value — an attacker with access to the ciphertext learns frequency. Use deterministic encryption only for low-cardinality fields where frequency leakage is acceptable, and never for high-entropy secrets where equality itself is sensitive. Key separation is critical: a deterministic key must not also be used for randomized encryption.

Order-preserving and order-revealing encryption (OPE/ORE). These allow range queries on ciphertext but leak order and, in most constructions, approximate distance. They are weaker than deterministic encryption and have been broken in practice when the adversary has auxiliary data. For backend systems, prefer the alternative: encrypt the field with randomized AEAD and maintain a separate, access-controlled index or searchable tag (e.g., a bucketed range token or a blind index via HMAC with a separate key). The index leaks less than OPE and can be rotated independently.

Format-preserving encryption (FPE, NIST SP 800-38G, FF3-1). Preserves the format of the plaintext (e.g., a 16-digit card number encrypts to a 16-digit ciphertext) so legacy systems that validate format do not break. FPE is substantially slower and more complex than AEAD and requires careful domain-size analysis — short domains (e.g., 5-digit ZIP codes) have brute-force risk. Use FPE only at system boundaries where format compatibility is non-negotiable; inside your own services, change the schema to accept the ciphertext format instead.

Rule of thumb. If you can avoid deterministic or searchable encryption by changing the query pattern — decrypt on read, filter in the application

layer, or use tokenization — do that. Every searchable construction leaks more than randomized AEAD, and the leakage compounds across columns. Document the leakage model explicitly in your threat model (Ch 11) before adopting any of them.

Failure modes and incident patterns

Nonce reuse across restarts. A service that uses a counter nonce and persists the counter to a file will reuse nonces if the file is lost on crash or if two replicas share the counter file via NFS with stale caching. Prefer XChaCha20-Poly1305 with random 192-bit nonces where persistence is unreliable, or store the counter in a strongly consistent store (ZooKeeper, etcd) with fencing.

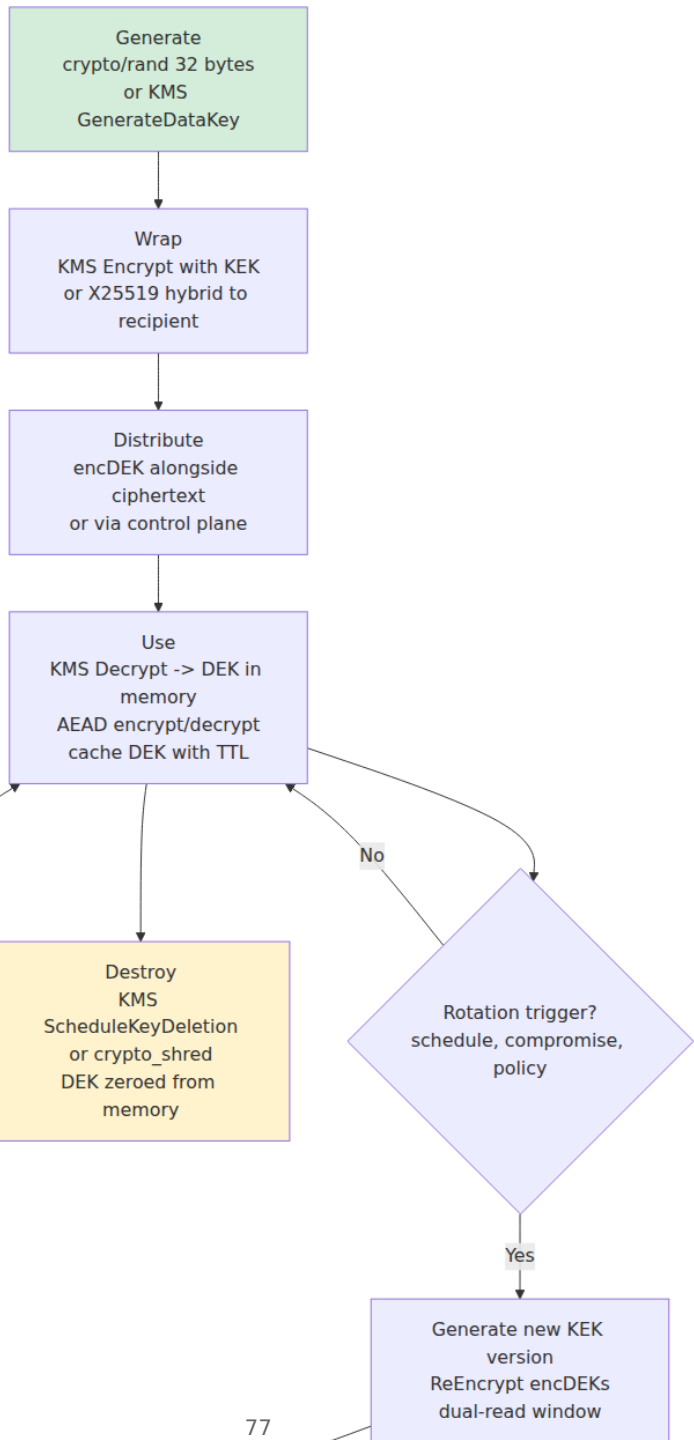
AAD binding failures. Forgetting to include the tenant ID, record version, or key ID in the AAD lets an attacker transplant a ciphertext from one context to another (a "cut and paste" attack). For example, a payment record encrypted with `aad = "v1"` can be replayed as `aad = "v2"` if the version is not bound, or a row from tenant A can be transplanted to tenant B if the tenant is not in the AAD. Bind every security-relevant header into the AAD and verify it on decrypt — the AAD is the authenticated context.

Key caching and zeroization. Unwrapped DEKs cached in process memory for performance must be zeroed when evicted. In Go, `clear(dek)` explicitly zeroes the slice; in languages with GC, pin the key material and overwrite it rather than relying on collection. DEKs must not be logged, included in error messages, or serialized to heap dumps. Audit your error paths: a common leakage is `fmt.Errorf("decrypt failed for key %x: %w", dek, err)` that writes the key to logs.

Algorithm confusion. A decryptor that accepts both AES-GCM and ChaCha20-Poly1305 based on a header field controlled by the attacker can be tricked into interpreting a ChaCha ciphertext as AES-GCM, producing predictable plaintext differences before the tag check fails. In practice the tag check still fails, but the error path may leak timing. Prefer a single AEAD per key scope; if agility is required, bind the algorithm choice into the KDF's `info` parameter and into the AAD so the wrong algorithm fails closed.

Key lifecycle at fleet scale

Encryption is not one key — it is a lifecycle. Every key moves through generation, distribution, use, rotation, and destruction, and each transition is a distributed-systems problem.



Generation must use the OS CSPRNG (`crypto/rand` , `randombytes_buf`). Never derive DEKs from passwords, timestamps, or `math/rand` . For envelope encryption, generate DEKs locally — do not call KMS to generate every DEK if latency matters; the KMS call is for wrapping, not generation.

Distribution of encrypted DEKs is not secret — they are ciphertext. Store them alongside the data they protect (one envelope per row/file). Distribution of KEKs is via the KMS control plane, replicated across regions with the KMS's own consistency guarantees. Do not copy KEK material to application hosts.

Rotation has two layers: KEK rotation (re-wrap DEKs, cheap) and DEK rotation (re-encrypt bulk data, expensive). KEK rotation can be frequent (quarterly or on incident); DEK rotation is lazy. Both require a dual-read window where verifiers accept both old and new versions, keyed by `kek_version` or `kid` — the same pattern as Ch 2's hash agility and Ch 5's JWT key rotation.

Destruction is where envelope encryption pays off again: deleting the KEK (or scheduling its deletion in KMS with a 7-30 day window) cryptographically shreds all data encrypted under it, without needing to overwrite every storage replica. This is the mechanism behind "right to be forgotten" implementations and secure offboarding. KMS deletion is recoverable within the window — after that, the data is unrecoverable by anyone, including you.

Performance and hardware

Operation	Throughput (single core, 2024 x86)	Notes
AES-256-GCM (AES-NI)	~4-6 GB/s	Requires AES-NI + CLMUL; falls to ~0.3 GB/s without
ChaCha20-Poly1305 (AVX2)	~2-3 GB/s	No hardware dependency; constant-time
X25519 ECDH	~100k ops/s	~10 μ s per exchange
RSA-2048 OAEP encrypt	~10k ops/s	Public-key op is fast; private-key (decrypt) is ~500 ops/s

Operation	Throughput (single core, 2024 x86)	Notes
RSA-3072 OAEP decrypt	~200 ops/s	Private-key op dominates; use for low-rate paths only
Argon2id (64 MiB, t=3)	~2 ops/s/core	Intentionally slow — not comparable to bulk encryption

For bulk data paths (storage, replication, backups), AES-GCM with AES-NI is the throughput winner. For per-message encryption on heterogeneous fleets (mobile clients, edge workers, Wasm), ChaCha20-Poly1305 avoids the "does this host have AES-NI?" question. For key exchange at connection setup, X25519 is fast enough to do per-connection ephemeral exchange — which is exactly what TLS 1.3 (RFC 8446) and Noise do for forward secrecy.

Distributed-systems lens

Encryption at the single-service level is a library call. Encryption across a fleet is a coordination, availability, and blast-radius problem.

KMS as a dependency. Envelope encryption makes the KMS a read-path dependency for cold DEKs (those not in cache). Design for KMS unavailability: cache unwrapped DEKs with a TTL, serve reads from cache when KMS is down, and alert on KMS error rates rather than failing open. For critical paths, consider multi-region KMS replication (AWS multi-region keys, GCP global KMS) so a regional KMS outage does not halt decryption fleet-wide.

Consistency of key versions. A rolling deploy that introduces a new KEK version while old replicas still encrypt with the previous version is safe *only* if decryptors accept both versions. Deploy decryptors first (dual-read), then switch encryptors, then deprecate the old KEK — the same expand-contract pattern as schema migrations (Vol 5, Ch 14). Reversing the order causes decrypt failures on mixed-version fleets.

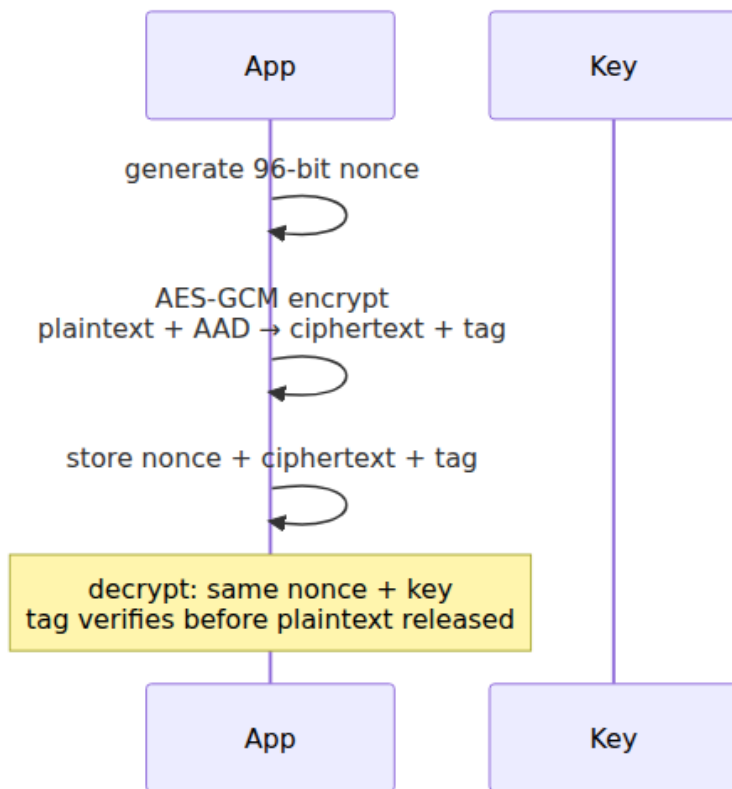
Blast radius. One KEK wrapping many DEKs means one KEK compromise exposes many items. Scope KEKs by service, environment, and sensitivity tier — `kek/payments/prod`, `kek/analytics/staging` — so compromise of one tier does not affect another. Within a tier, scope DEKs

narrowly (per-row or per-file for sensitive data, per-partition for bulk data) to limit how much a single DEK compromise exposes.

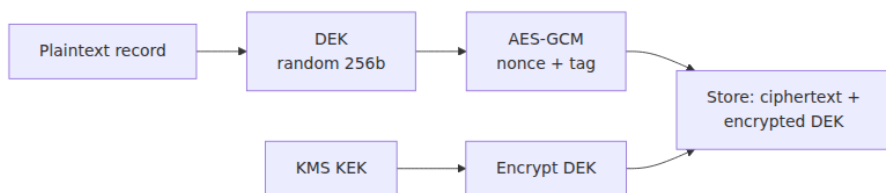
Forward secrecy for stored data. Unlike transport encryption where ephemeral keys provide forward secrecy directly (EC-DHE in TLS 1.3), stored data has no session to make ephemeral. The closest approximation is prompt DEK rotation and re-encryption after KEK compromise, plus short DEK lifetimes for high-sensitivity data. For messaging (Vol 10), per-message DEKs with prompt deletion after delivery approximate forward secrecy more closely.

When not to encrypt at the application layer. If TLS (Vol 3, Ch 6) already protects data in transit and your storage layer provides at-rest encryption (EBS, GCS, database TDE), application-layer field encryption adds defense-in-depth but also complexity, key-management overhead, and query limitations (encrypted fields cannot be indexed or filtered without special constructions like deterministic or order-preserving encryption, both of which weaken security). Encrypt at the application layer when: the storage layer is untrusted or shared, the data must remain opaque to operators/DBAs, or regulatory scope requires it (e.g., PCI DSS cardholder data, HIPAA PHI). Otherwise, transport + storage-layer encryption may be sufficient — see Ch 4 for the decision framework.

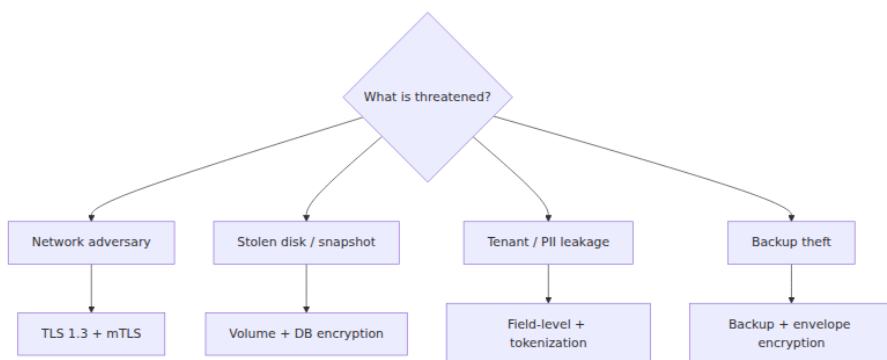
AES-GCM Encrypt and Decrypt



KMS Envelope for At-Rest



Encryption Scope Decision



Key takeaways

- AEAD (RFC 5116) is the only safe symmetric-encryption interface — it binds confidentiality and integrity with a 16-byte tag and fails closed on any tampering. Use AES-GCM where AES-NI is available and ChaCha20-Poly1305 (RFC 8439) elsewhere; both provide 128-bit security.
- Unauthenticated modes (AES-CBC, CTR alone) are broken for backend use — they are malleable and vulnerable to padding oracles. Migrate to AEAD; if you must compose manually, use encrypt-then-MAC with independent keys and constant-time verification, but prefer AEAD.
- Nonce reuse with the same key is catastrophic for both AES-GCM and ChaCha20-Poly1305. For distributed encryptors sharing a key, use XChaCha20-Poly1305 (192-bit random nonce), per-replica HKDF subkeys, or partitioned nonce spaces — never shared-key random 96-bit nonces without coordination.
- Envelope encryption (random DEK per item + KMS-held KEK) decouples bulk-encryption performance from KMS calls, enables cheap KEK rotation without bulk re-encrypt, and supports cryptographic shredding on deletion. Cache unwrapped DEKs with a TTL and scope KEKs by service/tier.
- For asymmetric encryption, prefer X25519-based hybrid encryption (RFC 7748 + HKDF + AEAD) or `libsodium crypto_box_seal` — small keys, fast, no padding oracle. Use RSA-OAEP (RFC 8017) only for interop; never RSA PKCS#1 v1.5 (Bleichenbacher/ROBOT). Raw ECDH output is not a key — always HKDF it.

- Key lifecycle — generation via `crypto/rand`, wrapping, distribution alongside ciphertext, dual-read rotation, and scheduled destruction — is a distributed coordination problem. Every encrypted value should carry a key version (`kek_version`, `kid`) for agility, and rotation must follow an expand-contract deploy order (decryptors first, then encryptors).

Further reading

- **Standards:** RFC 5116 (AEAD), RFC 5869 (HKDF), RFC 7748 (X25519/X448), RFC 8032 (Ed25519), RFC 8439 (ChaCha20-Poly1305), RFC 9106 (Argon2), RFC 8446 (TLS 1.3), RFC 8017 (RSA PKCS#1 / OAEP), NIST SP 800-38D (GCM), NIST SP 800-57 (Key Management), FIPS 180-4 (SHA-2), FIPS 202 (SHA-3).
- Ferguson, Schneier, Kohno — *Cryptography Engineering* (Wiley, 2010) — AEAD, nonce management, and key lifecycle at practitioner depth.
- Latacora — *Cryptographic Right Answers* (<https://latacora.micro.blog/2018/04/03/cryptographic-right-answers.html>) — opinionated primitive choices, regularly updated.
- libsodium documentation — <https://doc.libsodium.org/> — sealed boxes, XChaCha20-Poly1305, and correct-by-default APIs.
- age specification — <https://github.com/C2SP/C2SP/blob/main/age.md> — a clean, modern hybrid file-encryption design (X25519 + ChaCha20-Poly1305 + HKDF) worth reading as a reference construction.
- Go `crypto/*` and `golang.org/x/crypto` docs — <https://pkg.go.dev/crypto> — authoritative for service implementations.

- AWS KMS / GCP KMS envelope encryption docs — <https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html> — operational patterns for KEK/DEK at cloud scale.

Chapter 4 — Certificates, PKI, and TLS Operations

What this chapter covers. TLS protects every byte your backend sends — between users and edge, between services, between services and databases. But TLS without a working public-key infrastructure is just expensive math. This chapter is the operational companion to Volume 3, Chapter 6: where Vol 3 describes the TLS 1.3 wire protocol (handshake, record layer, 0-RTT per RFC 8446) and the cryptographic guarantees it provides, this chapter describes how certificates are issued, chained, verified, revoked, and rotated at scale — the system that makes TLS usable and, when neglected, the most common cause of production outages. We build a CA from scratch with OpenSSL, automate issuance through ACME and cert-manager, configure TLS termination correctly, and design rotation strategies that do not require a fleet-wide restart at 3 a.m.

Learning goals — after this chapter you should be able to:

- Parse an X.509 certificate (RFC 5280): distinguish Subject, Issuer, SAN, Key Usage, Extended Key Usage, Basic Constraints, and Authority/Key Identifier extensions, and explain why the legacy Common Name (CN) is insufficient.
- Build and verify a chain from leaf through intermediate to root, explain why every production PKI uses at least one intermediate, and describe how path validation and name constraints work.
- Create a private CA hierarchy with OpenSSL: generate a root, an intermediate, CSRs, and signed leaf certificates with SANs — and verify the chain with `openssl verify`.
- Operate revocation (CRL, OCSP per RFC 6960, OCSP stapling per RFC 6961, and short-lived certificates) and explain why revocation is the least reliable part of the Web PKI and how to design around it.

- Configure a TLS server (nginx, Go `tls.Config`, Envoy) with safe defaults: TLS 1.2 floor, correct cipher ordering, ALPN, OCSP stapling, and session ticket hygiene.
- Design certificate rotation at fleet scale — automated issuance via ACME/cert-manager, hot reload without connection drops, dual-certificate deploys, expiry monitoring, and emergency rotation after compromise — and explain why short-lived certificates (hours, not months) are the modern direction.
- Diagnose the most common PKI failures: expiry, incomplete chains, clock skew, intermediate rotation, and private-key leakage.

Boundary notes. The TLS *wire protocol* — handshake state machine, record framing, cipher suites, key exchange (ECDHE), 0-RTT, and session resumption per RFC 8446 — is covered in **Vol 3, Ch 6**. Read that for what happens on the wire between two peers. This chapter covers the *operational PKI* around that protocol: certificate lifecycle, chain-of-trust management, automation, and deployment. The *intra-service* mutual-TLS (mTLS) control plane — SPIFFE/SPIRE, workload identity, and zero-trust service mesh — is covered in **Ch 10**; this chapter introduces mTLS as a server-TLS extension and leaves identity federation to Ch 10. For supply-chain signing PKI (Fulcio, Sigstore) see Companion Book 5, Ch 3.

X.509 in one page

An X.509 v3 certificate (RFC 5280) is a signed binding between a public key and an identity. It contains:

- **Subject** and **Issuer** distinguished names (DNs). For a leaf, Subject is the entity (e.g., `CN=api.example.com` — legacy) and Issuer is the CA that signed it. For a root, they are identical (self-signed).
- **Subject Public Key Info (SPKI)** — the public key (RSA-2048+, ECDSA P-256/P-384, or Ed25519) and its algorithm identifier.
- **Validity** — `NotBefore` / `NotAfter` (UTCtime or GeneralizedTime). Every TLS implementation rejects certificates outside this window; clock skew is a real failure mode.
- **Serial number** — unique per CA, used for revocation lookups.
- **Extensions** (the part that actually matters for verification):
- **Subject Alternative Name (SAN)** — the set of DNS names, IP addresses, URIs, or wildcards the cert is valid for. Since RFC 6125

and CA/Browser Forum Baseline Requirements, clients validate SAN, not CN. Every leaf must have a SAN containing every name it serves. A CN-only certificate is rejected by modern browsers and Go's `crypto/tls`.

- **Basic Constraints** — `CA:TRUE/FALSE` plus `pathLenConstraint`. A leaf must be `CA:FALSE`; violating this has caused real CA mis-issuance incidents.
- **Key Usage (KU) and Extended Key Usage (EKU)** — KU declares what operations the key may be used for (`digitalSignature`, `keyEncipherment` for RSA, `keyAgreement` for ECDH); EKU declares the *purpose* (`serverAuth`, `clientAuth`, `codeSigning`, `OCSPSigning`). A cert with `EKU=serverAuth` must not be accepted for client authentication.
- **Authority Key Identifier (AKI) and Subject Key Identifier (SKI)** — hash of the issuer/subject public key, used to build the chain unambiguously when a leaf could chain to multiple intermediates with the same DN.
- **Authority Information Access (AIA)** — URL of the issuer's certificate and OCSP responder.
- **CRL Distribution Points (CRLDP)** — where to fetch the revocation list.
- **Subject Information Access / Certificate Transparency** is carried via the `ct_preced_scts` extension (RFC 6962) on precertificates.

```
# Inspect any certificate you are given – memorize this incantation
$ openssl x509 -in leaf.pem -noout -text | head -n 80
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            04:8e:3a:7b:91:c2:00:1f:44:d3:9a:11:b7:06:aa:2e
        Signature Algorithm: ecdsa-with-SHA384
        Issuer: C=US, O=Example Corp Intermediate CA, CN=Example Interm
ediate G1
        Validity
            Not Before: Aug 20 00:00:00 2026 GMT
            Not After : Nov 18 23:59:59 2026 GMT
        Subject: C=US, O=Example Corp, CN=api.example.com
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
            Public-Key: (256 bit)
            pub:
                04:a1:...:9f
            ASN1 OID: prime256v1
            NIST CURVE: P-256
        X509v3 extensions:
            X509v3 Basic Constraints: critical, CA:FALSE
            X509v3 Key Usage: critical, Digital Signature
            X509v3 Extended Key Usage: TLS Web Server Authentication
            X509v3 Subject Alternative Name: critical,
                DNS:api.example.com, DNS:api.internal.example.com
            X509v3 Authority Key Identifier: keyid:8F:2A:...
            X509v3 Subject Key Identifier: 3C:91:...
            Authority Information Access:
                CA Issuers - URI:http://aia.example.com/
intermediate.crt
                OCSP - URI:http://ocsp.example.com
```

If you cannot read the output above and spot a missing SAN, a `CA:TRUE` leaf, or an expired `NotAfter`, you cannot review a TLS deployment. Practice on real certificates until you can.

How chains and trust work

No client ships every leaf certificate. Clients ship a **trust store** — a set of self-signed root certificates deemed trustworthy (Mozilla NSS, Apple, Microsoft, Go's `x509.SystemCertPool()` backed by the OS). A leaf is trusted if a chain can be built from the leaf through zero or more intermediates to a root in the trust store, and *every* signature and constraint in the chain validates.

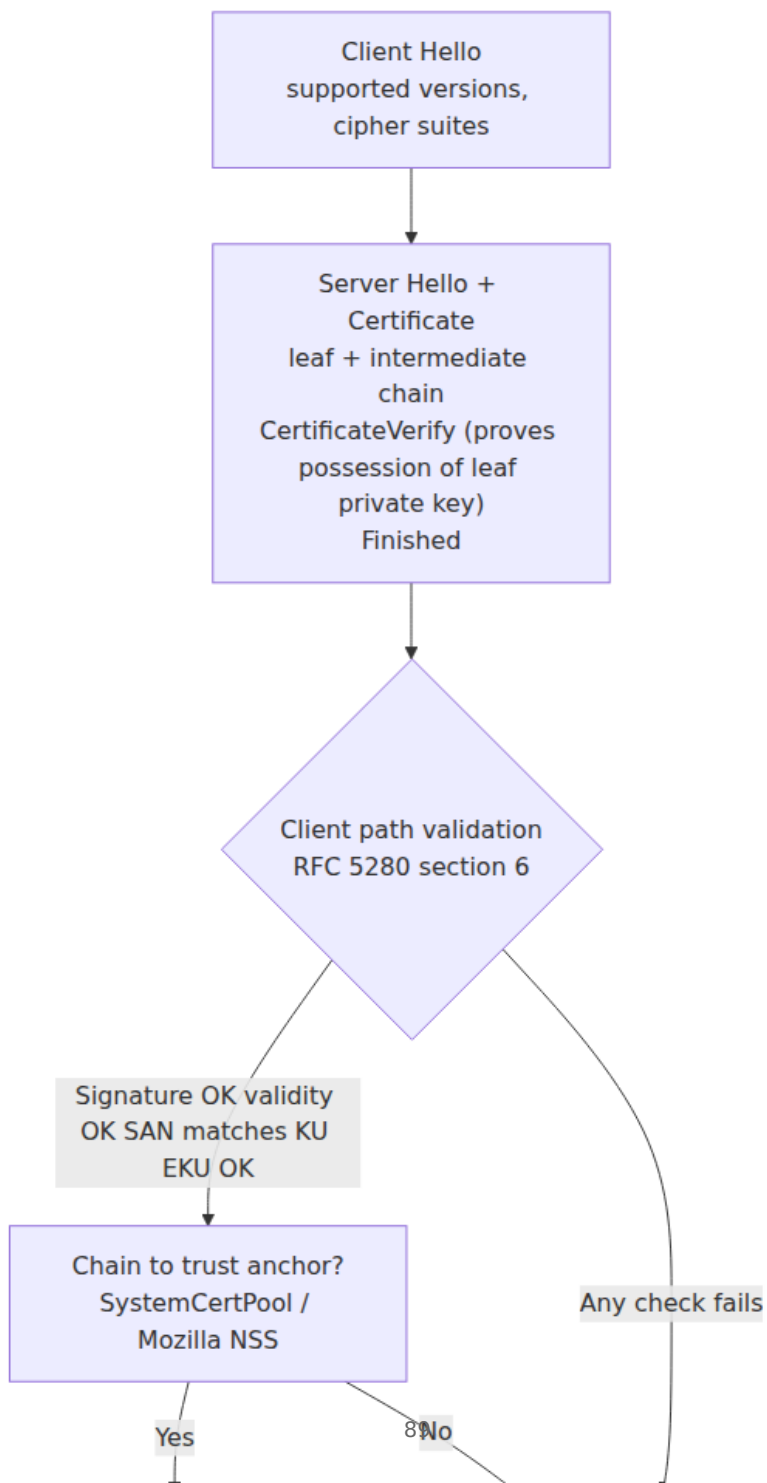
```
Root CA (self-signed, in trust store)
└─ Intermediate CA (signed by Root, CA:TRUE, pathLen 0 or 1)
    └─ Leaf (signed by Intermediate, CA:FALSE, SAN=api.example.com)
```

Why an intermediate exists at all: the **root private key is the most sensitive material in the PKI**. It is kept offline in an HSM or air-gapped host, often under a quorum ceremony (two or more custodians present). Issuing directly from the root would require bringing the root key online for every certificate — unacceptable risk. Instead the root signs a small number of intermediates, the intermediates (online, in HSMs in data centers) sign leaves, and if an intermediate key is compromised, the root can revoke it and issue a new intermediate without replacing the trust anchor on every client. This is the same reason KMS hierarchies use a root KEK to wrap data KEKs (Ch 3) — compartmentalization of the blast radius.

Path validation (RFC 5280 §6) checks, for the full chain:

1. Each certificate's signature verifies with the issuer's public key.
2. Each certificate is within its validity window (requires correct clocks).
3. `BasicConstraints` and `KeyUsage` are satisfied at each level (a leaf cannot act as a CA; an intermediate with `pathLenConstraint=0` cannot issue a subordinate CA).
4. Name constraints, if present on an intermediate, are satisfied (e.g., an intermediate constrained to `permitted;DNS:.example.com` cannot issue for `evil.com`).
5. Revocation status is checked (CRL/OCSP — see below).
6. The chain terminates at a trust anchor.

Most verification failures at 02:00 UTC are item 6: the server sent an incomplete chain (leaf without intermediate), the client has no path to a trust anchor, and the error surfaces as `x509: certificate signed by unknown authority` (Go) or `ERR_CERT_AUTHORITY_INVALID` (Chrome). Servers must send the full chain minus the root — `leaf + intermediate` concatenated — not just the leaf. Clients must not need to fetch intermediates via AIA chasing in the hot path; if they do, the first connection before the fetch fails.



Building a CA hierarchy with OpenSSL

Theory is not enough — you should build a chain once so the abstractions become concrete. What follows is a minimal, correct hierarchy. For production, replace file-based keys with an HSM (YubiHSM, AWS CloudHSM) and OpenSSL's `engine` or `provider` interface, but the certificate logic is identical.

Version pins. OpenSSL 3.0+ (current stable 3.3/3.4). Flags shown are stable across 3.0–3.4. Go `crypto/x509` behavior notes reference Go 1.22+.

```
#!/usr/bin/env bash
set -euo pipefail
mkdir -p pki/{root,intermediate,leaf} && cd pki

# — 1. Root CA: 4096-bit RSA, self-signed, 10-year lifetime, offline
in reality —
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out root/
root-ca.key
chmod 400 root/root-ca.key

cat > root/root-ca.cnf <<'CNF'
[ req ]
distinguished_name = dn
x509_extensions    = v3_ca
prompt             = no
[ dn ]
C = US
O = Example Corp Root CA
CN = Example Root G1
[ v3_ca ]
basicConstraints    = critical, CA:TRUE, pathlen:1
keyUsage            = critical, keyCertSign, cRLSign
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always,issuer
CNF

openssl req -x509 -new -nodes \
    -key root/root-ca.key -sha384 -days 3650 \
    -config root/root-ca.cnf -extensions v3_ca \
    -out root/root-ca.crt

# — 2. Intermediate CA: ECDSA P-256, CSR signed by root —————
openssl ecparam -genkey -name prime256v1 -noout -out intermediate/
intermediate.key
chmod 400 intermediate/intermediate.key

cat > intermediate/intermediate-csr.cnf <<'CNF'
[ req ]
distinguished_name = dn
prompt             = no
[ dn ]
C = US
O = Example Corp Intermediate CA
CN = Example Intermediate G1
CNF

openssl req -new -key intermediate/intermediate.key \
    -config intermediate/intermediate-csr.cnf -out intermediate/
intermediate.csr
```

```
cat > intermediate/intermediate-ext.cnf <<'CNF'
basicConstraints      = critical, CA:TRUE, pathlen:0
keyUsage              = critical, keyCertSign, cRLSign
extendedKeyUsage      = serverAuth, clientAuth
subjectKeyIdentifier  = hash
authorityKeyIdentifier = keyid:always,issuer
crlDistributionPoints  = URI:http://crl.example.com/root.crl
authorityInfoAccess    = caIssuers;URI:http://aia.example.com/root.crt,
OCSP;URI:http://ocsp.example.com
CNF
```

```
openssl x509 -req -in intermediate/intermediate.csr \
  -CA root/root-ca.crt -CAkey root/root-ca.key -CAcreateserial \
  -sha384 -days 1825 -extfile intermediate/intermediate-ext.cnf \
  -out intermediate/intermediate.crt
```

— 3. Leaf: ECDSA P-256, SAN is mandatory

```
openssl ecparam -genkey -name prime256v1 -noout -out leaf/leaf.key
chmod 400 leaf/leaf.key
```

```
cat > leaf/leaf-csr.cnf <<'CNF'
[ req ]
distinguished_name = dn
req_extensions      = v3_req
prompt              = no
[ dn ]
C = US
O = Example Corp
CN = api.example.com
[ v3_req ]
subjectAltName = @alt_names
[ alt_names ]
DNS.1 = api.example.com
DNS.2 = api.internal.example.com
CNF
```

```
openssl req -new -key leaf/leaf.key \
  -config leaf/leaf-csr.cnf -out leaf/leaf.csr
```

```
cat > leaf/leaf-ext.cnf <<'CNF'
basicConstraints      = critical, CA:FALSE
keyUsage              = critical, digitalSignature
extendedKeyUsage      = serverAuth
subjectAltName        = @alt_names
subjectKeyIdentifier  = hash
authorityKeyIdentifier = keyid,issuer
crlDistributionPoints  = URI:http://crl.example.com/intermediate.crl
authorityInfoAccess    = caIssuers;URI:http://aia.example.com/
intermediate.crt, OCSP;URI:http://ocsp.example.com
[ alt_names ]
```

```

DNS.1 = api.example.com
DNS.2 = api.internal.example.com
CNF

openssl x509 -req -in leaf/leaf.csr \
  -CA intermediate/intermediate.crt -CAkey intermediate/
intermediate.key -CAcreateserial \
  -sha256 -days 90 -extfile leaf/leaf-ext.cnf \
  -out leaf/leaf.crt

# — 4. Verify – the command that should be in your CI pipeline —
echo "=== Verify leaf against root (via intermediate) ==="
openssl verify -CAfile root/root-ca.crt -untrusted intermediate/
intermediate.crt leaf/leaf.crt
# Expected: leaf/leaf.crt: OK

echo "=== Full chain file for servers (leaf + intermediate, NOT root)
=== "
cat leaf/leaf.crt intermediate/intermediate.crt > leaf/fullchain.pem

echo "=== TLS test with s_server / s_client ==="
# In one terminal: openssl s_server -cert leaf/fullchain.pem -key leaf/
leaf.key -www -port 8443
# In another:      openssl s_client -connect localhost:8443 -CAfile
root/root-ca.crt -servername api.example.com

```

What to test in CI: every certificate your pipeline produces should pass `openssl verify -CAfile root.crt -untrusted intermediate.crt leaf.crt` and a SAN check (`openssl x509 -noout -ext subjectAltName`). If either fails, the build fails — do not let an unverifiable certificate reach staging.

Private keys are the crown jewels. `chmod 400` is the minimum; in production they live in an HSM and never touch disk. If a private key is ever written to a log, a heap dump, an error message, or a world-readable file, treat it as compromised and rotate immediately — there is no way to prove it was not exfiltrated. This is the same discipline as Ch 3's KMS hygiene.

Revocation: the hard part

Issuance is the easy half of PKI. Revocation — telling the world that a previously valid certificate is no longer trustworthy because its key was compromised or its identity is no longer valid — is where the Web PKI is weakest.

Mechanism	How it works	Failure mode
CRL (Certificate Revocation List, RFC 5280)	CA periodically publishes a signed list of revoked serial numbers. Clients fetch and cache it.	CRLs grow without bound; fetches are slow, cached, and often skipped. Many clients soft-fail (accept the cert if the CRL cannot be fetched).
OCSP (RFC 6960)	Client queries the CA's OCSP responder in real time: is serial X revoked?	Adds a blocking network round-trip to every handshake; responder must be highly available; privacy leak (CA learns which sites you visit); soft-fail again dominates.
OCSP stapling (RFC 6961, <code>status_request</code> / <code>status_request_v2</code>)	Server periodically fetches a signed OCSP response from the CA and staples it to the handshake. Client verifies the staple instead of calling the CA.	Requires server configuration and periodic refresh (response is valid 24–72 h); if the staple is missing, clients still soft-fail unless Must-Staple is set.
Must-Staple (RFC 7633, <code>1.3.6.1.5.5.7.1.24</code>)	Leaf extension telling clients to hard-fail if no valid staple is present.	Opt-in per certificate; if the server misconfigures stapling, the site becomes unreachable — high operational cost, low adoption.
CRLite / OneCRL / CRLSets	Browser vendors aggregate revocations centrally and push compact filters to browsers.	Covers only the browser ecosystem; not available to backend service-to-service verification.

Mechanism	How it works	Failure mode
Short-lived certificates	Validity of hours to days instead of months; expiry <i>is</i> revocation.	Requires robust automation; clock skew becomes the dominant failure. Let's Encrypt's move toward 6-day certs and ACME renewals is this direction.

The practical consequence: **revocation is unreliable for immediate containment**. If a leaf key is compromised, revoking the cert and hoping clients check is not a containment plan. The real containment is rotating the key and certificate, and — for high-value keys — rotating the intermediate. For backend services, add an explicit blocklist (revoked serials / SPKI hashes) distributed through your config plane or service mesh control plane, checked on every connection; this gives you hard-fail revocation with your own availability properties instead of depending on the Web PKI's soft-fail semantics.

Configuring TLS termination

nginx (1.25+, OpenSSL 3.x)


```

# /etc/nginx/conf.d/tls.conf
server {
    listen 443 ssl;
    server_name api.example.com;

    # Leaf + intermediate concatenated; key never world-readable
    ssl_certificate      /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key  /etc/nginx/certs/leaf.key;

    # Protocol floor: TLS 1.2 minimum, prefer 1.3. Disable 1.0/1.1.
    ssl_protocols TLSv1.2 TLSv1.3;

    # TLS 1.2 cipher suite: ECDHE + AEAD + PFS only. No CBC, no RC4, no
    # SHA-1.
    # Order matters when ssl_prefer_server_ciphers is on.
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-
    SHA384:
        ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-AES128-GCM-
    SHA256:
        ECDHE-RSA-AES256-GCM-SHA384:ECDHE-RSA-CHACHA20-
    POLY1305;
    ssl_prefer_server_ciphers on;

    # TLS 1.3 ciphers are negotiated separately (no cipher config
    # needed;
    # Go/OpenSSL choose TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384,
    # TLS_CHACHA20_POLY1305_SHA256 automatically)

    # ALPN for HTTP/2 and future HTTP/3 – must match Vol 3 Ch 7
    # nginx negotiates h2/http/1.1 via ALPN with the client.

    # OCSP stapling – nginx fetches and staples on behalf of clients
    ssl_stapling on;
    ssl_stapling_verify on;
    ssl_trusted_certificate /etc/nginx/certs/intermediate.crt;
    resolver 1.1.1.1 valid=300s;
    resolver_timeout 5s;

    # Session tickets: rotate keys, or disable and use session IDs with
    # shared cache
    # For 0-RTT (Vol 3 Ch 6) do NOT enable without replay-safe design;
    # prefer to leave off
    # for sensitive APIs. If needed: ssl_session_tickets on with
    # external ticket key rotation.

    ssl_session_cache shared:SSL:50m;
    ssl_session_timeout 4h;

```

```
# HSTS – after verifying TLS is correct, add preload
add_header Strict-Transport-Security "max-age=63072000; includeSubD
omains; preload" always;

location / { proxy_pass http://upstream; }
}
```

Go `crypto/tls` (Go 1.22+, `crypto/tls` Config)

Go negotiates TLS 1.3 by default and does not expose TLS 1.3 cipher suite configuration — the right default. Hardening is about floors, verification, and hot reload:

```

package main

import (
    "crypto/tls"
    "crypto/x509"
    "log"
    "net/http"
    "os"
    "time"
)

func tlsConfig(certFile, keyFile, caFile string) *tls.Config {
    // System roots + private CA for internal PKI
    pool, err := x509.SystemCertPool()
    if err != nil {
        log.Fatal(err)
    }
    if caFile != "" {
        pem, err := os.ReadFile(caFile)
        if err != nil {
            log.Fatal(err)
        }
        if !pool.AppendCertsFromPEM(pem) {
            log.Fatal("failed to append private CA")
        }
    }

    // Load leaf + chain. Go's LoadX509KeyPair handles
    leaf+intermediate bundles
    // if fullchain.pem is leaf followed by intermediate.
    cert, err := tls.LoadX509KeyPair(certFile, keyFile)
    if err != nil {
        log.Fatal(err)
    }

    return &tls.Config{
        Certificates: []tls.Certificate{cert},

        // Enforce TLS 1.2 floor. TLS 1.3 is preferred automatically
        when both sides support it.
        MinVersion: tls.VersionTLS12,

        // Curve preferences – X25519 first, then P-256. RSA key
        exchange is not offered in TLS 1.3.
        CurvePreferences: []tls.CurveID{tls.X25519, tls.CurveP256},

        // Root pool for outbound verification (client side) and for
        mTLS client-cert verification.
        RootCAs: pool,
    }
}

```

```

    // OCSP stapling and SCTs are handled by the server
    automatically if the cert contains them.

    // Session Tickets: disable at the server if you do not need
    resumption,
    // or rotate SessionTicketKey externally. Never leave a static
    ticket key in source.
    // SessionTicketsDisabled: false, // enable only with managed
    rotation
    }
}

func main() {
    cfg := tlsConfig("/etc/certs/fullchain.pem", "/etc/certs/
leaf.key", "/etc/certs/root.crt")

    srv := &http.Server{
        Addr:      ":8443",
        TLSConfig:  cfg,
        ReadHeaderTimeout: 5 * time.Second,
        IdleTimeout: 120 * time.Second,
    }

    // Hot reload: watch cert files and swap TLSConfig atomically via
    GetCertificate.
    // See "Rotation at fleet scale" below for the GetCertificate
    pattern.
    log.Fatal(srv.ListenAndServeTLS("", "")) // certs already in
    TLSConfig
}

// Hot-reload pattern – prefer GetCertificate over static Certificates
for long-lived processes.
func reloadingCert(certFile, keyFile string)
func(*tls.ClientHelloInfo) (*tls.Certificate, error) {
    return func(*tls.ClientHelloInfo) (*tls.Certificate, error) {
        cert, err := tls.LoadX509KeyPair(certFile, keyFile)
        if err != nil {
            return nil, err
        }
        return &cert, nil
    }
}

// Usage: cfg.GetCertificate = reloadingCert("/etc/certs/
fullchain.pem", "/etc/certs/leaf.key")

```

Envoy (1.29+) — typical for service-mesh sidecars

Envoy validates TLS via SDS (Secret Discovery Service) so certificates can be rotated without restarting the proxy:

```
# envoy.yaml – transport socket with SDS-backed rotation
transport_sockets:
- name: envoy.transport_sockets.tls
  typed_config:
    "@type": type.googleapis.com/
    envoy.extensions.transport_sockets.tls.v3.DownstreamTlsContext
    common_tls_context:
      tls_certificate_sds_secret_configs:
        - name: "leaf-and-chain"
          sds_config:
            resource_api_version: V3
            api_config_source:
              api_type: GRPC
              grpc_services:
                - envoy_grpc: { cluster_name: sds_cluster }
      combined_validation_context:
        default_validation_context:
          match_typed_subject_alt_names:
            - san_type: DNS
              matcher: { exact: "api.example.com" }
          validation_context_sds_secret_config:
            name: "root-ca"
            sds_config: { resource_api_version: V3, api_config_source: {
api_type: GRPC, grpc_services: [{ envoy_grpc: { cluster_name: sds_clust
er }}}}]}
```

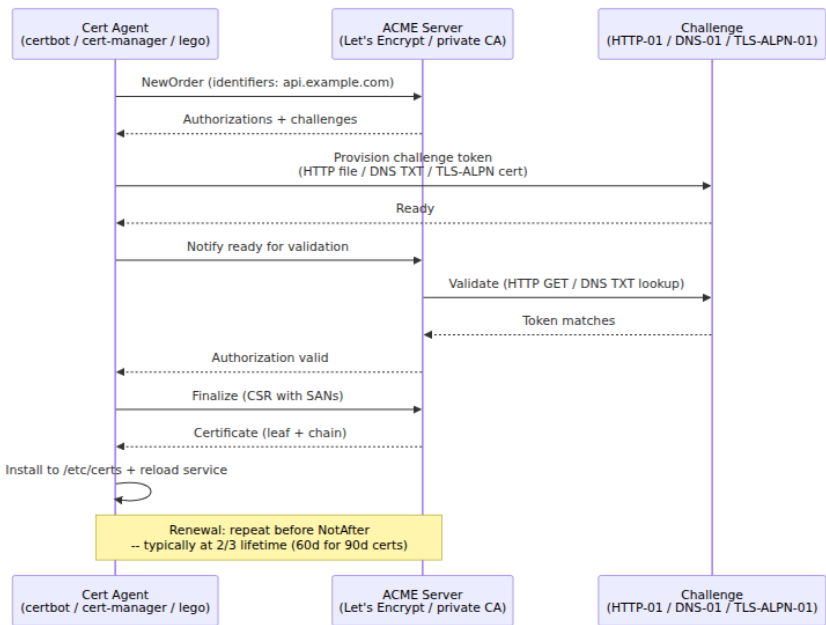
Hardening checklist

- **TLS 1.2 floor, 1.3 preferred.** Disable TLS 1.0/1.1. Clients that cannot do 1.2 are unacceptably broken.
- **ECDHE + AEAD only** for TLS 1.2. No **RSA** key exchange (no forward secrecy), no **CBC** (padding oracle risk), no **SHA1**.
- **ALPN** — negotiate **h2/http/1.1** correctly; misconfigured ALPN causes silent fallback to HTTP/1.1 and confuses load balancers.
- **HSTS** — enable after proving the chain is stable; include **preload** only when you are sure every subdomain serves valid TLS.
- **Certificate Transparency** — SCTs embedded by the CA or delivered via OCSP/TLS extension; required by Chrome since 2018.
- **Private key file permissions** **400** or **HSM**; never commit to git; scan with **gitleaks** / **truffleHog** (see Ch 8).

Automation: ACME, Let's Encrypt, and cert-manager

Manual issuance does not scale past a handful of hosts and guarantees an expiry incident. Automation via ACME (RFC 8555) is the standard.

Version pins. ACME RFC 8555; Let's Encrypt current chain ISRG Root X1/X2; CA/Browser Forum baseline lifetime is shrinking — ~90 days (Let's Encrypt default) moving toward 47 days and eventually days/hours. Design for 30-day renewal windows regardless of current max lifetime.



Challenge types:

Challenge	How it proves control	When to use
HTTP-01 (RFC 8555 §8.3)	Agent places a file at <code>http://domain/.well-known/acme-challenge/<token></code>	Single host, port 80 reachable. Not suitable for wildcard certs.

Challenge	How it proves control	When to use
DNS-01 (RFC 8555 §8.4)	Agent creates a <code>_acme-challenge.domain</code> TXT record	Wildcards (<code>*.example.com</code>), internal hosts not reachable from the internet, Kubernetes clusters. Requires DNS API credentials — treat them as high-value secrets.
TLS-ALPN-01 (RFC 8737)	Agent serves a special cert on port 443 during validation	Alternative to HTTP-01 when port 80 is blocked; less commonly used.

cert-manager on Kubernetes (v1.14+)

cert-manager is the dominant ACME controller for Kubernetes and a good template for non-Kubernetes automation too — the pattern (Issuer → Certificate → Secret, with renewal controller) generalizes.

```

# 1. ClusterIssuer – single ACME account for the cluster
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: platform@example.com
    privateKeySecretRef:
      name: le-account-key
    solvers:
      - dns01:
          cloudDNS:
            project: prod-network
            serviceAccountSecretRef:
              name: clouddns-sa
              key: key.json
          selector:
            dnsZones: ["example.com"]
      - http01:
          ingress:
            class: nginx
---
# 2. Certificate – what to issue and where to store it
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: api-example-com
  namespace: api
spec:
  secretName: api-tls          # resulting Secret: tls.crt, tls.key,
                               # ca.crt
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  commonName: api.example.com
  dnsNames:
    - api.example.com
    - api.internal.example.com
  duration: 2160h              # 90d – explicit; do not rely on CA
                               # default
  renewBefore: 720h           # renew when 30d remain (1/3 of
                               # duration)
  privateKey:
    algorithm: ECDSA
    size: 256
  usages:
    - server auth
---

```



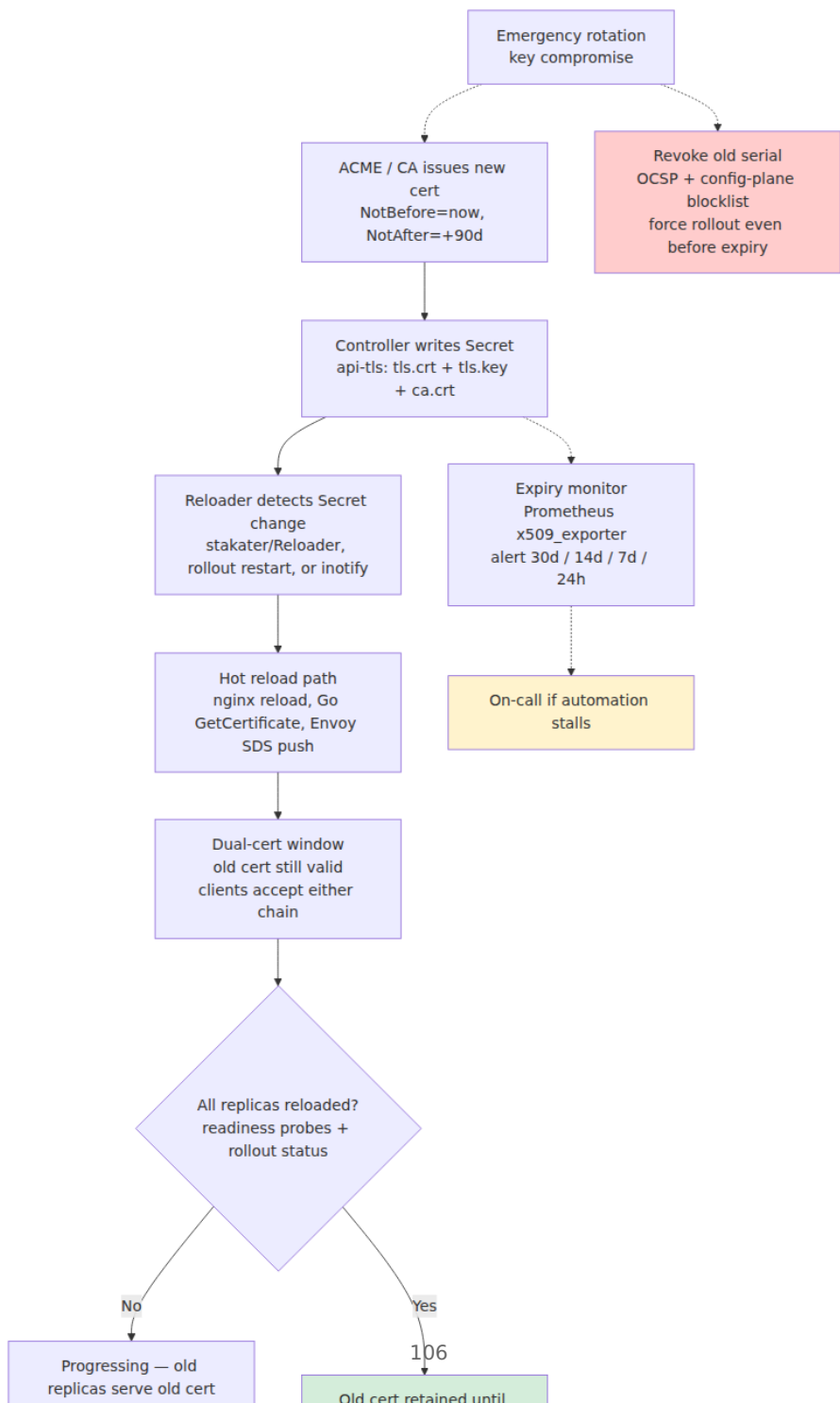
```
# 3. Ingress consuming the Secret – nginx example
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
  namespace: api
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
  - hosts: [api.example.com]
    secretName: api-tls
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: { name: api, port: { number: 8080 }}
```

cert-manager's renewal controller watches `NotAfter` and re-issues at `renewBefore`. The updated `Secret` triggers a rolling update of any pod that mounts it — but only if you wire that trigger (see rotation below).

For private PKI (internal mTLS), replace the ACME `ClusterIssuer` with a `CA` or `Vault` issuer pointing at your intermediate, or use SPIRE (Ch 10) for workload certificates with even shorter lifetimes (hours).

Rotation at fleet scale

A certificate that is valid for 90 days and renewed every 60 days is healthy. A certificate that expires in 20 minutes and is renewed by an automation that pages at 03:00 is an incident waiting to happen. Rotation must be designed as a system, not a cron job.



Patterns that actually work

1. Hot reload, not restart. Every TLS server in the fleet must reload certificates without dropping connections.

Server	Mechanism
nginx	<code>nginx -s reload</code> — zero-downtime reload; existing connections keep the old cert until they close, new connections use the new cert.
Go net/http	<code>tls.Config.GetCertificate</code> — called on every handshake, so swapping the underlying <code>tls.Certificate</code> is atomic and lock-free.
Envoy	SDS — the control plane pushes the new <code>Secret</code> over xDS; Envoy swaps without listener drain.
Java (Jetty/Netty)	<code>SSLContext</code> reload via <code>KeyManagerFactory</code> reinit, or sidecar Envoy termination.
Kubernetes	<code>stakater/Reloader</code> or <code>Rollout</code> annotation <code>secret.reloader.stakater.com/reload: "true"</code> — or better, mount via SDS/cert-manager <code>cainjector</code> rather than <code>Secret</code> rollout.

2. Dual-certificate deploys. During rotation, both the old and new certificates are valid (their validity windows overlap). Clients must accept either chain. Deploy verifiers (clients that pin or cache the old leaf) *after* servers have the new cert, or use chain-level trust (verify to root, not to leaf) so new leaves are trusted automatically. This is the same expand-contract pattern as Ch 3's encryption key rotation — decryptors/verifiers first, then encryptors/signers.

3. Short-lived certs as the endgame. A certificate valid for 6 hours that is renewed every 4 hours reduces the revocation problem to near-irrelevance — expiry is revocation. The trade-off is tighter coupling to automation availability. For internal workloads, SPIFFE/SPIRE (Ch 10) issues SVIDs with 1-hour lifetimes via the Workload API, rotated automatically by the SPIRE agent. For public leaves, ACME clients that renew at 2/3 lifetime with jittered scheduling already approximate this.

4. Monitoring that prevents the 3 a.m. page.

```
# Prometheus alert – fires before the cert actually expires
groups:
- name: tls-certs
  rules:
  - alert: TLSCertExpiringSoon
    expr: (x509_cert_not_after - time()) / 86400 < 30
    for: 1h
    labels: { severity: warning }
    annotations: { summary: "Certificate {{ $labels.cn }} expires in <30d" }
  - alert: TLSCertExpiringCritical
    expr: (x509_cert_not_after - time()) / 86400 < 7
    for: 15m
    labels: { severity: critical }
    annotations: { summary: "Certificate {{ $labels.cn }} expires in <7d – rotation stalled" }
  - alert: TLSCertInvalid
    expr: x509_cert_verify_success == 0
    for: 5m
    labels: { severity: critical }
    annotations: { summary: "Certificate {{ $labels.cn }} fails verification" }
```

5. Emergency rotation. When a private key is compromised, waiting for expiry is not an option:

1. Revoke the old certificate (OCSP + CRL) — but assume clients soft-fail, so this is not containment by itself.
2. Push the compromised serial/SPKI hash to your config-plane blacklist; service-mesh proxies and internal verifiers hard-fail on it.
3. Issue a new key pair (new private key — rotating only the cert with the same key defeats the purpose) and certificate with a new serial.
4. Force a rolling update fleet-wide before the old cert's `NotAfter`, even if automation would have waited.
5. Rotate the intermediate if the compromise scope warrants it, and audit whether the root ceremony needs to be invoked. Keep the root offline ceremony documented and rehearsed — you do not want to be writing it during an incident.

Private PKI at scale (beyond public Web PKI)

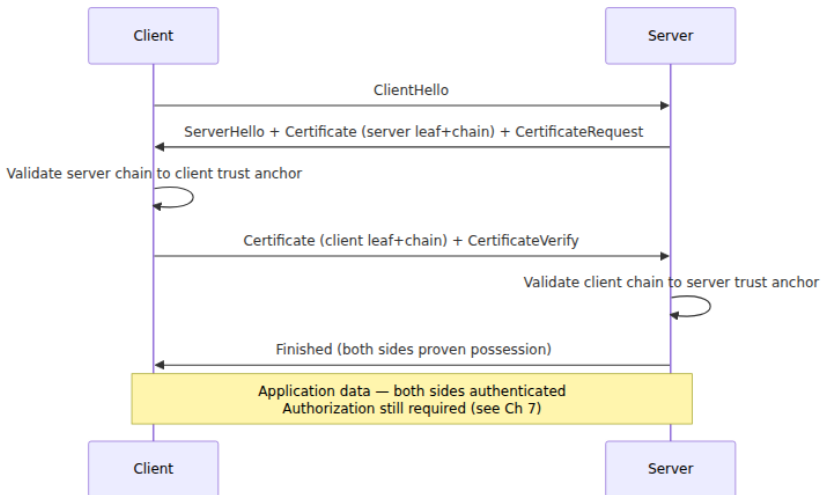
For service-to-service TLS, the Web PKI is the wrong tool — you control both sides. Options:

- **Private CA with Vault PKI / step-ca / EJBCA** — issue internal certs via ACME or Vault API, with short lifetimes and automated renewal. Clients trust your private root via `SystemCertPool` augmentation or Envoy `validation_context`.
- **SPIFFE/SPIRE (Ch 10)** — workload identity via SVID (X.509 or JWT) with attestation, rotation, and federation. The preferred path for Kubernetes fleets.
- **cert-manager CA/Vault issuers** — reuse the cert-manager control loop for internal certs, same `Certificate` CRD, different `Issuer`.

Do not use the same CA for public-facing and internal certs. Scope and separation limit blast radius exactly as with KEK scoping (Ch 3).

mTLS in one section (bridge to Ch 10)

Mutual TLS reuses the same PKI machinery but verifies certificates in both directions: the server presents a leaf verified against the client's trust store, and the client presents a leaf verified against the server's trust store.



In Go, enabling mTLS is one field — `ClientAuth` — but the operational complexity is in trust-store distribution and client certificate provisioning:

```
pool := x509.NewCertPool()
pem, _ := os.ReadFile("/etc/certs/internal-root.crt")
pool.AppendCertsFromPEM(pem)

cfg := &tls.Config{
    Certificates: []tls.Certificate{serverCert},
    ClientAuth:   tls.RequireAndVerifyClientCert, // hard-fail without
    valid client cert
    ClientCAs:   pool,                          // trust anchor for
    client certs
    MinVersion:  tls.VersionTLS12,
}
```

Do not conflate mTLS authentication with authorization — proving the client holds a key for `service-a.prod.example.com` does not tell you what `service-a` is allowed to do (Ch 7). And do not use mTLS with long-lived client certs baked into images; issue short-lived certs via SPIRE or Vault Agent and rotate them like server certs. The full workload-identity story, including attestation and federation, is Ch 10.

Failure modes and incident patterns

Expiry. Still the most common TLS incident. A cert issued 90 days ago with a renewal job that silently failed 30 days ago expires at `NotAfter` and every new handshake fails simultaneously — a fleet-wide outage with no gradual signal. Mitigations: monitor at 30/14/7/1 days, test renewal in staging with the same automation, and use short-lived certs where the renewal loop is exercised so often that failures are caught quickly.

Incomplete chain. Server sends leaf without intermediate. Works in browsers that have cached the intermediate from another site, fails for fresh clients, mobile apps, and `curl/Go` with a minimal trust store. Symptom: `x509: certificate signed by unknown authority` that is intermittent. Fix: send `fullchain.pem` (leaf + intermediate), test with `openssl s_client -connect host:443 -servername host -showcerts | openssl verify` from a clean environment.

Clock skew. A service whose clock is hours behind will reject a freshly issued cert (it thinks `NotBefore` is in the future); a service whose clock is

ahead will accept expired certs. NTP/chrony must be healthy before TLS can be healthy. Monitor `chrony tracking` alongside cert expiry.

Intermediate rotation. A CA rotates its intermediate (new key, new cert). Servers still serving the old chain are fine until clients prune the old intermediate from their trust-path caches or the old intermediate expires. Servers that fetch intermediates via AIA on every handshake hide the problem until AIA is slow. Pin automation to track the issuer's current intermediate and alert on issuer change.

Private-key leakage. Keys committed to git, echoed in CI logs, included in heap dumps, or stored in world-readable `Secret` mounts. Detection: `gitleaks`, `truffleHog`, `detect-secrets` in pre-commit and CI; OPA/Kyverno policy `deny` on `Secret` without `type: kubernetes.io/tls` + RBAC that restricts `Secret` reads. Remediation is rotation, not redaction.

Distributed-systems lens

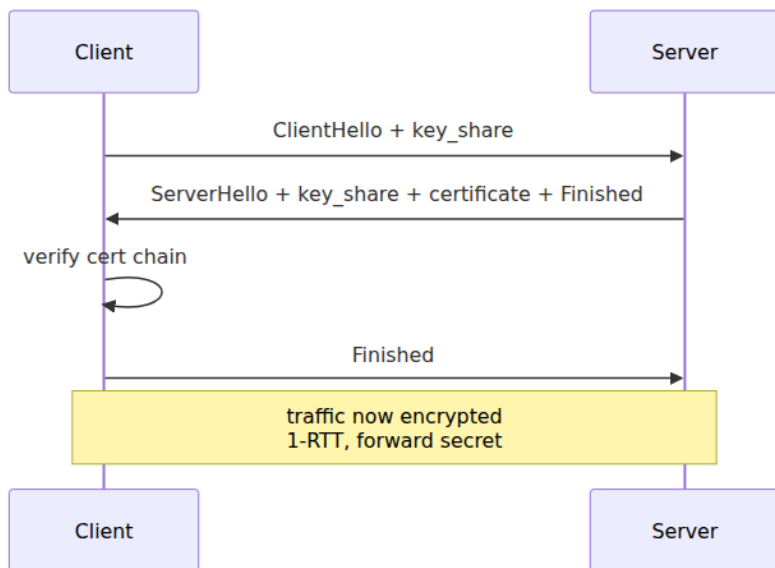
PKI at single-host scale is `openssl req`. PKI at fleet scale is a distributed scheduling, monitoring, and blast-radius problem.

- **Issuance as a distributed cron.** Every certificate has a renewal deadline; the renewal controller (cert-manager, lego, Vault agent) is a distributed cron that must fire exactly once per cert, handle leader election on restart, and retry with backoff on CA unavailability. Treat renewal failures as `critical` — a missed renewal is a future outage, not a warning.
- **Consistency window on rotation.** For a brief window, some replicas serve the old cert and some serve the new cert. Both must be valid. Clients that verify to the root handle this automatically; clients that pin the old leaf or cache the old SPKI do not. Design verification around stable trust anchors (roots), not leaves.
- **KMS/HSM as a dependency for private keys.** If private keys live in an HSM or KMS (as they should for intermediates), issuance depends on HSM availability. Cache issued certs with slack (do not issue with `NotAfter` hours away if HSM availability is regional), and replicate HSM-backed CAs across regions or use multi-region intermediates.
- **Config-plane propagation for trust anchors.** Rotating a root or adding a new trust anchor requires pushing the new PEM to every verifier — every Go service's `SystemCertPool` augmentation, every

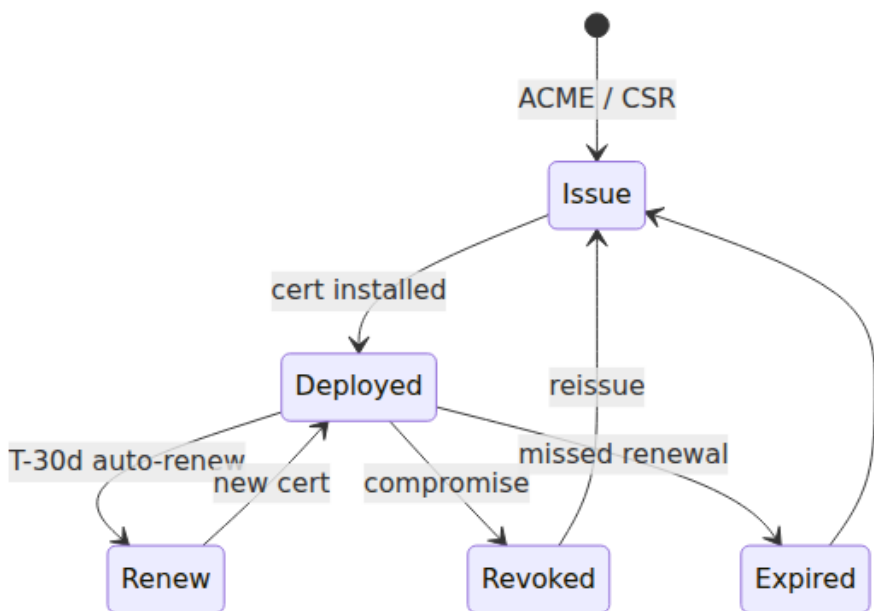
Envoy `validation_context`, every nginx `ssl_trusted_certificate`, every Java truststore. Use your config plane (Kubernetes ConfigMap with reload, Envoy xDS, Vault Agent templates) and version the anchor bundle (`ca-bundle:v3`) so you can roll back.

- **Observability.** Export `x509_cert_not_after`, `x509_cert_verify_success`, `ocsp_staple_expiry`, and `tls_handshake_errors` as metrics from every terminator. Alert on time-to-expiry, stapling staleness, and handshake error rate — a spike in `unknown_ca` after a deploy is the signal that a chain was broken by the change.

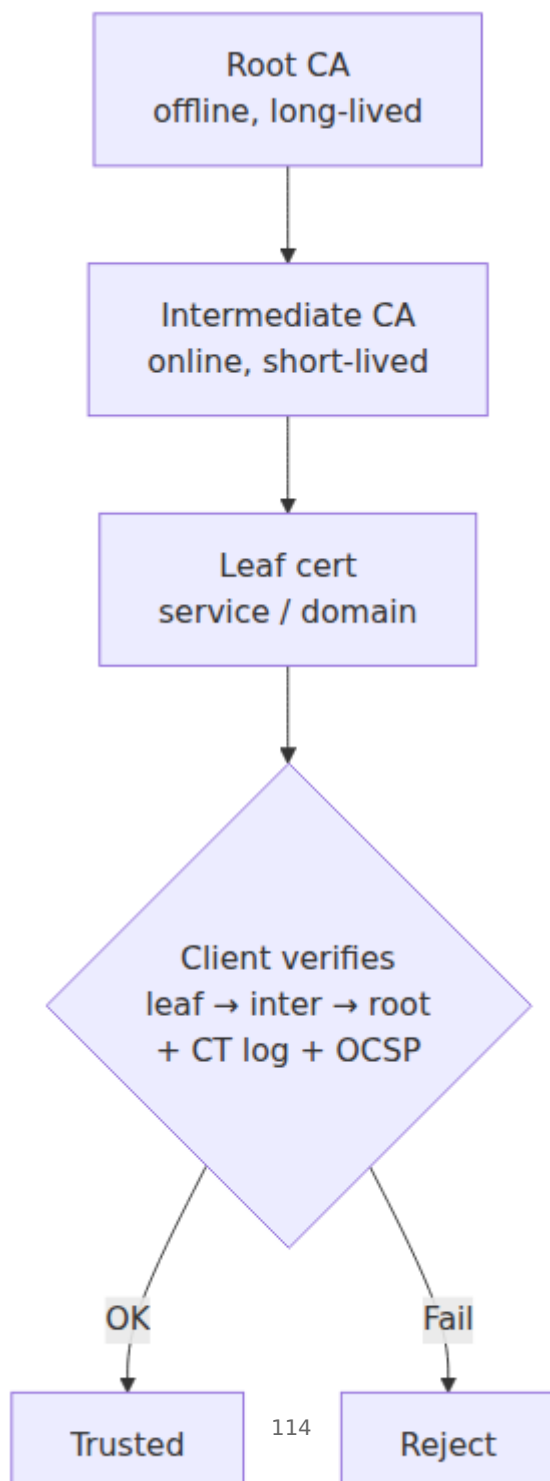
TLS 1.3 Handshake



Certificate Lifecycle



Chain of Trust



Key takeaways

- Every TLS certificate is an X.509 v3 binding of a public key to identities via SANs — not CN — with KU/EKU and Basic Constraints that restrict how it may be used. Clients validate SAN, validity window, chain signatures, and constraints before trusting a leaf; servers must send leaf + intermediate, not just leaf.
- Chains exist for blast-radius compartmentalization: an offline root signs a small number of intermediates, intermediates sign leaves. Compromise of an intermediate is recoverable without replacing the trust anchor on every client. The same KEK/DEK scoping principle from Ch 3 applies.
- OpenSSL can build a correct hierarchy (root → intermediate → leaf) in a few commands; verification is `openssl verify -CAfile root.crt -untrusted intermediate.crt leaf.crt`. Wire this into CI — no unverifiable certificate should reach staging.
- Revocation (CRL, OCSP, stapling, CRLite) is soft-fail and unreliable for immediate containment. For backend systems, add a hard-fail blacklist on serial/SPKI distributed through your config plane, and prefer short-lived certificates (hours-days) where expiry is revocation.
- TLS termination must enforce a TLS 1.2 floor (1.3 preferred), ECDHE + AEAD only, ALPN, OCSP stapling, and HSTS — whether in nginx, Go `tls.Config`, or Envoy SDS. Private keys live in HSMs or `400`-permissioned files, never in git or logs.
- Automation via ACME (RFC 8555) and cert-manager is mandatory past a handful of hosts. Use DNS-01 for wildcards, HTTP-01 for single hosts, and renew at 2/3 lifetime with jitter. For internal PKI, use private CA issuers or SPIFFE/SPIRE (Ch 10).
- Rotation at fleet scale requires hot reload (nginx reload, Go `GetCertificate`, Envoy SDS), a dual-cert validity window, expiry monitoring at 30/14/7/1 days, and an emergency playbook that includes key replacement (not just cert replacement) and config-plane blocklists.

Further reading

- **Standards:** RFC 5280 (X.509 PKI), RFC 6960 (OCSP), RFC 6961 (OCSP stapling / multi-stapling), RFC 7633 (Must-Staple), RFC 8555 (ACME), RFC 6962 (Certificate Transparency), RFC 8446 (TLS 1.3), RFC 6125 (Name validation), CA/Browser Forum Baseline Requirements (<https://cabforum.org/baseline-requirements/>).
- Dierks, Rescorla — *The Transport Layer Security (TLS) Protocol Version 1.2* (RFC 5246) and Rescorla — *TLS 1.3* (RFC 8446) — read alongside Vol 3 Ch 6 for wire vs ops context.
- Barnes et al. — *ACME* (RFC 8555) — concise spec for automation.
- Langley, Eckersley — *Certificate Transparency* (RFC 6962) — log structure and SCT validation.
- cert-manager docs — <https://cert-manager.io/docs/> — the operational reference for Kubernetes fleets.
- smallstep `step-ca` / `step-cli` docs — <https://smallstep.com/docs/step-ca> — private CA with ACME and JWK provisioners; a good alternative to Vault PKI for smaller fleets.
- `openssl(1)`, `openssl-x509(1)`, `openssl-verify(1)` man pages (OpenSSL 3.3) — authoritative for command flags.
- Go `crypto/tls` and `crypto/x509` docs — <https://pkg.go.dev/crypto/tls> — the reference for Go server and client TLS behavior.

Chapter 5 — Authentication: Sessions, Tokens, and JWTs

What this chapter covers. Every request that reaches your backend carries a claim — "I am user 4821" or "I am service `payments-prod`" — and the system's job is to decide whether to believe it. This chapter is the mechanism behind that decision: how a successful login becomes a session or token, how that artifact is presented on every request, and how the server validates it efficiently and safely at scale. We contrast server-side sessions (opaque handles backed by a store) with client-side bearer tokens (JWTs per RFC 7519), show exactly when to use each, demonstrate secure cookie discipline, CSRF defense, and token binding, and implement issued-and-verified JWTs in Go and Python that you can

run and test. The distributed-systems lens is front and center: session stickiness vs shared stores, cache coherence, clock skew for expiry, revocation of stateless tokens, and key rotation without logging everyone out.

Learning goals — after this chapter you should be able to:

- Distinguish authentication (who you are) from authorization (what you may do — Ch 7) and from session management (how the proof of authentication is carried across requests).
- Compare server-side sessions and client-side tokens (JWTs) by trust model, revocation, size, and failure mode — and choose correctly per use case rather than defaulting to one.
- Implement secure session cookies: `HttpOnly`, `Secure`, `SameSite`, `__Host-` prefix, rotation on privilege change, and CSRF defense via double-submit or `SameSite`.
- Issue, verify, and rotate JWTs correctly: `alg`, `kid`, `exp` / `nbf` / `iat` discipline, JWKS (RFC 7517) distribution, asymmetric (RS256/ES256) vs symmetric (HS256) trade-offs, and why `alg: none` and key confusion are not theoretical.
- Design session storage for a fleet — sticky vs shared (Redis/database), cache-coherent verification, and the consistency window that matters.
- Handle revocation and logout for both stateful sessions and stateless JWTs, including short-lived access tokens + refresh tokens, blocklists, and versioned token binding.
- Audit an auth flow for the classic failures: session fixation, session hijacking via XSS, CSRF, replay, and leaked tokens in logs/URLs.

Boundary notes. The *transport* that protects authentication artifacts in transit — TLS 1.3 (RFC 8446), certificates, and PKI operations — is covered in **Vol 3, Ch 6** (wire) and **Ch 4** (operations) of this volume. This chapter assumes TLS is already in place and focuses on the *application-layer* proof of identity carried inside that channel. Delegated authorization (letting one service act on a user's behalf) is **Ch 6** (OAuth 2.0 / OIDC); this chapter is *first-party* authentication — the user's relationship with your system. Service-to-service identity (mTLS, SPIFFE/SPIRE) is **Ch 10**.

Why authentication is a distributed-systems problem

On a single server, authentication is trivial: validate a password (Ch 2), set a cookie, keep a map from session ID to user in memory. In a fleet it is a coordination, consistency, and availability problem:

- A login handled by instance A must be verifiable by instance B on the next request — even if the request is routed to a different availability zone, even if A has crashed and restarted and lost its memory.
- A password change or logout must invalidate proofs that are already in the wild — on other replicas, in caches, on the client — within a bounded window, not "eventually when the token expires."
- A signing key rotation must not invalidate every in-flight session at once, and verifiers that have cached the old public key must learn the new one without a fleet-wide restart.

Every mechanism in this chapter trades consistency, availability, and latency for the authentication proof. The right trade-off depends on the product: a banking session has different revocation and expiry requirements than a read-heavy content API. We will call those trade-offs out explicitly.

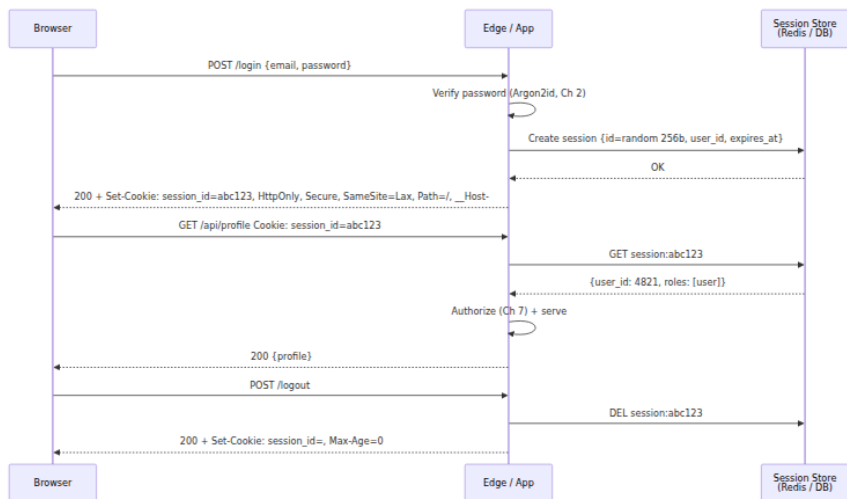
The request lifecycle



The artifact that bridges those two phases — cookie or token — and the server's method of verifying it are the design decisions this chapter covers.

Server-side sessions

A server-side session treats the cookie as an opaque handle. The browser stores `session_id=abc123;` the server stores the mapping `abc123 → {user_id, roles, issued_at, ...}` in a durable store. The handle itself carries no meaning and requires a lookup on every request.



Generating and storing the handle

The session ID must be unpredictable — it is a bearer secret equivalent to a password for the life of the session.

```
// Go – correct session ID generation (Go 1.22+)
package main

import (
    "crypto/rand"
    "encoding/base64"
    "net/http"
    "time"
)

func generateSessionID() string {
    b := make([]byte, 32) // 256 bits
    if _, err := rand.Read(b); err != nil {
        panic(err) // CSPRNG failure is fatal – do not fall back
    }
    return base64.RawURLEncoding.EncodeToString(b) // 43 chars, URL-
safe
}

func setSessionCookie(w http.ResponseWriter, sessionID string) {
    http.SetCookie(w, &http.Cookie{
        Name:     "__Host-session_id", // __Host- prefix: requires
Secure + Path=/ + no Domain
        Value:    sessionID,
        Path:     "/",
        MaxAge:    3600, // 1h – short; refresh via sliding window or
refresh token
        HttpOnly: true, // JS cannot read – mitigates XSS exfiltration
        Secure:    true, // TLS only – never over plaintext
        SameSite: http.SameSiteLaxMode, // CSRF mitigation; Strict for
sensitive apps
        // Domain deliberately absent with __Host- prefix
    })
}
```

Cookie attribute	Why it matters
HttpOnly	Prevents <code>document.cookie</code> reads — an XSS that can run JS still cannot steal the cookie via JS. Not a fix for XSS, but reduces blast radius.
Secure	Browser sends only over HTTPS. Without it, an HTTP downgrade or mixed-content page leaks the session.

Cookie attribute	Why it matters
<code>SameSite=Lax</code>	Browser omits the cookie on cross-site <code>POST</code> (CSRF defense). <code>Strict</code> omits even on top-level <code>GET</code> navigations — stronger but breaks some OAuth flows; choose per product.
<code>__Host-</code> prefix	Browser enforces <code>Secure</code> , <code>Path=/</code> , no <code>Domain</code> — prevents a sibling subdomain from overwriting the cookie. Use on every auth cookie.
<code>__Secure-</code> prefix	Weaker variant: requires <code>Secure</code> but allows <code>Domain</code> . Prefer <code>__Host-</code> .

On the server, the session store is typically Redis, Memcached, or a relational row. The store must:

- Support TTL/expiry natively (Redis `SET session:abc EX 3600` or DB `expires_at` + periodic sweep).
- Be available on the request path — a session store outage is an auth outage. Replicate across AZs and fail with `503` (not "anonymous") if unreachable; failing open is worse than failing closed.
- Be keyed by a hash of the session ID if you want to avoid storing the raw bearer value verbatim — though most teams store it directly and rely on TLS + store ACLs.

The Redis session helper (Python, redis-py 5.x)

```
# sessions.py – production-shaped session helpers (Python 3.12, redis-  
py 5.0+)  
import secrets, json, time  
from typing import Optional  
import redis  
  
r = redis.Redis(host="sessions.internal", port=6379, ssl=True, decode_r  
esponses=True)  
  
SESSION_TTL = 3600          # 1h  
REFRESH_THRESHOLD = 600    # refresh in background when <10m remain  
  
def create_session(user_id: int, roles: list[str]) -> str:  
    sid = secrets.token_urlsafe(32) # 256 bits, uses os.urandom via  
secrets  
    payload = json.dumps({"user_id": user_id, "roles": roles, "iat": in  
t(time.time())})  
    # NX ensures no collision (vanishingly unlikely but correct)  
    r.set(f"session:{sid}", payload, ex=SESSION_TTL, nx=True)  
    return sid  
  
def get_session(sid: str) -> Optional[dict]:  
    raw = r.get(f"session:{sid}")  
    if raw is None:  
        return None  
    data = json.loads(raw)  
    # Sliding window: extend TTL in the background when close to expiry  
  
# Do this async (Celery/RQ) or piggyback a pipeline EXPIRE to avoid  
extra RTT  
    ttl = r.ttl(f"session:{sid}")  
    if 0 < ttl < REFRESH_THRESHOLD:  
        r.expire(f"session:{sid}", SESSION_TTL)  
    return data  
  
def revoke_session(sid: str) -> None:  
    r.delete(f"session:{sid}")  
  
def revoke_all_for_user(user_id: int) -> None:  
    # Requires secondary index or scan – see "Revocation" below  
    for key in r.scan_iter(match="session:*", count=500):  
        raw = r.get(key)  
        if raw and json.loads(raw).get("user_id") == user_id:  
            r.delete(key)
```

In production the `scan_iter` loop is expensive — maintain a `user_sessions:{user_id}` set as a secondary index, or scope revocation to JWTs with a version check instead (see below).

CSRF: the session's shadow

Server-side sessions that rely on cookies are automatically sent by the browser — which is exactly the property Cross-Site Request Forgery exploits. An attacker page `evil.com` that triggers `POST https://api.example.com/transfer` will carry the victim's cookie if the browser considers it same-site-eligible.

Two defenses, use both in depth:

1. **SameSite=Lax or Strict** — the browser omits the cookie on cross-site POST. This is the primary defense and, for most teams, sufficient. Verify every auth cookie sets it; audit with `curl -v` and browser DevTools.
2. **Double-submit / synchronizer token** — issue a `csrf_token` (random 256 bits, stored server-side or as `__Host-csrf_token` with `SameSite=Lax`) and require the client to send it as a header (`X-CSRF-Token`) that JavaScript must set explicitly — a cross-origin page cannot set custom headers without CORS preflight, which the server can deny. Frameworks (Django, Rails, Spring) do this automatically; wire it yourself if you are not using one.

APIs consumed only by non-browser clients (mobile apps, service-to-service) that use `Authorization: Bearer` tokens are not vulnerable to CSRF — the browser does not auto-send `Authorization` headers — so they do not need CSRF tokens. But any endpoint that accepts cookies must defend.

Bearer tokens and JWTs

A bearer token is a self-contained proof: the client holds `Authorization: Bearer <token>` and the server verifies it without a per-request store lookup (or with a cached-key lookup). JWT (JSON Web Token, RFC 7519) is the dominant format — a signed (JWS, RFC 7515) or encrypted (JWE, RFC 7516) JSON object with a well-known serialization.

A JWT has three Base64url parts: `header.payload.signature`.

```
eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6Im5xIn0 header:
{"alg":"ES256","typ":"JWT","kid":"k1"}
eyJzdWIiOiI0ODIxIiwiaXNzIjoiaHR0cHM6Ly9hdXRoLmV4YW1wbGUuY29tIiwiaXVkJjo
iYXBpIiwiaXNzIjoiaXNzI0MTk... payload
MEUCIQDx... signature over header.payload with ES256 private key k1
```

JWT libraries exist for every language — but JWT misuse is the most audited auth vulnerability class. The rules below are non-optional.

Header and payload discipline

```
{
  "alg": "ES256",
  "typ": "JWT",
  "kid": "k1"
}
```

```
{
  "sub": "4821",
  "iss": "https://auth.example.com",
  "aud": "api",
  "exp": 1724197200,
  "nbf": 1724193600,
  "iat": 1724193600,
  "jti": "b2e8b2c0-9b0a-4f3a-9c1a-8f3d2e1a0b9c",
  "roles": ["user"]
}
```

Claim	Required?	Why
<code>alg</code>	Yes	Declares the signature algorithm. Verifier must <i>allowlist</i> acceptable <code>alg</code> values — never trust the token's <code>alg</code> blindly (see <code>alg:none</code> and key confusion).
<code>kid</code>	Yes at fleet scale	Identifies which key signed the token. Without <code>kid</code> , rotation requires trying every key or coupling token issuance to verifier deploys.
<code>exp</code>	Yes	Absolute expiry — verifier rejects after this. Keep access tokens short (5–15 min).
<code>nbf</code> / <code>iat</code>	Yes	<code>nbf</code> prevents use before issuance; <code>iat</code> supports clock-skew leeway and audit.

Claim	Required?	Why
<code>iss</code> / <code>aud</code>	Yes	Bind the token to its issuer and intended audience — a token for <code>aud: analytics</code> must not be accepted by <code>aud: payments</code> .
<code>jti</code>	For revocation	Unique token ID — the handle a blocklist can reference.
<code>sub</code>	Yes	The subject the token asserts.

A token without `exp` is a forever-credential. A token without `aud` is a cross-service replay. A token whose `alg` is not allowlisted is a signature-bypass waiting to happen.

Issuing and verifying JWTs

Version pins. PyJWT 2.8+ (Python), Go github.com/golang-jwt/jwt/v5 + golang.org/x/crypto for ECDSA, `cryptography` 42+ or `PyJWT[crypto]` for ES256. JWT specs: RFC 7519 (JWT), RFC 7515 (JWS), RFC 7517 (JWK/JWKS), RFC 7518 (JWA).

Choose **asymmetric** (RS256/ES256/EdDSA) for tokens verified by multiple services — verifiers hold the *public* key, so compromise of a verifier does not let it forge tokens. Use **symmetric** (HS256) only when the issuer and the single verifier share a secret and no other service touches the token (rare at fleet scale).

ES256 (ECDSA P-256) is the default for new systems: small keys and signatures, fast, constant-time in well-maintained libraries, and no RSA padding decisions to get wrong.

Python — issue and verify (PyJWT 2.8+, cryptography for ES256)

```

# jwt_auth.py – issue + verify with ES256 and JWKS rotation (Python
3.12)
import time, uuid, json
import jwt # PyJWT 2.8+
from jwt import PyJWKClient
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import serialization

ISSUER = "https://auth.example.com"
AUDIENCE = "api"
KID = "k1"

# — Key generation (once, stored in KMS/HSM in production) —————
private_key = ec.generate_private_key(ec.SECP256R1())
private_pem = private_key.private_bytes(
    serialization.Encoding.PEM,
    serialization.PrivateFormat.PKCS8,
    serialization.NoEncryption(),
)
public_pem = private_key.public_key().public_bytes(
    serialization.Encoding.PEM,
    serialization.PublicFormat.SubjectPublicKeyInfo,
)

# — Issue —————
def issue_token(user_id: str, roles: list[str], ttl: int = 600) -> str:
    now = int(time.time())
    payload = {
        "sub": user_id,
        "iss": ISSUER,
        "aud": AUDIENCE,
        "exp": now + ttl,
        "nbf": now,
        "iat": now,
        "jti": str(uuid.uuid4()),
        "roles": roles,
    }
    return jwt.encode(payload, private_pem, algorithm="ES256",
        headers={"kid": KID, "typ": "JWT"})

# — Verify – allowlist alg, verify iss/aud/exp, fetch key by kid ———
# In production, cache the JWKS response (Cache-Control / Expires) and
# reuse the PyJWKClient across requests – do not fetch JWKS per
request.
jwks_client = PyJWKClient(f"{ISSUER}/.well-known/jwks.json")

def verify_token(token: str) -> dict:
    # Fetch signing key by kid from JWKS (cached internally by
    PyJWKClient)

```

```

    signing_key = jwks_client.get_signing_key_from_jwt(token)
    return jwt.decode(
        token,
        signing_key.key,
        algorithms=["ES256"],           # allowlist – never accept
        "none" or HS256 here
        issuer=ISSUER,
        audience=AUDIENCE,
        leeway=30,                      # 30s clock skew tolerance
        options={"require": ["exp", "iat", "iss", "aud", "sub"]},
    )

# Demo
if __name__ == "__main__":
    tok = issue_token("4821", ["user"])
    print(tok)
    print(json.dumps(verify_token(tok), indent=2))
    # Tamper test – flipping a byte must fail verification:
    # bad = tok[:-4] + "AAAA"
    # verify_token(bad) # -> jwt.InvalidSignatureError

```


Go — verify with JWKS and `kid` (`net/http` middleware, Go 1.22+)

```

package auth

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "strings"
    "sync"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

// JWKS cache – refresh on kid miss or TTL, not per request.
type JWKSCache struct {
    mu      sync.RWMutex
    url     string
    keys    map[string]any // kid -> public key
    fetched time.Time
    ttl     time.Duration
}

func NewJWKSCache(url string) *JWKSCache {
    return &JWKSCache{url: url, keys: make(map[string]any), ttl: 10 * time.Minute}
}

func (c *JWKSCache) KeyFunc(token *jwt.Token) (any, error) {
    // Hard allowlist – reject anything that is not ES256
    if token.Method.Alg() != jwt.SigningMethodES256.Alg() {
        return nil, fmt.Errorf("unexpected alg: %s",
            token.Header["alg"])
    }
    kid, _ := token.Header["kid"].(string)
    if kid == "" {
        return nil, fmt.Errorf("missing kid")
    }
    c.mu.RLock()
    k, ok := c.keys[kid]
    fresh := time.Since(c.fetched) < c.ttl
    c.mu.RUnlock()
    if ok && fresh {
        return k, nil
    }
    if err := c.refresh(); err != nil {
        return nil, err
    }
    c.mu.RLock()
    defer c.mu.RUnlock()

```

```

    k, ok = c.keys[kid]
    if !ok {
        return nil, fmt.Errorf("unknown kid: %s", kid)
    }
    return k, nil
}

func (c *JWKSCache) refresh() error {
    // Fetch JWKS JSON (RFC 7517), parse jwkSet, populate c.keys by kid
    // Omitted for brevity – use github.com/MicahParks/keyfunc or
    // golang.org/x/oauth2/jws patterns; handle HTTP timeout + retry.
    // Pseudocode:
    // resp, _ := http.Get(c.url) ; var set jwkSet ;
    json.NewDecoder(resp.Body).Decode(&set)
    // for _, jwk := range set.Keys { c.keys[jwk.Kid] =
    jwk.PublicKey() }
    return fmt.Errorf("refresh: implement JWKS fetch (see keyfunc
library)")
}

// Middleware – use as http.Handler wrapper
func RequireJWT(cache *JWKSCache, issuer, audience string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            tok := strings.TrimPrefix(r.Header.Get("Authorization"), "Bearer ")
            if tok == "" {
                http.Error(w, "missing bearer token", http.StatusUnauthorized)
                return
            }
            claims := jwt.MapClaims{}
            _, err := jwt.ParseWithClaims(tok, claims, cache.KeyFunc,
                jwt.WithIssuer(issuer),
                jwt.WithAudience(audience),
                jwt.WithLeeway(30*time.Second),
                jwt.WithExpirationRequired(),
            )
            if err != nil {
                http.Error(w, "invalid token", http.StatusUnauthorized)
                return
            }
            ctx := context.WithValue(r.Context(), "claims", claims)
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

var _ = json.Marshal // keep import

```

Library note. The `refresh()` stub is the only part you should not hand-roll in production — use `github.com/MicahParks/keyfunc` (JWKS auto-refresh) or `github.com/lestrrat-go/jwx/v2/jwk` which handles caching, rotation, and `kid` lookup out of the box. The `KeyFunc` allowlist pattern above is what to audit regardless of library.

The JWT pitfalls you must block

Pitfall	Mechanism	Defense
alg: none	Attacker sets <code>alg</code> to <code>none</code> , removes signature; naive verifiers that trust the header accept an unsigned token.	Allowlist <code>algorithms=["ES256"]</code> (or your one chosen alg) — never accept <code>none</code> . Libraries that default to allowlisting (PyJWT <code>algorithms=required</code>) are safer; audit every <code>jwt.decode</code> / <code>jwt.Parse</code> call.
Key confusion (RS256 ↔ HS256)	Attacker sets <code>alg: HS256</code> and signs with the RSA <i>public</i> key as if it were an HMAC secret — verifier that accepts both RS256 and HS256 will verify with the public key as the HMAC key.	Accept exactly one algorithm family. If you use ES256, the allowlist is <code>["ES256"]</code> — not <code>["ES256", "HS256"]</code> . Never mix asymmetric and symmetric verification in the same code path.
Weak HS256 secret	HS256 secret is a short password; attacker brute-forces the HMAC key offline.	If you must use HS256, the secret must be 256+ random bits (<code>secrets.token_bytes(32)</code>), stored in a KMS/secret manager, never in source. Prefer ES256 to avoid the problem entirely.
Missing exp / aud	Forever-token or cross-service replay.	<code>require: ["exp", "aud"]</code> and verify both. Keep <code>exp</code> short (minutes).
Large payload / header DoS	Attacker stuffs many claims or a huge header to exhaust parser memory.	Enforce token size limits (e.g., 8 KB) before parsing; reject tokens with unexpected claims.

Pitfall	Mechanism	Defense
Leaked tokens in logs/URLs	GET /api?token=... or Authorization header logged verbatim.	Tokens travel in Authorization header or Secure / HttpOnly cookie only — never query params or fragment. Scrub Authorization from access logs; alert on its presence in error messages.

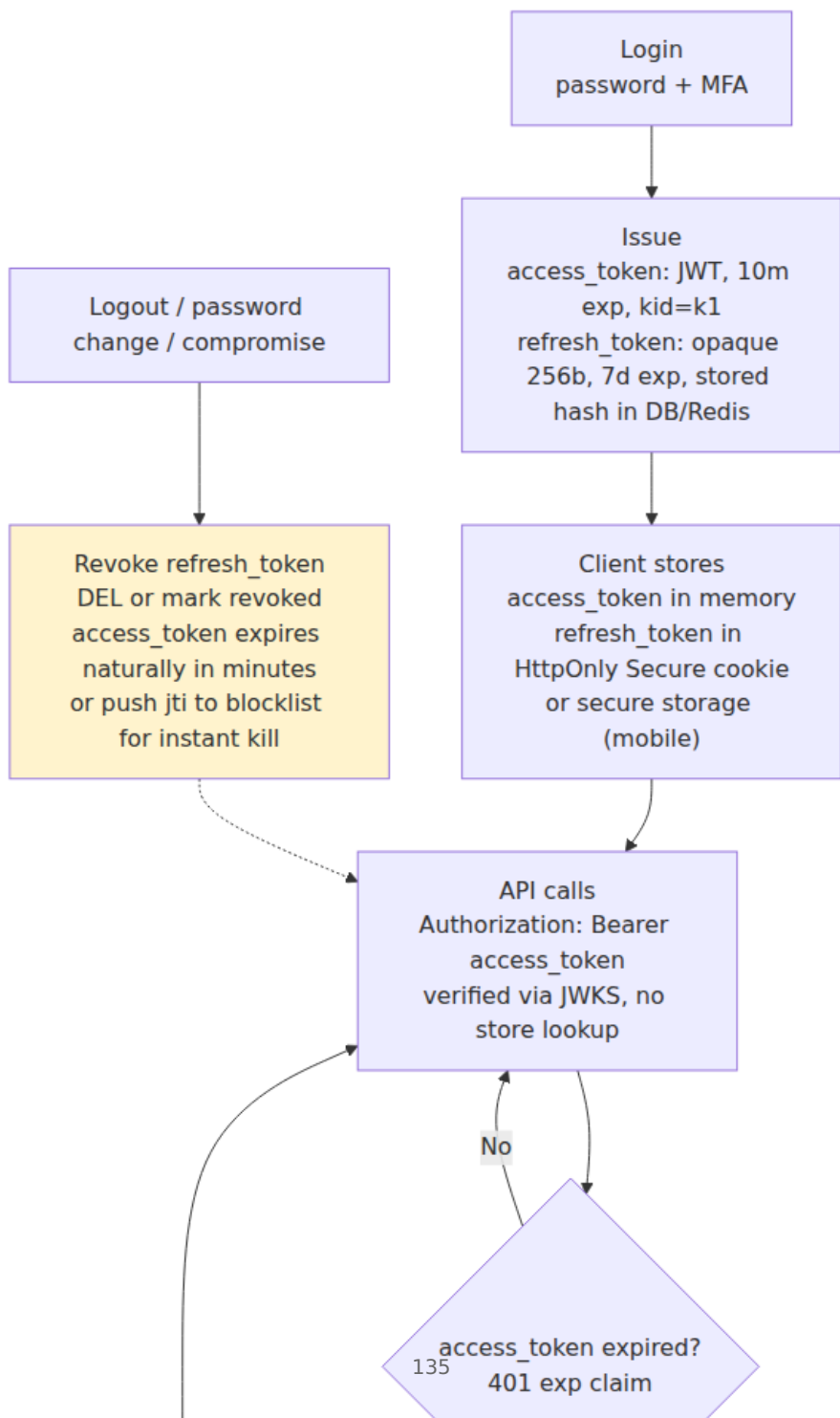
Sessions vs JWTs — the decision

Dimension	Server-side session	JWT (asymmetric)
State	Stateful — store lookup per request	Stateless verification — no per-request store hit if JWKS is cached
Size	Cookie ~50 bytes	Token ~500-1500 bytes (header + claims + signature) — too large for many cookies, fine in Authorization header
Revocation	Immediate — DEL session:abc	Delayed — must wait for exp or maintain a blocklist (see below)
Horizontal scale	Requires shared store (Redis) or sticky routing	Any replica with the JWKS verifies — no store on the hot path
Key compromise	Store ACL compromise vs session theft	Private signing key compromise forges any token — protect with HSM/KMS
Best for	Traditional web apps with cookies, admin consoles, flows that need instant logout	Microservice APIs, mobile/SPA backends, service-to-mesh calls where replicas scale rapidly

Most production systems use **both**: a session cookie for the web app (where instant logout and CSRF discipline matter) and short-lived JWTs for the API layer (where stateless verification at the edge matters). They are not mutually exclusive.

Refresh tokens and the token lifecycle

A JWT that is stolen is valid until `exp`. The mitigation is to keep `exp` short (5-15 min) and use a **refresh token** — a longer-lived, opaque, revocable handle that can mint new access tokens — so compromise of an access token has a bounded window and refresh can be gated and monitored.



Refresh tokens **must** be:

- **Opaque and random** — not JWTs. Generated with `secrets.token_urlsafe(32)` (256 bits), stored as a hash (SHA-256) in the database/Redis, looked up on refresh. Leaking the DB should not leak usable tokens.
- **Bound** — to a device fingerprint, IP class, or DPoP proof (RFC 9449) so theft requires more than the token alone. At minimum, store the issuing `user_agent` hash and alert on mismatch.
- **Single-use or versioned** — on each refresh, either rotate (issue a new refresh token, invalidate the old one — detects replay if the old one is reused) or maintain a `token_version` per user (see Ch 6) so password change invalidates all outstanding refresh tokens in one increment.
- **Scoped and short-lived for their type** — 7–30 days, not months. Require re-authentication for sensitive actions regardless of refresh validity.


```

# refresh.py - minimal refresh token store (PostgreSQL + Redis cache
# sketch)
import hashlib, secrets, time
import psycopg # psycopg 3.x

def hash_token(tok: str) -> str:
    return hashlib.sha256(tok.encode()).hexdigest()

def issue_refresh(user_id: str, ttl: int = 7*86400) -> str:
    raw = secrets.token_urlsafe(32)
    h = hash_token(raw)
    exp = int(time.time()) + ttl
    with psycopg.connect("dbname=auth") as conn:
        conn.execute(
            "INSERT INTO refresh_tokens(hash, user_id, expires_at,
            created_at) VALUES (%s,%s,to_timestamp(%s), now())",
            (h, user_id, exp),
        )
    return raw # send to client over TLS only

def verify_refresh(raw: str) -> str | None:
    h = hash_token(raw)
    with psycopg.connect("dbname=auth") as conn:
        row = conn.execute(
            "SELECT user_id FROM refresh_tokens WHERE hash=%s AND
            expires_at > now() AND revoked_at IS NULL",
            (h,),
        ).fetchone()
    return row[0] if row else None

def revoke_refresh(raw: str) -> None:
    h = hash_token(raw)
    with psycopg.connect("dbname=auth") as conn:
        conn.execute("UPDATE refresh_tokens SET revoked_at=now() WHERE
            hash=%s", (h,))

def revoke_all_for_user(user_id: str) -> None:
    with psycopg.connect("dbname=auth") as conn:
        conn.execute("UPDATE refresh_tokens SET revoked_at=now() WHERE
            user_id=%s AND revoked_at IS NULL", (user_id,))

```

Revocation for stateless tokens

The fundamental tension: JWTs are valuable because they are stateless, but revocation is a stateful operation. Options, in order of practicality:

1. **Short expiry + refresh** — the default. Revoke the refresh token; outstanding access tokens expire in minutes. For most products this is sufficient and the simplest. The window is `exp` — keep it short.
2. **Versioned tokens** — embed `token_version` (or `auth_version`) in the JWT, store the current version per user in Redis/DB, reject tokens whose version does not match. Password change or "log out everywhere" increments the version and invalidates all outstanding tokens in one write, with one lookup added to verification (cacheable):

```
python # On verify: after signature checks if
claims["ver"] != redis.get(f"auth_ver:{claims['sub']}"): raise
InvalidToken("stale token version - re-authenticate")
```
3. **JTIs blacklist (denylist)** — on logout/compromise, push the `jti` to a Redis set with TTL = remaining `exp`. Verifiers check the set (single `SISMEMBER` / `EXISTS`). Scales to the number of *revoked* tokens, not total tokens — manageable if revocation is rare, expensive if you revoke on every logout in a high-traffic system (prefer versioned tokens or short-TTL there).
4. **Stateful introspection** — verifiers call the auth service to check token validity (RFC 7662 token introspection). Correct but reintroduces a store lookup on every request — at which point you have rebuilt server-side sessions with JWT serialization overhead.

Do not try to have it both ways: if your threat model requires instant revocation of individual access tokens at high rate, use sessions or versioned tokens, not a naive JWT that you then bolt a per-request DB check onto.

Session and token binding

A bearer token that can be replayed from any machine is a password-equivalent secret in transit. Binding raises the bar:

- **DPoP (Demonstrating Proof-of-Possession, RFC 9449, 2023)** — the client generates an ephemeral key pair, sends the public key (as a `jwk` in the DPoP header), and signs a proof JWT over the request method, URI, and a server nonce. The auth server binds the access

token to the DPoP public key (`cnf.jkt` thumbprint); verifiers check that the request's DPoP proof matches the bound key. Stolen tokens cannot be replayed without the DPoP private key. Increasingly required for high-value APIs and FAPI profiles — implement with `dpop` libraries rather than hand-rolling the proof.

- **mTLS binding (RFC 8705, OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens)** — bind the access token to the client's mTLS certificate thumbprint (`cnf.x5t#S256`). Verification is that the same client certificate is presented on the TLS connection. Natural fit alongside Ch 4/Ch 10 mTLS.
- **Cookie** `__Host-` + `Secure` + `HttpOnly` is itself a weak binding — it ties the session to the TLS origin and prevents JS exfiltration, but not replay.

For backend service-to-service tokens, DPoP or mTLS binding is strongly recommended. For browser sessions, `__Host-` cookies plus short-lived JWTs plus refresh with rotation is the pragmatic baseline.

Clock skew, expiry, and the distributed clock

Every `exp / nbf` check compares the verifier's clock to the issuer's clock. In a fleet with NTP/chrony drift, a token that is valid on the issuer may be considered expired on a verifier whose clock is ahead, or not-yet-valid on a verifier whose clock is behind. The fix:

- Keep clocks synchronized — `chrony` with PTP-backed NTP, alert on `chrony tracking offset > 50 ms`. This is an infrastructure invariant TLS (Ch 4), sessions, and token expiry all depend on.
- Allow **leeway** (30–60 s is typical) — `leeway=30` in PyJWT, `WithLeeway(30*time.Second)` in Go `jwt`. This is not "accept expired tokens" — it is "tolerate bounded clock disagreement." Do not set leeway to minutes; fix the clocks instead.
- Log `iat` skew on verification — `verifier_now - claims["iat"]` — as a time-series. A growing positive skew means the verifier is ahead and will start rejecting fresh tokens soon.

Distributed-systems lens — two architectures

Sticky sessions (avoid if you can)

Route each user to the same backend via consistent hashing or `Set-Cookie` affinity (`affinity: cookie` in nginx/ALB). Then sessions can live in process memory. It is simple but fragile: instance failure loses sessions, scaling requires draining affinity, and canary deploys leak sticky users across versions. Use only for legacy state you cannot externalize. The modern default is a shared store or stateless JWT.

Shared store + cache-coherent verification



- Session lookups are `GET session:{id}` to Redis — single-digit milliseconds in the same AZ, ~10 ms cross-AZ. Pipeline the session fetch with the business query to hide the RTT, or cache hot sessions in-process with a short TTL (5-30 s) and accept a bounded staleness window on revocation.
- JWKS is fetched from the auth service's `/.well-known/jwks.json` (RFC 8414 / OIDC Discovery), cached in each verifier with a TTL, and refreshed on `kid` miss. The JWKS endpoint must be highly available — it is on the verification path for cold `kid`s. Serve it from a CDN or replicated cache, not a single auth instance, and version keys (`kid: k1, k2, ...`) so rotation is additive.
- `auth_ver` / JTI blocklist checks are one more Redis lookup on the hot path. Weigh that against the security requirement: if instant revocation is not required, skip it and rely on short `exp` + refresh. Every lookup you add is latency and a failure mode.

Failure modes and incident patterns

Session fixation. Attacker sets `session_id=known_value` in the victim's browser (via `Set-Cookie` in an HTTP response, subdomain cookie, or XSS), victim logs in and the app associates the victim's identity with the attacker-known session ID, attacker reuses it. Defense: **regenerate the**

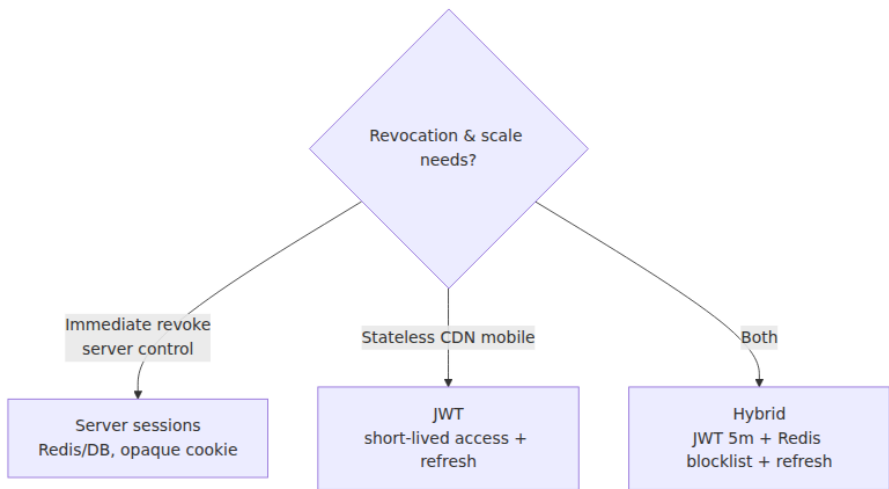
session ID on authentication — issue a fresh random ID after login, invalidate the old one. Test: log in twice and assert the cookie value changed.

Session hijacking via XSS. Any XSS that can run `fetch(document.cookie)` steals `HttpOnly`-less cookies and, if `Secure` is missing, can exfiltrate over HTTP. Defense is layered: `HttpOnly` + `Secure` + `__Host-` as above, plus Content Security Policy (Ch 9) to prevent the XSS in the first place, plus short TTLs so stolen sessions die quickly. Audit: no auth cookie may be readable from JS — verify with DevTools → Application → Cookies.

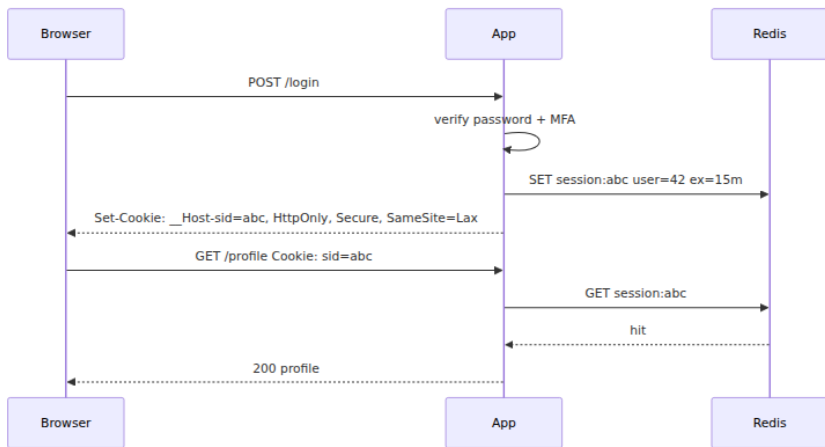
JWT replay after theft. Stolen `Authorization` header from a log file or a compromised edge. Defenses: short `exp`, DPoP/mTLS binding, do not log `Authorization` headers, encrypt tokens at rest on the client (mobile Keychain/Keystore, not `localStorage`), and prefer `HttpOnly` cookies for refresh tokens so JS cannot read them.

Key rotation without `kid`. New signing key deployed, old key deleted, every in-flight token fails verification and every user is logged out. Defense: publish both keys in JWKS with distinct `kid`s, issue new tokens with the new `kid`, keep the old key verifiable until every token signed with it has expired (max `exp` window), then remove it. This is the same dual-read pattern as Ch 3 and Ch 4. Monitor `kid` distribution in issued tokens to know when the old `kid` is safe to retire.

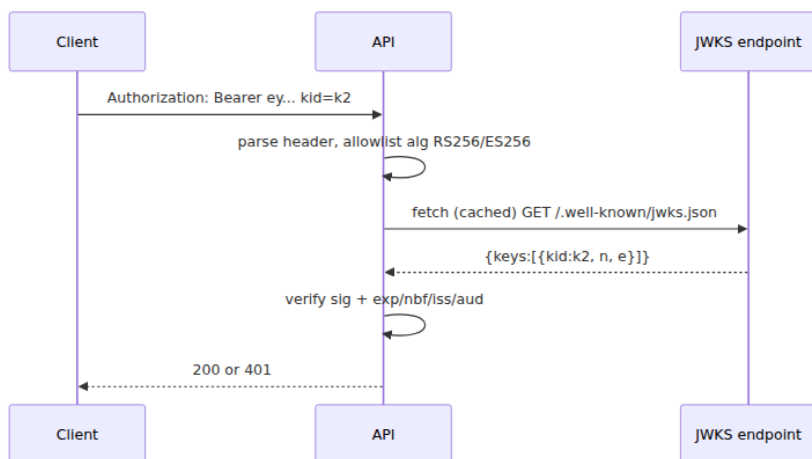
Session vs JWT Decision



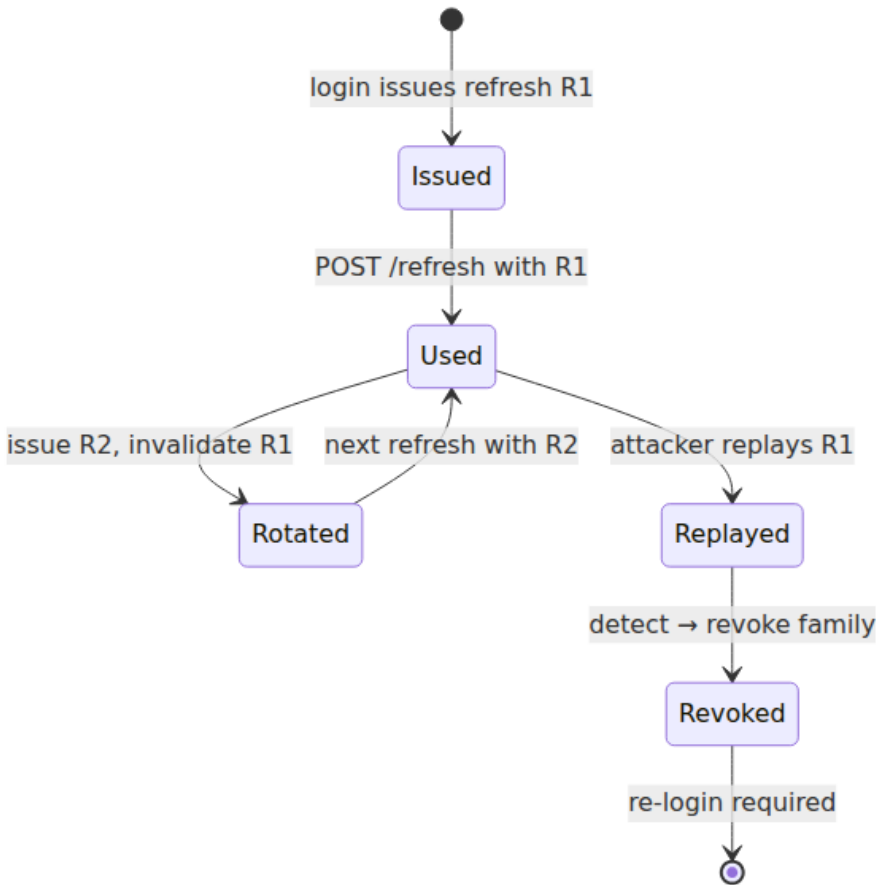
Login and Session Creation



JWT Validation via JWKS



Refresh Token Rotation



Key takeaways

- Server-side sessions (opaque handle + store lookup) and JWTs (self-contained signed object, RFC 7519) are not rivals — they are different trade-offs. Sessions give instant revocation at the cost of a store on the hot path; JWTs give stateless verification at the cost of delayed revocation bounded by `exp`. Most fleets use sessions for web (cookies) and short-lived JWTs for APIs.
- Secure cookies require `HttpOnly` + `Secure` + `SameSite=Lax` (or `Strict`) + `__Host-` prefix, with session IDs from a CSPRNG (`crypto/rand`, `secrets.token_urlsafe`) and regeneration on login to prevent fixation.

- JWTs must be issued with `exp`, `iss`, `aud`, `kid`, and `jti`, verified with an allowlisted `alg` (one value, e.g., `ES256`), checked `iss` / `aud` / `exp`, and distributed via JWKS (RFC 7517). Never accept `alg: none`, never mix RS256 and HS256 in the same verifier, and never trust the header's `alg` without an allowlist.
- Keep access tokens short (5–15 min), use opaque random refresh tokens (256 bits, stored as hashes) bound to a fingerprint, and detect replay via rotation or versioning. Long-lived bearer tokens are a breach waiting to happen.
- Revocation for JWTs is bounded by `exp` unless you add state: versioned tokens (`auth_ver` per user) or JTI blocklists give instant revocation with one extra lookup; introspection reintroduces per-request state. Choose based on whether your product actually needs instant individual-token revocation.
- Bind high-value tokens with DPoP (RFC 9449) or mTLS (RFC 8705) so theft does not give replay; at minimum, do not leak tokens via URLs or logs and keep refresh tokens in `HttpOnly` cookies or secure device storage.
- At fleet scale, session stores (Redis) and JWKS are availability dependencies — replicate, cache with TTL, refresh on `kid` miss, tolerate 30–60 s clock skew with leeway, and version/overlap signing keys so rotation does not invalidate in-flight tokens. Every `exp` / `nbf` check depends on healthy NTP/chrony.

Further reading

- **Standards:** RFC 7519 (JWT), RFC 7515 (JWS), RFC 7517 (JWK/JWKS), RFC 7518 (JWA), RFC 7638 (JWK Thumbprint), RFC 7662 (Token Introspection), RFC 8705 (mTLS Token Binding), RFC 9449 (DPoP), RFC 6750 (Bearer Token Usage), RFC 6265 / RFC 6265bis (Cookies), RFC 6797 (HSTS), OWASP ASVS 3.x (Session Management) and OWASP Cheat Sheet Series (Session Management, JWT, CSRF Prevention).
- Lodderstedt et al. — *OAuth 2.0 Security Best Current Practice* (IETF draft-ietf-oauth-security-topics, updates RFC 6749/6750) — even for first-party auth, §4 describes token hygiene that applies directly.

- OWASP — *JWT Cheat Sheet for Java / JSON Web Token Cheat Sheet* — concise pitfall catalog (alg:none, key confusion); readable as a code-review checklist outside Java.
- PyJWT docs — <https://pyjwt.readthedocs.io/> (2.8+) and `golang-jwt/jwt` — <https://github.com/golang-jwt/jwt> — authoritative for library behavior and `kid` handling.
- cryptography (Python) — <https://cryptography.io/> — for correct ES256 key handling; avoids foot-guns in raw `ecdsa` libraries.
- Jones, Bradley, Sakimura — *JSON Web Token (JWT) RFC 7519 + JWS RFC 7515 + JWK RFC 7517* — read as a trio; JWT alone is underspecified without them.

Chapter 6 — OAuth 2.0 and OpenID Connect

What this chapter covers. Chapter 5 solved first-party authentication — the user proves who they are to *your* system. But modern backends rarely live alone: a mobile app needs to call your API on the user's behalf without seeing their password, a partner service needs scoped access to one user's data, and your own microservices need to act as a user across service boundaries. OAuth 2.0 (RFC 6749 + RFC 6750) is the delegation protocol for "let app A act on user U's behalf, within scope S, for time T, revocably." OpenID Connect (OIDC 1.0) is the identity layer on top — "and tell app A who U is, with a verifiable ID token." This chapter implements both at the level a backend engineer needs: the authorization code flow with PKCE (RFC 7636) end-to-end, the threat model (code interception, token leakage, open redirector), OIDC Discovery and JWKS, token handling (JWT vs opaque, introspection per RFC 7662, revocation), and the distributed-systems reality of an authorization server fleet that must issue, verify, and revoke tokens under load without becoming a single point of failure.

Learning goals — after this chapter you should be able to:

- Explain OAuth 2.0's four roles, four grant types, and why only authorization code + PKCE is recommended for new systems (per OAuth 2.0 Security Best Current Practice, draft-ietf-oauth-security-topics).

- Implement and verify the authorization code flow with PKCE: `code_challenge` / `code_verifier`, `state` binding, `redirect_uri` exact-match validation, and token exchange with client authentication (or public-client PKCE without a secret).
- Distinguish OIDC's ID token (identity assertion, JWT, validated by the client) from OAuth's access token (delegation credential, validated by the resource server) and from the UserInfo endpoint — and explain when each is used.
- Operate OIDC Discovery (`.well-known/openid-configuration` per RFC 8414 / OIDC Discovery 1.0) and JWKS rotation (RFC 7517) so clients and resource servers discover keys without redeploy.
- Handle tokens correctly: short-lived JWT access tokens vs opaque tokens, refresh rotation, introspection (RFC 7662) vs local JWT verification, audience/scope enforcement at every resource server, and why `implicit` and `password` grants must not be used.
- Run a minimal OAuth authorization server and a confidential client in Go/Python, and integrate a relying party via `golang.org/x/oauth2` or `authlib`.
- Design the authorization server for scale — stateless JWT issuance with cached JWKS vs stateful introspection, horizontal scaling of the code/token stores, revocation propagation, and multi-region discovery.

Boundary notes. The *transport* that protects every OAuth redirect and token exchange — TLS 1.3 (RFC 8446) and PKI operations — is covered in **Vol 3, Ch 6** (wire) and **Ch 4** (operations) of this volume; every endpoint in this chapter must be `https://` and both sides must validate certificates (Ch 4) — OAuth over plaintext is broken by definition. *First-party* session/token mechanics (cookies, JWT structure, `kid` / exp discipline, JWKS caching) are **Ch 5**; this chapter reuses that machinery for delegated access and adds the redirect dance and cross-party trust. *Service-to-service* identity without a user (workload identity, SPIFFE/SPIRE, mTLS) is **Ch 10**; OAuth client-credentials is the user-absent grant covered here, but workload attestation is Ch 10. *Authorization* (what a token's `scope` / `claims` allow — RBAC/ABAC/ReBAC) is **Ch 7**.

Why delegation, not password sharing

Before OAuth, the only way for app A to act on user U's behalf was to get U's password — the "password anti-pattern" that still appears in legacy password grants and screen-scraping. The password gives *full* access, cannot be scoped or revoked per app, and compromise of A leaks U's credential for every service where it is reused. OAuth replaces it with a scoped, revocable token whose lifetime is independent of the password and whose compromise is bounded to one client and one scope.

The pattern appears everywhere in a distributed backend even when the spec language is not used:

- A web app's frontend (SPA) calling its own API gateway — the gateway validates an access token scoped to that user.
- A partner integration ("connect your calendar") — the partner receives a token scoped to `calendar:read` that the user can revoke without changing their password.
- An internal job that fans out across services as a user — the job propagates the user's access token (or exchanges it via token exchange, RFC 8693) and each service enforces `aud / scope` locally.

All of these are OAuth-shaped, whether they speak RFC 6749 on the wire or a proprietary analogue. Learning the spec is learning the shape.

Roles, tokens, and grant types

The four roles

Role	What it is	Example
Resource Owner	The user (or entity) that owns the resource	Alice, <code>user_id=4821</code>
Client	The app requesting delegated access on the owner's behalf	<code>calendar-sync</code> partner app, your SPA, a backend service
Authorization Server (AS)	Issues tokens after authenticating the owner and obtaining consent	<code>https://auth.example.com</code>

Role	What it is	Example
Resource Server (RS)	Hosts the protected resource, validates the access token	<code>https://api.example.com</code> — validates <code>Authorization: Bearer ...</code>

In many production systems the AS and RS are operated by the same team (your auth service + your API), but the spec keeps them distinct because in federated deployments they are not — Google's AS issues tokens that your RS validates via Google's JWKS.

The two tokens that matter

- **Access token** (RFC 6749 §1.4, RFC 6750) — the credential the client presents to the RS. Short-lived (minutes), scoped (`scope=read:profile`), audience-bound (`aud=api.example.com`). The RS validates signature, `exp`, `aud`, and `scope` on every request. Format may be JWT (Ch 5) or opaque — both are specified; the choice is a distributed-systems trade-off (see below).
- **Refresh token** (RFC 6749 §1.5) — a longer-lived opaque handle that the client exchanges at the AS for a new access token without involving the user. Stored as a hash server-side (like Ch 5's refresh), bound to the client, optionally rotated on each use to detect replay.
- **Authorization code** — not a token but a single-use, short-lived (30–60 s), opaque handle that the AS returns via redirect and the client exchanges for tokens. The indirection is the security boundary: the code traverses the browser redirect (where it could be intercepted), the tokens traverse a direct back-channel between client and AS (where they cannot).

OIDC adds a third: the **ID token** — a JWT that asserts *who the user is* (`sub`, `iss`, `aud=client_id`, `exp`, `nonce`, `acr/amr` for auth strength). It is consumed by the *client*, not the RS, and must be validated as a JWT (Ch 5: allowlisted `alg`, `iss/aud/exp` checks, `kid`-based JWKS lookup). The ID token is proof of authentication; the access token is proof of delegation — conflating them leaks identity claims to resource servers that should not see them or grants delegation power to an identity assertion.

Grant types — and which to use

Version pins. OAuth 2.0 RFC 6749 + Bearer RFC 6750 (2012, RFC 6749 is being revised as draft-ietf-oauth-v2-1). PKCE RFC 7636 (2015, required for authorization code by OAuth 2.1 and Security BCP). Token Introspection RFC 7662, Token Revocation RFC 7009, AS Metadata RFC 8414, JWT Access Token profile RFC 9068. OIDC Core/Discovery 1.0 (2014, with errata; implement against current Final).

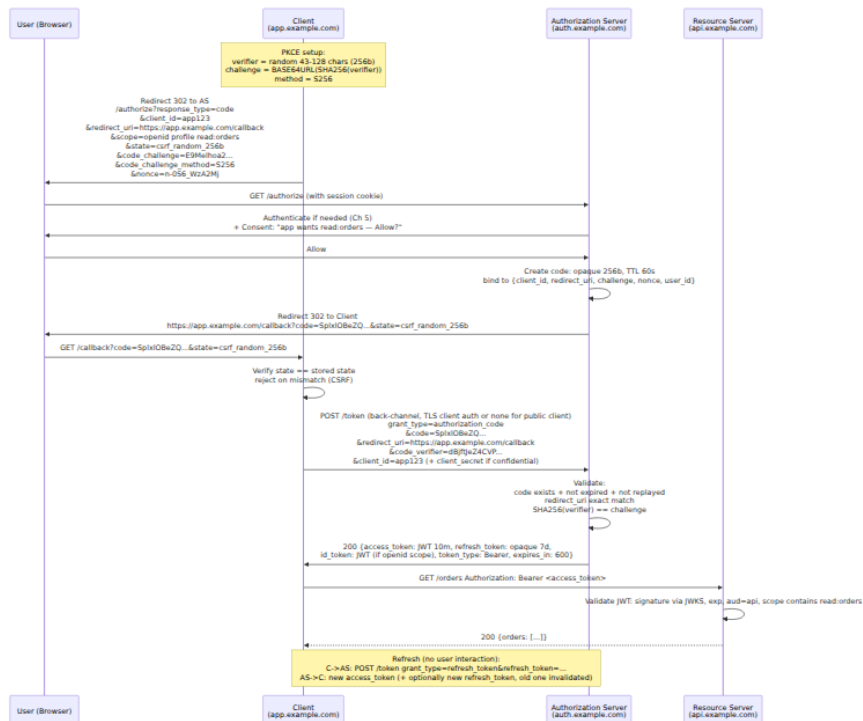
Grant	Flow	When to use	Status
Authorization code + PKCE (RFC 6749 §4.1 + RFC 7636)	Browser redirect → code → back-channel token exchange with <code>code_verifier</code>	Every user-facing delegation — web, mobile, SPA, native apps	Recommended for all new clients — public or confidential
Client credentials (RFC 6749 §4.4)	Client authenticates with its own credential, no user	Service-to-service / machine-to-machine (no resource owner)	Correct for M2M — scoped to the client, not a user
Device code (RFC 8628)	User enters a code on a second device	TVs, CLI tools, IoT with limited input	Correct for input-constrained devices
Refresh token (RFC 6749 §6)	<code>grant_type=refresh_token</code> with the stored refresh handle	Renew an access token without user interaction	Correct — with rotation and binding (Ch 5)

Grant	Flow	When to use	Status
Implicit (<code>response_type=token</code> , RFC 6749 §4.2)	Token returned directly in the redirect fragment	Legacy SPAs before PKCE existed	Must not be used — token exposed in URL/history, no PKCE, no client auth (Security BCP §2.1.2)
Resource Owner Password Credentials (<code>password</code> , RFC 6749 §4.3)	Client collects the user's password and exchanges it for a token	Migration from password anti-pattern only	Must not be used except for narrow legacy migration; removed in OAuth 2.1

If your design includes `response_type=token` or `grant_type=password` , stop and redesign around authorization code + PKCE.

Authorization code + PKCE, step by step

PKCE (Proof Key for Code Exchange, RFC 7636, pronounced "pixy") closes the code-interception attack: even if an attacker intercepts the authorization code in the redirect, they cannot exchange it without the `code_verifier` that never left the legitimate client's memory.



Why every field matters

Field	Role	If omitted or wrong
state	CSRF binding between the authorize request and the callback — random 256 bits, stored in <code>__Host-</code> cookie or server session, compared exactly on callback	Attacker initiates <code>code</code> for their account, victim completes it and links their session to attacker's identity (login CSRF / code injection)
nonce (OIDC)	Replay binding for the ID token — client sends <code>nonce</code> , AS echoes it in <code>id_token.nonce</code> , client validates	ID token replay: attacker replays a captured ID token to impersonate the user

Field	Role	If omitted or wrong
<code>redirect_uri</code>	Exact-match validated by the AS against the client's pre-registered set — not prefix, not regex, not "starts with"	Open redirector: attacker registers <code>https://evil.com?redirect_uri=https://app.example.com/callback@evil.com</code> or exploits lax matching to steal the code
<code>code_challenge</code> / <code>code_verifier</code>	PKCE — proves the client that requested the code is the same client exchanging it	Code interception: on mobile, a malicious app registers the same redirect scheme and captures the code; without PKCE, it can exchange it
<code>code</code> single-use	AS deletes the code on first exchange; replay attempts fail	Code replay — attacker that observed the code reuses it before the legitimate client
<code>client_secret</code> / <code>private_key_jwt</code>	Authenticates a <i>confidential</i> client (server that can keep a secret); public clients (SPA, mobile) use PKCE alone and must not be issued a static secret	Secret leakage via mobile binary decompilation or JS bundle; treating a public client as confidential gives false assurance

OIDC: identity on top of delegation

OIDC (OpenID Connect Core 1.0) adds a thin, well-specified identity layer to the OAuth 2.0 code flow. When the client includes `scope=openid` (plus optionally `profile`, `email`, `address`, `phone`), the AS:

1. Returns an **ID token** (JWT) in the token response alongside the access token. The ID token's `aud` is the `client_id` (it is for the client), `iss` is the issuer URL, `sub` is the user, `nonce` echoes the request nonce, and `exp` is short (minutes). The client validates it (Ch 5 JWT discipline: allowlisted `alg`, `iss` / `aud` / `exp` / `nonce` checks, `kid` JWKS lookup) and uses it to establish the user's identity — typically to look up or create a local session.
2. Exposes a **UserInfo endpoint** (`GET /userinfo` with the access token) that returns claims (`sub`, `name`, `email`, `email_verified`, ...)

as JSON, governed by the granted scopes. The client should prefer the ID token for basic identity and call UserInfo only for claims that may have changed since issuance or that are too large for a JWT.

The RS does not consume the ID token. The client does not send the ID token to the RS. The access token (JWT per RFC 9068 or opaque) is what the RS validates; the ID token is what the client validates to know who logged in. Mixing them — sending the ID token as a bearer to the RS, or treating the access token as an identity assertion — is a common bug that leaks `email / profile` to every RS and grants identity-token audience to delegation.

Discovery and JWKS — how clients find the truth

OIDC Discovery 1.0 and AS Metadata (RFC 8414) let a client discover the AS's endpoints and keys from a single issuer URL, without hardcoding:

```
GET https://auth.example.com/.well-known/openid-configuration
200 {
  "issuer": "https://auth.example.com",
  "authorization_endpoint": "https://auth.example.com/authorize",
  "token_endpoint": "https://auth.example.com/token",
  "userinfo_endpoint": "https://auth.example.com/userinfo",
  "jwks_uri": "https://auth.example.com/.well-known/jwks.json",
  "scopes_supported": ["openid", "profile", "email", "read:orders"],
  "response_types_supported": ["code"],
  "grant_types_supported": ["authorization_code", "refresh_token", "client_credentials"],
  "code_challenge_methods_supported": ["S256"],
  "token_endpoint_auth_methods_supported": ["client_secret_basic", "private_key_jwt", "none"],
  ...
}

GET https://auth.example.com/.well-known/jwks.json (RFC 7517)
200 {"keys": [
  {"kty": "EC", "crv": "P-256", "x": "...", "y": "...", "kid": "k1", "alg": "ES256", "use": "sig"},
  {"kty": "EC", "crv": "P-256", "x": "...", "y": "...", "kid": "k2", "alg": "ES256", "use": "sig"}
]}
```

Clients and RSs cache both responses by `Cache-Control / Expires` or a local TTL (5–15 min), refresh on `kid` miss, and pin the `issuer` to the `iss` claim. In a multi-region deploy, the discovery document and JWKS

are static, CDN-cacheable assets replicated to the edge — they must not depend on a single AS instance's liveness.

A minimal authorization server (Go)

What follows is deliberately stripped to the mechanics — storage is in maps for readability, adapted to Redis/DB at fleet scale. For production, prefer a hardened AS (ORY Hydra / Ory Kratos, Keycloak, Auth0/Okta) or a library (`ory/fosite` for Go, `node-oidc-provider`) over hand-rolling — the spec edge cases (exact `redirect_uri` matching, token binding, DPoP, JAR/PAR) are where hand-rolled servers break. Read this to understand the shape, then use a vetted implementation.

Version	pins.	Go	1.22+,	github.com/golang-jwt/jwt/v5 ,
		golang.org/x/crypto	for S256,	google/uuid for token IDs.

```

// Minimal AS – authorization code + PKCE (S256), token issuance (Go
1.22+)
// Focus: PKCE verification, code single-use, redirect_uri exact match,
JWT minting.
package main

import (
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "strings"
    "sync"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "github.com/google/uuid"
)

// — Domain —————
type Client struct {
    ID            string
    RedirectURIs  map[string]bool // exact-match set – not prefix, not
regex
    IsPublic      bool             // true: no secret, PKCE required
    Secret        string           // only for confidential clients
}

type codeEntry struct {
    ClientID      string
    RedirectURI   string
    Challenge     string // code_challenge
(BASE64URL(SHA256(verifier)))
    UserID        string
    Scope         string
    Nonce         string
    ExpiresAt     time.Time
}

// In-memory stores – replace with Redis/DB in production
var (
    clients = map[string]Client{
        "app123": {ID: "app123", RedirectURIs: map[string]bool{"https://
app.example.com/callback": true}, IsPublic: true},
        "svc-payments": {ID: "svc-payments", RedirectURIs: map[string]b
ool{}, IsPublic: false, Secret: "s3cr3t-not-in-source-use-vault"},
    }

```

```

    codes    = map[string]codeEntry{}
    codesMu sync.Mutex
    // refreshStore: hash -> {user_id, client_id, exp} – store hashes,
    not raw tokens (Ch 5)
)

// — Helpers —
func randURLSafe(n int) string {
    b := make([]byte, n)
    if _, err := rand.Read(b); err != nil {
        panic(err)
    }
    return base64.RawURLEncoding.EncodeToString(b)
}

// JWKS key – in production load from KMS/HSM, rotate via kid
var hmacKey = []byte(randURLSafe(32)) // HS256 demo key – use ES256
asymmetric in production

func mintAccessToken(userID, clientID, scope string) (string, error) {
    now := time.Now()
    claims := jwt.MapClaims{
        "sub": userID,
        "iss": "https://auth.example.com",
        "aud": "api",
        "scope": scope,
        "exp": now.Add(10 * time.Minute).Unix(),
        "iat": now.Unix(),
        "jti": uuid.NewString(),
    }
    tok := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    tok.Header["kid"] = "k1"
    return tok.SignedString(hmacKey)
}

func mintIDToken(userID, clientID, nonce string) (string, error) {
    now := time.Now()
    claims := jwt.MapClaims{
        "sub": userID,
        "iss": "https://auth.example.com",
        "aud": clientID, // ID token audience is the client, not the
RS
        "exp": now.Add(10 * time.Minute).Unix(),
        "iat": now.Unix(),
        "nonce": nonce,
    }
    tok := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    tok.Header["kid"] = "k1"
    return tok.SignedString(hmacKey)
}

```

```

// — Handlers —————
// GET /authorize?
response_type=code&client_id=...&redirect_uri=...&scope=...&state=...&code_challenge=...&code_challenge_method=S256&nonce=...
func authorizeHandler(w http.ResponseWriter, r *http.Request) {
    q := r.URL.Query()
    if q.Get("response_type") != "code" {
        http.Error(w, "unsupported_response_type", http.StatusBadRequest)
        return
    }
    clientID, redirectURI := q.Get("client_id"), q.Get("redirect_uri")
    cl, ok := clients[clientID]
    if !ok || !cl.RedirectURIs[redirectURI] {
        http.Error(w, "unauthorized_client or invalid redirect_uri", http.StatusBadRequest)
        return
    }
    if q.Get("code_challenge") == "" ||
q.Get("code_challenge_method") != "S256" {
        http.Error(w, "code_challenge (S256) required", http.StatusBadRequest)
        return
    }
    if q.Get("state") == "" {
        http.Error(w, "state required", http.StatusBadRequest)
        return
    }
    // Authenticate the user (Ch 5) – stubbed as ?user_id=... for demo
    userID := q.Get("user_id")
    if userID == "" {
        // In production: redirect to login, then back here with session
        http.Error(w, "login required", http.StatusUnauthorized)
        return
    }
    code := randURLSafe(32)
    codesMu.Lock()
    codes[code] = codeEntry{
        ClientID:    clientID,
        RedirectURI:  redirectURI,
        Challenge:    q.Get("code_challenge"),
        UserID:       userID,
        Scope:        q.Get("scope"),
        Nonce:        q.Get("nonce"),
        ExpiresAt:   time.Now().Add(60 * time.Second),
    }
    codesMu.Unlock()

    // Consent would happen here – omitted for brevity
    loc := fmt.Sprintf("%s?code=%s&state=%s", redirectURI, code,

```

```

q.Get("state"))
    http.Redirect(w, r, loc, http.StatusFound)
}

// POST /token
grant_type=authorization_code&code=...&redirect_uri=...&code_verifier=...
..&client_id=...
func tokenHandler(w http.ResponseWriter, r *http.Request) {
    if err := r.ParseForm(); err != nil {
        http.Error(w, "invalid_request", http.StatusBadRequest)
        return
    }
    if r.Form.Get("grant_type") != "authorization_code" {
        http.Error(w, "unsupported_grant_type", http.StatusBadRequest)
        return
    }
    codeStr := r.Form.Get("code")
    codesMu.Lock()
    entry, ok := codes[codeStr]
    if ok {
        delete(codes, codeStr) // single-use - delete immediately
    }
    codesMu.Unlock()
    if !ok || time.Now().After(entry.ExpiresAt) {
        http.Error(w, `{"error":"invalid_grant"}`, http.StatusBadRequest)
        return
    }
    if r.Form.Get("redirect_uri") != entry.RedirectURI {
        http.Error(w, `{"error":"invalid_grant","error_description":"re
direct_uri mismatch"}`, http.StatusBadRequest)
        return
    }
    cl := clients[entry.ClientID]
    // Confidential client: verify secret (or private_key_jwt -
omitted)
    if !cl.IsPublic {
        _, pass, _ := r.BasicAuth()
        if pass == "" {
            pass = r.Form.Get("client_secret")
        }
        if pass != cl.Secret {
            http.Error(w, `{"error":"invalid_client"}`, http.StatusUnau
thorized)
            return
        }
    }
    // PKCE: S256(verifier) == challenge
    verifier := r.Form.Get("code_verifier")
    if verifier == "" {
        http.Error(w, `{"error":"invalid_grant","error_description":"co

```

```

de_verifier required"}`, http.StatusBadRequest)
    return
}
h := sha256.Sum256([]byte(verifier))
computed := base64.RawURLEncoding.EncodeToString(h[:])
if computed != entry.Challenge {
    http.Error(w, `{"error":"invalid_grant","error_description":"pk
ce verification failed"}`, http.StatusBadRequest)
    return
}

access, _ := mintAccessToken(entry.UserID, entry.ClientID, entry.Sc
ope)
resp := map[string]any{
    "access_token": access,
    "token_type":   "Bearer",
    "expires_in":   600,
    "scope":        entry.Scope,
}
// OIDC: include id_token when openid scope was requested
if containsScope(entry.Scope, "openid") {
    idTok, _ := mintIDToken(entry.UserID, entry.ClientID, entry.Non
ce)
    resp["id_token"] = idTok
}
// Refresh token would be added here (opaque random, stored hash)
w.Header().Set("Content-Type", "application/json")
w.Header().Set("Cache-Control", "no-store")
json.NewEncoder(w).Encode(resp)
}

func containsScope(scope, want string) bool {
    for _, s := range splitScope(scope) {
        if s == want {
            return true
        }
    }
    return false
}

func splitScope(s string) []string {
    return strings.Fields(s)
}

func main() {
    http.HandleFunc("/authorize", authorizeHandler)
    http.HandleFunc("/token", tokenHandler)
    http.HandleFunc("/.well-known/jwks.json", func(w http.ResponseWriter, r *http.Request) {
        // Minimal JWKS – expose only the public part in production
        (ES256 JWk)
        w.Header().Set("Content-Type", "application/json")
    })
}

```



```

w.Header().Set("Cache-Control", "public, max-age=600")
json.NewEncoder(w).Encode(map[string]any{"keys": []map[string]a
ny{
    {"kty": "oct", "k": base64.RawURLEncoding.EncodeToString(hm
acKey), "kid": "k1", "alg": "HS256"},
    {}}
})
log.Println("AS on :8081 - authorize at /authorize, token at /
token")
log.Fatal(http.ListenAndServe(":8081", nil))
}

```

At fleet scale, the `codes` and `refreshStore` maps become Redis hashes with TTLs (code: 60 s, refresh: 7 d), sharded by code/refresh ID so any AS instance can validate. Token minting moves to `ES256` with a KMS-backed private key — the `hmacKey` above is for brevity only — and JWKS serves the public key set with cache headers.

The client — three shapes

**Confidential client (Go, server-side web app,
`golang.org/x/oauth2` v0.20+)**

```

package main

import (
    "context"
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "fmt"
    "log"
    "net/http"

    "golang.org/x/oauth2"
)

var conf = &oauth2.Config{
    ClientID:     "app123",
    ClientSecret: "", // public client with PKCE – no secret (or set
for confidential)
    Scopes:       []string{"openid", "profile", "read:orders"},
    Endpoint:     oauth2.Endpoint{
        AuthURL:  "https://auth.example.com/authorize",
        TokenURL: "https://auth.example.com/token",
    },
    RedirectURL: "https://app.example.com/callback",
}

func handleLogin(w http.ResponseWriter, r *http.Request) {
    verifier := randURLSafe(32)
    h := sha256.Sum256([]byte(verifier))
    challenge := base64.RawURLEncoding.EncodeToString(h[:])
    state := randURLSafe(16)

    // Persist verifier + state in session (Ch 5) keyed by a short-
lived cookie
    http.SetCookie(w, &http.Cookie{Name: "__Host-oauth_state", Value: s
tate, Path: "/", Secure: true, HttpOnly: true, MaxAge: 300, SameSite: h
ttp.SameSiteLaxMode})
    http.SetCookie(w, &http.Cookie{Name: "__Host-pkce_verifier",
Value: verifier, Path: "/", Secure: true, HttpOnly: true, MaxAge: 300,
SameSite: http.SameSiteLaxMode})

    url := conf.AuthCodeURL(state,
        oauth2.SetAuthURLParam("code_challenge", challenge),
        oauth2.SetAuthURLParam("code_challenge_method", "S256"),
        oauth2.SetAuthURLParam("nonce", randURLSafe(16)),
    )
    http.Redirect(w, r, url, http.StatusFound)
}

func handleCallback(w http.ResponseWriter, r *http.Request) {

```

```

    wantState, _ := r.Cookie("__Host-oauth_state")
    if r.URL.Query().Get("state") != wantState.Value {
        http.Error(w, "state mismatch – possible CSRF", http.StatusBadRequest)
        return
    }
    verifier, _ := r.Cookie("__Host-pkce_verifier")
    code := r.URL.Query().Get("code")
    // oauth2 lib handles code verifier injection via WithPKCEVerifier
    tok, err := conf.Exchange(
        r.Context(), code,
        oauth2.VerifierOption(verifier.Value),
    )
    if err != nil {
        http.Error(w, fmt.Sprintf("token exchange: %v", err), http.StatusBadRequest)
        return
    }
    _ = tok // tok.AccessToken, tok.RefreshToken, tok.Extra("id_token")
    // Validate id_token (iss/aud/exp/nonce, kid JWK) before creating
    local session
    http.Redirect(w, r, "/dashboard", http.StatusFound)
}

func randURLSafe(n int) string {
    b := make([]byte, n)
    rand.Read(b)
    return base64.RawURLEncoding.EncodeToString(b)
}

func main() {
    http.HandleFunc("/login", handleLogin)
    http.HandleFunc("/callback", handleCallback)
    log.Fatal(http.ListenAndServe(":8080", nil))
}

```

Public client (SPA / mobile — no secret, PKCE only)

The flow is identical except `ClientSecret` is empty, the `code_verifier` is kept in `sessionStorage` (SPA) or secure storage (mobile), and the token exchange uses `client_id` without `client_secret`. The AS treats the client as public and relies on PKCE + exact `redirect_uri` matching alone to prevent code interception. Native apps must use a claimed `https` redirect (Android App Links / iOS Universal Links) or loopback `http://127.0.0.1:PORT/callback` (RFC 8252 §7.3), not a custom scheme that any app can register.

Service client (client credentials, Go)

```
import (
    "context"
    "golang.org/x/oauth2/clientcredentials"
)

var svcConf = clientcredentials.Config{
    ClientID:     "svc-payments",
    ClientSecret: "from-vault-not-source",
    TokenURL:     "https://auth.example.com/token",
    Scopes:       []string{"payments:write"},
}

func callPaymentsAPI(ctx context.Context) {
    tok, err := svcConf.Token(ctx) // POST /token
    grant_type=client_credentials
    if err != nil {
        log.Fatal(err)
    }
    req, _ := http.NewRequestWithContext(ctx, "GET", "https://
api.example.com/charges", nil)
    req.Header.Set("Authorization", "Bearer "+tok.AccessToken)
    // ...
}
```

Python relying party (Flask + `authlib 1.3+`)

```
# app.py – Flask OIDC relying party (authlib 1.3+, Python 3.12)
from flask import Flask, redirect, session, request, url_for
from authlib.integrations.flask_client import OAuth

app = Flask(__name__)
app.secret_key = "change-me-via-vault" # used to sign session cookies

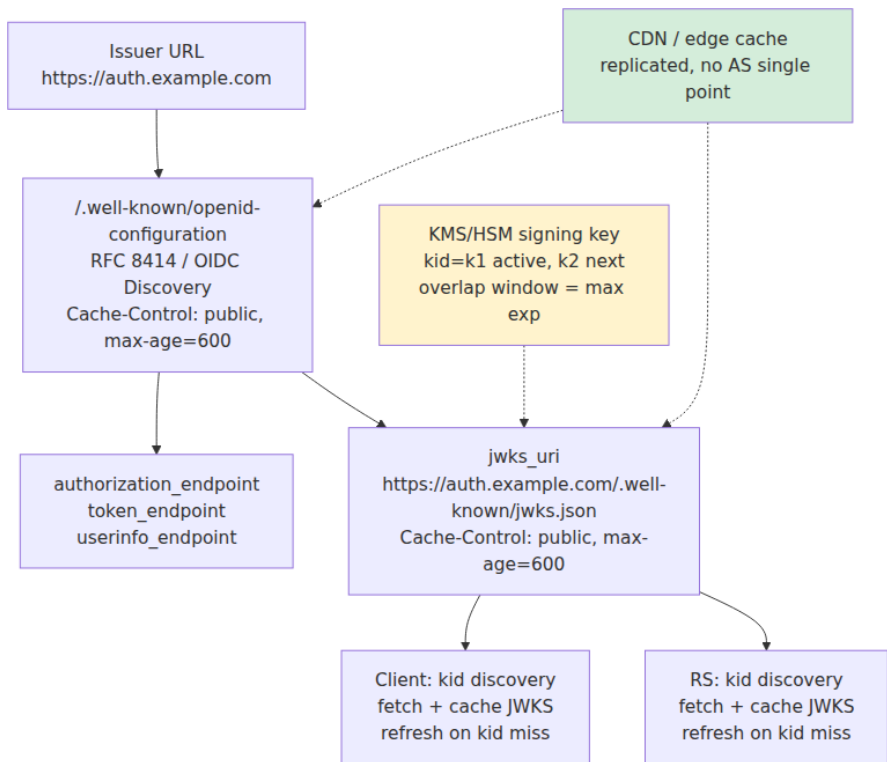
oauth = OAuth(app)
oauth.register(
    name="example",
    server_metadata_url="https://auth.example.com/.well-known/openid-configuration",
    client_id="appl23",
    client_secret=None, # public client with PKCE
    client_kwargs={"scope": "openid profile read:orders", "code_challenge_method": "S256"},
)

@app.route("/login")
def login():
    redirect_uri = url_for("callback", _external=True)
    return oauth.example.authorize_redirect(redirect_uri)

@app.route("/callback")
def callback():
    token = oauth.example.authorize_access_token()
    # validates state + PKCE + id_token
    # token["access_token"], token["id_token"], token["refresh_token"]
    session["user"] = oauth.example.parse_id_token(token) # validates iss/aud/exp/nonce/kid
    return redirect("/dashboard")

@app.route("/orders")
def orders():
    token = oauth.example.fetch_access_token() # uses refresh if needed (if configured)
    resp = oauth.example.get("https://api.example.com/orders", token=token)
    return resp.json()
```

OIDC discovery as a distributed contract



Clients and RSs discover endpoints and keys from the issuer URL alone — no hardcoded key material. The discovery document and JWKS are CDN-cachable static JSON; `kid` rotation is additive and observed as a distribution histogram across issued tokens before the old `kid` is retired (see Ch 5). For private PKI, the same pattern serves `/.well-known/openid-configuration` from the internal AS with a private issuer URL.

Token design for the fleet

JWT access tokens (RFC 9068) vs opaque tokens

Dimension	JWT (RFC 9068 profile)	Opaque (random handle)
Verification	Local — RS checks signature via cached JWKS, no AS call	Remote — RS calls <code>POST /introspect</code> (RFC 7662) on the hot path
Latency / availability	One-time JWKS fetch, then no dependency — RS is autonomous	Every request pays an AS round-trip; AS is a critical dependency
Size	500-1500 bytes — fine in <code>Authorization</code> header, too large for cookies/URLs	~43 chars — storable anywhere
Revocation	Bounded by <code>exp</code> (minutes) or <code>auth_ver</code> / JTI blocklist (Ch 5)	Immediate — AS flips a bit, introspection returns <code>active: false</code>
Confidentiality	Claims visible to anyone with the token (Base64url) — do not put secrets in JWT claims; encrypt (JWE) if you must	Claims stay on the AS — token is meaningless without introspection

The modern default for high-throughput APIs is **JWT access tokens with short `exp` (5-15 min) + opaque refresh tokens**. This gives local verification at the edge (no AS call per request) with bounded revocation via short expiry and refresh revocation. If your threat model requires instant per-token revocation at high rate, either shorten `exp` further or use opaque tokens with introspection caching (short TTL, e.g., 30 s — reintroduces a bounded staleness window on revocation).

Audience and scope enforcement

Every RS **must** validate `aud` and `scope` — an access token for `aud: analytics` must not be accepted by `payments`, and a token with `scope: read:orders` must not allow `write:orders`. The checks are inexpensive and the most commonly skipped.


```
// RS middleware – aud + scope (Go)
func requireScope(aud, scope string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            claims := r.Context().Value("claims").(jwt.MapClaims)
            if claims["aud"] != aud {
                http.Error(w, "invalid audience", http.StatusForbidden)
                return
            }
            if !hasScope(claims["scope"].(string), scope) {
                http.Error(w, "insufficient_scope", http.StatusForbidden)
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}
```

Introspection (RFC 7662) and revocation (RFC 7009)

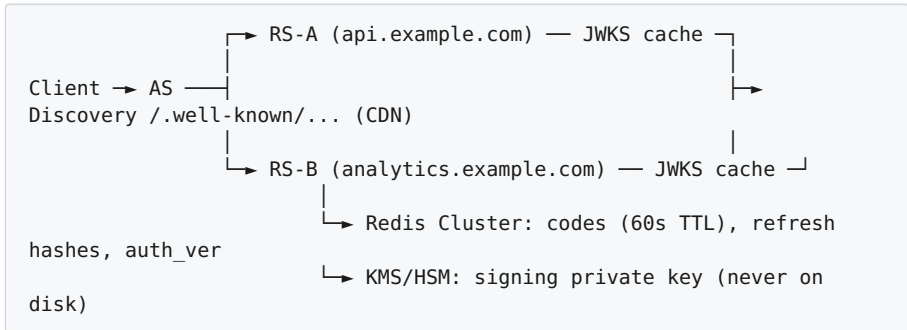
When the RS cannot (or does not want to) verify JWTs locally, it calls the AS:

```
# Introspection – RS asks AS: is this token active and what are its claims?
$ curl -u svc-payments:secret -X POST https://auth.example.com/
introspect \
  -d token=Sp1xl0BeZQ... \
  -d token_type_hint=access_token
{
  "active": true,
  "sub": "4821",
  "client_id": "app123",
  "scope": "read:orders",
  "aud": "api",
  "exp": 1724197200,
  "iat": 1724193600
}

# Revocation – client or AS revokes a refresh or access token
$ curl -u app123:secret -X POST https://auth.example.com/revoke \
  -d token=refresh_abc... -d token_type_hint=refresh_token
# 200 – token is now inactive; introspection returns active:false
```

Cache introspection responses for a short TTL (e.g., 30 s) at the RS to avoid per-request AS load, but understand the revocation delay that caching introduces.

Distributed-systems lens — the authorization server at scale



- **Horizontal scale of the AS.** Authorization codes, refresh hashes, and `auth_ver` counters live in Redis Cluster with short TTLs; any AS instance can validate any code/refresh. Token *minting* is stateless given the signing key (held in KMS/HSM), so AS instances are interchangeable — put them behind a load balancer and scale horizontally. The bottleneck is the refresh/code store, not minting.
- **JWKS as a CDN asset.** The `/.well-known/jwks.json` and `/.well-known/openid-configuration` responses are static JSON with `Cache-Control: public, max-age=600`. Serve them from the edge (CloudFront/Cloud CDN) or a replicated cache, not a single AS pod — a JWKS outage blinds every RS that needs to verify a new `kid`. Version `kid`s additively and monitor `kid` distribution to know when an old key is safe to retire (same overlap pattern as Ch 3/Ch 4/Ch 5).
- **Refresh token rotation and replay detection.** Issue a new refresh token on every `grant_type=refresh_token` exchange, keep the previous one valid for a short grace window (10–30 s) to handle concurrent refresh races (two replicas refreshing simultaneously), and detect replay outside that window as token theft — revoke the token family and alert. This is the same fencing used for distributed idempotency (Vol 6, Ch 9).

- **Multi-region AS.** Run the AS in ≥ 2 regions with the same issuer URL (anycast or latency-routed DNS), replicate the signing key via KMS multi-region keys, and replicate the discovery/JWKS documents. Codes and refresh state can be regional if clients are sticky to a region, or globally replicated if clients roam — choose based on latency vs consistency requirements. The discovery `issuer` must be the same string everywhere — `iss` validation fails if regions disagree.
- **Token binding at scale.** DPoP (RFC 9449) and mTLS binding (RFC 8705) add per-request proof that the presenter holds the key bound to the token (`cnf.jkt` / `cnf.x5t#S256`). At fleet scale, the RS validates the binding proof on every request — no AS call — so binding adds CPU but not a dependency. Roll it out per-sensitive scope (e.g., require DPoP only for `payments:write`) rather than globally to control cost.

Failure modes and incident patterns

Open redirect via `redirect_uri` mismatch. The AS validates `redirect_uri` against a prefix or regex instead of exact equality, or preserves query parameters, letting an attacker craft `redirect_uri=https://app.example.com/callback@evil.com` or `redirect_uri=https://app.example.com/callback?next=//evil.com`. The authorization code is delivered to the attacker, who exchanges it (with or without PKCE, depending on client type). Defense: exact string equality against a pre-registered set, no wildcards, no prefix matching — one line in the AS, enforced in code review and tested with a redirect-URI fuzzer.

Code interception on mobile / loopback. A malicious app registers the same custom scheme (`myapp://callback`) and captures the code. Without PKCE, it exchanges the code directly. With PKCE, it fails because it does not have the `verifier`. Defense: PKCE `S256` is mandatory for every client — confidential or public — and custom schemes must be replaced with claimed `https` redirects (App Links / Universal Links) where the OS verifies domain ownership.

Token leakage via referrer / logs / history. The code or access token appears in `Referer` headers, browser history, or server access logs because the callback URL or an API URL included it in a query parameter. Defense: tokens travel in `POST` bodies and `Authorization`

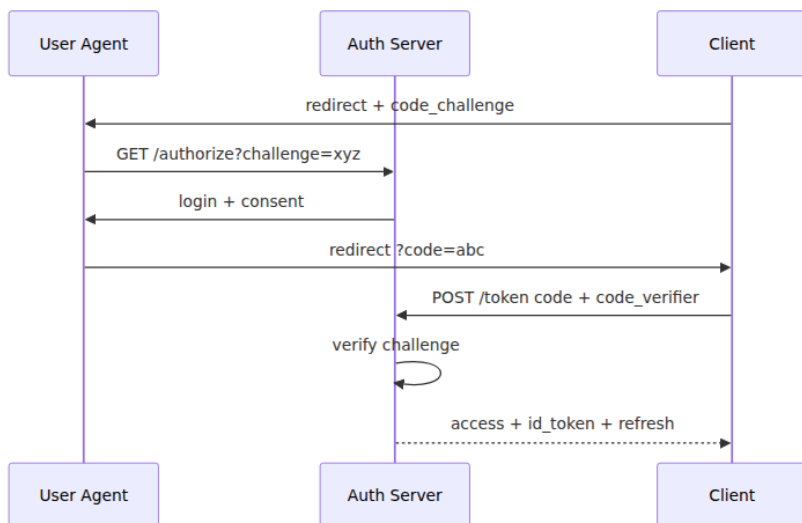
headers only, never query params; set `Referrer-Policy: no-referrer` on the callback; scrub `Authorization` / `code` from logs; return `Cache-Control: no-store` on token responses (RFC 6749 §5.1).

Refresh token replay / theft. A stolen refresh token is replayed to mint new access tokens indefinitely. Defenses layered: bind refresh tokens to a client fingerprint and `aud`, rotate on each use with a short grace window, detect replay outside the window and revoke the token family, keep refresh lifetime bounded (7-30 d), and require re-authentication for sensitive scopes. Monitor `token_refresh_rate` per client/user — a spike is the signal.

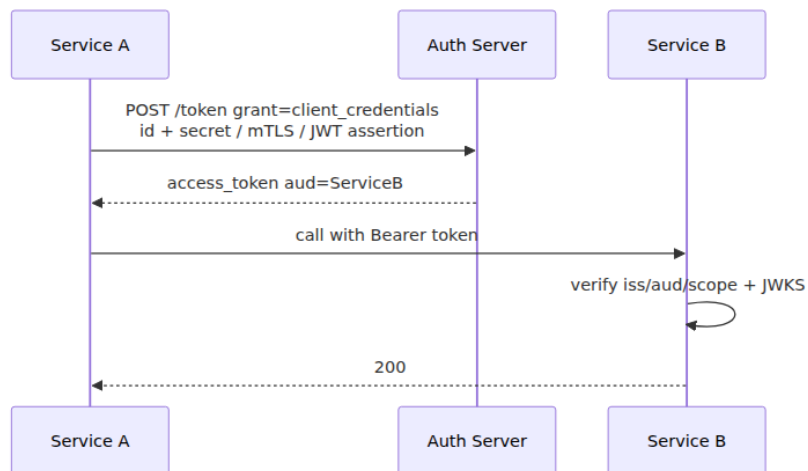
Confused deputy / audience confusion. A token issued for `aud: analytics` is accepted by `aud: payments` because the RS skips `aud` validation. Defense: every RS validates `aud == itself` and `scope` contains the required scope for the endpoint — fail closed on missing or mismatched `aud / scope`. Test with a token for one RS against another in CI.

Authorization code replay. The code is used twice — the legitimate client and an attacker both exchange it. Defense: single-use codes deleted on first read (the `delete(codes, codeStr)` in the AS above), with the first exchange winning; the second gets `invalid_grant`. Log and alert on replay attempts — they are near-certain attacks or bugs.

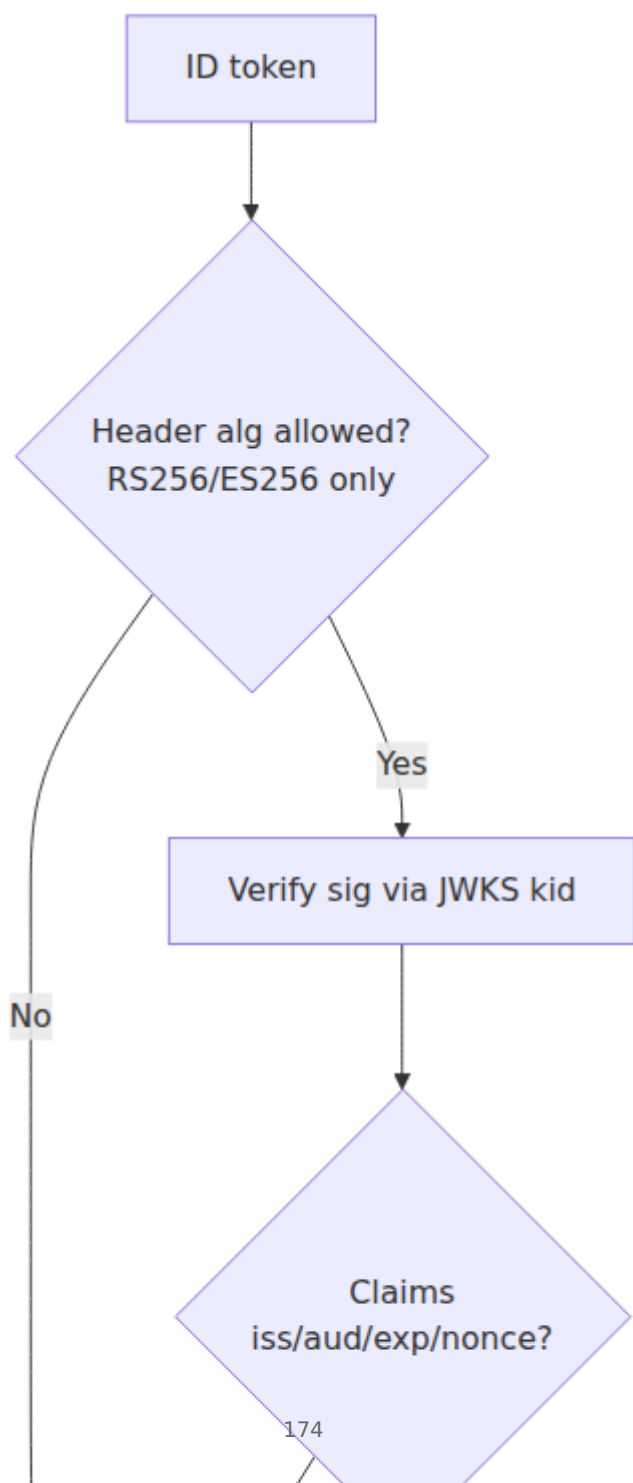
Authorization Code with PKCE



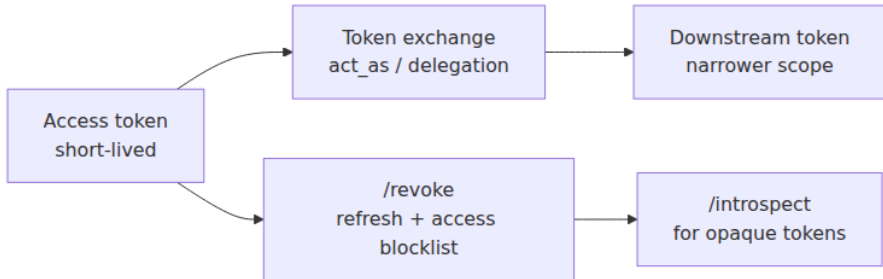
Client Credentials Flow



OIDC ID Token Validation



Token Exchange and Revocation



Key takeaways

- OAuth 2.0 (RFC 6749/6750) delegates authority via scoped, revocable access tokens; OIDC (1.0) adds identity via ID tokens and Discovery. The access token is for the RS (`aud: api`, `scope-checked`), the ID token is for the client (`aud: client_id`, `nonce-checked`) — they are not interchangeable.
- Only authorization code + PKCE (RFC 7636 S256) is recommended for new user-facing clients — public or confidential — with `state` (CSRF), `nonce` (OIDC replay), exact `redirect_uri` matching, single-use codes (60 s TTL), and `code_verifier` that never leaves the client. `implicit` and `password` grants must not be used (Security BCP).
- OIDC Discovery (`/.well-known/openid-configuration` per RFC 8414 / OIDC Discovery 1.0) and JWKS (RFC 7517) let clients and RSs discover endpoints and keys without redeploy — cache them, refresh on `kid` miss, and serve them from the edge.
- Prefer short-lived JWT access tokens (5–15 min, RFC 9068) verified locally via cached JWKS for high-throughput RSs, and opaque refresh tokens (256 bits, stored as hashes) rotated on each use. Use introspection (RFC 7662) or revocation (RFC 7009) only where instant per-token revocation is required, and cache with a bounded staleness window.
- Every RS must validate `aud`, `scope`, and `exp` on every request — skipping any of them is a confused-deputy or privilege-escalation bug. Token binding (DPoP RFC 9449 or mTLS RFC 8705) ties the token to the holder's key and prevents replay of stolen tokens.

- The AS at scale is a stateless minter (KMS/HSM signing key, horizontally scalable) backed by a shared store for codes/refresh and a CDN-cached JWKS. `kid`-versioned keys with an overlap window, refresh rotation with a grace period, and regional replication of the `issuer` string are the fleet-scale invariants.
- Test redirects with a fuzzer, keep every token out of URLs/logs/referrers (`Cache-Control: no-store` , `Referrer-Policy`), and alert on code replay, refresh replay, and `aud` mismatches — they are the near-certain signals of an attack.

Further reading

- **Standards:** RFC 6749 (OAuth 2.0), RFC 6750 (Bearer Token Usage), RFC 7636 (PKCE), RFC 7662 (Token Introspection), RFC 7009 (Token Revocation), RFC 8414 (AS Metadata), RFC 9068 (JWT Access Tokens), RFC 8693 (Token Exchange), RFC 8705 (mTLS Binding), RFC 9449 (DPoP), RFC 8252 (OAuth for Native Apps), RFC 9126 (Pushed Authorization Requests, PAR), RFC 9396 (Rich Authorization Requests, RAR); OIDC Discovery 1.0, OIDC Core 1.0, OIDC JWT Secured Authorization Response Mode (JARM).
- Lodderstedt, Bradley, Labunets, Fett — *OAuth 2.0 Security Best Current Practice* (draft-ietf-oauth-security-topics, replaces RFC 6819) — the single most important companion to RFC 6749; read §2–§4 before shipping any flow.
- Parecki — *OAuth 2.0 Simplified* (<https://aaronparecki.com/oauth-v2/>) — concise flow diagrams aligned with the Security BCP.
- [golang.org/x/oauth2 docs](https://golang.org/x/oauth2/docs) — <https://pkg.go.dev/golang.org/x/oauth2> — `AuthCodeURL` / `Exchange` with PKCE verifier support.
- `authlib` (Python) — <https://docs.authlib.org/> — OIDC discovery, JWKS, and DPoP in Python.
- ORY Hydra / `ory/fosite` — <https://www.ory.sh/hydra> + <https://github.com/ory/fosite> — production AS and Go OAuth2 framework; the `fosite` examples are the reference for handler patterns.

- Fett, Küsters, Schmitz — *A Comprehensive Formal Security Analysis of OAuth 2.0* (CCS 2016) — the formal underpinning of the Security BCP's recommendations, useful for protocol reasoning.

Chapter 7 — Authorization: RBAC, ABAC, and ReBAC (Zanzibar)

What this chapter covers. Authentication (Ch 5) proves *who* you are; OAuth/OIDC (Ch 6) lets you delegate that proof. Authorization decides *what* you may do once your identity is established — the `allow` or `deny` at every API endpoint, every row, every field. At single-service scale this is an `if user.isAdmin` check. At fleet scale — thousands of services, millions of resources, hierarchical ownership, sharing, and revocation with bounded staleness — it is a distributed data and consistency problem whose wrong answer is a privilege escalation or a data leak. This chapter builds the three models every backend engineer must be able to design and operate: Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), and Relationship-Based Access Control (ReBAC) as realized by Google Zanzibar (2019 paper) and its open-source incarnation AuthZed SpiceDB (v1.35+, CNCF). We implement all three, compare their expressiveness and operational cost, and show how centralized policy decision points scale without becoming a single point of failure or a p99 latency cliff.

Learning goals — after this chapter you should be able to:

- Distinguish authentication, authorization, and audit — and explain why `scope / claims` in a token are not an authorization decision, only an input to one.
- Design and implement RBAC (core + hierarchical) with role explosion mitigations, and explain where RBAC breaks (resource-instance granularity, contextual conditions).
- Design and implement ABAC with attribute-based policies (OPA/Rego v0.70+, Cedar 4.x) — policy-as-code, decision inputs, and bundle distribution.
- Model and operate ReBAC with Zanzibar/SpiceDB: schema, relation tuples, `Check / Expand / LookupResources`, Zookeeper-based consistency (`at_least_as_fresh`), and caveated relationships.

- Choose among RBAC, ABAC, and ReBAC (and their hybrids) by comparing expressiveness, latency, consistency, and operability at fleet scale.
- Architect a distributed authorization plane — PDP/PEP separation, decision caching, bulk checks, and revocation propagation — and test it for confused-deputy, TOCTOU, and inheritance-escalation bugs.

Boundary notes. *Authentication* — proving identity via sessions, JWTs, and PKI — is **Ch 4-6** (PKI/TLS ops, sessions/JWTs, OAuth 2.0/OIDC). This chapter assumes the caller is already authenticated and the `sub / iss / aud / exp` on the presented credential have been validated; it decides what that authenticated principal may do. *Service-to-service* workload identity that feeds authorization (SPIFFE/SPIRE, mTLS) is **Ch 10**. *Policy-as-code for admission and compliance* (OPA/Kyverno for Kubernetes) shares tooling with ABAC but targets platform policy — covered here as policy engines, in the Companion Book 6 for cloud-native admission. *Threat modeling* the authorization boundary as an attack surface is **Ch 11**; data-layer row/column enforcement (RLS, views) is **Vol 5, Ch 5-6** (transactions/isolation is not access control). Secrets that gate access (Ch 8) are inputs to authorization, not authorization itself.

Authorization at the request boundary

Every authenticated request converges on the same question inside the handler:

```
principal (who) + action (what) + resource (which) + context (when/
where/how) → allow | deny
```

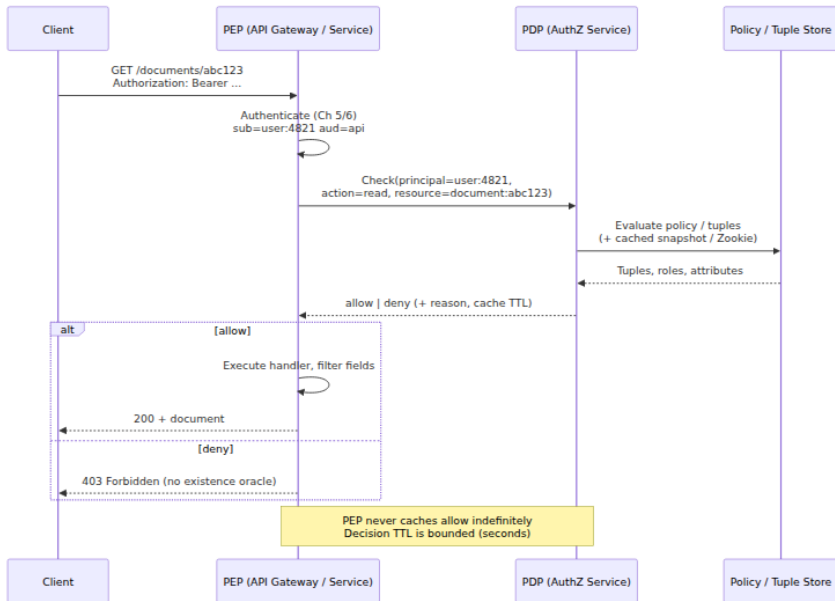
The principal is a `sub` (user `4821`), a service identity (`spiffe://prod/payments`), or a group membership expanded from the token. The action is `read`, `write`, `delete`, `publish`, `admin`. The resource is `document:abc123`, `org:acme/billing`, or `GET /v1/documents/{id}`. Context is time, IP, device posture, or a caveat (`only during business hours`).

Getting this check wrong is silent: the service returns `200` with data the caller should not have seen. There is no crash signal — only an access log that looks normal until an audit or a breach report reveals the leak. This is why authorization is tested as a *negative* property (enumerate what must be denied), not just a positive one.

In a monolith, authorization lives next to the data:

```
func GetDocument(w http.ResponseWriter, r *http.Request) {
    user := auth.FromContext(r.Context()) // validated in Ch 5/6
    doc := store.Get(r.PathValue("id"))
    if !canRead(user, doc) { // ← authorization
        http.Error(w, "forbidden", http.StatusForbidden)
        return
    }
    // ...
}
```

In a fleet, `canRead` cannot be a local function. The data it needs — roles, attributes, relationships — lives in other services, changes concurrently, and must be evaluated with a bounded staleness guarantee. The enforcement point (PEP) and the decision point (PDP) separate.



Two security properties govern the PEP/PDP contract:

1. **Fail-closed.** Any error in the PDP (timeout, store unavailable) denies by default. Fail-open authorization is a bypass.
2. **No oracle.** A `403` must not reveal whether the resource exists. `404` vs `403` distinguishes existence — return `404` for both "not found" and "not visible" when enumeration matters, or `403` uniformly when

you do not need to hide existence. Choose per resource class and document it.

RBAC: roles as the indirection layer

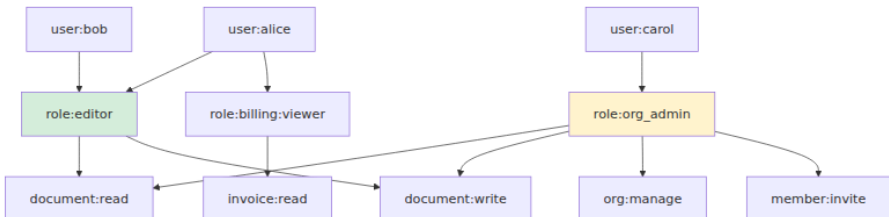
Core model

RBAC (ANSI INCITS 359-2004, NIST RBAC) replaces per-principal, per-resource ACLs with two indirection tables:

Users $\xrightarrow{\text{(UA)}}$ Roles $\xrightarrow{\text{(PA)}}$ Permissions (action $\times$ resource class)

- **UA** (user assignment): `alice  $\rightarrow$  {editor, billing:viewer}`.
- **PA** (permission assignment): `editor  $\rightarrow$  {document:read, document:write}`.
- A **session** activates a subset of a user's roles (relevant when role sets are large or mutually exclusive constraints exist).

The payoff is administrative: when a permission changes, you update the role, not every user.



Implementing RBAC in a service

The simplest correct implementation is a table plus middleware — no external PDP required — and is the right starting point for most teams.

```

-- Postgres 16 – RBAC core tables
CREATE TABLE roles (
    id text PRIMARY KEY,          -- 'editor', 'billing:viewer'
    description text NOT NULL
);
CREATE TABLE permissions (
    id text PRIMARY KEY,          -- 'document:read'
    action text NOT NULL,
    resource_class text NOT NULL
);
CREATE TABLE role_permissions (
    role_id text REFERENCES roles(id) ON DELETE CASCADE,
    permission_id text REFERENCES permissions(id) ON DELETE CASCADE,
    PRIMARY KEY (role_id, permission_id)
);
CREATE TABLE user_roles (
    user_id text NOT NULL,
    role_id text REFERENCES roles(id) ON DELETE CASCADE,
    scope_id text,                -- NULL = global; 'org:acme' =
    scoped role
    granted_by text NOT NULL,
    granted_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, role_id, coalesce(scope_id, ''))
);
CREATE INDEX ON user_roles(user_id);

```

```

// Go 1.22 – RBAC middleware (net/http). Validates aud/scope are not
// confused with authorization – Ch 6's access token claims are
// *inputs*,
// not the decision.

type Authorizer struct {
    db *sql.DB
    // Decision cache: userID+permission → bool, TTL 30s.
    // Revocation reduces TTL via pub/sub invalidation (see below).
    cache *lru.Cache
}

func (a *Authorizer) Require(permission string) func(http.Handler)
http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Req
uest) {
            p := auth.FromContext(r.Context())
            if p == nil {
                http.Error(w, "unauthenticated", http.StatusUnauthoriz
ed)
                return
            }
            ok, err := a.hasPermission(r.Context(), p.Subject, permissi
on, p.OrgID)
            if err != nil {
                // Fail-closed: PDP error → deny, not 500 with data.
                slog.Error("authz check failed", "err", err, "sub", p.S
ubject)
                http.Error(w, "forbidden", http.StatusForbidden)
                return
            }
            if !ok {
                // No oracle: 403 without confirming resource
                existence.
                http.Error(w, "forbidden", http.StatusForbidden)
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

func (a *Authorizer) hasPermission(ctx context.Context, subject, perm,
scope string) (bool, error) {
    key := subject + ":" + perm + ":" + scope
    if v, ok := a.cache.Get(key); ok {
        return v.(bool), nil
    }
    var exists bool

```

```

err := a.db.QueryRowContext(ctx, `
    SELECT EXISTS(
        SELECT 1 FROM user_roles ur
        JOIN role_permissions rp ON rp.role_id = ur.role_id
        WHERE ur.user_id = $1
              AND rp.permission_id = $2
              AND (ur.scope_id IS NULL OR ur.scope_id = $3)
    )`, subject, perm, scope).Scan(&exists)
if err != nil {
    return false, err
}
a.cache.Add(key, exists)
time.AfterFunc(30*time.Second, func() { a.cache.Remove(key) })
return exists, nil
}

// Usage:
// mux.Handle("GET /v1/documents/{id}",
//     authz.Require("document:read")(handleGetDocument))

```

Performance note: cache TTL is a *security* parameter, not just a latency one. A 30 s TTL means a revocation (removing alice from editor) is visible fleet-wide within 30 s plus replication lag. For sensitive permissions (org:manage, billing:write), use 5 s or bypass the cache and serve from a replicated read model. Expose `authz_decision_cache_hit` and `authz_decision_latency_ms` as metrics — a sudden hit-rate drop is a revocation-burst or cache-poisoning signal.

Hierarchical RBAC and its limits

Hierarchical RBAC (ANSI RBAC-2) adds role inheritance:

```

org_admin ⊃ editor ⊃ viewer           viewer ⊂ editor ⊂ org_admin

```

Inheritance is convenient (grant `org_admin` once, get all child permissions) but introduces the first scaling trap — **role explosion**. When teams need editor of folder X but viewer of folder Y, the instinct is to mint new roles per resource: `editor:folder:abc`, `viewer:folder:xyz`. With R resource instances and P permission classes, the role space grows as $O(R \times P)$. At thousands of folders and dozens of teams, the role catalog becomes ungovernable, and every new product feature ("share a document with one user for 24 hours") requires a schema migration.

Two mitigations before abandoning RBAC:

- **Scoped roles.** Store `scope_id` alongside the role assignment (`alice` is `editor` scoped to `folder:abc`). The middleware checks (`user`, `role`, `scope`) rather than minting a new role per scope. This keeps the role catalog small (P roles) and pushes cardinality into the assignment table (which is indexed).
- **Team/Group indirection.** Assign roles to groups (`team:backend` → `editor`), and users to groups. A join replaces a flood of per-user rows, and team changes propagate without per-user writes.

When scoped roles still force per-instance rows and business logic needs contextual conditions ("*allow if owner or if shared and not expired*"), move to ABAC or ReBAC rather than minting more roles.

ABAC: policy as code

Attribute-Based Access Control (NIST SP 800-162) replaces the role indirection with a **policy evaluated over attributes** of the principal, the resource, the action, and the environment:

```
policy: allow if
  principal.department == resource.department
  && action == "read"
  && environment.time ∈ business_hours
  && resource.classification ≤ principal.clearance
```

The PDP evaluates these policies per request. Two dominant policy engines in 2024–2026:

Engine	Language	Distribution	Strength
OPA / Styra (OPA v0.70+, Rego)	Rego — Datalog-inspired, JSON over HTTP	Bundle (tar.gz over HTTPS) or sidecar (<code>opa</code> container)	Mature, large ecosystem, Envoy/Istio integration
Cedar (AWS, v4.x, Apache-2.0)	Cedar — typed, formally verified authorization	Embedded library (Rust/Go/Java)	Strong typing, formal analysis, analyzer tool

Both are version-pinned here because policy semantics change across releases — pin the engine exactly as you pin compilers.

OPA/Rego example: document access

Policies live in versioned bundles, not in application code. The application sends a JSON `input` and reads `result.allow`.

```
# bundle/policy/document.rego – OPA v0.70+, Rego
package document.authz

import future.keywords.in

default allow := false

# Rule 1: org_admin on the same org can do anything
allow if {
    input.principal.roles[_] == "org_admin"
    input.principal.org_id == input.resource.org_id
}

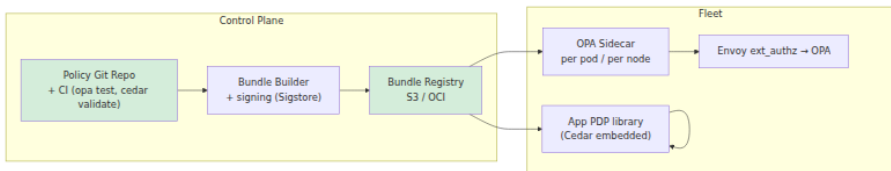
# Rule 2: owner can read/write their document
allow if {
    input.action in {"read", "write"}
    input.principal.sub == input.resource.owner_id
}

# Rule 3: shared-with user can read if share is active and not expired
allow if {
    input.action == "read"
    some share in input.resource.shares
    share.principal == input.principal.sub
    share.revoked == false
    time.now_ns() < time.parse_rfc3339_ns(share.expires_at)
}

# Rule 4: deny if resource classification exceeds clearance
deny_reason["insufficient clearance"] if {
    input.resource.classification == "confidential"
    input.principal.clearance != "confidential"
}
allow if {
    not deny_reason[_]
    # ... positive rules above
}
```

```
// Go - OPA sidecar check (OPA bundle served at localhost:8181)
func opaAllow(ctx context.Context, input any) (bool, error) {
    body, _ := json.Marshal(map[string]any{"input": input})
    req, _ := http.NewRequestWithContext(ctx, "POST",
        "http://localhost:8181/v1/data/document/authz/allow", bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return false, err // fail-closed: treat PDP error as deny
    }
    defer resp.Body.Close()
    var out struct{ Result bool `json:"result"` }
    if err := json.NewDecoder(resp.Body).Decode(&out); err != nil {
        return false, err
    }
    return out.Result, nil
}
```

Deployment for ABAC at scale:



- **Bundle distribution** is the consistency problem. OPA polls bundles over HTTPS with ETag; Cedar libraries reload from S3. Use bundle signing and `bundle.activate` only after `opa test` passes in CI. Canary bundle rollout per region before fleet-wide.
- **Latency.** OPA sidecar adds ~1-3 ms p50 per check on the same host (Unix socket preferred). For p99-sensitive paths, evaluate inline (Cedar embedded library, zero hop) or batch checks.
- **Debuggability.** Every deny must log a *reason* (`deny_reason` above) at `debug` level with a request correlation ID — otherwise ABAC denials become unactionable and operators widen policies to "make it work."

ABAC's weakness is operational: when every permission is a policy expression over arbitrary attributes, the policy set grows without a data model. Review becomes hard ("does any policy grant `alice` access to

`doc:xyz ?`"), and performance depends on attribute fetch latency. ReBAC gives that policy set a graph structure.

ReBAC: relationships as the data model

ReBAC treats authorization as a **graph question**: is there a path from the principal to the resource through typed relationships? Google introduced the model in the *Zanzibar: Google's Consistent, Global Authorization System* paper (USENIX ATC 2019), the system behind Drive, Calendar, YouTube, and ~500 other Google services. AuthZed SpiceDB (CNCF, v1.35+ in 2025–2026) is the widely adopted open-source Zanzibar implementation; OpenFGA (CNCF) and Keto offer compatible models.

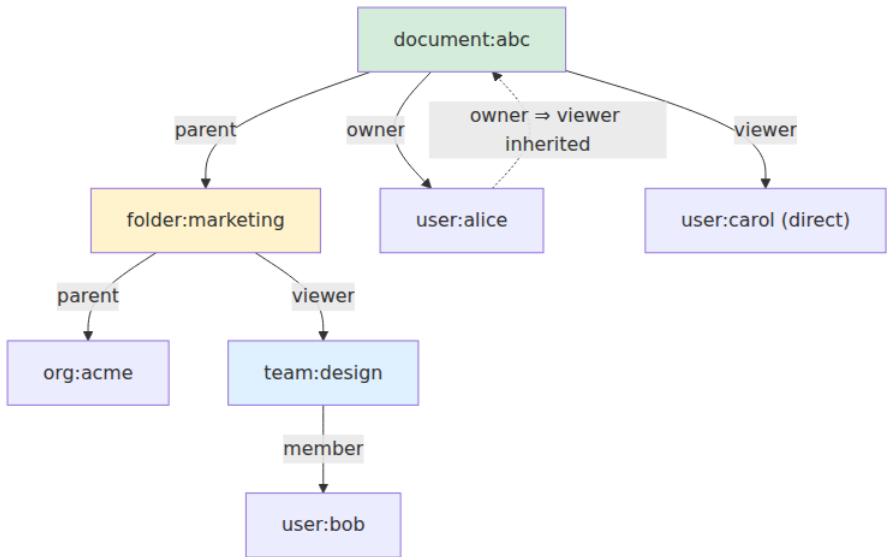
The Zanzibar mental model

Every check is `Check(subject, relation, object)` — "does `user:alice` have `viewer` on `document:abc`?"

Relationships are **tuples** of the form `object#relation@subject`:

```
document:abc#owner@user:alice
document:abc#viewer@user:bob
folder:marketing#viewer@team:design
team:design#member@user:bob           // bob is a member of team:design
n
document:abc#parent@folder:marketing // abc lives in folder marketing
g
```

Schema defines how relations compose — union, intersection, exclusion, and indirection through other relations:



Reading the graph: `user:bob` has `viewer` on `document:abc` not because of a direct tuple, but because `document:abc#parent@folder:marketing`, `folder:marketing#viewer@team:design`, and `team:design#member@user:bob` chain transitively. This transitivity is declared in schema, evaluated by the server.

SpiceDB schema

SpiceDB schemas (v1.35+ DSL, validated with `zed validate`) declare object types, relations, and permission composition:

```

// schema.zed - SpiceDB v1.35+ (AuthZed)
// Validate: zed schema validate schema.zed
// Write:    zed schema write schema.zed

definition user {}

definition team {
    relation member: user
}

definition org {
    relation admin: user
    relation member: user | team#member
    // org member includes direct users and members of teams assigned
    to the org
    permission view = member + admin
    permission manage = admin
}

definition folder {
    relation parent: org
    relation viewer: user | team#member | org#member
    relation editor: user | team#member | org#admin
    relation owner: user

    // Permissions compose relations + parent indirection:
    permission view = viewer + editor + owner + parent->view
    permission edit = editor + owner + parent->manage
}

definition document {
    relation parent: folder | org
    relation owner: user
    relation viewer: user | team#member
    relation editor: user | team#member
    relation viewer_via_share: user // populated by share service,
    with caveat

    // viewer via share is caveated (expiry check) - see below
    permission view = viewer + viewer_via_share + editor + owner + parent->view
    permission edit = editor + owner + parent->edit
    permission delete = owner + parent->manage
}

```

Key composition operators:

- **+** — union (any path grants).
- **&** — intersection (all paths must grant, e.g., `member & not_blocked`).

- `-` — exclusion (e.g., `viewer - banned`).
- `->` — indirection through a relation on another object (`parent->view` means "if the parent grants `view`, so do we").

Tuples, checks, and caveats

Tuples are the data plane. The control plane is tiny; the data plane holds billions of tuples at Google scale and millions in a typical SpiceDB deployment.

```
# AuthZed SpiceDB 1.35+ — zed CLI (github.com/authzed/zed v0.20+)
# Assumes spicedb running locally (docker run quay.io/authzed/spicedb
serve ...)

# Write tuples
zed relationship create document:abc#owner@user:alice
zed relationship create document:abc#parent@folder:marketing
zed relationship create folder:marketing#parent@org:acme
zed relationship create folder:marketing#viewer@team:design
zed relationship create team:design#member@user:bob
zed relationship create document:abc#viewer@user:carol

# Check: does bob have viewer on document:abc? (via folder->team
indirection)
zed permission check document:abc#view@user:bob
# → true (via folder:marketing#viewer@team:design#member)

# Expand: show *why* bob has viewer (for debuggability / audit)
zed permission expand document:abc#view

# Bulk check: which documents can bob view? (fan-out for list
endpoints)
zed permission lookup-resources document#view --subject user:bob
```

Caveated relationships (SpiceDB v1.30+; Zanzibar "contextual tuples") attach a condition evaluated at check time — the bridge from pure ReBAC to ABAC-style context:

```
// Caveat definition in schema
caveat expiry_caveat(expires_at timestamp) {
    expires_at > now()
}

definition document {
    // ...
    relation viewer_via_share: user with expiry_caveat
    permission view = viewer + viewer_via_share + editor + owner + parent->view
}
```

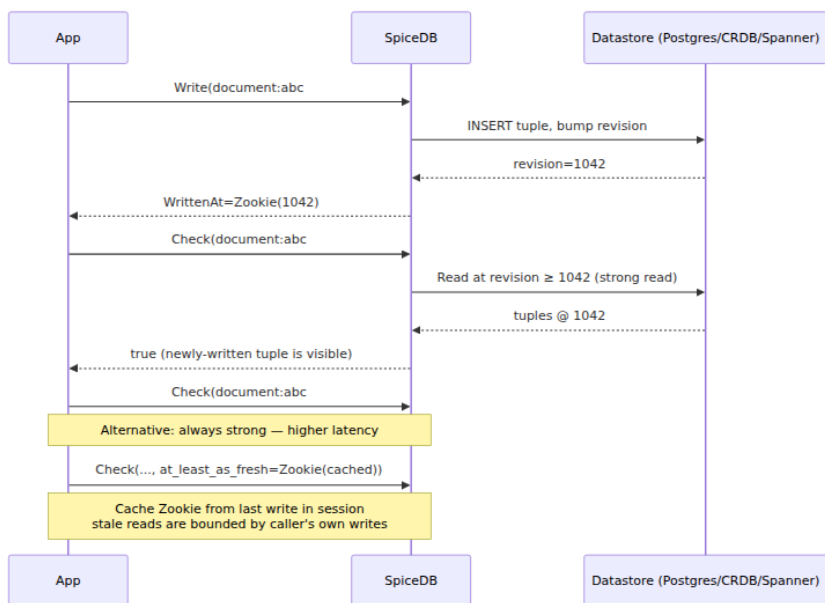
```
# Tuple with caveat context supplied at check time
zed relationship create document:abc#viewer_via_share@user:dave \
    --caveat expiry_caveat --context '{"expires_at":
    "2026-09-01T00:00:00Z"}'

# Check supplies `now()` from server time; expired shares deny
automatically.
zed permission check document:abc#view@user:dave
```

This keeps the tuple store from needing background reapers for expiry — the condition is evaluated live, and revocation is a tuple delete.

Consistency: Zookies and `at_least_as_fresh`

Zanzibar's central contribution is a consistency token — the **Zookie** (SpiceDB's `ZedToken`, an opaque string encoding a datastore revision). Every write returns a Zookie; every read accepts one as a freshness bound:



In practice:

- **After a write you control** (share a document, revoke access), pass the returned Zookie as `at_least_as_fresh` on the next check. The user who just shared a doc sees it shared immediately — causal consistency without paying strong-read latency on every check.
- **On the hot path**, cache a Zookie per request or per session and reuse it — SpiceDB can serve from a read replica at that revision, preserving horizontal read scaling.
- **fully_consistent=true** forces a strong read from the primary — use for sensitive checks (delete, billing) where staleness is unacceptable, and accept the latency cost.

SpiceDB exposes this as:


```
// Go - authzed-go v1.0+ (github.com/authzed/authzed-go v1.2+)
import (
    pb "github.com/authzed/authzed-go/proto/authzed/api/v1"
    "github.com/authzed/authzed-go/v1"
)

client, _ := authzed.NewClient("localhost:50051",
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    grpcutil.WithInsecureBearerToken("s3cr3t"), // preshared key; mTLS
    in prod (Ch 10)
)

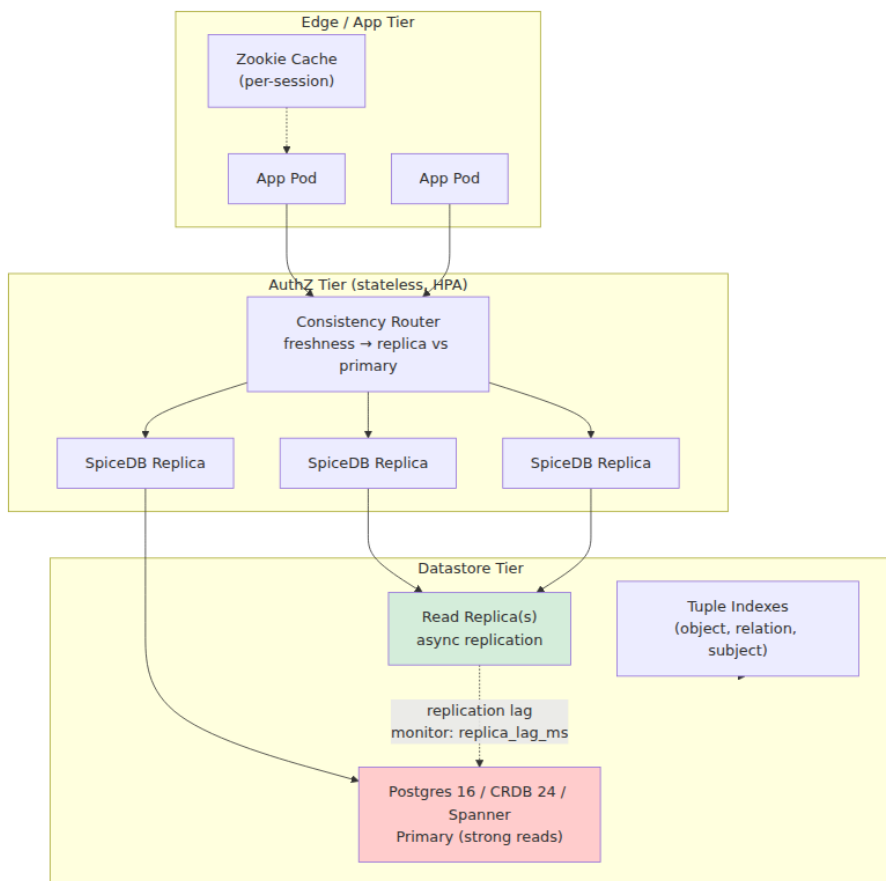
// Check with Zookie from a prior write
resp, err := client.CheckPermission(ctx, &pb.CheckPermissionRequest{
    Resource: &pb.ObjectReference{ObjectType: "document", ObjectId: "
abc"},
    Permission: "view",
    Subject: &pb.SubjectReference{Object: &pb.ObjectReference{Object
Type: "user", ObjectId: "bob"}},
    Consistency: &pb.Consistency{
        Requirement: &pb.Consistency_AtLeastAsFresh{
            AtLeastAsFresh: zookieFromPriorWrite, // *pb.ZedToken or
nil
        },
    },
})
// resp.Permissionship == PERMISSIONSHIP_HAS_PERMISSION

// Bulk check for list endpoints - which of 100 docs can bob view?
bulk, _ := client.BulkCheckPermission(ctx, &pb.BulkCheckPermissionReque
st{
    Consistency: &pb.Consistency{Requirement: &pb.Consistency_MinimizeL
atency{MinimizeLatency: true}},
    Items: items, // []*BulkCheckPermissionRequestItem, up to ~100 per
call
})
```

For HTTP list filtering without N+1 checks, prefer `LookupResources` — the server does the fan-out:

```
# Which documents can bob view? (server-side graph traversal)
zed permission lookup-resources document#view --subject user:bob --
consistency minimize_latency
```

Distributed Zanzibar at scale



- **Stateless serving tier.** SpiceDB replicas are stateless — they dispatch graph traversal (`Watch` + `dispatch`) across the cluster. Scale with HPA on `spicedb_check_latency_p99`. The bottleneck is the datastore, not the dispatch tier.
- **Datastore choice.** Postgres 16 (single-region, simpler), CockroachDB 24.x or Spanner (multi-region, strong reads without a single primary). SpiceDB's datastore interface abstracts this; choose by region requirements.
- **Indexing.** Every tuple lookup is `(object_type, object_id, relation)` or `(subject_type, subject_id)`. Undersized indexes turn a `Check` from `~5 ms` into a table scan. SpiceDB's `migrate` creates them — do not drop them.

- **Watch API for cache invalidation.** SpiceDB's `Watch` streams tuple changes as Zookies. Use it to invalidate PEP decision caches — `Watch` from the last seen Zookie, invalidate cache entries whose resource appeared in the stream.

Choosing among RBAC, ABAC, and ReBAC

Dimension	RBAC	ABAC	ReBAC (Zanzibar/SpiceDB)
Model	<code>user-role-permission</code>	<code>policy(attributes)</code> → <code>allow</code>	<code>graph(subject-relation-object)</code> → <code>allow</code>
Granularity	Class-level ("editors can edit docs")	Arbitrary conditions	Instance-level ("bob can view doc abc via folder")
Delegation / sharing	New role or assignment	New policy rule	New tuple (cheap, no deploy)
Hierarchy / inheritance	Role hierarchy (limited)	Explicit in policy	<code>parent->permission</code> in schema
Context (time, IP, expiry)	No	First-class	Via caveats (ABAC inside ReBAC)
Audit ("who can access X?")	Query <code>user_roles</code>	Policy analysis (hard)	<code>Expand / LookupResources</code> (graph query)
Latency	Cacheable DB lookup (µs-ms)	Policy eval + attribute fetch (ms-10s ms)	Graph traversal + datastore read (5-20 ms p50)
Consistency	DB replication lag	Bundle + attribute staleness	Zookie-bounded (causal)
Ops cost	Low	Medium (policy lifecycle)	Medium-High (tuple store, schema migrations)

Practical guidance at fleet scale:

- **Default to RBAC** for coarse, class-level permissions (`org_admin` , `billing:read`). It is simple, fast, and sufficient for most service-level gates.
- **Add ABAC** when decisions need environmental context (clearance, time window, device posture) that does not fit a relation. OPA/Cedar policies can call ReBAC as one input (`spicedb_check` as a built-in).
- **Adopt ReBAC** when resources are hierarchical (org → folder → document), sharing is user-driven, or you need to answer "which resources can this principal access?" efficiently. If your permission model has ever required a `shared_with` table that joins through folders and teams, ReBAC is that join as a service.
- **Hybrid is normal.** A mature platform runs RBAC for coarse gates, ReBAC for resource-instance decisions, and a thin ABAC policy layer for cross-cutting conditions (blocklists, geo-fencing). Keep the hybrid explicit — document which layer owns which permission, or operators will check the wrong one during an incident.

Failure modes and incident patterns

Role explosion → privilege creep. Teams mint `viewer_plus_one_permission` roles to avoid a policy change, and the catalog grows until no one can audit it. Mitigation: cap the role catalog (e.g., ≤ 20 roles), require scoped assignments instead of new roles, and alert on role count growth.

Confused deputy. Service A presents a token with `scope: document:read` to Service B, but B serves `document:write` because it checks "is the caller authenticated?" not "does the caller's scope cover this action?" Mitigation: every PEP validates `scope / permission` against the requested action, not just identity; test cross-scope tokens in CI (request `write` with a `read` token, expect 403).

TOCTOU (time-of-check, time-of-use). Check passes, then the resource is deleted or permissions change before the use. A `Check` followed by a `Get` without atomicity leaks a deleted resource or bypasses a revocation. Mitigation: where consistency matters, bundle check+fetch via the same Zookie / same transaction, or re-check at use time. For bulk operations, `LookupResources` already returns only currently-visible resources.

Inheritance escalation. Schema `permission view = viewer + parent->view` plus an overly broad `folder:marketing#viewer@user:*` grants every document under the folder to every user. Mitigation: never use wildcard tuples in production; model "everyone" as an explicit group (`team:all`) whose membership is audited, and review schema `parent->` chains for transitive blast radius.

Cache-poisoned allow. A PEP caches `allow=true` for `TTL=5 min`; a revocation deletes the tuple but the cache still allows. Mitigation: short TTLs (5-30 s), `Watch`-based invalidation, and `at_least_as_fresh` on sensitive paths. Monitor `authz_stale_allow_total` estimated from `Watch` lag.

Schema migration without backfill. Adding `permission view = ... + new_relation` before tuples for `new_relation` exist silently narrows access; removing a composition silently widens it. Mitigation: SpiceDB schema migrations are additive — deploy new schema, backfill tuples, then rely on the new permission. Test schema changes with `zed validate` and a shadow check (compare old vs new schema results on sampled requests) before promoting.

Distributed-systems lens

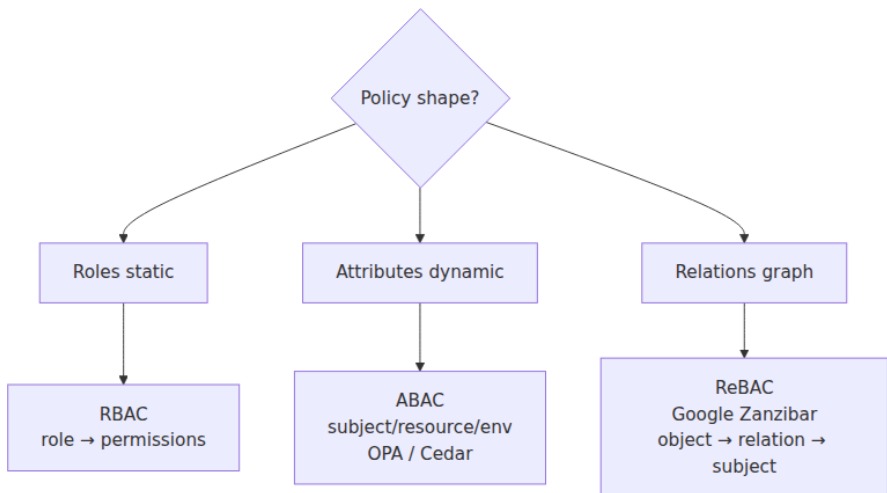
Authorization at scale is a *data system* problem with availability and partition implications distinct from authentication:

- **Authorization data is a hot, graph-structured dataset with strong consistency needs on revocation.** A login (Ch 5) can tolerate seconds of staleness; a revoke ("remove eve from document") must be visible before the next `Check` that matters. Zookies give you the knob — causal freshness per caller without forcing every check to the primary. Use it.
- **Decision fan-out on list endpoints.** `GET /documents?filter=mine` naively checks `N` resources with `N` round-trips — an `N+1` that becomes a p99 cliff at `N=100`. `LookupResources` / `BulkCheck` pushes the fan-out to the `authz` tier, which parallelizes traversal against the tuple store. Push filter-then-authorize queries down; never filter in the app loop.
- **Cross-region authorization.** Tuple replication lag across regions delays revocation visibility. Options: (a) serve checks from the write region for sensitive resources (higher latency, strong), (b) accept

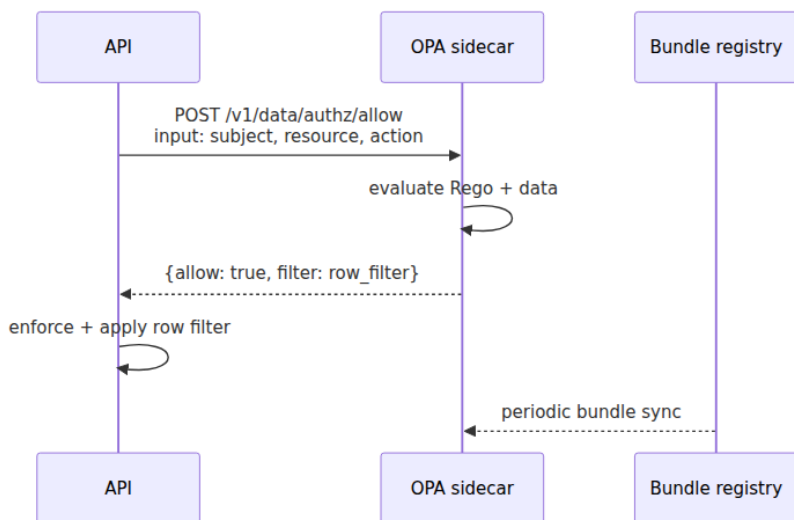
bounded staleness (low latency, needs TTL discipline), or (c) use CRDB/Spanner with strong cross-region reads. Do not pretend a single-region tuple store is multi-region consistent — document the staleness SLA per resource class.

- **Bulkhead the PDP.** The authz tier is a hard dependency — every request validates against it. Apply the same resilience patterns as any critical dependency (Vol 7, Ch 11; Vol 11, Ch 10): circuit breaker (fail-closed), bulkhead (isolate authz thread pool), timeout (50 ms budget for a check, 200 ms for LookupResources), and a degraded mode only where explicitly approved (e.g., read-only cache for view but never for delete). A PDP outage must not become an authz bypass.
- **Observability.** Emit `authz_check_total{result=allow|deny, permission, cached}`, `authz_check_latency_ms`, `spicedb_datastore_replica_lag_ms`, and `authz_policy_version`. Alert on deny rate spikes (misconfigured rollout), p99 latency regression (missing index), and replica lag exceeding the decision TTL.

RBAC vs ABAC vs ReBAC



OPA Policy Evaluation



Key takeaways

- Authorization answers `principal × action × resource × context → allow|deny`. Authentication (Ch 5–6) is a prerequisite, not a substitute — validating a token's `aud / exp / scope` is only the input to the decision.
- RBAC (`user→role→permission`, ANSI 359) is the right default for class-level gates. Implement with `(user_roles, role_permissions)` tables, scoped assignments (`scope_id`), and short-TTL decision caches (5–30 s) that fail closed on PDP error and hide existence on deny.
- ABAC (OPA/Rego v0.70+, Cedar 4.x) expresses arbitrary attribute conditions (`clearance, time, classification`) as versioned policy bundles. Ship bundles via CI with signing, sidecar or embedded evaluation, and per-decision `deny_reason` for operability.
- ReBAC (Zanzibar 2019, SpiceDB v1.35+) models permissions as a typed graph (`object#relation#subject` tuples) with schema-declared composition (`+`, `&`, `-`, `parent->permission`). It is the correct model for hierarchical resources and user-driven sharing.
- SpiceDB's Zookie (`ZedToken`) gives causal consistency without forcing every check to the primary: `pass at_least_as_fresh=Zookie(write)` after a write you control,

`fully_consistent` for sensitive paths, `minimize_latency` otherwise. Use `LookupResources` / `BulkCheck` for list endpoints to avoid N+1.

- At fleet scale, authorization is a tiered architecture — coarse RBAC + instance-level ReBAC + thin ABAC for context — served by a stateless dispatch tier (SpiceDB HPA) over a replicated tuple store (Postgres 16 / CRDB 24) with `Watch`-based cache invalidation and circuit-broken, fail-closed PEPs.

Further reading

- **Papers:** Pang et al. — *Zanzibar: Google's Consistent, Global Authorization System* (USENIX ATC 2019) — the definitive ReBAC paper; read §2-§4 for model and consistency, §5 for scale. Sandhu et al. — *The NIST Model for Role-Based Access Control* (ACM RBAC 2000) and ANSI INCITS 359-2004 — RBAC reference model. NIST SP 800-162 — *Guide to ABAC Definition and Considerations*.
- **Standards:** ANSI INCITS 359 (RBAC), NIST SP 800-162 (ABAC). OPA Policy Reference (<https://www.openpolicyagent.org/docs/latest/policy-reference/>) — Rego semantics. Cedar Language Reference (<https://docs.cedarpolicy.com/>) — typed policy, validation, and formal analysis.
- **SpiceDB / Zanzibar:** AuthZed SpiceDB docs (<https://authzed.com/docs/spicedb>) — schema DSL, consistency modes, `zed` CLI, datastore backends. `authzed-go` v1.2+ (<https://github.com/authzed/authzed-go>). OpenFGA docs (<https://openfga.dev/docs>) — alternative Zanzibar implementation; compare schema ergonomics. Google Zanzibar whitepaper errata and AuthZed blog — caveats and `LookupResources` performance notes.
- **OPA/Cedar:** OPA v0.70+ docs — <https://www.openpolicyagent.org/docs/latest/> — bundle lifecycle, Envoy `ext_authz` integration, `opa test`. Cedar 4.x docs — <https://docs.cedarpolicy.com/> — schema, entities, and the `cedar validate` / `cedar analyze` toolchain.
- **Operational:** *Ship authorization as a platform* — AuthZed blog series on `Watch`, bulk checks, and CRDB/Spanner deployment.

Chapter 8 — Secrets Management

What this chapter covers. Every backend system is a lattice of secrets — database passwords, API keys, TLS private keys (Ch 4), signing keys (Ch 3), OAuth client secrets (Ch 6), encryption DEKs/KEKs (Ch 3), and the tokens that carry identity (Ch 5). A secret is any value whose disclosure breaks a security property, and managing secrets is managing their entire lifecycle: generation, distribution, storage, rotation, revocation, and audit — across dozens of services, multiple environments, CI/CD pipelines, and ephemeral compute that may live for seconds. Hard-coded credentials and long-lived static secrets remain the most common breach vector not because engineers do not know better, but because the easy path leaks. This chapter builds a disciplined secrets plane: envelope encryption, Vault (HashiCorp Vault 1.15+ / OpenBao) as the central secrets manager, dynamic and short-lived credentials, distribution via agent/sidecar/CSI, rotation without downtime, detection and remediation of leaked secrets, and the distributed-systems reality of secret replication, blast radius, and availability when the secrets store itself is a dependency of every service.

Learning goals — after this chapter you should be able to:

- Classify secrets by kind, lifetime, and blast radius — and explain why static, long-lived secrets are the wrong default for any credential that can be made dynamic or short-lived.
- Operate Vault/OpenBao 1.15+ for KV v2, Transit, PKI, and database dynamic credentials — policies, auth methods (AppRole, Kubernetes, JWT/OIDC, AWS IAM), response wrapping, and audit devices.
- Distinguish encryption at rest (envelope encryption with KMS/HSM), encryption in transit (TLS per Ch 4), and secret distribution — and implement each correctly.
- Distribute secrets to workloads via Vault Agent, Agent Injector, Secrets Store CSI Driver, and External Secrets Operator — and compare their lifecycle and failure modes.

- Implement rotation — automated, staggered, dual-credential, and lease-based — for database passwords, API keys, and certificates without downtime, and plan emergency revocation.
- Detect leaked secrets in git, logs, and artifacts (gitleaks, TruffleHog, GitHub secret scanning) and remediate by rotation, not redaction.
- Design the secrets plane for scale — HA Vault clusters (Raft integrated storage), replication, seal/unseal with auto-unseal (KMS/Shamir), and the availability trade-off when Vault is a hard dependency.

Boundary notes. *Cryptographic primitives* (AEAD, KDFs, hashing) are **Ch 1-3** — this chapter consumes them via envelope encryption and compares secret storage to key management (Ch 3's KEK/DEK hierarchy reappears as Vault's barrier key / recovery key). *PKI and TLS private keys* are **Ch 4** (certificate lifecycle, rotation, mTLS); this chapter covers how those private keys are stored and distributed, not how X.509 chains validate. *Authentication tokens* (sessions, JWTs, OAuth) are **Ch 5-6**; this chapter stores the secrets that *mint* those tokens (signing keys, client secrets) and the dynamic credentials that replace static passwords. *Build/CI secrets* (pipeline poisoning, OIDC workload identity for CI) are covered in Companion Book 4; this chapter focuses on runtime secrets, including CI as one consumer. *Workload identity* (SPIFFE/SPIRE, mTLS) is **Ch 10** — the preferred replacement for many long-lived secrets. *Threat modeling* secrets as assets is **Ch 11**.

Why secrets management is a lifecycle, not a vault

Putting secrets in a vault is necessary but not sufficient. A secret has six phases, and a failure at any one is a breach:

Generate → Store → Distribute → Use → Rotate → Revoke/Audit

- **Generate** with a CSPRNG (Ch 1) or via the secrets manager itself (Vault `gen/creds`). Never derive from weak entropy, timestamps, or `math/rand`.
- **Store** encrypted at rest with envelope encryption, with access control (Vault policies, not broad IAM), and with audit logging on every read.

- **Distribute** over TLS (Ch 4) to the workload that needs it, with minimal scope (one service gets one database role, not `root`), minimal lifetime (minutes-hours, not months), and no persistence in env vars that leak to `/proc` or crash dumps.
- **Use** in memory only, never logged, never echoed in error messages, never returned in API responses. Mask in observability.
- **Rotate** automatically, with dual-credential overlap so rotation does not require coordinated downtime.
- **Revoke/audit** instantly on suspicion, with a bounded propagation window and a log of who read what, when, from where.

The naive path — `.env` file, Kubernetes `Secret` (base64), environment variable, checked into git — fails at every phase: generation is ad-hoc, storage is plaintext (etcd at rest is not encrypted by default), distribution is a copy, use leaks to logs, rotation is manual (so it never happens), and revocation is "change the password and redeploy everything at 2 a.m."

This chapter replaces each phase with a concrete mechanism.

What counts as a secret

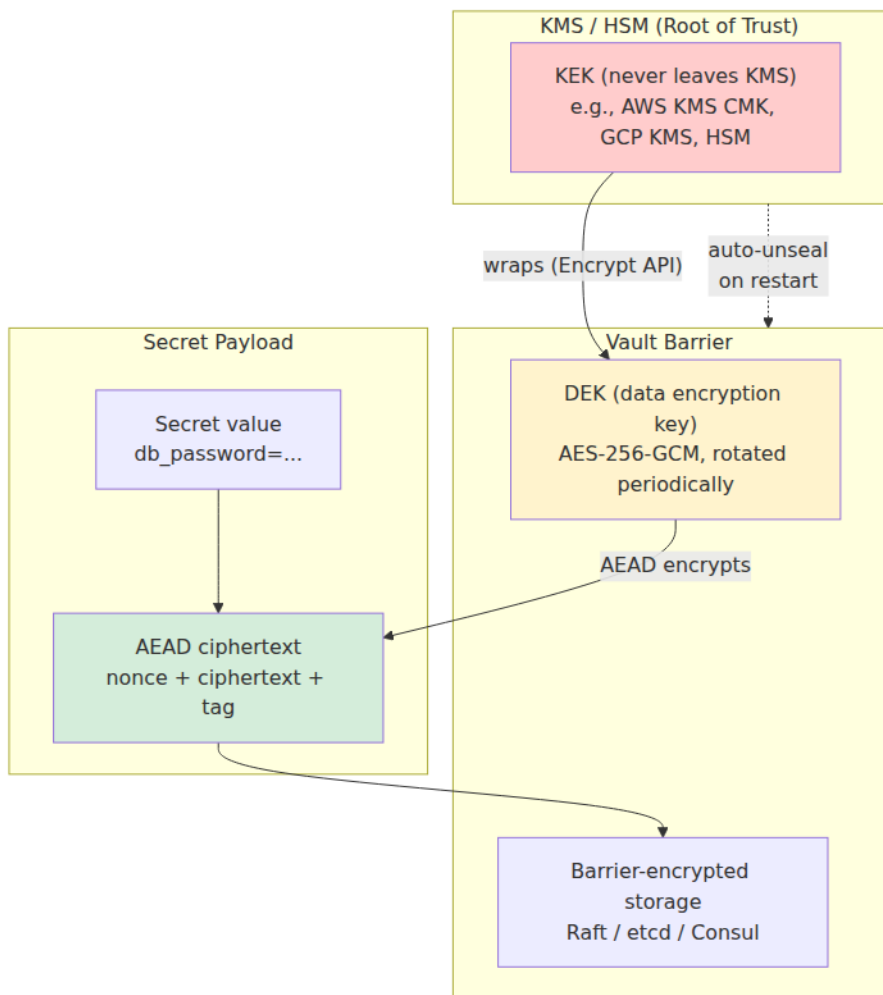
Class	Examples	Lifetime that is safe	Replacement
Database credentials	Postgres/MySQL passwords, connection strings	Minutes-hours (dynamic)	Vault database engine — per-service, per-pod ephemeral creds
API keys / tokens	Stripe, AWS, GitHub PATs, internal service tokens	Hours-days, scoped	Vault KV + rotation, or OIDC workload identity (Ch 10)
TLS private keys	Server leaf keys, mTLS client keys (Ch 4)	Hours (SPIRE) to 90 days (ACME)	Vault PKI / SPIRE SVID — short-lived, auto-renewed
Signing keys	JWT signing keys (Ch 5), image signing keys (Companion Book 5)	Days-months, versioned (<code>kid</code>)	Vault Transit / KMS — sign via API, key never leaves HSM

Class	Examples	Lifetime that is safe	Replacement
Encryption keys	DEKs, KEKs (Ch 3)	DEK per object, KEK per scope	Envelope encryption — KEK in KMS/HSM, DEK wrapped
CI/CD secrets	Registry passwords, deploy keys, cloud credentials	Minutes (OIDC-federated)	OIDC workload identity — no static secret at all
Human credentials	Break-glass passwords, recovery codes	One-time, sealed	Vault Cubbyhole / Shamir split, break-glass audit

Rule of thumb: if the secret *can* be made dynamic and short-lived, it *should* be. Static secrets are technical debt that accrues breach risk with age.

Envelope encryption: how secrets rest safely

A secrets manager does not just "encrypt the database" — it uses **envelope encryption**, the same KEK/DEK hierarchy from Ch 3, now applied to the storage layer.



- The **KEK** lives in a KMS/HSM and never leaves it. Vault calls `Encrypt / Decrypt` via the KMS API. Compromise of the Vault host does not expose the KEK.
- The **DEK** (Vault calls it the *barrier key*) encrypts the Raft storage. Rotating the DEK re-wraps storage without re-encrypting every secret — only the DEK wrapper changes.
- **Auto-unseal** (Vault 1.15+ `seal "awskms"` / `seal "gcpkms"` / `seal "transit"`) removes the manual Shamir unseal ceremony. The Vault process fetches the barrier key from KMS on start. For break-glass

without KMS, Shamir seals (`threshold=3, shares=5`) remain an option — store shards with separate custodians.

For application-layer secrets (e.g., encrypting a column before it reaches Vault), use the same pattern explicitly — Vault Transit is the service that does it:

```
# Vault Transit 1.15+ – envelope encryption as a service
vault secrets enable transit
vault write -f transit/keys/app-dek exportable=false type=aes256-gcm96

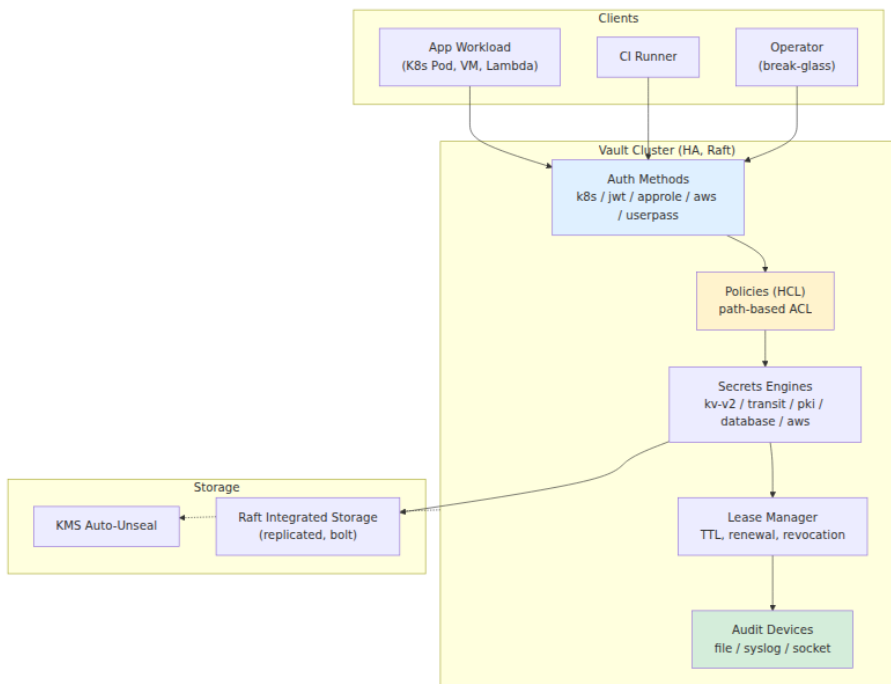
# App encrypts via Transit – DEK never leaves Vault
vault write transit/encrypt/app-dek plaintext=$(base64 <<< "sensitive
payload")
# → ciphertext: vault:v1:...

vault write transit/decrypt/app-dek ciphertext="vault:v1:..."
```

This keeps DEKs out of application memory and makes rotation a Vault API call, not a re-encryption migration.

Vault as the control plane

HashiCorp Vault (1.15+; OpenBao is the open-source fork under the same API — either is fine) is the de facto control plane. Its architecture maps directly to the lifecycle above.



Policies (version-pinned: HCL, Vault 1.15+)

Policies are path-based ACLs. Every token's capabilities derive from exactly the policies attached to it — no policy, no access.

```

# vault/policy/app-payments.hcl - scoped to one service, one env
path "kv/data/payments/prod/*" {
    capabilities = ["read"]
}
path "kv/data/payments/prod/api-keys/*" {
    capabilities = ["read"]
}
# Transit encrypt/decrypt - key never leaves Vault
path "transit/encrypt/app-dek" { capabilities = ["update"] }
path "transit/decrypt/app-dek" { capabilities = ["update"] }
# Dynamic DB creds - read-only role
path "database/creds/payments-ro" {
    capabilities = ["read"]
}
# Renew own lease only
path "sys/leases/renew" { capabilities = ["update"] }
path "sys/leases/revoke" { capabilities = ["update"] }

# Explicit deny trumps allow - block prod secrets from staging identity
path "kv/data/payments/prod/*" {
    capabilities = ["deny"]
    # Applied via a staging policy that is never attached to prod roles
}

# Write:
# vault policy write app-payments vault/policy/app-payments.hcl

```

Attach policies to roles, not to tokens directly. Roles bind an auth method to a policy set:


```

# Kubernetes auth – Vault 1.15+ (vault auth enable kubernetes)
vault auth enable kubernetes
vault write auth/kubernetes/config kubernetes_host="https://$KUBERNETES
_PORT_443_TCP_ADDR:443"

vault write auth/kubernetes/role/payments-prod \
    bound_service_account_names=payments \
    bound_service_account_namespaces=prod \
    policies=app-payments \
    ttl=15m max_ttl=1h period=15m \
    token_type=batch # batch tokens: lightweight, non-renewable, ~30%
lower overhead

# AppRole for VMs / non-K8s
vault auth enable approle
vault write auth/approle/role/payments-prod \
    token_policies=app-payments token_ttl=15m token_max_ttl=1h \
    secret_id_ttl=10m token_type=batch

# JWT/OIDC for CI (GitHub Actions, GitLab) – no static secret at all
vault auth enable jwt
vault write auth/jwt/config oidc_discovery_url="https://
token.actions.githubusercontent.com"
vault write auth/jwt/role/github-deploy \
    bound_audiences="https://github.com/example-org" \
    bound_claims="{\"repository\":\"example-org/payments\"}" \
    policies=ci-deploy ttl=5m max_ttl=5m

```

The `ttl` / `max_ttl` on the role is the blast-radius control — a leaked token is useful for at most `max_ttl`.

Auth methods: choosing correctly

Method	Use when	Token type	Rotation
Kubernetes (k8s)	Workload is a Pod with a ServiceAccount JWT	batch (short)	Automatic via SA token projection (hourly)
JWT/OIDC	CI runner, serverless, any OIDC issuer	batch	Per-run, no persistence
AppRole	VM, bare metal, non-K8s host	batch or service	<code>secret_id</code> rotation via automation

Method	Use when	Token type	Rotation
AWS IAM	EC2/ECS/Lambda with instance role	batch	IAM role session (hours)
Userpass / LDAP	Break-glass human	service (renewable)	Manual + MFA (TOTP per Ch 5)

Prefer `batch` tokens for workloads — they are not persisted in Vault storage, cannot be renewed, and have lower overhead. Use `service` tokens only for long-lived human or controller identities that need renewal.

Secrets engines

KV v2 (`kv/data/...`) — the generic encrypted store. Versioned, with `max_versions`, `delete_version_after`, and `cas` (check-and-set) for safe concurrent writes:

```

vault secrets enable -path=kv kv-v2
vault kv put kv/payments/prod/db host=db.prod.example.com password="$(openssl rand -base64 32)"
vault kv get kv/payments/prod/db
vault kv metadata put -max-versions=10 -delete-version-after=720h kv/payments/prod/db

```

Transit — encryption as a service (above). Use for app-layer envelope encryption and for signing (`transit/sign`, `transit/hmac`).

PKI — internal CA for short-lived certs (Ch 4). Issue SPIFFE-compatible SVIDs or private TLS leaves with `ttl=1h`:

```

vault secrets enable pki
vault write pki/root/generate internal common_name="prod.example.com" ttl=87600h
vault write pki/roles/app-server allow_any_name=true max_ttl=24h server_flag=true client_flag=false
vault write pki/issue/app-server common_name="payments.prod.example.com" ttl=1h

```

Database — dynamic, ephemeral credentials. Vault creates a temporary user, grants a role, and revokes on lease expiry:

```

vault secrets enable database
vault write database/config/pg-prod \
  plugin_name=postgresql-database-plugin \
  connection_url="postgresql://{{username}}:{{password}}@pg.prod.example.com:5432/payments?sslmode=require" \
  username="vault_admin" password="$VAULT_ADMIN_PW" \
  allowed_roles="payments-ro,payments-rw"

vault write database/roles/payments-ro \
  db_name=pg-prod \
  creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL '{{expiration}}'; GRANT SELECT ON ALL TABLES IN SCHEMA public TO \"{{name}}\";" \
  revocation_statements="DROP ROLE IF EXISTS \"{{name}}\";" \
  default_ttl=15m max_ttl=1h
```

Every `vault read database/creds/payments-ro` mints a unique (username, password) valid for 15 m, auto-dropped on lease expiry. No static password to leak.

Distributing secrets to workloads

Storing secrets in Vault is half the job; getting them to the workload without reintroducing the leaks you just fixed is the other half. Four patterns dominate in 2024-2026:

Pattern	How it works	Secret appears as	Rotation	Failure mode
Vault Agent (sidecar)	Sidecar authenticates, renders templates to tmpfs	File on emptyDir: medium: Memory	Agent re-renders on lease renewal	Agent crash → start monitor vault_agent_ca
Agent Injector (mutating webhook)	Webhook injects Agent sidecar + annotations	File (same)	Same as Agent	Webhook down → start without secret closed is correct)
Secrets Store CSI Driver	CSI volume mounts secrets as files	File (tmpfs)	CSI rotation (rotationPollInterval: 2m)	Driver DaemonSet → mount fails

Pattern	How it works	Secret appears as	Rotation	Failure mode
External Secrets Operator (ESO)	Controller syncs Vault → K8s <code>Secret</code>	Native <code>Secret</code> (etcd, base64)	<code>refreshInterval: 1h</code>	ESO down → stale <code>Secret</code> ; etcd at must be encrypted

All four are production-grade; the choice is blast-radius and ergonomics:

- For **new K8s workloads**, prefer **CSI Driver** or **Agent Injector** — secrets never become Kubernetes `Secret` objects, never land in etcd, and are `tmpfs`-only.
- For **existing workloads that already consume `Secret` env vars**, **ESO** is the migration path — it replaces manual `kubectl create secret` with automated sync and gives you a single control plane while you migrate to file mounts.

```

# Vault Agent Injector – annotation-driven (Vault 1.15+, vault-k8s
1.4+)
apiVersion: apps/v1
kind: Deployment
metadata: { name: payments, namespace: prod }
spec:
  template:
    metadata:
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/agent-inject-secret-db: "kv/data/payments/
prod/db"
        vault.hashicorp.com/agent-inject-template-db: |
          {{`{- with secret "kv/data/payments/prod/db" -}}`}}
          host={{`{{ .Data.data.host }}`}}
          password={{`{{ .Data.data.password }}`}}
          {{`{- end -}}`}}
        vault.hashicorp.com/role: "payments-prod"
        vault.hashicorp.com/agent-cache-enable: "true"
        # Template rendered to /vault/secrets/db (tmpfs), re-rendered
on renewal
    spec:
      serviceAccountName: payments
      containers:
        - name: app
          image: registry.example.com/payments:1.42.0
          # App reads /vault/secrets/db – never an env var
          volumeMounts: [{ name: vault-secrets, mountPath: /vault/
secrets }]
      ---
# Secrets Store CSI Driver – Vault provider (secrets-store.csi.k8s.io
v1.4+)
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata: { name: vault-db, namespace: prod }
spec:
  provider: vault
  parameters:
    vaultAddress: "https://vault.prod.example.com:8200"
    roleName: "payments-prod"
    objects: |
      - objectName: "db-password"
        secretPath: "kv/data/payments/prod/db"
        secretKey: "password"
    # Optional: sync to K8s Secret for legacy consumers – avoid if
possible
    # secretObjects:
    # - secretName: payments-db
    #   type: Opaque
    #   data:

```

```

# - objectName: db-password
#   key: password
---
# ESO - Vault → K8s Secret sync (external-secrets.io v0.9+)
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: payments-db, namespace: prod }
spec:
  refreshInterval: 5m
  secretStoreRef: { name: vault-backend, kind: ClusterSecretStore }
  target:
    name: payments-db           # K8s Secret that is created/updated
    creationPolicy: Owner
    deletionPolicy: Retain
  data:
    - secretKey: password
      remoteRef: { key: kv/payments/prod/db, property: password }

```

Application consumption — read from file, never from env:

```

// Go 1.22 - read secret from Vault Agent / CSI tmpfs file
// File is rendered as KEY=VALUE or JSON; never logged, never returned
// in errors.

func loadDBPassword(path string) (string, error) {
    b, err := os.ReadFile(path) // /vault/secrets/db or /mnt/secrets-
    store/db-password
    if err != nil {
        return "", fmt.Errorf("read secret: %w",
err) // do not include file content
    }
    // If template rendered as KEY=VALUE, parse; if raw value, trim
    space.
    // Keep in memory only as long as needed; zero after use if held in
    []byte.
    pw := strings.TrimSpace(string(b))
    // Explicitly clear the byte slice that held the raw read
    for i := range b {
        b[i] = 0
    }
    if pw == "" {
        return "", errors.New("empty secret")
    }
    return pw, nil
}

```

Hardening checklist for distribution:

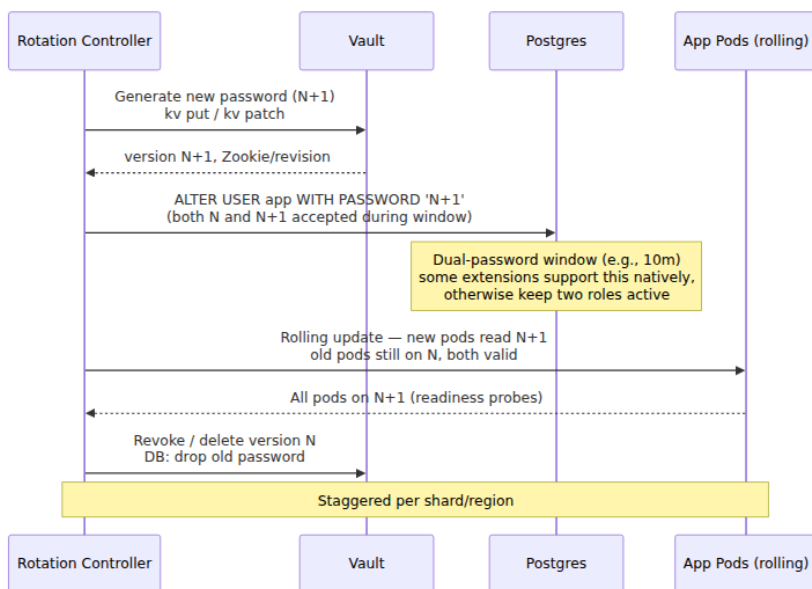
- **No env vars for secrets.** Env vars leak to `/proc/<pid>/environ`, crash dumps, and child processes. Files on `tmpfs` with `0400` are the correct target.
- **tmpfs only.** `emptyDir: { medium: Memory }` or CSI `tmpfs` — never a persistent volume. Secrets must not survive pod rescheduling on disk.
- **No logging.** Scrub `password`, `token`, `secret` keys from structured logs. Test with a canary secret and grep logs for its value in CI.
- **Response wrapping for bootstrapping.** When a human or CI must hand a secret to a workload, use Vault response wrapping (`vault read -wrap-ttl=5m ...`) — the wrapping token is single-use and short-lived, not the secret itself.

Rotation: the only remediation that matters

Redacting a leaked secret from git history does not revoke it. **Rotation is remediation** — every other action is hygiene.

Principles

1. **Dual-credential overlap.** At any moment, `N` and `N+1` are both valid for a window (minutes-hours). Writers use `N+1` immediately; readers accept both; after the window, `N` is revoked. No coordinated restart.
2. **Automated, not manual.** If rotation requires a human, it will not happen before the incident. Automate via Vault leases, `ESO refreshInterval`, or a rotation controller; alert on `vault_lease_expiry` and `secret_age_days`.
3. **Staggered.** Rotate one shard/region/service at a time. A fleet-wide rotation that breaks one service should not break all of them simultaneously.
4. **Lease-based where possible.** Dynamic credentials (Vault database engine) rotate by construction — the lease is the rotation. Prefer leases over cron.



Concrete rotation patterns

Database — dynamic credentials (preferred, no rotation needed beyond lease):


```

// Go - Vault database dynamic creds with lease renewal
import vault "github.com/hashicorp/vault/api"

func dbWithDynamicCreds(vaultAddr, role string) (*sql.DB, error) {
    vc, _ := vault.NewClient(&vault.Config{Address: vaultAddr})
    // Auth via K8s / JWT / AppRole - Vault Agent handles this; app
    // reads VAULT_TOKEN file
    secret, err := vc.Logical().Read("database/creds/" + role) //
    "payments-ro"
    if err != nil || secret == nil {
        return nil, fmt.Errorf("mint creds: %w", err)
    }
    username := secret.Data["username"].(string)
    password := secret.Data["password"].(string)
    leaseID := secret.LeaseID
    leaseDuration := time.Duration(secret.LeaseDuration) * time.Second

    dsn := fmt.Sprintf("postgres://%s:%s@pg.prod.example.com:5432/
    payments?sslmode=require",
        url.QueryEscape(username), url.QueryEscape(password))
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        return nil, err
    }

    // Renew lease in background; on failure, re-mint (Vault revokes
    // old user on lease expiry)
    go func() {
        ticker := time.NewTicker(leaseDuration / 2)
        defer ticker.Stop()
        for range ticker.C {
            s, err := vc.Sys().Renew(leaseID, 0)
            if err != nil || s == nil {
                slog.Error("lease renewal failed, re-minting", "err", e
rr)
                // Re-mint path: Read new creds, open new *sql.DB,
                // drain old pool
                return
            }
            // Update ticker to half of new duration
            ticker.Reset(time.Duration(s.LeaseDuration/2) *
time.Second)
        }
    }()
    return db, nil
}

```

Static API key — automated rotation via Vault + controller:

```

# Kubernetes CronJob that rotates a Stripe-like API key every 30d
(staggered)
apiVersion: batch/v1
kind: CronJob
metadata: { name: rotate-stripe-key, namespace: prod }
spec:
  schedule: "0 3 1 * *" # monthly – plus on-demand via `kubectl
create job`
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: rotator
          restartPolicy: OnFailure
          containers:
            - name: rotator
              image: registry.example.com/rotator:1.2.0
              env:
                - name: VAULT_ADDR
                  value: "https://vault.prod.example.com:8200"
              command:
                - /bin/sh
                - -c
                - |
                  set -euo pipefail
                  # 1. Mint new key at provider (Stripe example)
                  NEW_KEY=$(stripe_generate_key --scope payments)
                  # 2. Write to Vault as new version (KV v2 keeps history)
                  vault kv put kv/payments/prod/stripe api_key="$NEW_KEY"
rotated_at="$(date -u +%FT%TZ)"
                  # 3. Dual-key window: keep old key valid at provider for
1h
                  # (provider-specific – Stripe: keep both keys active
briefly)
                  # 4. Trigger rolling update so pods pick up new version
                  kubectl rollout restart deployment/payments -n prod
                  kubectl rollout status deployment/payments -n prod --
timeout=300s
                  # 5. Revoke old key at provider after rollout succeeds
                  stripe_revoke_key "$OLD_KEY_ID"

```

JWKS / signing keys — see Ch 5. Rotate by adding `kid=N+1` alongside `kid=N`, deploy verifiers (RS) first, then start signing with `N+1`, then retire `N` after `max_token_lifetime`.

Emergency revocation

When a secret is known-compromised, rotation speed is containment:

1. **Revoke the lease / delete the version** — `vault lease revoke -prefix database/creds/` or `vault kv delete kv/payments/prod/db + vault kv destroy` (permanent). For dynamic DB creds, lease revocation drops the database user immediately.
2. **Push a config-plane blocklist** — for tokens/JWTs, add the `jti / kid` to a blocklist distributed via config (Ch 4 pattern). Assume Vault revocation alone is not instant for already-issued tokens.
3. **Force workload restart** — `kubectrl rollout restart` fleet-wide. Do not wait for `refreshInterval` — the compromised value is in memory.
4. **Rotate the KEK if scope warrants it** — if the Vault barrier or transit KEK is suspected, re-wrap via KMS. This is rare but must be rehearsed.

Detecting leaked secrets

Prevention fails without detection. Three layers:

Pre-commit / pre-push:

```
# .pre-commit-config.yaml - gitleaks 8.18+ (github.com/gitleaks/gitleaks)
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks:
  - id: gitleaks
# TruffleHog 3.70+ as CI fallback
# trufflehog git --since-commit HEAD~1 --fail --only-verified
```

CI / repo scanning:

```
# gitleaks in CI – scans the diff, not just HEAD
gitleaks detect --source . --verbose --redact --max-target-megabytes 10
0

# TruffleHog verified-only (reduces false positives – only reports
# creds that actually authenticate)
trufflehog git file://. --only-verified --fail

# GitHub secret scanning (push protection) – enable at org level:
# Settings → Code security → Secret scanning → Push protection (blocks
# push that contains a secret)
```

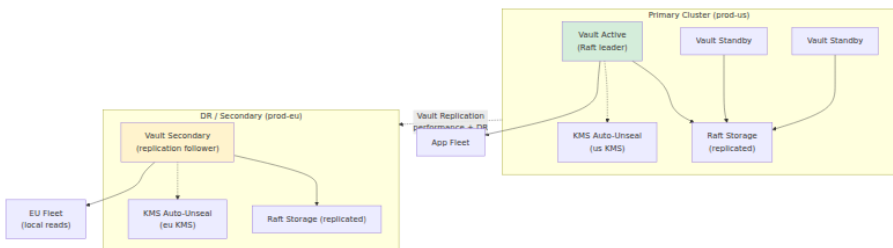
Runtime / artifact scanning:

```
# Scan container images and logs for accidental secret inclusion
trufflehog docker --image registry.example.com/payments:1.42.0 --only-
verified
gitleaks detect --source /var/log/app --no-git
```

When a leak is found, the runbook is the same every time: **rotate, do not redact**. Rewriting git history (`filter-repo`) without rotating leaves the secret valid in every clone, fork, and CI cache that already fetched it. Rotate first, then clean history to reduce future exposure.

Vault at scale: HA, replication, and the availability trade-off

Vault is a hard dependency — if Vault is down, no pod can mint a new database password, no sidecar can render a new secret, and no rotation can complete. Design for its availability explicitly.



- **Raft integrated storage** (Vault 1.15+ default) — no external Consul/etcd. Three or five voters, `retry_join` for auto-clustering, `autopilot` for dead-server cleanup. Back up Raft snapshots to S3/GCS (`vault operator raft snapshot save`).

- **Performance replication** — secondary clusters serve reads locally (KV, Transit) without forwarding to primary. Required for multi-region latency — a 5 ms local read vs 80 ms cross-region.
- **DR replication** — async WAL shipping for failover. Promote secondary if primary region is lost. Test promotion — a DR cluster you have never promoted is a hope, not a plan.
- **Seal HA.** KMS auto-unseal must itself be multi-region or the KMS region outage seals Vault. Use KMS multi-region keys or Shamir as fallback.
- **Caching and graceful degradation.** Vault Agent cache (`cache { use_auto_auth_token = true }`) and `proxy_cache` let workloads survive a brief Vault outage with cached leases. Size cache TTLs to cover your p99 Vault recovery time, and alert on `vault_core_unsealed` and `vault_raft_autopilot_healthy`.

Operational metrics to export and alert on:

```
# Prometheus alerts for the secrets plane
groups:
- name: vault
  rules:
  - alert: VaultSealed
    expr: vault_core_unsealed == 0
    for: 1m
    labels: { severity: critical }
  - alert: VaultLeaseExpiryStalled
    expr: increase(vault_expire_num_leases{success="false"}[5m]) > 0
    labels: { severity: warning }
  - alert: SecretAgeTooHigh
    expr: (time() - vault_kv_version_created_time) / 86400 > 30
    for: 1h
    labels: { severity: warning }
    annotations: { summary: "Secret {{ $labels.path }} not rotated in 30d" }
  - alert: VaultRaftLag
    expr: vault_raft_autopilot_healthy == 0
    for: 5m
    labels: { severity: critical }
```

Kubernetes and cloud-native hardening

- **etcd encryption at rest.** Kubernetes `Secret` is base64, not encrypted, unless you enable `EncryptionConfiguration` with a KMS provider:

```
# EncryptionConfiguration – KMS provider (Kubernetes 1.29+)
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources: [secrets]
  providers:
  - kms:
      apiVersion: v2
      name: aws-kms
      endpoint: unix:///var/run/kms-plugin/socket.sock # aws-
        encryption-provider
      cachesize: 1000
      timeout: 3s
  - aesgcm: # fallback – key in local file, weaker than KMS
  - name: fallback
    keys: [{ name: key1, secret: <base64-32-bytes> }]
```

Without this, anyone with etcd read or snapshot access reads every Secret in plaintext.

- **RBAC on Secrets.** No `get/list` on `Secrets` for broad roles. Audit `ClusterRole` bindings that include `secrets` :

```
kubectl auth can-i --list --as=system:serviceaccount:default:default |
grep secrets
# Any unexpected "yes" is a finding.
```

- **Cloud workload identity over static cloud keys.** No `AWS_ACCESS_KEY_ID` in Vault if you can use IRSA (EKS), Workload Identity (GKE), or Managed Identity (AKS) — Vault's `aws` secrets engine can vend STS credentials instead, or eliminate the secret entirely via OIDC federation (Ch 10).

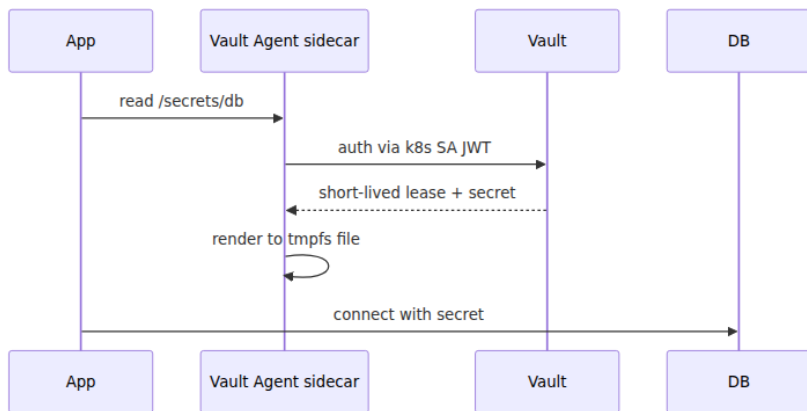
Distributed-systems lens

- **Secrets are a consistency and blast-radius problem.** A database password change must propagate to every replica's connection pool within a bounded window. Dynamic credentials with short leases make the window explicit (lease TTL); static secrets with long TTLs make it unbounded. Prefer the explicit window.
- **Cold start vs steady state.** At pod start, the secret must be available before the app becomes ready (readiness probe should fail if the secret file is absent). Warm, the secret refreshes on lease

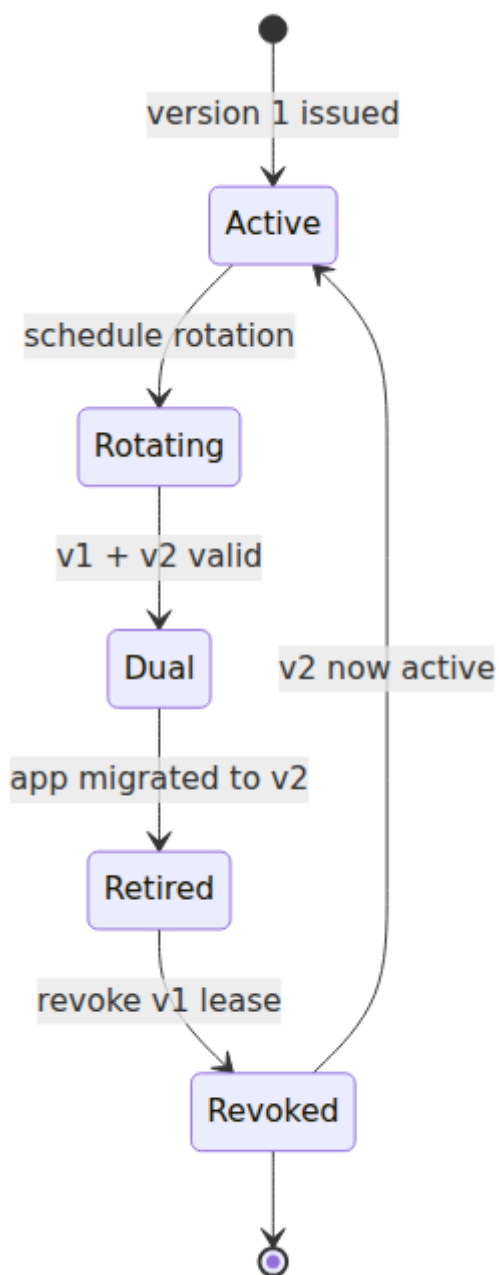
renewal. Design both paths — a pod that starts without a secret must not become `Ready` and serve unauthenticated fallback behavior.

- **Thundering herd on rotation.** A fleet-wide rotation that expires `N` secrets at the same second causes `N` simultaneous Vault reads. Jitter renewal ($\text{lease_duration}/2 \pm 10\%$) and stagger rotation (one shard at a time) to smooth the load.
- **Region affinity for latency.** A service in `eu-west-1` that fetches secrets from `us-east-1` Vault pays cross-region latency on every cold start and renewal. Performance replication or a regional Vault cluster is not optional for multi-region fleets.
- **Audit as a forensic requirement.** Every read, write, and lease revocation must be auditable (Vault audit device → SIEM). In a breach, "which workloads read this secret, when, from which IP, with which token?" is the first question. If you cannot answer it, the blast radius is "everything that could have."

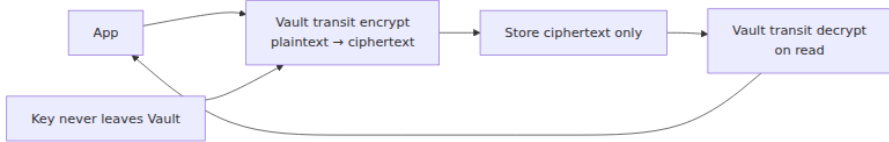
Secrets Retrieval with Sidecar



Secret Rotation Lifecycle



Transit Encryption



Key takeaways

- A secret's lifecycle is generate → store → distribute → use → rotate → revoke/audit. Optimizing only "store" (putting it in a vault) while leaving distribution as env vars and rotation as manual is the common failure mode.
- Envelope encryption (KEK in KMS/HSM wrapping a DEK that AEAD-encrypts storage) is the correct at-rest model — for Vault's barrier, for Transit, and for app-layer encryption. Auto-unseal via KMS removes the manual Shamir ceremony; keep Shamir as break-glass.
- Vault 1.15+ (or OpenBao) organizes the control plane as auth methods → policies (HCL, per-path) → secrets engines (KV v2, Transit, PKI, database) → leases. Bind policies to roles (k8s, JWT/OIDC, AppRole), use `batch` tokens for workloads (15 m TTL), and enforce `ttl / max_ttl` as the blast-radius bound.
- Prefer dynamic, short-lived credentials (database engine, PKI) over static secrets — the lease is the rotation. Where static secrets remain, automate rotation with dual-credential overlap, staggered rollout, and forced restarts; never rely on manual rotation.
- Distribute via file on `tmpfs` (Vault Agent sidecar/Injector, CSI Driver) rather than Kubernetes `Secret` / env vars. ESO is the migration path for legacy `Secret` consumers. In all cases, the app reads a file with `0400`, never logs the value, and fails readiness if the file is absent.
- Detect leaks with gitleaks 8.18+ and TruffleHog 3.70+ in pre-commit and CI, plus GitHub push protection. Remediation is always rotation first, history cleanup second — redaction without rotation leaves the secret valid in every clone and cache.
- Vault at scale is an HA Raft cluster (3/5 voters) with performance replication for multi-region reads, DR replication for failover, KMS auto-unseal, Agent caching for brief outages, and alerts on seal status, lease failures, secret age, and Raft health. Enable etcd/KMS

encryption at rest for Kubernetes Secrets and lock down RBAC on `get/list` secrets.

Further reading

- **Vault:** HashiCorp Vault 1.15+ docs — <https://developer.hashicorp.com/vault/docs> — auth methods, policies, audit, replication, Raft storage, seal. OpenBao docs — <https://openbao.org/docs/> — open-source Vault lineage. Vault Agent / Injector / CSI provider docs — <https://developer.hashicorp.com/vault/docs/platform/k8s>.
- **Kubernetes secrets plane:** *Kubernetes Secrets* (<https://kubernetes.io/docs/concepts/configuration/secret/>) and *Encrypting Secret Data at Rest* (<https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>). Secrets Store CSI Driver (<https://secrets-store-csi-driver.sigs.k8s.io/>) and Vault CSI Provider. External Secrets Operator (<https://external-secrets.io/latest/>) — `ExternalSecret` / `ClusterSecretStore`.
- **Cloud workload identity:** AWS IRSA (<https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html>), GCP Workload Identity (<https://cloud.google.com/kubernetes-engine/docs/how-to/workload-identity>), Azure Workload Identity — the path to eliminating static cloud keys entirely (see Ch 10 for SPIFFE generalization).
- **Detection:** gitleaks 8.18+ (<https://github.com/gitleaks/gitleaks>), TruffleHog 3.70+ (<https://github.com/trufflesecurity/trufflehog>), GitHub Secret Scanning & Push Protection (<https://docs.github.com/en/code-security/secret-scanning>).
- **Standards & papers:** NIST SP 800-57 Part 1 Rev. 5 — *Recommendation for Key Management* — key lifecycle and cryptoperiods. NIST SP 800-53 — controls for media protection and

key management. Envelope encryption pattern — see Google Cloud KMS / AWS KMS docs for KEK/DEK reference architectures.

Chapter 9 — Application Security: OWASP, Injection, SSRF, and Deserialization

What this chapter covers. Cryptography (Ch 1–4), authentication (Ch 5–6), authorization (Ch 7), and secrets (Ch 8) form the perimeter and the control plane. Application security is what happens *inside* the handler — where user-controlled input meets string interpolation, query builders, HTTP clients, template engines, and deserializers. A single unsanitized concatenation can become SQL injection; a single unchecked URL fetch can become Server-Side Request Forgery (SSRF) that reaches the cloud metadata service; a single `pickle.loads` on untrusted bytes can become remote code execution. These are not exotic bugs — they are the top entries on the OWASP Top 10 (2021, with 2023 API Top 10 and the emerging 2025 draft) year after year, and they dominate breach reports because they bypass every perimeter control at once. This chapter builds a systematic AppSec practice for backend engineers: input handling as a discipline, injection prevention by construction (parameterized queries, not escaping), SSRF defense in depth for cloud-native backends, deserialization and template-injection safety, and the surrounding controls — security headers, dependency and supply-chain risk, and verification (SAST/DAST/IAST, fuzzing) — that make the practice durable at fleet scale.

Learning goals — after this chapter you should be able to:

- Map OWASP Top 10 2021 and OWASP API Security Top 10 2023 to concrete backend risks — and explain where each risk traverses the Fleet/Vol 7–8 boundary vs where it lives inside a single handler.
- Prevent injection by construction — parameterized queries (SQL), safe query builders (NoSQL/LDAP/XPath), and output encoding (XSS) — and explain why escaping/blacklisting is the wrong primitive.

- Defend against SSRF in cloud-native backends — URL parsing pitfalls, allowlist-based egress, metadata-service hardening (IMDSv2), and per-request egress policy.
- Eliminate dangerous deserialization — `pickle / yaml.load / Java deserialization` as RCE — and safely handle untrusted structured input (JSON schema validation, strict codecs).
- Apply the remaining OWASP controls that matter for backends — authentication/authorization failures, security misconfiguration, vulnerable dependencies, and logging/monitoring — with version-pinned tooling.
- Operate verification — SAST, DAST, IAST, SCA, and fuzzing — in CI/CD without drowning in false positives, and measure AppSec posture with actionable metrics.

Boundary notes. *Cryptography and hashing* that underpin AppSec controls (CSRF tokens, signed URLs) are **Ch 1-3**; *session and token mechanics* (CSRF defense via SameSite/double-submit, JWT validation) are **Ch 5**; *delegation* (OAuth/OIDC) is **Ch 6**; *authorization* (IDOR/BOLA, access-control checks) is **Ch 7**; *secrets that AppSec must not leak* (keys, tokens in logs) are **Ch 8**; *mTLS and workload identity* that harden service-to-service calls (relevant for SSRF egress) are **Ch 10**; *threat modeling* the application as an attack surface is **Ch 11**. *Supply-chain depth* (SBOMs, SLSA, signing, CI/CD hardening) is the Companion Series (Books 1-6) — this chapter covers the *application* side of dependency risk (SCA, reachability), not the *pipeline* side. *Database isolation and RLS* are **Vol 5, Ch 5-6**; *API design* (idempotency, pagination, error handling) is **Vol 8**; *observability* (logging at scale, SIEM) is **Vol 11, Ch 3**.

The OWASP lens for backend engineers

Version pins. **OWASP Top 10 2021** (current stable as of 2025-2026; 2025 draft in public review) — the primary map used here. **OWASP API Security Top 10 2023** — the API-specific overlay (BOLA, BFLA, etc.) that matters for every backend that exposes REST/gRPC/GraphQL. **OWASP ASVS 4.0.3** — the verification standard for control depth. Tool versions pinned inline (Semgrep, CodeQL, ZAP, etc.) as of 2025-2026.

OWASP Top 10 2021 — the eight categories that dominate backend risk (two are frontend/broad and noted):

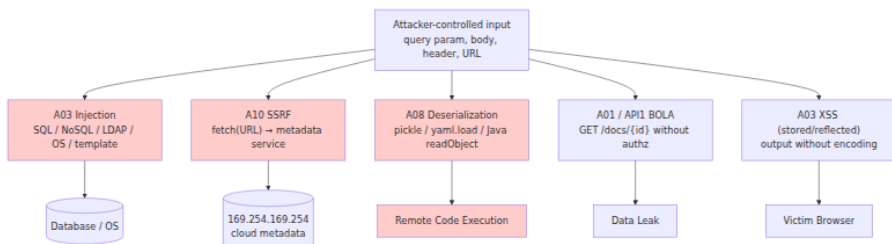
#	Category (2021)	Backend meaning	Where it lives
A01	Broken Access Control	Missing or bypassable authorization checks — Ch 7's problem, AppSec's most exploited entry	Every handler (Ch 7)
A02	Cryptographic Failures	Weak TLS, leaked secrets, insufficient encryption — Ch 3-4, Ch 8	Transport + storage
A03	Injection	SQL/NoSQL/LDAP/XPath/OS command, XSS, template injection	This chapter — § Injection
A04	Insecure Design	Missing threat model, no security requirements — Ch 11	Design phase
A05	Security Misconfiguration	Default creds, verbose errors, open S3 buckets, permissive CORS	This chapter — § Misconfig
A06	Vulnerable Components	Outdated libs with known CVEs — SCA/reachability	This chapter — § Dependencies
A07	Auth Failures	Broken session, credential stuffing, MFA gaps — Ch 5-6	AuthN (Ch 5-6)
A08	Software & Data Integrity Failures	Deserialization, unsigned updates — CI/CD + § Deserialization	Pipeline + this chapter
A09	Logging & Monitoring Failures	Insufficient detection, no alerting — Vol 11, Ch 3-5	Observability
A10	SSRF	Server fetches attacker-controlled URL — cloud metadata exfil	This chapter — § SSRF

OWASP API Security Top 10 2023 adds precision for backends:

- **API1 BOLA** (Broken Object Level Authorization) — `GET /v1/documents/{id}` without verifying the caller may access *that* `id` (IDOR). Ch 7 + this chapter.

- **API3 BFLA** (Broken Function Level Authorization) — unprotected admin/debug endpoints. Ch 7.
- **API8 Security Misconfiguration** — excessive CORS, missing rate limits (Vol 7, Ch 9), verbose GraphQL introspection.

The Top 10 is a *prioritization* tool, not a checklist. Use it to allocate review and testing effort: every handler gets injection + access-control review; every outbound fetch gets SSRF review; every deserialization boundary gets a codec review.



Injection: the failure to separate code from data

Every injection flaw has the same root cause: **user input is concatenated into a string that is later parsed as code** — SQL, shell, LDAP filter, XPath, template directive, or HTML. The fix is the same in every case: **never build code by string interpolation; use a parameterized API that keeps code and data on separate channels.**

Escaping and blocklisting are not fixes — they are filters that the next encoding bypasses. Parameterization is construction.

SQL injection

The canonical example — and still the most damaging at scale because it exfiltrates or mutates the entire database:

```
// □ VULNERABLE – Go 1.22, database/sql – string interpolation
func getUserVulnerable(db *sql.DB, username string) (*User, error) {
    // Attacker sends username = ` ' OR '1'='1' --` → returns all users
    // or `'; DROP TABLE users; --` → destructive
    query := fmt.Sprintf("SELECT id, email FROM users WHERE username = '%s'", username)
    row := db.QueryRow(query)
    var u User
    err := row.Scan(&u.ID, &u.Email)
    return &u, err
}

// □ SAFE – parameterized query (prepared statement), same driver
func getUserSafe(db *sql.DB, ctx context.Context, username string) (*User, error) {
    // Data travels as a bound parameter, never parsed as SQL.
    row := db.QueryRowContext(ctx,
        "SELECT id, email FROM users WHERE username = $1", // Postgres
        $1; MySQL ?, SQLite ?
        username,
    )
    var u User
    if err := row.Scan(&u.ID, &u.Email); err != nil {
        return nil, err
    }
    return &u, nil
}
```

```
# Python – psycopg 3.1+ (psycopg[binary]) – same principle
# □ VULNERABLE
cur.execute(f"SELECT id, email FROM users WHERE username = '{username}'")

# □ SAFE – server-side parameter binding
cur.execute("SELECT id, email FROM users WHERE username = %s", (username,))

# □ SAFE – SQLAlchemy 2.0+ – expression builder, no string concatenation
stmt = select(User).where(User.username == username) # bound parameter
```

This extends beyond `SELECT`:

- **Dynamic table/column names** cannot be parameterized — they are *identifiers*, not *values*. Allowlist them:

```

var allowedSortCols = map[string]bool{"created_at": true, "name":
true, "email": true}

func listUsersSafe(db *sql.DB, sortCol string) ([]User, error) {
    if !allowedSortCols[sortCol] {
        return nil, fmt.Errorf("invalid sort column: %q", sortCol)
    }
    // Identifier is allowlisted, not parameterized – safe after
validation.
    query := fmt.Sprintf("SELECT id, email FROM users ORDER BY %s", sor
tCol)
    rows, err := db.Query(query)
    // ...
}

```

- **ORMs are not automatically safe.** `User.objects.raw("SELECT ... %s" % username)` (Django) or `query("SELECT ... " + input)` (any string-concatenated ORM call) reintroduces injection. Use the ORM's parameterized API (`filter(username=...)`) and lint for `raw/extra/RawSQL`.
- **Second-order injection.** Data that was safely stored (via parameterized `INSERT`) is later concatenated into a new query without parameterization. The store does not sanitize — the *use* must parameterize.

NoSQL, LDAP, XPath, OS command

The same code-vs-data confusion appears wherever a backend builds a query or command from input:


```
// □ MongoDB injection – Node.js mongodb driver 6.x
// Attacker sends { "username": { "$ne": null }, "password": { "$ne": null } } as JSON body
db.collection("users").findOne({ username: req.body.username,
password: req.body.password })
// → matches any user when $ne is parsed as an operator

// □ SAFE – validate type, use strict equality, or use an ODM with schema
if (typeof req.body.username !== "string" || typeof req.body.password !== "string") {
    return res.status(400).send("invalid input");
}
db.collection("users").findOne({ username: req.body.username,
password: hash(req.body.password) })

// □ OS command injection – Go os/exec
cmd := exec.Command("sh", "-c", "convert "+inputPath+"
"+outputPath) // inputPath="; rm -rf /;"

// □ SAFE – no shell, args as separate elements, allowlisted paths
cmd := exec.Command("convert", validatedInputPath, validatedOutputPath)
// validatedInputPath must be allowlisted (e.g., regex ^/data/uploads/[a-z0-9_-]+\.(png$)
```

LDAP filter injection (* () \ \ \0), XPath (' or '1'='1), and template injection (Jinja2 {{ }}) follow the same rule — use the engine's parameterized/bound API, never string concatenation.

XSS as an injection at the output boundary

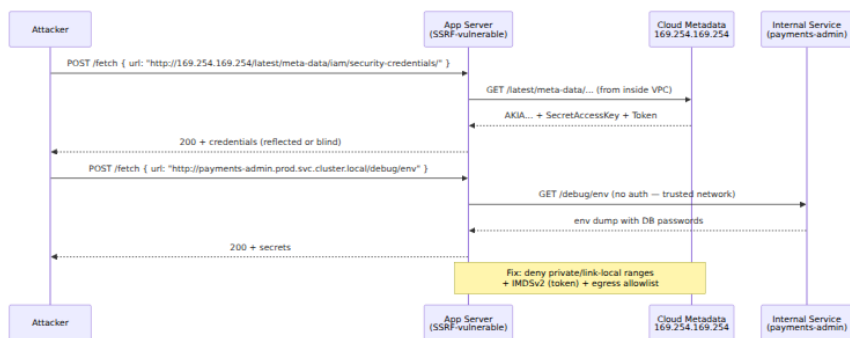
Cross-Site Scripting is injection where the *output* is parsed as code by the victim's browser. The backend fix is **context-aware output encoding** at every render boundary:

- **HTML body:** `html/template` (Go, auto-escapes), Jinja2 `autoescape`, React JSX (auto-escapes by default — `dangerouslySetInnerHTML` is the escape hatch that reintroduces the bug).
- **HTML attribute / URL / JS context:** different encoders — `url.QueryEscape`, `jsEncoder`. Encoding for the wrong context is still vulnerable.
- **Content Security Policy (CSP)** as defense in depth — not a substitute for encoding, but limits blast radius when encoding is missed. See § Security Headers below.

SSRF: the server as a confused deputy

SSRF (OWASP A10 2021, after climbing from the API Top 10) is the backend analogue of open redirect — the server fetches a URL that the attacker controls, and that fetch originates from *inside* the trust boundary. In a cloud-native backend, SSRF reaches:

- **Cloud metadata service** — `http://169.254.169.254/latest/meta-data/` (AWS IMDSv1), `http://metadata.google.internal/` (GCP), `http://169.254.169.254/metadata/` (Azure). With IMDSv1, a single SSRF leaks IAM credentials. IMDSv2 (token-required) mitigates but is not universally enforced.
- **Internal services** — `http://internal-payments.prod.svc.cluster.local/admin`, Redis, Elasticsearch, or any unauthenticated internal endpoint reachable from the fetching pod.
- **File scheme / gopher / dict** — `file:///etc/passwd`, `gopher://` (if the HTTP client supports it) — depending on client library.



Why naive validation fails

```
// □ BROKEN – blocklist of private IPs, checked before DNS – bypassed
// by:
// - DNS rebinding: attacker domain resolves to 1.2.3.4 (allowed) then
//   to 169.254.169.254 on fetch
// - 0xA9FEA9FE (hex), 2852039166 (decimal), http://0.0.0.0, http://
//   [::ffff:169.254.169.254]
// - Redirect: allowed URL 302-redirects to metadata service
// - URL parser mismatch: Go net/url vs HTTP client parse differently
func isAllowedBypassable(raw string) bool {
    u, _ := url.Parse(raw)
    host := u.Hostname()
    ip := net.ParseIP(host)
    if ip != nil && ip.IsPrivate() {
        return false
    }
    return true
}
```

Defense in depth for SSRF (cloud-native)

No single check is sufficient. Layer all of these:

1. Allowlist-based egress — the primary control.

If the feature is "fetch a webhook URL" or "fetch an image for processing," the set of legitimate destinations is small. Allowlist it.

```

// Go 1.22 - SSRF-safe fetch with allowlist + no redirects + private-IP
deny
var allowedHosts = map[string]bool{
    "hooks.example.com": true,
    "cdn.example.com":   true,
}

func fetchAllowlisted(ctx context.Context, rawURL string) ([]byte, error) {
    u, err := url.Parse(rawURL)
    if err != nil {
        return nil, err
    }
    if u.Scheme != "https" {
        return nil, errors.New("only https allowed")
    }
    if !allowedHosts[u.Hostname()] {
        return nil, errors.New("host not in allowlist")
    }
    // Resolve DNS ourselves and deny private/link-local/metadata
    ranges
    ips, err := net.DefaultResolver.LookupIP(ctx, "ip", u.Hostname())
    if err != nil {
        return nil, err
    }
    for _, ip := range ips {
        if isBlockedIP(ip) {
            return nil, fmt.Errorf("resolved IP %s is blocked", ip)
        }
    }
    // No redirects - attacker-controlled redirect bypasses the
    allowlist
    client := &http.Client{
        CheckRedirect: func(*http.Request, []*http.Request) error {
            return http.ErrUseLastResponse
        },
        Timeout: 5 * time.Second,
    }
    req, _ := http.NewRequestWithContext(ctx, "GET", u.String(), nil)
    resp, err := client.Do(req)
    if err != nil {
        return nil, err
    }
    defer resp.Body.Close()
    if resp.StatusCode >= 300 && resp.StatusCode < 400 {
        return nil, errors.New("redirects not followed")
    }
    return io.ReadAll(io.LimitReader(resp.Body, 5<<20)) // 5 MiB cap
}

```

```

func isBlockedIP(ip net.IP) bool {
    blocked := []net.IPNet{
        {IP: net.ParseIP("10.0.0.0"), Mask: net.CIDRMask(8, 32)},
        {IP: net.ParseIP("172.16.0.0"), Mask: net.CIDRMask(12, 32)},
        {IP: net.ParseIP("192.168.0.0"), Mask: net.CIDRMask(16, 32)},
        {IP: net.ParseIP("169.254.0.0"), Mask: net.CIDRMask(16, 32)}, /
    / link-local + metadata
        {IP: net.ParseIP("127.0.0.0"), Mask: net.CIDRMask(8, 32)},
        {IP: net.ParseIP "::1"), Mask: net.CIDRMask(128, 128)},
        {IP: net.ParseIP("fc00::"), Mask: net.CIDRMask(7, 128)}, //
    unique local
        {IP: net.ParseIP("fe80::"), Mask: net.CIDRMask(10, 128)}, //
    link-local v6
    }
    for _, n := range blocked {
        if n.Contains(ip) {
            return true
        }
    }
    return false
}

```

```

# Python – equivalent allowlist with ipaddress (stdlib)
import ipaddress, socket
from urllib.parse import urlparse

ALLOWED_HOSTS = {"hooks.example.com", "cdn.example.com"}
BLOCKED_NETS = [ipaddress.ip_network(cidr) for cidr in
    ["10.0.0.0/8", "172.16.0.0/12", "192.168.0.0/16", "169.254
    .0.0/16", "127.0.0.0/8"]]

def fetch_allowlisted(url: str) -> bytes:
    u = urlparse(url)
    if u.scheme != "https" or u.hostname not in ALLOWED_HOSTS:
        raise ValueError("URL not allowlisted")
    for _, _, _, _ (ip_str, *_ ) in socket.getaddrinfo(u.hostname, 443)
    :
        if any(ipaddress.ip_address(ip_str) in net for net in BLOCKED_N
    ETS):
        raise ValueError(f"resolved IP {ip_str} is blocked")
    # Use httpx with follow_redirects=False, timeout, and size cap
    import httpx
    with httpx.Client(follow_redirects=False, timeout=5.0) as c:
        r = c.get(url)
        if 300 <= r.status_code < 400:
            raise ValueError("redirects not followed")
        r.raise_for_status()
        return r.content[: 5 * 1024 * 1024]

```

2. IMDSv2 and metadata hardening (cloud control).

- AWS: enforce IMDSv2 (`HttpTokens: required` , `HttpPutResponseHopLimit: 1` — so a container cannot hop to the host's metadata). Prefer IMDSv2-only at the org SCP level. Use IRSA/ Workload Identity (Ch 10) so pods have no host-level IAM role to leak.
- GCP: `metadata concealment` is deprecated — use Workload Identity and disable legacy metadata endpoints.
- Azure: IMDS requires `Metadata: true` header — reject that header at the egress proxy if the app should never contact IMDS.

3. Egress proxy / service mesh policy.

Even with allowlists in code, enforce at the network layer — code allowlists have parser bugs, network policy does not:

```
# Kubernetes NetworkPolicy – default deny egress, allow only listed
hosts (via EgressGateway or Cilium)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-metadata-egress, namespace: prod }
spec:
  podSelector: { matchLabels: { app: payments } }
  policyTypes: [Egress]
  egress:
    - to: [{ namespaceSelector: { matchLabels: { name: prod } } }] #
      internal
      ports: [{ port: 443 }, { port: 5432 }]
    - to: [{ ipBlock: { cidr: 0.0.0.0/0, except: [169.254.0.0/16, 10.0.0.
0/8] } }]
      ports: [{ port: 443 }] # external only, no link-local
  ---
# Cilium CiliumNetworkPolicy – DNS-aware allowlist (Cilium 1.15+)
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata: { name: payments-egress-allowlist, namespace: prod }
spec:
  endpointSelector: { matchLabels: { app: payments } }
  egress:
    - toFQDNs: [{ matchName: hooks.example.com }, { matchName: cdn.exempl
e.com }]
      toPorts: [{ ports: [{ port: "443", protocol: TCP } ] }]
# No other egress – metadata IP never matches a FQDN
```

Deserialization and template injection

Deserialization of untrusted bytes into objects that execute code during construction is **remote code execution by design**. The fix is to never deserialize untrusted input with a codec that permits code execution — use a strict, schema-validated codec instead.

The danger list (do not use on untrusted input)

Language	Dangerous codec	What it can execute	Safe replacement
Python	<code>pickle.loads</code> , <code>marshal.loads</code> , <code>yaml.load</code> (without <code>SafeLoader</code>)	Arbitrary Python via <code>__reduce__</code>	<code>json.loads</code> + <code>jsonschema</code> , <code>yaml.safe_load</code> , <code>msgpack</code> with <code>strict</code>
Java	<code>ObjectInputStream.readObject</code> , <code>Jackson.enableDefaultTyping</code> , <code>SnakeYAML</code>	Gadget chains → RCE	Jackson without default typing + <code>allowlist</code> , <code>jackson-databind 2.15+</code> <code>BlockHound</code>
Node.js	<code>unserialize</code> (PHP-compat), <code>eval</code> , <code>vm.runInNewContext</code> on input	JS execution	<code>JSON.parse</code> + <code>zod / joi</code> validation
Ruby	<code>Marshal.load</code> , <code>YAML.load</code>	Object instantiation + method call	<code>JSON.parse</code> , <code>YAML.safe_load</code>
Go	<code>encoding/gob</code> on untrusted bytes (less common)	Type confusion	<code>encoding/json</code> + <code>go-playground/validator</code>

```

# ☐ RCE – Python pickle on untrusted input (e.g., from a queue, cache,
or cookie)
import pickle, base64
payload = base64.b64decode(user_input)
# attacker crafts a pickle that calls os.system
obj = pickle.loads(payload) # executes during
deserialization

# ☐ SAFE – JSON with schema validation (jsonschema 4.21+)
import json, jsonschema

schema = {
    "type": "object",
    "properties": {
        "action": {"type": "string", "enum": ["create", "update"]},
        "document_id": {"type": "string", "pattern": "^[a-z0-9-]
{8,64}$"},
        "title": {"type": "string", "maxLength": 200},
    },
    "required": ["action", "document_id"],
    "additionalProperties": False, # reject unexpected fields – fail-
closed
}
data = json.loads(user_input)
jsonschema.validate(data, schema) # raises ValidationError on mismatch

# YAML – safe variant
import yaml
data = yaml.safe_load(user_input) # never yaml.load without
Loader=yaml.SafeLoader

```



```
// □ VULNERABLE – Java Jackson with default typing (gadget chain via @class)
ObjectMapper mapper = new ObjectMapper();
mapper.enableDefaultTyping(); // attacker sends
{"@class": "com.sun.org.apache..."}
Object obj = mapper.readValue(untrustedJson, Object.class);

// □ SAFE – Jackson 2.15+ with BlockHound + explicit allowlist, no default typing
ObjectMapper safe = new ObjectMapper();
safe.activateDefaultTyping(
    BasicPolymorphicTypeValidator.builder()
        .allowIfBaseType(AllowedBase.class) // only subtypes of an allowlisted base
        .build(),
    ObjectMapper.DefaultTyping.NON_FINAL
);
// Or better: no polymorphic typing at all – deserialize to a concrete, validated DTO
DocumentRequest req = safe.readValue(untrustedJson, DocumentRequest.class);
validator.validate(req); // jakarta.validation – @NotNull, @Pattern, @Size
```

Template injection follows the same pattern — user input interpolated into a template that is then rendered as code:

```
# □ SSTI – Jinja2 (Flask) – user_input =
"{{ config.__class__.__init__.__globals__ }}"
from jinja2 import Template
Template("Hello " + user_input).render() # executes template directives

# □ SAFE – autoescape + no string concatenation into template source
from jinja2 import Environment, select_autoescape
env = Environment(autoescape=select_autoescape())
template = env.from_string("Hello {{ name }}") # name is a variable, not template source
template.render(name=user_input) # user_input is escaped, not executed
```

For Java (Thymeleaf, Freemarker) and Go (html/template vs text/template — html/template auto-escapes, text/template does not — always use html/template for HTML), the same rule holds: user input is a variable, never template source.

Security headers, misconfiguration, and the surrounding controls

Headers are not a primary defense, but they are a cheap, fleet-wide hardening layer that limits blast radius when a handler bug slips through:

```
# Envoy / nginx / API Gateway – security headers (all responses,
including errors)
# Envoy HttpConnectionManager.response_headers_to_add (Envoy 1.30+)
response_headers_to_add:
- header: { key: "Strict-Transport-Security", value: "max-
age=31536000; includeSubDomains; preload" }
- header: { key: "X-Content-Type-Options", value: "nosniff" }
- header: { key: "X-Frame-Options", value: "DENY" }
- header: { key: "Referrer-Policy", value: "no-referrer" }
- header: { key: "Permissions-Policy", value: "camera=(), microphone=()
, geolocation=()" }
- header: { key: "Content-Security-Policy", value: "default-src
'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-
ancestors 'none'" }
# CSP: start with report-only, then enforce – report-uri / report-to
for violation telemetry
- header: { key: "Content-Security-Policy-Report-Only", value: "default
-src 'self'; report-uri /csp-report" }
```

```
// Go 1.22 – security headers middleware (net/http)
func securityHeaders(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        h := w.Header()
        h.Set("Strict-Transport-Security", "max-age=31536000;
includeSubDomains")
        h.Set("X-Content-Type-Options", "nosniff")
        h.Set("X-Frame-Options", "DENY")
        h.Set("Referrer-Policy", "no-referrer")
        h.Set("Content-Security-Policy", "default-src 'self'; script-
src 'self'; object-src 'none'")
        h.Set("Permissions-Policy", "camera=(), microphone=(),
geolocation=()")
        // No Cache-Control here – per-handler (Ch 5: no-store on token
responses)
        next.ServeHTTP(w, r)
    })
}
```

Header	What it prevents	Notes
HSTS (Strict-Transport-Security)	SSL stripping, cookie over HTTP	<code>includeSubDomains</code> + preload; requires HTTPS everywhere (Ch 4)
CSP (Content-Security-Policy)	XSS payload execution, even if injection occurred	<code>default-src 'self'</code> baseline; <code>report-uri</code> for telemetry; nonce/hash for inline scripts
X-Content-Type-Options: nosniff	MIME-sniffing XSS	Cheap, always on
X-Frame-Options / CSP frame-ancestors	Clickjacking	<code>DENY</code> unless framing is intentional
Referrer-Policy	Token/URL leakage via <code>Referer</code>	<code>no-referrer</code> on OAuth callbacks (Ch 6)
Permissions-Policy	Unwanted browser features	Disable <code>camera / mic</code> unless needed

Misconfiguration (OWASP A05 / API8) is the rest of the hardening surface:

- **CORS** — `Access-Control-Allow-Origin: *` with `Allow-Credentials: true` is a data leak. Allowlist origins exactly; prefer `Vary: Origin`; never reflect `Origin` without validation. For non-browser backends, CORS is irrelevant — do not add it "just in case."
- **Verbose errors** — stack traces, SQL errors, and internal hostnames in `500` responses leak internals. Return `{"error": "internal_error", "request_id": "..."}` externally; log details with the request ID internally.
- **Unprotected debug endpoints** — `/debug/pprof`, `/metrics`, `/admin`, `/.env` must be behind mTLS/network policy, not just "undocumented." Scan your own fleet with `nmap / nuclei` for exposed endpoints.
- **Rate limiting** (Vol 7, Ch 9) — every auth and mutation endpoint needs it; without it, credential stuffing and enumeration bypass all other controls.

Vulnerable dependencies: SCA and the supply-chain edge

OWASP A06 (Vulnerable Components) and the Companion Series overlap here. For AppSec, the question is narrower: *which vulnerable dependencies are actually reachable and exploitable in your service?*

```
# SCA in CI – OSV-Scanner 1.8+ (google/osv-scanner), pinned to OSV DB
osv-scanner --recursive --format sarif --output osv.sarif ./...

# Reachability-aware – CodeQL / Semgrep supply-chain (filter to called code)
# Semgrep 1.60+ – supply-chain + reachability (semgrep supply-chain)
semgrep ci --supply-chain --sarif --output semgrep.sarif

# Dependency update automation – Renovate / Dependabot with grouping + auto-merge for patches
# + osv-scanner as a required check before auto-merge
```

- **Pin versions** — `requirements.txt` / `go.mod` / `package-lock.json` with hashes (Ch 1 of Companion). `npm ci`, `pip install --require-hashes`, `go mod download` with `GOSUMDB`.
- **Reachability, not just presence.** A CVE in a transitive dependency that your service never calls is lower priority than a CVE on the hot path. Tools that trace call graphs (CodeQL, Semgrep supply-chain) reduce alert fatigue by an order of magnitude — use them.
- **Container base images** (Companion Book 6) — distroless/minimal bases reduce the CVE surface before SCA even runs. Scan images with `grype` (Anchore, 0.80+) or `trivy` (Aqua, 0.55+) in CI and as an admission gate.

Verification: SAST, DAST, IAST, and fuzzing

No single verification method catches all AppSec bugs. Layer them by phase and cost:



Method	What it finds	When	False-positive rate
SAST (Semgrep, CodeQL, SonarQube)	Injection, hardcoded secrets, weak crypto, taint flow	CI, per PR	Medium — tune rules, use taint mode
SCA (osv-scanner, grype, trivy)	Known CVEs in dependencies	CI, nightly, admission	Low (with reachability)
Secrets (gitleaks, TruffleHog)	Leaked creds in diff	Pre-commit + CI	Low (verified-only)
DAST (ZAP, Nuclei)	Runtime misconfig, exposed endpoints, SSRF probes	Staging, nightly	Medium — needs authenticated crawl
Fuzzing (<code>go test -fuzz</code> , libFuzzer, Jazzer)	Parser bugs, deserialization, input validation	CI (minutes) + nightly (hours)	Very low — every finding is a crash
IAST (Contrast, Seeker)	Same as SAST but with runtime context	Staging (agent)	Low — but agent overhead

Tuning for signal, not noise:

- **SAST taint rules** — prefer taint-tracking rules (`source → sink`) over generic pattern matches. Semgrep `taint` mode and CodeQL `Security` suite are high-signal; generic `hotspot` rules are noisy — run them as warnings, not blocking checks.
- **DAST with auth** — crawl with a real session/JWT, not anonymous. An unauthenticated DAST scan misses every BOLA/IDOR behind auth (API1) — which is most of them.
- **Fuzz the parser boundary** — every handler that parses user input (JSON, XML, URL, query string) is a fuzz target:

```
// Go 1.22 – native fuzzing (go test -fuzz FuzzParseWebhookURL)
func FuzzParseWebhookURL(f *testing.F) {
    f.Add("https://hooks.example.com/callback")
    f.Fuzz(func(t *testing.T, raw string) {
        // Must not panic, must not allow SSRF – only allowlisted hosts
        data, err := fetchAllowlisted(context.Background(), raw)
        if err != nil {
            return // rejected input – expected for fuzz noise
        }
        _ = data
        // If fetchAllowlisted returns without error, raw must be
        allowlisted
        // and must not resolve to a blocked IP – assert that here
    })
}
```

```
go test -fuzz FuzzParseWebhookURL -fuzztime 60s ./internal/fetch
# Run nightly for 30m+ – every crash is a bug, no triage needed
```

WAF/RASP (ModSecurity, AWS WAF, Cloudflare, Google Cloud Armor) are **defense in depth**, not a fix. They buy time while a code fix ships, and they mitigate 0-days in dependencies — but they have bypasses, add latency, and create a false sense of coverage if used as the primary control. Deploy in `log / count` mode first, tune to your traffic, then enforce per-rule.

Distributed-systems lens

- **Input validation at the edge, authorization at the resource.**

Validate shape (schema, length, charset) at the API gateway / BFF (Vol 7, Ch 8) to reject malformed input before it fans out; enforce authorization (BOLA/BFLA) at the owning service (Ch 7) where the resource graph lives. Gateway validation without service-level authz is bypassable via service-to-service calls.

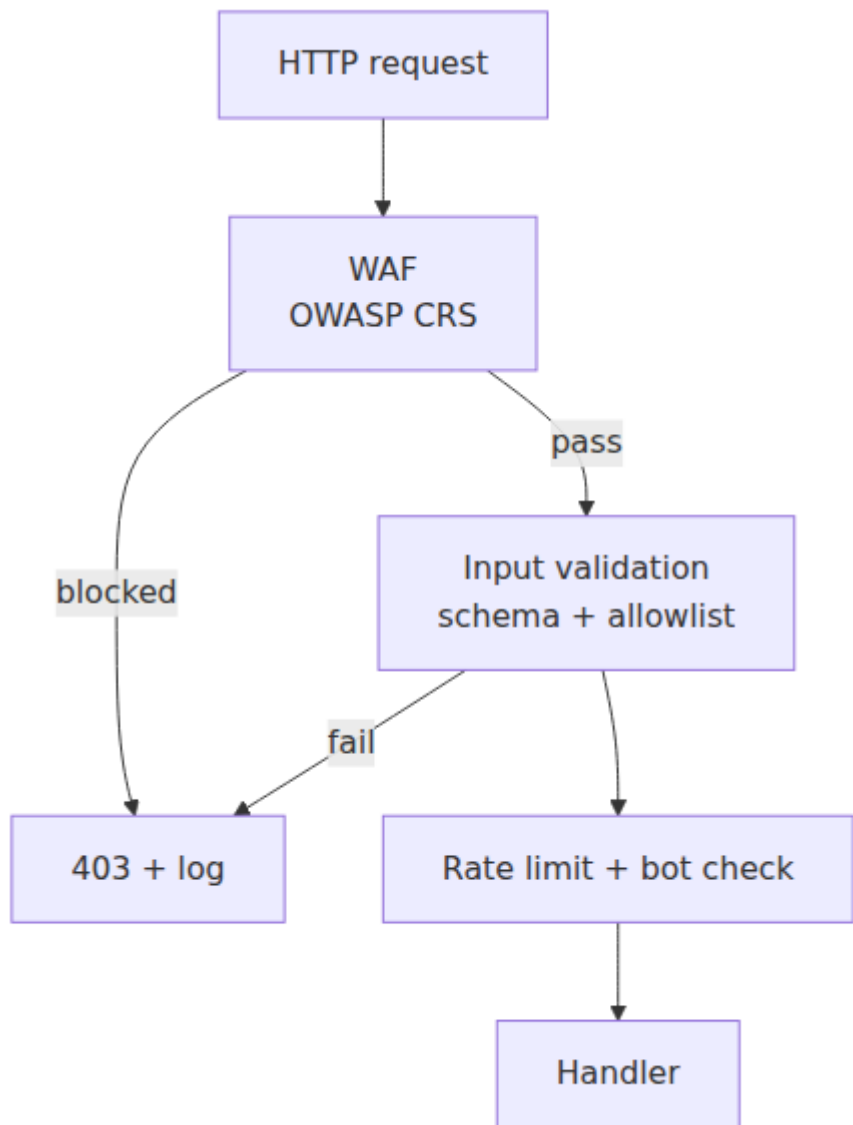
- **SSRF blast radius in a mesh.** In a flat network, one SSRF-vulnerable service can reach every internal endpoint.

Microsegmentation (Kubernetes `NetworkPolicy`, Cilium

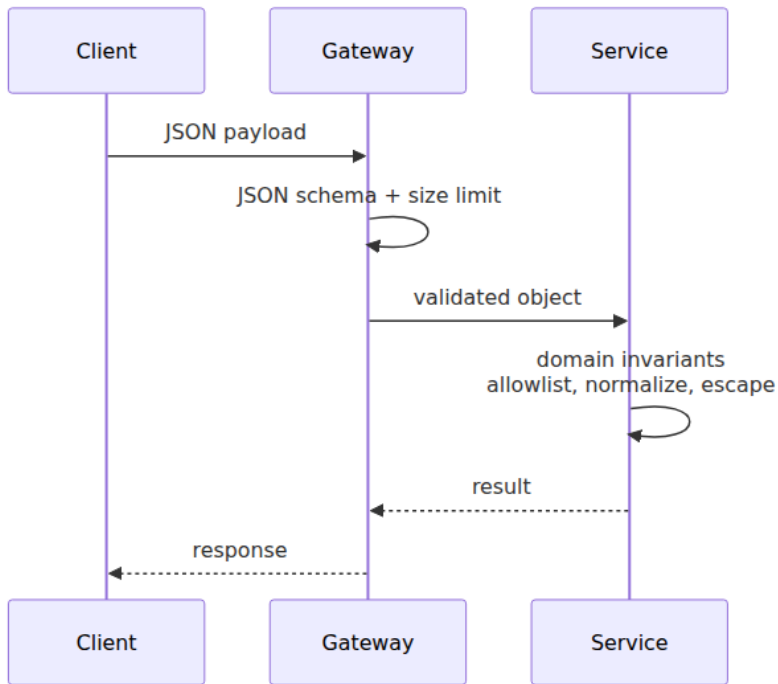
`CiliumNetworkPolicy`, mTLS with `AuthorizationPolicy` in Istio) limits SSRF to the compromised pod's network scope. mTLS alone does not fix SSRF — the fetch originates from a legitimate identity — but network policy does.

- **Consistency of security controls.** A WAF rule, a CSP header, or a CORS policy that is enforced in `us-east-1` but not in `eu-west-1` is a bypass. Distribute controls via the same config plane as routing (Envoy xDS, Gateway API) and verify parity with a cross-region probe.
- **Centralized logging for AppSec signals.** Injection probes (`' OR 1=1` , `{{7*7}}` , `169.254.169.254`), deserialization errors, and authz denials must flow to a SIEM (Vol 11, Ch 3) with alerts on rate spikes — a burst of `403` with `reason: BOLA` is an active enumeration, not background noise.
- **Dependency blast radius at scale.** One vulnerable transitive dependency shared across 200 services is a fleet-wide incident. Centralize SCA results in a dependency inventory (OSV + SBOM per Companion Book 3), prioritize by reachability, and roll out patches via the same progressive delivery as feature flags (Vol 11, Ch 9) — canary, then fleet.

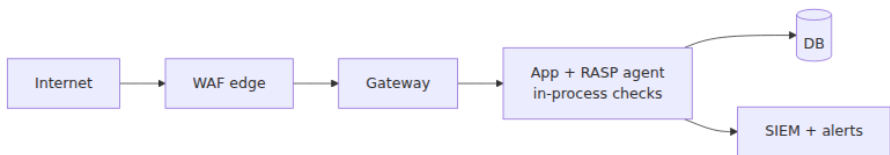
OWASP Request Filtering



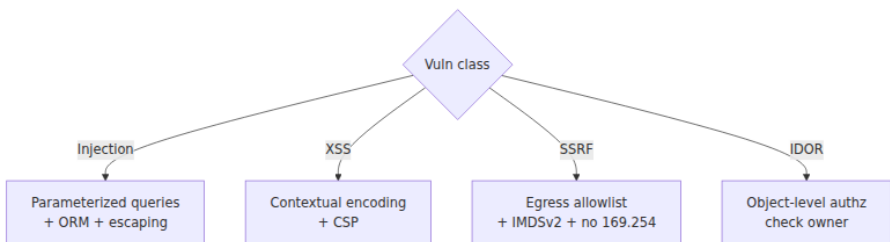
Input Validation Pipeline



WAF and RASP Architecture



Threat Mitigation Decision



Key takeaways

- OWASP Top 10 2021 (with API Top 10 2023 overlay) is the prioritization map: A01/BOLA (access control, Ch 7), A03 (injection), A10 (SSRF), and A08 (deserialization) are the handler-level risks this chapter owns. Every handler gets injection + authz review; every outbound fetch gets SSRF review; every deserialization boundary gets a codec review.
- Injection is code-vs-data confusion — the fix is parameterized APIs (SQL `$1 / ?`, MongoDB strict types, `exec.Command` without shell) and allowlisted identifiers for dynamic names. Escaping and blocklisting are not fixes. Second-order injection (stored data later concatenated) requires parameterization at the *use*, not just the store.
- SSRF in cloud-native backends requires defense in depth: allowlist-based egress (small host set), DNS resolution + private/link-local/metadata IP deny, no redirects, IMDSv2 (`HttpTokens: required`, `HopLimit: 1`), and network-layer enforcement (`NetworkPolicy / CiliumNetworkPolicy / mTLS AuthorizationPolicy`). Naive blocklists and parser-dependent checks are bypassable.
- Deserialization of untrusted bytes with `pickle / yaml.load / readObject` / template concatenation is RCE. Replace with strict codecs (`json + jsonschema`, `yaml.safe_load`, Jackson without default typing, `html/template` with variables) and `additionalProperties: false` / allowlisted DTOs. User input is always a variable, never template source.
- Security headers (HSTS, CSP, `X-Content-Type-Options`, `Referrer-Policy`) and misconfiguration hardening (exact CORS, no verbose errors, no exposed debug endpoints, rate limits) are cheap, fleet-wide blast-radius limiters — distribute via Envoy/Gateway and verify parity cross-region.
- Dependencies are AppSec — pin with hashes, scan with `osv-scanner / gype / trivy` in CI and admission, prioritize by reachability (CodeQL/Semgrep supply-chain), and roll out patches progressively. Container minimal bases reduce surface before scanning.
- Verification layers as SAST (Semgrep/CodeQL taint, CI), SCA + secrets (`osv-scanner`, `gitleaks/TruffleHog`), DAST (ZAP/Nuclei, authenticated crawl, staging), fuzzing (`go test -fuzz`, `libFuzzer/ Jazzer`, nightly), and WAF as defense in depth — tune for signal (taint

mode, verified-only, authenticated crawl) and measure via `authz_deny_rate`, `injection_probe_rate`, and `mean_time_to_patch_critical`.

Further reading

- **Standards:** OWASP Top 10 2021 (<https://owasp.org/Top10/>) and OWASP API Security Top 10 2023 (<https://owasp.org/API-Security/>) — read both; the API Top 10 is the backend overlay. OWASP ASVS 4.0.3 (<https://owasp.org/www-project-application-security-verification-standard/>) — control depth per level. OWASP Cheat Sheet Series (<https://cheatsheetseries.owasp.org/>) — injection, SSRF, deserialization, XSS prevention — concise, actionable.
- **Injection & XSS:** OWASP *SQL Injection Prevention Cheat Sheet* and *XSS Prevention Cheat Sheet*. Go `database/sql` docs (<https://pkg.go.dev/database/sql>) — prepared statements. `psycopg 3.1+` docs — parameter binding. MongoDB driver docs — BSON type safety.
- **SSRF:** OWASP *SSRF Prevention Cheat Sheet*. AWS IMDSv2 docs (<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html>) — `HttpTokens: required`. HackerOne SSRF reports — real bypasses (DNS rebinding, parser mismatch, redirect). Cilium NetworkPolicy docs (<https://docs.cilium.io/en/stable/security/policy/>) — DNS-aware egress.
- **Deserialization:** OWASP *Deserialization Cheat Sheet*. Python `pickle` docs — `__reduce__` warning. Jackson `deserialization` docs (<https://github.com/FasterXML/jackson-databind>) — BlockHound, polymorphic typing. `jsonschema 4.21+` (<https://python-jsonschema.readthedocs.io/>).
- **Verification:** Semgrep 1.60+ (<https://semgrep.dev/docs/>) — taint mode, supply-chain. CodeQL 2.17+ (<https://codeql.github.com/docs/>) — security queries. OWASP ZAP 2.15+ (<https://www.zaproxy.org/docs/>) — authenticated crawl. `go test -fuzz` (<https://go.dev/doc/fuzz/>) — native fuzzing. gitleaks 8.18+ / TruffleHog 3.70+ — secret scanning.
- **Headers & misconfig:** Mozilla Web Security (https://infosec.mozilla.org/guidelines/web_security) — HSTS, CSP, CORS. Envoy `response_headers_to_add` (<https://www.envoyproxy.io/docs/>)

envoy/latest/configuration/http/http_conn_man/headers) — fleet-wide header injection.

Chapter 10 — Zero Trust and Service-to-Service Authentication: mTLS and SPIFFE/SPIRE

What this chapter covers. Perimeter security assumes a hard outside and a soft inside — once traffic crosses the firewall, services trust each other implicitly because they are "inside" the network. Zero trust rejects that assumption: every service-to-service call is authenticated and authorized as if it traversed the public internet, regardless of where the caller sits in the VPC, the cluster, or the mesh. The network is not a trust boundary; identity is. This chapter builds that model from first principles to fleet reality. We start with why network-bound trust (VPC firewalls, perimeter TLS, shared secrets) fails at scale, define zero trust as workload identity plus mutual authentication plus least-privilege authorization, and then implement the two load-bearing pieces: mutual TLS (mTLS) — how TLS client authentication gives each workload a cryptographic identity — and SPIFFE/SPIRE — how a fleet issues, rotates, and federates short-lived X.509 SVIDs and JWT-SVIDs at scale without long-lived secrets. We show Envoy/Istio mTLS in depth (peer authentication, authorization policy, certificate rotation), raw Go/Java mTLS with `crypto/tls` and `javax.net.ssl`, SPIRE server/agent deployment with Kubernetes and `Workload API`, and federation across trust domains. Every pattern is anchored in runnable, version-pinned config and code and viewed through the distributed-systems lens where rotation without restarts, revocation without CRLs, and blast-radius isolation determine whether zero trust is a posture or just a diagram.

Learning goals — after this chapter you should be able to:

- Define zero trust for backend systems — workload identity as the trust anchor, mutual authentication on every hop, and authorization by identity (not by network) — and explain why perimeter/VPC trust breaks under lateral movement and single-service compromise.
- Establish mTLS correctly — TLS 1.2+ mutual authentication (`CertificateRequest` + `CertificateVerify`), peer-certificate

validation (`verify_peer` , `SAN / SPIFFE ID matching`), cipher/TLS-version pinning, and why server-only TLS is not zero trust.

- Operate SPIFFE/SPIRE — `SPIFFE ID ( spiffe://trust-domain/workload/... )` as the universal workload identity, `X509-SVID` and `JWT-SVID` issuance via the `Workload API` , attestation (Kubernetes `k8s_psat` , `k8s_sat` , `join_token` , `x509pop`), node vs workload attestation, registration entries, and automatic rotation.
- Deploy SPIRE on Kubernetes (SPIRE server with `k8s_psat` node attestation, agents as DaemonSets, `spire-agent` `Workload API` socket) and issue identities to pods without injecting long-lived secrets or restarting workloads on rotation.
- Configure service-mesh and raw mTLS enforcement: Istio `PeerAuthentication / AuthorizationPolicy` (or Cilium/Linkerd equivalents), Envoy `transport_socket + validation_context` with `SPIFFE` bundle, and Go/Java `tls.Config` with `GetCertificate / GetClientCertificate + VerifyPeerCertificate` pinning to a SPIFFE bundle.
- Federate trust domains (`spiffe://prod.example.com ↔ spiffe://partner.example.net`) via bundle exchange and JWKS/SPIFFE federation, handle CA rotation and bundle updates without downtime, and reason about revocation via short TTLs rather than CRLs/OCSP.
- Reason through the distributed-systems lens — certificate rotation without restarts, hot-reload of trust bundles, the blast radius of a compromised workload, audit and observability of identity (logging SPIFFE IDs, not IPs), and the interaction with secrets (Ch 8), authorization (Ch 7), and AppSec egress (Ch 9).

Boundary notes. *TLS/PKI mechanics* (handshake, cert chains, CA hierarchies) are **Ch 4 (Certificates, PKI, and TLS Operations)** — this chapter assumes them. *OAuth/OIDC and human/CI identity* are **Ch 6**; *authorization by workload identity* builds on **Ch 7 (RBAC/ABAC/ReBAC)** with SPIFFE ID as the principal. *Secrets that zero-trust replaces or reduces* are **Ch 8**. *AppSec egress that rides on mTLS* is **Ch 9 (SSRF)**, and *supply-chain workload identity* (keyless signing, Fulcio) is the **Companion Series, Book 5** — SPIFFE complements, not replaces, that layer.

Zero trust — the model and why perimeters fail

The perimeter assumption

The classic network assumes:

```
Internet —[ firewall ]— DMZ —[ firewall ]— private VPC — trusted services
```

Inside the VPC, services call each other over plaintext or server-only TLS, authenticate with a shared secret or a bearer token that any insider can copy, and authorize by "if you reached me, you must be allowed." Identity, where it exists, is the *network location* — source IP, security group, namespace, or the fact that the caller is inside the mesh.

This fails in three routine ways:

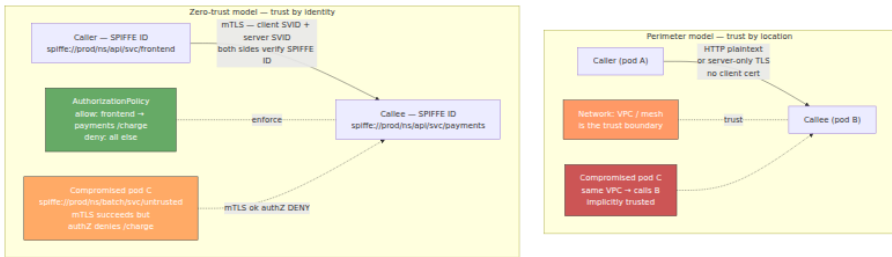
1. **Lateral movement.** One compromised pod (SSRF, deserialization, leaked secret — Ch 9) can call any other service because nothing authenticates the caller. The blast radius is the entire VPC.
2. **Insider and supply-chain.** A malicious or compromised dependency that runs inside the network is already "trusted" — the perimeter cannot distinguish it from a legitimate caller.
3. **Cloud and multi-cluster.** Workloads span VPCs, regions, clusters, and providers. There is no single perimeter to draw — and even if there were, east-west traffic already dominates north-south.

Zero trust collapses the perimeter to *each connection*: the network is assumed hostile, every caller proves its identity cryptographically, and every callee authorizes the caller by that identity — not by where the bytes arrived from.

The three tenets for backend systems

Tenet	Means in practice	Replaces
Workload identity	Every workload (pod, VM, job, function) has a stable, non-secret, cryptographically verifiable identity — the SPIFFE ID	IP, hostname, API key, shared secret

Tenet	Means in practice	Replaces
Mutual authentication	Every hop authenticates <i>both</i> sides with short-lived credentials bound to that identity (mTLS)	Server-only TLS + bearer token
Identity-based authorization	Every callee authorizes by the caller's identity (SPIFFE ID / JWT-SVID claim), not by network or token possession alone	Security-group / subnet / "inside the cluster"



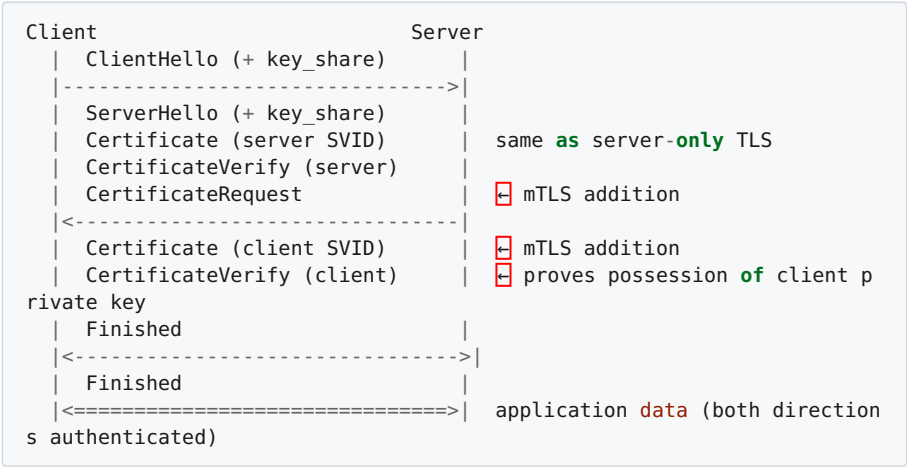
Two consequences matter for backend design:

- **Identity is not a secret.** A SPIFFE ID (`spiffe://trust-domain/...`) is public — like a username, not a password. Possession of the *SVID* (the X.509 cert bound to the private key) proves the identity; knowledge of the ID alone does not. This is why SPIFFE replaces API keys — the private key never leaves the workload's `Workload API` socket.
- **Trust is explicit and enumerable.** An `AuthorizationPolicy` that says "only `spiffe://prod/ns/api/svc/frontend` may call `POST /v1/charge` " is auditable — unlike a security-group rule that says "any pod in subnet `10.4.2.0/24` may call payments."

Mutual TLS — the authentication hop

What mTLS adds to TLS (and what Ch 4 already covered)

Vol 3, Ch 6 and Ch 4 of this volume cover the TLS 1.3 handshake, cipher suites, and PKI. mTLS changes one thing: the server requests and verifies a *client* certificate. The handshake becomes:



The two `CertificateVerify` messages each sign the transcript with the sender's private key — proving possession without ever transmitting the key. A middlebox that sees the wire learns the SPIFFE IDs (they are in the certs) but cannot impersonate either side.

Why server-only TLS is not zero trust. With server-only TLS, only the client verifies the server. A compromised client can still call the server — the server has no cryptographic reason to refuse. mTLS gives the *server* the same proof the client has.

What to validate — the checklist

Check	What it means	Failure mode if skipped
Peer cert chain validates to a trusted bundle	<code>verify_peer</code> against the SPIFFE trust bundle (or mesh CA bundle)	Any self-signed cert accepted
Leaf SAN is the expected SPIFFE ID (or URI SAN prefix)	<code>URI:spiffe://prod/ns/api/svc/frontend</code> exact match or <code>spiffe://prod/ns/api/*</code> pattern	Any workload in the trust domain can impersonate any other

Check	What it means	Failure mode if skipped
Cert is not expired and not yet valid	<code>NotBefore</code> ≤ now ≤ <code>NotAfter</code> ; short TTL (1h-24h) expected	Clock skew hides stale creds; long TTL widens compromise window
TLS version and cipher pinned	TLS 1.2+ , AEAD ciphers only (<code>TLS_AES_128_GCM_SHA256</code> , <code>TLS_CHACHA20_POLY1305_SHA256</code> , <code>TLS_ECDHE_*_WITH_AES*_GCM</code>)	Downgrade to weak ciphers
SNI / ALPN consistent (gRPC/HTTP2)	<code>h2</code> / <code>grpc</code> ALPN; SNI matches expected service if used	Protocol confusion
Rotation without restart	Hot-reload cert and bundle on <code>Workload API</code> update	Rotation requires pod restart → rollout or outage

Go — raw mTLS with SPIFFE bundle (no mesh)

```

// Go 1.22+ - mTLS client and server using a SPIFFE bundle as the trust
// anchor.
// In production, certs come from the Workload API (see SPIRE section
// below);
// this shows the tls.Config shape with hot-reload.

package mtls

import (
    "crypto/tls"
    "crypto/x509"
    "fmt"
    "net/http"
)

// newServerTLS returns a tls.Config that *requires* a client cert
// signed by the SPIFFE bundle
// and verifies the client's SPIFFE ID is in the allowed set.
func newServerTLS(cert tls.Certificate, bundle *x509.CertPool, allowedIDs
map[string]bool) *tls.Config {
    return &tls.Config{
        Certificates: []tls.Certificate{cert},
        ClientAuth:    tls.RequireAndVerifyClientCert,
        ClientCAs:    bundle, // SPIFFE bundle (trust domain root) -
// not the system pool
        MinVersion:    tls.VersionTLS12,
        VerifyPeerCertificate: func(rawCerts [][]byte, _ [][]*x509.Cert
ificate) error {
            if len(rawCerts) == 0 {
                return fmt.Errorf("no client certificate presented")
            }
            leaf, err := x509.ParseCertificate(rawCerts[0])
            if err != nil {
                return err
            }
            // SPIFFE ID is a URI SAN; Envoy/Go expose it as leaf.URIs
            if len(leaf.URIs) == 0 || leaf.URIs[0].Scheme != "spiffe" {
                return fmt.Errorf("client cert has no SPIFFE URI SAN")
            }
            id := leaf.URIs[0].String()
            if !allowedIDs[id] {
                return fmt.Errorf("SPIFFE ID %q not authorized", id)
            }
            return nil
        },
        GetCertificate: func(*tls.ClientHelloInfo) (*tls.Certificate, e
rror) {
            // In SPIRE deployments, fetch from Workload API here for
            // rotation without restart
            return &cert, nil
        },
    }
}

```

```

    },
}
}

// newClientTLS returns a tls.Config that presents a client cert and
// verifies the server's SPIFFE ID.
func newClientTLS(cert tls.Certificate, bundle *x509.CertPool, expected
ServerID string) *tls.Config {
    return &tls.Config{
        Certificates: []tls.Certificate{cert},
        RootCAs:      bundle,
        MinVersion:   tls.VersionTLS12,
        ServerName:   "payments.prod.example.com", // SNI – or leave
empty and verify in VerifyPeerCertificate
        VerifyPeerCertificate: func(rawCerts [][]byte, _ [][]*x509.Cert
ificate) error {
            if len(rawCerts) == 0 {
                return fmt.Errorf("no server certificate")
            }
            leaf, err := x509.ParseCertificate(rawCerts[0])
            if err != nil {
                return err
            }
            if len(leaf.URIs) == 0 || leaf.URIs[0].String() != expected
ServerID {
                return fmt.Errorf("server SPIFFE ID mismatch: got %v
want %q", leaf.URIs, expectedServerID)
            }
            // Also verify chain to bundle – Go does this automatically
when RootCAs is set
            // and VerifyPeerCertificate is nil; with a custom
verifier, do it explicitly:
            opts := x509.VerifyOptions{Roots: bundle, KeyUsages:
[]x509.ExtKeyUsage{x509.ExtKeyUsageServerAuth}}
            if _, err := leaf.Verify(opts); err != nil {
                return fmt.Errorf("server cert chain invalid: %w", err)
            }
            return nil
        },
    },
}

// Wiring – servers hot-reload certs by re-reading from the Workload
API or a file watcher.
// Clients reuse the same tls.Config with http.Transport or
grpc.WithTransportCredentials.
func Example() {
    // cert, bundle loaded from Workload API (see SPIRE section) or
files:
    // certPEM, keyPEM, bundlePEM := workloadAPI.GetX509SVID(ctx)
    // cert, _ := tls.X509KeyPair(certPEM, keyPEM)
}

```

```
// bundle := x509.NewCertPool();
bundle.AppendCertsFromPEM(bundlePEM)
_ = &http.Server{
    Addr:      ":8443",
    TLSConfig: newServerTLS(tls.Certificate{}, x509.NewCertPool(),
map[string]bool{"spiffe://prod/ns/api/svc/frontend": true}),
}
}
```

Production note. Above, `GetCertificate` / `GetClientCertificate` and a bundle watcher replace `cert` and `ClientCAs` / `RootCAs` without restarting the process. Libraries like `go-spiffe/v2` and `java-spiffe` do this by implementing `tls.Config.GetCertificate` against the `Workload API` stream — see the `SPIRE Workload API` section below. Never read `cert.pem` once at startup.

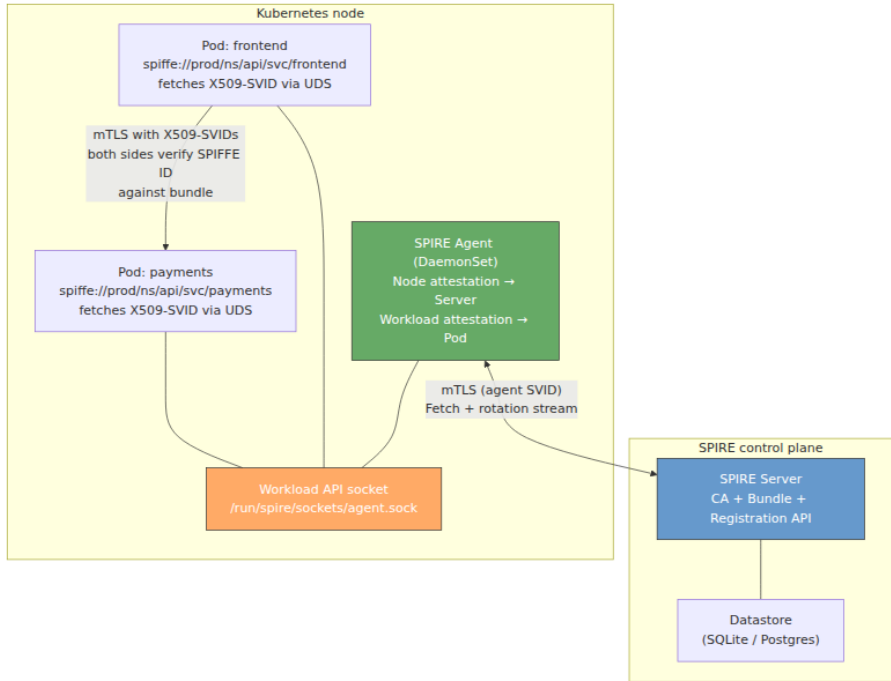
SPIFFE and SPIRE — workload identity at fleet scale

Shared secrets and long-lived certs do not scale — they leak, they require rotation ceremonies that need restarts, and they give every workload the same credential shape whether it is a deployment, a job, or a third-party integration. SPIFFE (Secure Production Identity Framework for Everyone, CNCF graduated) defines a *universal* workload identity and SPIRE (SPIFFE Runtime Environment) is its fleet-scale issuer.

The primitives

Primitive	What it is	Example
SPIFFE ID	URI <code>spiffe://trust-domain/path</code> — the identity, not the credential	<code>spiffe://prod.example.com/ns/payments/sa/payments-api</code>
Trust domain	The authority namespace — one per organization/environment that owns a root CA	<code>prod.example.com</code> , <code>staging.example.com</code>
X509-SVID	Short-lived X.509 cert (TTL 1h-24h) with SPIFFE ID in URI SAN + private key	What mTLS actually uses

Primitive	What it is	Example
JWT-SVID	Short-lived JWT (<code>aud</code> , <code>sub=SPIFFE ID</code> , <code>exp</code> in minutes) via <code>Workload API</code>	For non-mTLS hops (HTTP <code>Authorization: Bearer <JWT-SVID></code>)
Bundle	Trust domain's root CA(s) + JWT signing keys — what verifiers pin	<code>spiffe://prod.example.com bundle</code>
Workload API	Unix-domain-socket gRPC API (<code>/run/spire/sockets/agent.sock</code>) that workloads call to fetch/rotate SVIDs	<code>FetchX509SVID</code> , <code>FetchJWTsVID</code> , <code>FetchBundle</code>
Attestation	How SPIRE proves a workload is the SPIFFE ID it claims — node + workload attestors	<code>k8s_psat</code> , <code>k8s_sat</code> , <code>join_token</code> , <code>x509pop</code> , <code>aws_iid</code>



Node attestation vs workload attestation

SPIRE verifies identity in two stages:

1. **Node attestation** — the Agent proves the *node* to the Server. On Kubernetes, `k8s_psat` (Projected ServiceAccount Token) is the modern default — the Agent presents a projected SAT bound to the node's service account; the Server validates it against the Kubernetes API. Older `k8s_sat` and cloud attestors (`aws_iid`, `gcp_iit`, `azure_msi`) work the same way — the node proves it is what it claims.
2. **Workload attestation** — the Agent proves a *pod/process* to itself before handing over an SVID. Selectors like `k8s:ns:payments`, `k8s:sa:payments-api`, `k8s:pod-label:app=payments`, `unix:uid:1000` are matched against registration entries. Only if the pod's attested selectors match an entry does the Agent issue that entry's SPIFFE ID.

A pod cannot claim an arbitrary SPIFFE ID — it only receives the IDs whose selectors it satisfies.

SPIRE on Kubernetes — production config (SPIRE 1.9+)


```
# spire-server – StatefulSet (excerpt, SPIRE 1.9.x, Kubernetes 1.29+)
# Full chart: spire-helm / spiffe/spire; this is the minimal Server
config.
```

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: spire-server
```

```
  namespace: spire
```

```
data:
```

```
  server.conf: |
```

```
    server {
```

```
      bind_address = "0.0.0.0"
```

```
      bind_port = "8081"
```

```
      trust_domain = "prod.example.com"
```

```
      data_dir = "/run/spire/data"
```

```
      log_level = "INFO"
```

```
      ca_key_type = "ec-p256"
```

```
      ca_ttl = "24h"
```

```
      default_x509_svid_ttl = "1h"
```

```
      default_jwt_svid_ttl = "5m"
```

```
    }
```

```
    plugins {
```

```
      DataStore "sql" {
```

```
        plugin_data {
```

```
          database_type = "postgres"
```

```
          connection_string = "postgres://
```

```
spire:__VAULT_SECRET__@postgres.spire.svc.cluster.local/spire?
```

```
sslmode=require"
```

```
        }
```

```
      }
```

```
      NodeAttestor "k8s_psat" {
```

```
        plugin_data {
```

```
          clusters = {
```

```
            prod = {
```

```
              service_account_allow_list = ["spire:spire-agent"]
```

```
              audience = ["spire-server"] # must match the agent's
```

```
projected SAT audience
```

```
            }
```

```
          }
```

```
        }
```

```
      }
```

```
      KeyManager "disk" {
```

```
        plugin_data { keys_path = "/run/spire/data/keys.json" }
```

```
      }
```

```
      Notifier "k8sbundle" {
```

```
        plugin_data { namespace = "spire" } # publishes bundle as
```

```
ConfigMap spire-bundle
```

```
      }
```

```
    }
```

```
    health_checker {
```

```

    registration_uds_path = "/run/spire/sockets/server.sock"
  }
---
# spire-agent - DaemonSet (one per node)
apiVersion: v1
kind: ConfigMap
metadata:
  name: spire-agent
  namespace: spire
data:
  agent.conf: |
    agent {
      data_dir = "/run/spire/data"
      log_level = "INFO"
      server_address = "spire-server.spire.svc.cluster.local"
      server_port = "8081"
      trust_domain = "prod.example.com"
      trust_bundle_path = "/run/spire/bundle/bundle.crt" # from spire-
bundle ConfigMap
    }
    plugins {
      NodeAttestor "k8s_psat" {
        plugin_data {
          cluster = "prod"
          audience = ["spire-server"]
        }
      }
      WorkloadAttestor "k8s" {
        plugin_data {
          # uses kubelet /pods API; no extra config
        }
      }
      KeyManager "disk" {
        plugin_data { directory = "/run/spire/data" }
      }
    }
  }
---
# Registration entries - who gets which SPIFFE ID (SPIRE CRD / server
API)
# Prefer the Kubernetes CRD (spire-controller-manager) over CLI for
GitOps.
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: payments-api
spec:
  spiffeIDTemplate: "spiffe://prod.example.com/ns/
{{ .PodMeta.Namespace }}/sa/{{ .PodSpec.ServiceAccountName }}"
  podSelector:
    matchLabels:
      app: payments

```

workloadSelectorTemplates:

- "k8s:ns:payments"
- "k8s:sa:payments-api"
- "k8s:pod-label:app=payments"

```
# Alternative – server CLI (for non-CRD installs):  
# spire-server entry create \  
#   -spiffeID spiffe://prod.example.com/ns/payments/sa/payments-api \  
#   -parentID spiffe://prod.example.com/spire/agent/k8s_psat/prod/$  
#   (uuid) \  
#   -selector k8s:ns:payments -selector k8s:sa:payments-api -selector  
#   k8s:pod-label:app=payments \  
#   -x509SVIDTTL 3600
```

```
# Workload pod – no secret injection; just mount the Workload API  
socket  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: payments  
  namespace: payments  
spec:  
  template:  
    metadata:  
      labels: { app: payments }  
    spec:  
      serviceAccountName: payments-api  
      containers:  
        - name: payments  
          image: ghcr.io/acme/payments:1.4.2  
          volumeMounts:  
            - name: spire-agent-socket  
              mountPath: /run/spire/sockets  
              readOnly: true  
          env:  
            - name: SPIFFE_ENDPOINT_SOCKET  
              value: unix:///run/spire/sockets/agent.sock  
          volumes:  
            - name: spire-agent-socket  
              csi:  
                driver: csi.spiffe.io  # SPIFFE CSI driver – mounts the  
agent socket  
                readOnly: true
```

Workload API — fetching and rotating SVIDs

Workloads never write `cert.pem` to disk — they stream SVIDs over the Workload API Unix socket. Two idiomatic paths:

```

// Go - fetch X509-SVID via Workload API (github.com/spiffe/go-spiffe/
v2 2.3+, SPIRE 1.9+)
package main

import (
    "context"
    "crypto/tls"
    "log"
    "net/http"

    "github.com/spiffe/go-spiffe/v2/workloadapi"
    "github.com/spiffe/go-spiffe/v2/spiffetls/tlsconfig"
    "github.com/zeebo/errs"
)

func main() {
    ctx := context.Background()

    // X509Source watches the Workload API and hot-reloads certs +
    bundle.
    // No restart on rotation - the source pushes updates.
    source, err := workloadapi.NewX509Source(ctx, workloadapi.WithClien
tOptions(
        workloadapi.WithAddr("unix:///run/spire/sockets/agent.sock"),
    ))
    if err != nil {
        log.Fatalf("x509 source: %v", err)
    }
    defer source.Close()

    // Server: require any client in the trust domain (authZ refines
    this - see mesh/AuthZ below)
    tlsConfig := tlsconfig.MTLSServerConfig(source, source,
        tlsconfig.AuthorizeAny()) // or
    AuthorizeMemberOf("prod.example.com")

    // Client: present our SVID and verify the server's SPIFFE ID
    clientTLS := tlsconfig.MTLSClientConfig(source, source,
        tlsconfig.AuthorizeMemberOf("prod.example.com"))

    _ = &http.Server{
        Addr:      ":8443",
        TLSConfig: tlsConfig,
    }
    _ = &http.Client{Transport: &http.Transport{TLSClientConfig: client
    TLS}}

    // JWT-SVID for non-mTLS hops (aud is the intended audience):
    // jwtSource, _ := workloadapi.NewJWTSource(ctx, ...)
    // jwt, _ := jwtSource.FetchJWTsvid(ctx, jwtSvid.Params{Audience:

```

```

"payments"}})
    // req.Header.Set("Authorization", "Bearer "+jwt.Marshal())
    _ = errs.Wrap
}

```

```

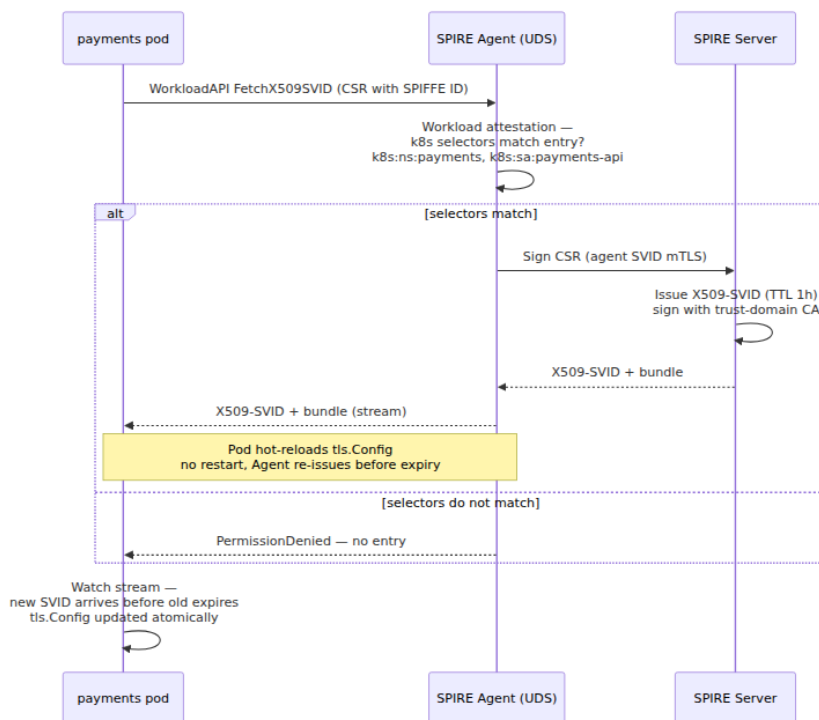
// Java – same shape (java-spiffe 0.8+, SPIRE 1.9+)
import io.spiffe.workloadapi.X509Source;
import io.spiffe.workloadapi.X509SourceConfig;
import io.spiffe.provider.SpiffeSslContext;
import javax.net.ssl.SSLContext;

X509Source source = X509Source.newSource(
    X509SourceConfig.newBuilder()
        .setWorkloadApiSocket("unix:///run/spire/sockets/agent.sock")
        .build());

SSLContext serverCtx = SpiffeSslContext.newBuilder()
    .withX509Source(source)
    .buildServerSSLContext(); // hot-reloads; no restart

SSLContext clientCtx = SpiffeSslContext.newBuilder()
    .withX509Source(source)
    .buildClientSSLContext("spiffe://prod.example.com/ns/payments/sa/
payments-api");
// For JWT-SVID: JwtSource + JwtSvid.getToken()

```



Enforcing — mesh policy and raw authZ

Identity is only useful if the callee enforces it. Two layers: mesh-level policy (Istio/Envoy/Cilium/Linkerd) for fleet-wide enforcement without code changes, and application-level SPIFFE ID checks for defense in depth.

Istio — PeerAuthentication + AuthorizationPolicy (Istio 1.22+, SPIRE as CA)

Istio's control plane (`istiod`) can consume SPIRE-issued SVIDs when SPIRE is the mesh CA (plug-in CA) or when `istiod` federates with SPIRE via `istio-spires` / `MCP`. The policy is the same shape:

```

# Enforce mTLS STRICT on payments – no plaintext allowed
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: payments-strict-mtls
  namespace: payments
spec:
  selector:
    matchLabels: { app: payments }
  mtls:
    mode: STRICT # PERMISSIVE during migration; STRICT after all
callers have SVIDs
---
# Allow only frontend → payments on POST /v1/charge
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: payments-allow-frontend-charge
  namespace: payments
spec:
  selector:
    matchLabels: { app: payments }
  action: ALLOW
  rules:
    - from:
      - source:
          principals: ["cluster.local/ns/api/sa/frontend"] # Istio
SPIFFE form; with SPIRE: spiffe://prod.example.com/ns/api/sa/frontend
        to:
      - operation:
          methods: ["POST"]
          paths: ["/v1/charge"]
      # Optional: also allow SRE break-glass identity with audit
    - from:
      - source:
          principals: ["spiffe://prod.example.com/ns/sre/sa/break-
glass"]
        to:
      - operation:
          methods: ["GET"]
          paths: ["/v1/orders/*"]
---
# During migration – log but don't yet deny non-mTLS
# AuthorizationPolicy with action: AUDIT / CUSTOM (Envoy ext_authz) for
canary

```

With SPIRE as the root, `principals` are full SPIFFE IDs (`spiffe://prod.example.com/...`). The Envoy sidecar validates the peer cert against

the SPIFFE bundle and extracts the SPIFFE ID for `source.principal` matching — no IP rules.

Envoy — raw mTLS with SPIFFE bundle (no full mesh)

When you do not run a mesh control plane, Envoy as a sidecar or gateway can still enforce mTLS with a SPIFFE bundle:


```

# Envoy listener – require client cert, verify SPIFFE URI SAN, route by
SPIFFE ID
# Envoy 1.31+, SPIRE bundle at /run/spire/bundle/bundle.crt (rotated by
SPIRE agent)
static_resources:
  listeners:
    - name: payments_listener
      address: { socket_address: { address: 0.0.0.0, port_value:
8443 } }
      filter_chains:
        - transport_socket:
            name: envoy.transport_sockets.tls
            typed_config:
              "@type": type.googleapis.com/
envoy.extensions.transport_sockets.tls.v3.DownstreamTlsContext
              require_client_certificate: true
              common_tls_context:
                tls_certificate_sds_secret_configs:
                  - name: "spiffe://prod.example.com/ns/payments/sa/
payments-api"
                    sds_config:
                      path_config_source: { path: "/run/spire/sockets/
agent.sock" } # SDS via Workload API
                validation_context_sds_secret_config:
                  name: "spiffe://prod.example.com"
                sds_config:
                  path_config_source: { path: "/run/spire/sockets/
agent.sock" }
                # Enforce TLS 1.2+, AEAD ciphers
                tls_params:
                  tls_minimum_protocol_version: TLSv1_2
                  cipher_suites: ["ECDHE-ECDSA-AES128-GCM-SHA256", "ECD
HE-RSA-AES128-GCM-SHA256"]
              filters:
                - name: envoy.filters.network.http_connection_manager
                  typed_config:
                    "@type": type.googleapis.com/
envoy.extensions.filters.network.http_connection_manager.v3.HttpConnect
ionManager
                    stat_prefix: payments
                    route_config:
                      name: payments_routes
                      virtual_hosts:
                        - name: payments
                          domains: ["*"]
                          routes:
                            # RBAC by SPIFFE ID – only frontend may POST /
v1/charge
                            - match: { prefix: "/v1/charge", headers: [{
name: ":method", exact_match: "POST" }] }

```

```

        route: { cluster: payments_local }
        typed_per_filter_config:
          envoy.filters.http.rbac:
            "@type": type.googleapis.com/
envoy.config.rbac.v3.RBACPerRoute
            rules:
              action: ALLOW
              policies:
                frontend:
                  principals:
                    - authenticated: {
principal_name: { exact: "spiffe://prod.example.com/ns/api/svc/
frontend" } }

                  permissions:
                    - any: true

            http_filters:
              - name: envoy.filters.http.rbac
                typed_config:
                  "@type": type.googleapis.com/
envoy.extensions.filters.http.rbac.v3.RBAC
                  rules: {} # per-route RBAC above
              - name: envoy.filters.http.router

```

Application-level — defense in depth behind the mesh

Mesh policy covers the fleet; application code still checks the SPIFFE ID it sees — so a misconfigured mesh does not silently grant access:

```

// Go – authorize by SPIFFE ID inside the handler (defense in depth)
func (h *Handler) Charge(w http.ResponseWriter, r *http.Request) {
    // tls.ConnectionState.PeerCertificates[0].URIs[0] is the caller
    SPIFFE ID
    // With go-spiffe, use spiffetls handshaker; with raw tls, parse
    leaf.URIs
    peerID := spiffeIDFromContext(r.Context()) // from
    tls.ConnectionState or gRPC peer.AuthInfo
    if peerID != "spiffe://prod.example.com/ns/api/svc/frontend" {
        http.Error(w, "forbidden: caller not authorized for /v1/
charge", http.StatusForbidden)
        return
    }
    // ... handle charge
}

// gRPC interceptor – same check for gRPC
func spiffeAuthInterceptor(allowed map[string]bool) grpc.UnaryServerInt
erceptor {
    return func(ctx context.Context, req any, _ *grpc.UnaryServerInfo,
handler grpc.UnaryHandler) (any, error) {
        p, ok := peer.FromContext(ctx)
        if !ok {
            return nil, status.Error(codes.Unauthenticated, "no peer")
        }
        tlsInfo, ok := p.AuthInfo.(credentials.TLSInfo)
        if !ok || len(tlsInfo.State.PeerCertificates) == 0 {
            return nil, status.Error(codes.Unauthenticated, "no client
certificate")
        }
        leaf := tlsInfo.State.PeerCertificates[0]
        if len(leaf.URIs) == 0 {
            return nil, status.Error(codes.Unauthenticated, "no SPIFFE
URI SAN")
        }
        id := leaf.URIs[0].String()
        if !allowed[id] {
            return nil, status.Errorf(codes.PermissionDenied, "SPIFFE
ID %q not authorized", id)
        }
        return handler(ctx, req)
    }
}

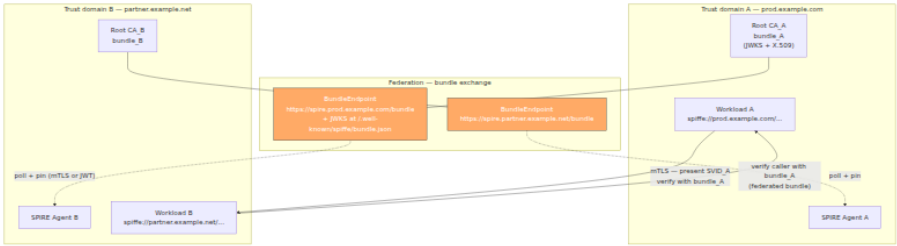
```

Federation — trust across domains

A single trust domain (`prod.example.com`) is a single root CA — if it is compromised, everything is. Federation lets two domains authenticate each other without sharing a CA: each domain publishes its bundle, the other imports it, and workloads in domain A can verify SVIDs from domain B.

Domain A: `spiffe://prod.example.com/ns/api/svc/frontend`
Domain B: `spiffe://partner.example.net/ns/fulfillment/svc/shipper`

Federation: A imports B's bundle; B imports A's bundle
Frontend (A) ⇄ Shipper (B): mTLS ⇄ shipper verifies frontend's SVID against A's bundle
Shipper ⇄ Frontend: reverse ⇄ frontend verifies shipper's SVID against B's bundle



```

# SPIRE server – federation config (SPIRE 1.9+)
server {
  trust_domain = "prod.example.com"
  federation {
    bundle_endpoint {
      address = "0.0.0.0"
      port = "8443"
      acme { domain_name = "spire.prod.example.com" }
    }
  }
  # TLS for bundle endpoint
  federates_with "partner.example.net" {
    bundle_endpoint_url = "https://spire.partner.example.net:8443"
    bundle_endpoint_profile "https_spiffe" {
      endpoint_spiffe_id = "spiffe://partner.example.net/spire/"
    }
  }
}

# Agent – fetches federated bundles for its trust domain
agent {
  trust_domain = "prod.example.com"
  # federated bundles appear alongside the local bundle via Workload
  API FetchBundle
}

```

Verification explicitly pins the federated bundle to a SPIFFE ID (`endpoint_spiffe_id`) or a CA — never `InsecureSkipVerify` . Rotation of a federated bundle is the same hot-reload path as the local bundle: the Agent watches `BundleEndpoint` and pushes updates to workloads via the Workload API stream.

JWT-SVID federation. JWT-SVIDs use the same bundle exchange but carry the federated `iss` (`https://spire.prod.example.com`) and a `kid` that maps to the federated bundle's JWKS. Verifiers must check `iss` against the expected trust domain before accepting `sub` .

Rotation, revocation, and the distributed-systems lens

Rotation without restarts

Workload certs rotate *before* they expire — typically at $\frac{1}{3}$ to $\frac{1}{2}$ of the TTL. With SPIRE, the Agent pushes a new SVID over the existing Workload API stream; the workload's `X509Source` swaps `tls.Config.Certificates` atomically; established connections continue with the old cert until they close, new handshakes use the new one. No pod restart, no rolling deploy for rotation.

The CA rotation is longer-lived (24h–7d for the signing CA, months for the root) and uses **bundle updates**: the new root is added to the bundle alongside the old, both are distributed, workloads trust either during the overlap window, then the old is removed. The same overlap handles a compromised CA — publish a new root, distribute, then stop signing with the old.

Revocation via short TTLs, not CRLs

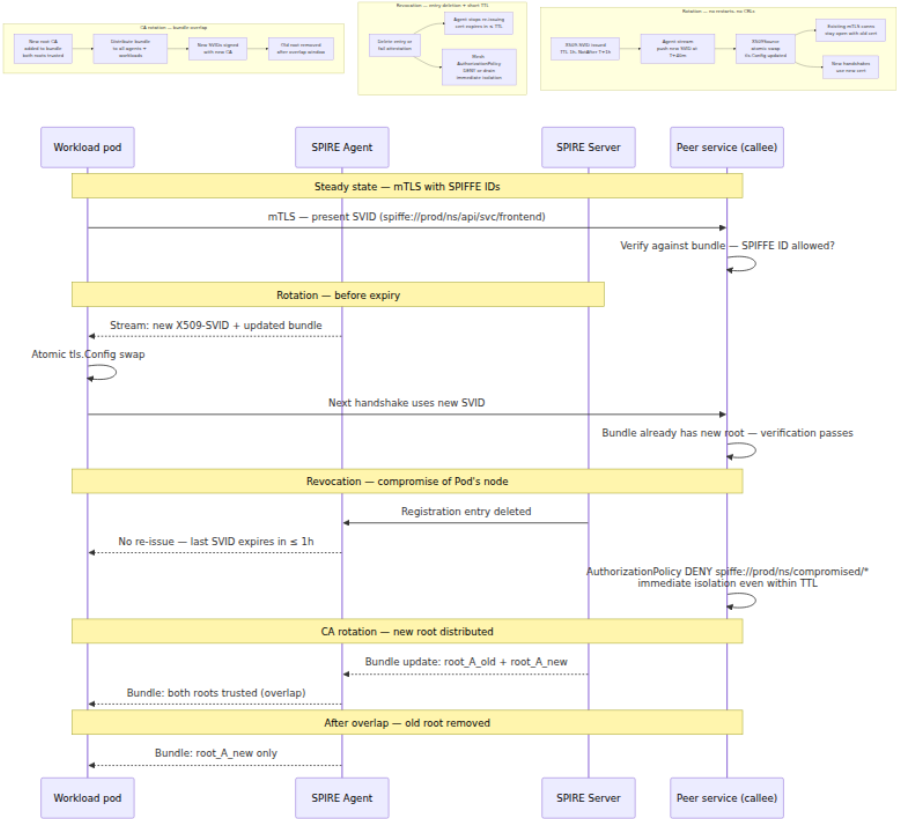
Zero trust at fleet scale does not use CRLs or OCSP for workload certs — distribution latency and size make them impractical for 10k-workload fleets with 1h certs. Instead, SPIRE issues the shortest TTL the fleet can tolerate (1h, 5m for JWT-SVIDs) and revokes by *not re-issuing*: delete the registration entry or fail workload attestation, and the cert expires within one TTL. For immediate isolation, combine with mesh-level `AuthorizationPolicy` deny or a data-plane close (drain connections, deny at the sidecar) — the cert may remain technically valid for minutes, but the workload is already denied at the next hop.

Blast radius and audit

- **Per-workload identity limits blast radius.** A compromised `payments` pod holds only `spiffe://prod/ns/payments/sa/payments-api` — it cannot forge `frontend`'s SVID, and mesh + application authZ denies it `frontend` → `payments` is not its grant. With shared secrets, one leaked key impersonates any service.
- **Log SPIFFE IDs, not IPs.** `source_ip: 10.4.2.88` is ephemeral and NAT'd; `caller_spiffe_id: spiffe://prod/ns/api/svc/frontend` is stable, auditable, and joins with deployment metadata. Emit it in

access logs, traces (`peer.service` in OpenTelemetry), and error reviews (Ch 7) — it is the `request_id` for identity.

- **Break-glass.** Keep an audited break-glass identity (`spiffe://prod/ns/sre/sa/break-glass`) with a deliberately short TTL (5m) and an approval workflow (PAM/OIDC — Ch 6) that mints a one-time SVID via the SPIRE API. Every use is logged and alerted — unlike a shared SSH key or a subnet allowlist that has no per-use audit.



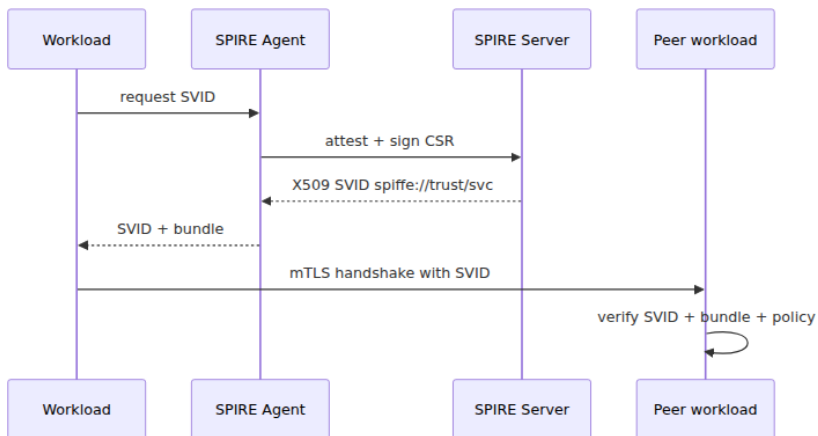
Putting it together — zero trust in a Kubernetes fleet

A minimal end-to-end checklist for a cluster that means it:

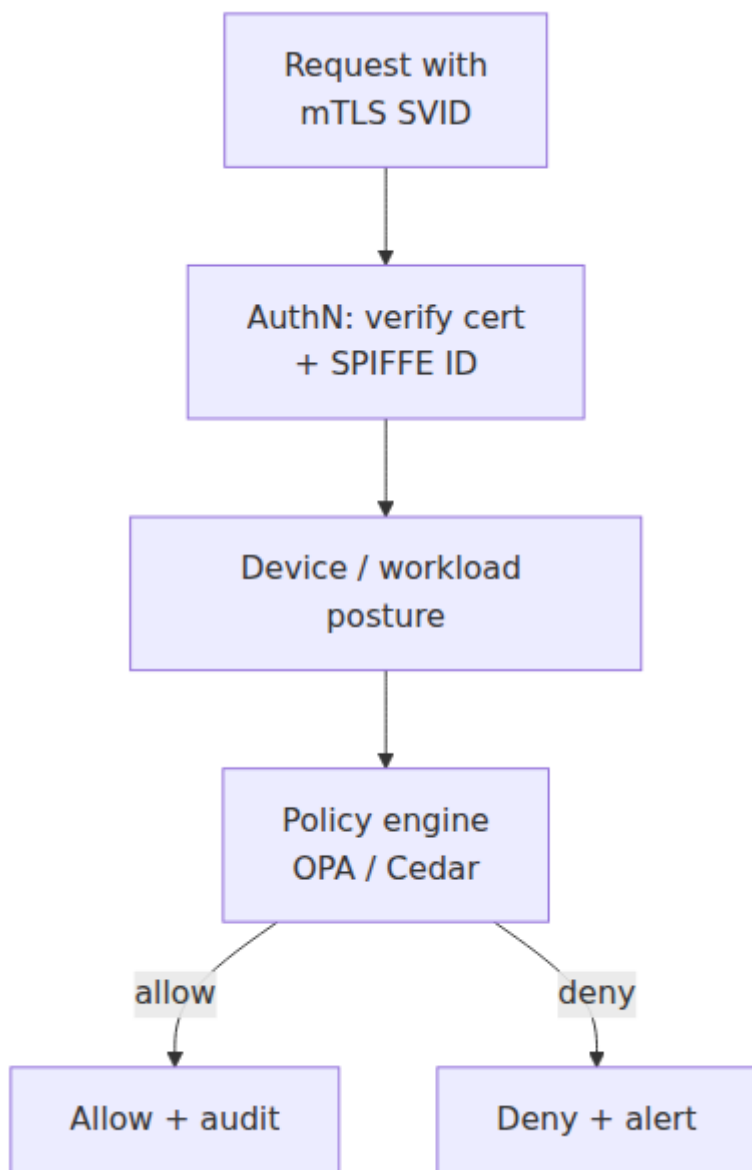
1. **Identity.** Deploy SPIRE server + agents (`k8s_psat` node attestation), `ClusterSPIFFEID` per workload. No workload holds a long-lived secret; every workload fetches via `Workload API`.

2. **Authentication.** Every service-to-service hop uses mTLS with X509-SVID — mesh (`PeerAuthentication STRICT`) or raw Envoy/go-spiffe. `PERMISSIVE` only during migration with a deadline and a burn-down metric.
3. **Authorization.** `AuthorizationPolicy` per callee by SPIFFE ID + path/method — not by IP/namespace. Application-level SPIFFE ID check as defense in depth.
4. **Rotation.** `default_x509_svid_ttl = 1h`, `default_jwt_svid_ttl = 5m`; workload `X509Source` hot-reloads; CA rotation via bundle overlap. No restart for rotation — alert if a pod's SVID is within 10m of expiry and not re-issued.
5. **Federation.** Publish `BundleEndpoint`, pin by SPIFFE ID/CA, import partner bundles — verify `iss` on JWT-SVIDs.
6. **Observability.** Access logs and traces carry `caller_spiffe_id` and `callee_spiffe_id`; metric `istio_requests_total{source_principal, destination_principal, response_code}` (Istio) or `envoy_cluster_ssl_connection_error` + RBAC `shadow_denied` for policy impact before enforcement.
7. **Break-glass.** Short-lived SVID mint via SPIRE API gated by OIDC + approval; every use audited; TTL 5m.

mTLS with SPIFFE



Zero Trust Policy Evaluation



Key takeaways

- Zero trust replaces network trust with workload identity + mutual authentication + identity-based authorization on every hop. The network is never a trust boundary — identity (SPIFFE ID) is.
- mTLS is the authentication hop: both sides present and verify an X.509 SVID, prove possession with `CertificateVerify`, and pin the peer's URI SAN (SPIFFE ID) against a trusted bundle. Server-only TLS is not zero trust — the server must also authenticate the caller.
- SPIFFE gives the universal identity (`spiffe://trust-domain/path`); SPIRE is the fleet issuer — node attestation (`k8s_psat`) proves the node, workload attestation (k8s selectors) proves the pod, the `Workload API` Unix socket delivers and rotates X.509-SVID s and JWT-SVID s, and the bundle is the trust anchor. Short TTLs (1h X.509, 5m JWT) replace CRLs.
- On Kubernetes, run SPIRE server (StatefulSet, `k8s_psat`, Postgres/SQLite, `disk` KeyManager) and agents (DaemonSet, `k8s` workload attestation), `ClusterSPIFFEID` per workload, and mount the `Workload API` socket via the SPIFFE CSI driver — workloads fetch with `go-spiffe / java-spiffe` `X509Source` and hot-reload on rotation without restarts.
- Enforce with mesh policy (`PeerAuthentication STRICT` + `AuthorizationPolicy` by principals SPIFFE IDs) and raw Envoy `transport_socket` + RBAC where no mesh control plane exists; keep an application-level SPIFFE ID check for defense in depth so a misconfigured mesh does not silently authorize.
- Federate trust domains by exchanging bundles via `BundleEndpoint` (pin by SPIFFE ID/CA), import federated bundles on agents, and verify JWT-SVID `iss` against the expected trust domain — no shared CA. Rotate CAs via bundle overlap (both roots trusted during the window).
- Operate the distributed-systems properties: rotation without restarts (`Workload API` stream + `X509Source` atomic swap), revocation by entry deletion + short TTL plus immediate mesh `DENY` /drain, logging and tracing by SPIFFE ID (not IP), per-principal metrics, canary `AUDIT` / `shadow` policy before `STRICT`, and a short-lived break-glass SVID gated by OIDC with full audit.

Further reading

- SPIFFE — Concepts, SPIFFE ID, SVID, Workload API, Federation. <https://spiffe.io/docs/latest/spiffe-about/overview/> / <https://github.com/spiffe/spiffe/blob/main/standards/SPIFFE.md>
- SPIRE — Architecture, deployment, attestation, registration, federation. <https://spiffe.io/docs/latest/spire-about/overview/> / <https://github.com/spiffe/spire>
- SPIRE on Kubernetes — `k8s_psat` , `ClusterSPIFFEID` , CSI driver. <https://spiffe.io/docs/latest/deploying/spire-for-k8s/>
- `go-spiffe/v2` / `java-spiffe` — `X509Source` , `JWTSource` , `tlsconfig` . <https://github.com/spiffe/go-spiffe> / <https://github.com/spiffe/java-spiffe>
- Envoy — TLS/mTLS, SDS, RBAC (`transport_socket` , `validation_context` , `SdsSecretConfig`). https://www.envoyproxy.io/docs/envoy/latest/api-v3/extensions/transport_sockets/tls/v3/tls.proto / <https://www.envoyproxy.io/docs/envoy/latest/configuration/security/rbac>
- Istio — `PeerAuthentication` , `AuthorizationPolicy` , SPIRE CA integration. https://istio.io/latest/docs/reference/config/security/peer_authentication/ / <https://istio.io/latest/docs/reference/config/security/authorization-policy/> / <https://istio.io/latest/docs/concepts/security/>
- NIST SP 800-207 — Zero Trust Architecture. <https://doi.org/10.6028/NIST.SP.800-207>
- BeyondCorp / Google Zero Trust — BeyondProd, context-aware access. <https://cloud.google.com/beyondcorp> / <https://research.google/pubs/beyondcorp-a-new-approach-to-enterprise-security/>
- Cilium / Linkerd — eBPF and mTLS/identity alternatives. <https://docs.cilium.io/en/stable/security/network/encryption/> / <https://linkerd.io/2/features/automatic-mtls/>
- Smallstep — `step-ca` , SPIFFE-aware CA, mTLS tooling. <https://smallstep.com/docs/step-ca/>
- RFC 8446 — TLS 1.3 (mTLS handshake). <https://www.rfc-editor.org/rfc/rfc8446.html>

- RFC 5280 — X.509 Certificates and CRLs. <https://www.rfc-editor.org/rfc/rfc5280.html>
- Vol 3, Ch 6 — TLS 1.3 and the Web PKI (handshake mechanics). Vol 9, Ch 4 — PKI/TLS operations. Vol 9, Ch 7 — Authorization by workload identity. Vol 9, Ch 8 — Secrets management.

Chapter 11 — Threat Modeling and Secure Design

What this chapter covers. Every control in Volumes 9 and the Companion Series — cryptography, authentication, authorization, mTLS, secrets, AppSec, signing — answers a *how*. Threat modeling answers *what* and *why*: what are we protecting, who are we protecting it from, what can go wrong, and what is worth fixing first. Without it, security is a checklist of controls applied uniformly and expensively, with gaps where the real risk lives. This is the capstone of Volume 9: we build a repeatable, team-scale threat-modeling discipline that turns architecture into prioritized, testable security requirements. We cover the STRIDE-per-element method (with data-flow diagrams), attack trees and kill chains, risk scoring (DREAD, CVSS, and qualitative severity $\times$ likelihood), trust boundaries and the data-flow diagram (DFD) as the modeling language, secure design principles (least privilege, defense in depth, fail-closed, complete mediation, economy of mechanism), and the operational loop — how a threat model lives inside design docs, code review, CI gates, and incident response. Every pattern is shown with realistic artifacts (DFD, STRIDE table, attack tree, mitigations mapped to controls from Ch 1-10) and viewed through the distributed-systems lens where a threat model must scale across dozens of services, teams, and deploys without becoming shelfware.

Learning goals — after this chapter you should be able to:

- Define threat modeling precisely — assets, threat actors, attack surface, trust boundaries, and risk — and explain why it precedes and prioritizes controls rather than following them.
- Draw a data-flow diagram (DFD) for a backend service or multi-service flow (processes, data stores, data flows, external entities, trust boundaries) and use it as the single modeling artifact.

- Apply STRIDE-per-element (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to enumerate threats systematically, with backend-relevant examples for each.
- Build attack trees and reason about kill chains / MITRE ATT&CK for backend-relevant techniques (initial access via SSRF/deserialization, lateral movement via flat network — Ch 10, credential access via leaked secrets — Ch 8).
- Score and prioritize risk with qualitative (severity × likelihood), DREAD, and CVSS, and map the result to fix-or-accept decisions, SLOs, and roadmaps.
- Apply secure design principles — least privilege, defense in depth, fail-closed/secure, complete mediation, separation of duties, economy of mechanism, and zero trust (Ch 10) — as design constraints, not afterthoughts.
- Run the operational loop: embed threat modeling in design docs/RFCs, generate testable security requirements, gate them in code review and CI (SAST/DAST/fuzz — Ch 9), and refresh the model on every material architecture change.

Boundary note. This chapter is the *method* that justifies every control in Volume 9: cryptography (Ch 1-3) for tampering/information disclosure, authentication (Ch 5-6) for spoofing, authorization (Ch 7) for elevation of privilege, secrets (Ch 8) for information disclosure, AppSec (Ch 9) for injection/tampering/SSRF, and zero trust/mTLS (Ch 10) for lateral movement and spoofing. The Companion Series, Books 1-8, threat-models the *supply chain* (dependencies, build, CI/CD, signing); this chapter models *runtime* backend systems. Volume 7 (System Design) and Volume 11 (Reliability/SRE) provide the architecture and incident-response context the model lives inside.

What threat modeling is — and what it is not

Threat modeling is a structured, design-time activity that answers four questions (Shostack's framing):

1. **What are we building?** — the DFD: components, data flows, trust boundaries.

2. **What can go wrong?** — threat enumeration (STRIDE, attack trees, ATT&CK).
3. **What are we going to do about it?** — mitigations mapped to design and controls.
4. **Did we do a good job?** — verification (tests, gates, review, red-team) and iteration.

What it is not:

- **Not a vulnerability scan.** Scans find known bugs in built code; threat modeling finds design flaws before code exists — missing authz on an internal endpoint, a trust boundary drawn in the wrong place, a secret that transits plaintext between two services.
- **Not a compliance checklist.** SOC 2 / ISO 27001 / SSDF ask *whether* controls exist; threat modeling asks *which* controls matter for *this* system and *this* adversary.
- **Not a one-time document.** A threat model is a living artifact tied to the architecture. When the architecture changes — a new service, a new data store, a new external integration, a new trust domain — the model is updated and re-scored.

The output is not the diagram. The output is **prioritized, testable security requirements** that enter the backlog, the design doc, and the CI gates.

The modeling language — data-flow diagrams and trust boundaries

A DFD has four element types plus the trust boundary that is the modeling primitive for backend systems:

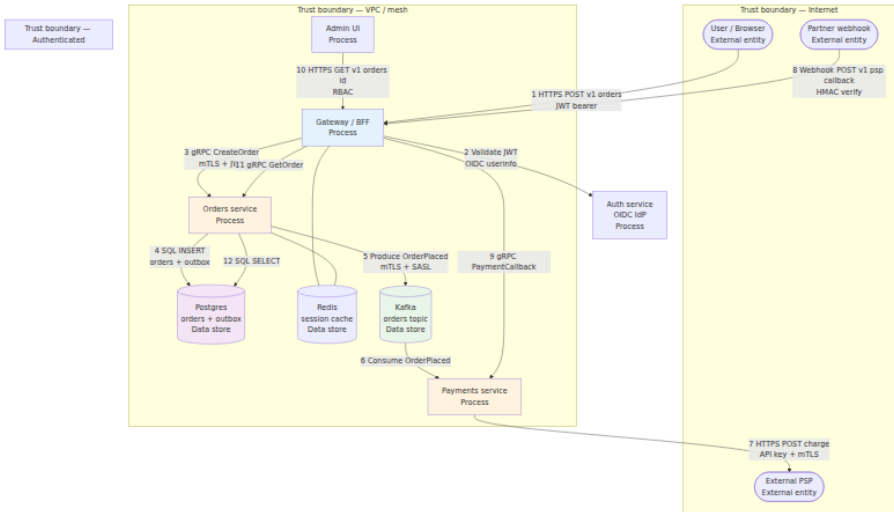
Element	Symbol	Meaning	Backend example
External entity	Rectangle	Outside the system's control	Browser, mobile app, partner API, attacker
Process	Circle/ rounded rect	Computation that transforms data	API gateway, <code>orders</code> service, <code>payments</code> service, worker

Element	Symbol	Meaning	Backend example
Data store	Parallel lines / cylinder	Persistent state	Postgres, Redis, S3, Kafka topic
Data flow	Arrow	Data in motion with protocol	HTTPS POST /v1/orders , gRPC Charge() , Kafka produce orders
Trust boundary	Dashed line	Privilege or control transition	Internet → VPC, VPC → service, service → service (zero trust — Ch 10), unauthenticated → authenticated

Trust boundaries are where threats live. Every flow that crosses a boundary is an attack surface. In a zero-trust architecture (Ch 10) *every service-to-service flow* is a trust boundary — which is the point.

Worked example — order placement flow

We will carry one example through the chapter: a user places an order via the gateway; the `orders` service writes Postgres (via the outbox — Vol 10, Ch 6), publishes `OrderPlaced` to Kafka, and the `payments` service charges via an external PSP. An admin UI reads orders for support.



Numbered flows 1-12 are the *attack surface*. Every flow is labeled with its protocol and auth — missing labels are findings.

STRIDE-per-element

STRIDE enumerates six threat classes. Applied per DFD element, it is systematic — every element is checked against the STRIDE categories that apply to it, so gaps become visible rather than relying on brainstorming.

Threat	Property violated	Applies to	Backend example
Spoofing	Authentication	External entity, Process	Attacker forges JWT, spoofs <code>X-User-ID</code> header, compromises pod identity (Ch 10)
Tampering	Integrity	Data flow, Data store, Process	MITM on HTTP between gateway and orders, SQL injection (Ch 9) mutates <code>orders</code> , Kafka message tampered
Repudiation	Non-repudiation	Process, Data store	Orders service denies it created order 1043 — no audit log, no signed event
Information disclosure	Confidentiality	Data flow, Data store, Process	<code>GET /v1/orders/{id}</code> returns another user's order (BOLA — Ch 7), Postgres snapshot exfiltrated, Kafka topic world-readable
Denial of service	Availability	Process, Data flow, Data store	<code>POST /v1/orders</code> without rate limit (Ch 9), poison message stalls partition (Vol 10, Ch 8), slow query locks <code>orders</code> table

Threat	Property violated	Applies to	Backend example
Elevation of privilege	Authorization	Process	Tenant A calls <code>POST /v1/orders</code> with Tenant B's <code>tenant_id</code> in body (BFLA), admin UI lacks RBAC and any user can refund

Applicability matrix

Not every STRIDE category applies to every element — this is what makes the method finite:

Element	S	T	R	I	D	E
External entity	✓		✓			
Process	✓	✓	✓	✓	✓	✓
Data flow		✓		✓	✓	
Data store		✓	✓	✓	✓	

A process is the richest element — all six apply.

STRIDE table for the order flow (abridged — full model has one row per flow × applicable STRIDE)

#	Element / Flow	S	T	R	I	D	E	Threat (one per row)	Mitigation (map Ch)
1	U→GW <code>POST /v1/orders</code>	✓						Attacker forges JWT or replays stolen token	OIDC issuer validation / <code>iss</code> check, short-lived JWT (5 <code>jti</code> replay cache 5-6)
1	U→GW		✓					MITM / request tampering on public internet	TLS 1.3, HSTS, certificate pinning mobile (Ch 4)
1	U→GW					✓		Unauthenticated flood — no rate limit	Per-IP + per-user bucket at gateway WAF (Ch 9, Vol 7)

#	Element / Flow	S	T	R	I	D	E	Threat (one per row)	Mitigation (map Ch)
1	U→GW						✓	Tenant A sets <code>tenant_id=B</code> in body	Tenant binding from JWT sub / tenant claim, not from body (Ch 7 ReBAC)
3	GW→Orders gRPC CreateOrder	✓						Compromised gateway or lateral movement — caller spoofs internal service	mTLS with SPIFFE <code>spiffe://prod/n/api/svc/gateway</code> AuthorizationPolicy allow only gateway orders (Ch 10)
3	GW→Orders		✓					gRPC payload tampered in transit (mesh MITM)	mTLS (encryption integrity), protobuf strict validation (Ch 8)
3	GW→Orders				✓			Internal gRPC leaks PII to logs	Structured logging with PII redaction pipeline classification (Vol 11 Ch 3)
4	Orders→DB INSERT orders		✓					SQL injection via order fields	Parameterized queries, ORM safe APIs, Sanitization (Ch 9)
5	Orders→Kafka Produce OrderPlaced		✓					Event tampered or forged — downstream acts on ghost order	Sign events (outbound idempotency key - Vol 10 Ch 6), SASL/SCRAM + ACLs on <code>order</code> topic (Vol 10 Ch 3)
5	Orders→Kafka				✓			<code>orders</code> topic world-readable — any service can consume PII	Kafka ACLs: only <code>payments</code> group can consume <code>orders</code> topic (Vol 11 Ch 3) encryption at rest (Ch 1-3)

#	Element / Flow	S	T	R	I	D	E	Threat (one per row)	Mitigation (map Ch)
6	Kafka→Payments Consume			✓				payments denies it processed order 1043 — no audit trail	Append-only audit (Vol 10 Ch 5 event sourcing), idempotent inbox with message dedup (Vol 6 Ch 9)
7	Payments→PSP POST /charge				✓			API key leaked in logs / env / exception	Secrets manager (Vault/AWS SM), no secret in env, SecretString type (Ch 8)
7	Payments→PSP					✓		PSP slow/down — payments handler blocks, partition stalls	Timeout (2s) + circuit breaker + non-blocking retry topic (Vol 10 Ch 8), backpressure (Vol 10 Ch 7)
8	P→GW Webhook / psp/callback	✓						Attacker forges PSP callback — marks unpaid order as paid	HMAC signature verification with PSP public key / shared secret from secrets manager, replay check on eventId
10	Admin→GW GET / v1/orders/{id}				✓			Support user reads another tenant's order (BOLA)	ReBAC check: user.can_read_order == user.tenant == order.tenant via Zanzibar (Ch 7)
10	Admin→GW						✓	Any authenticated user can POST /v1/admin/refund	RBAC: refund requires role=payments-admin ; tested with authz matrix (Ch 7)

A real threat model for this flow has 30–50 rows. The table is the artifact — every empty cell is a conscious *not applicable* or a missing mitigation.

Attack trees and kill chains

STRIDE enumerates *what* can go wrong per element; attack trees show *how* an attacker composes steps into a path to a goal. The root is the attacker's objective; children are sub-goals combined with **AND** (all required) or **OR** (any suffices).

Worked attack tree — forge a payment (mark order as paid without charging)



Reading the tree bottom-up, the mitigations that *cut the most branches* are the highest leverage:

- **HMAC verification + replay cache** on the webhook (T1) — cuts the cheapest external path.
- **Kafka ACLs + SASL scoped to topic+principal + mTLS** (T2) — without them, any compromised pod can forge events.
- **Parameterized queries + SAST taint** (T3a1) and **secrets manager + no env secrets** (T3a2) — the classic code-level cuts.
- **ReBAC on every resource access** (T4a2) — the BOLA cut that removes an entire class.

Kill chains and MITRE ATT&CK for backend

An attack tree is static; a kill chain orders the steps in time and maps them to ATT&CK techniques:

Phase	ATT&CK technique	Backend instance	Mitigation (Ch)
Initial access	T1190 Exploit Public-Facing App	SSRF in POST /v1/orders webhooks_url fetches 169.254.169.254	Allowlist egress + IMDSv2 + NetworkPolicy (Ch 9)

Phase	ATT&CK technique	Backend instance	Mitigation (Ch)
Execution	T1059 Command and Scripting	Deserialization RCE via <code>pickle / yaml.load</code>	Strict codecs, <code>additionalProperties: false</code> (Ch 9)
Credential access	T1552 Unsecured Credentials	DB password in env var dumped via <code>/ debug/vars</code>	Secrets manager, no <code>pprof / debug</code> in prod (Ch 8)
Lateral movement	T1021 Remote Services	Flat network — compromised <code>orders</code> pod calls <code>payments POST / charge</code> with no mTLS	mTLS + <code>AuthorizationPolicy</code> per hop (Ch 10)
Exfiltration	T1048 Exfiltration Over Alt Protocol	<code>SELECT * FROM orders</code> exfiltrated via DNS tunnel	Egress filtering, DB audit log, anomaly detection (Vol 11 Ch 5)
Impact	T1496 Resource Hijacking	Poison message stalls partition, lag grows, orders delayed	DLQ + non-blocking retries (Vol 10 Ch 8), backpressure (Vol 10 Ch 7)

Risk scoring — what to fix first

Enumeration without prioritization is a backlog that never shrinks. Three scoring models are common — use the lightest that still forces a decision.

Qualitative — severity × likelihood (recommended default)

	Low likelihood	Medium likelihood	High likelihood
High severity (PII leak, payment forgery, tenant isolation break)	High — fix soon	Critical — fix now	Critical — fix now + incident

	Low likelihood	Medium likelihood	High likelihood
Medium severity (DoS, non-PII leak, partial bypass)	Low — backlog	Medium — next sprint	High — fix soon
Low severity (verbose error, missing header)	Low — accept or backlog	Low — backlog	Medium — next sprint

Severity is *business impact* (what the attacker gains), not *CVSS base score*. A **CVSS 9.8** RCE in a dev-only service with no production data is lower severity than a **CVSS 6.5** BOLA that leaks every tenant's PII.

DREAD (when you need numbers for a roadmap)

Factor	1 (low)	5 (medium)	10 (high)
Damage	Verbose error	Single-tenant PII leak	Cross-tenant PII, payment forgery
Reproducibility	Race condition, 1 in 10k	Authenticated, needs setup	Anonymous, <code>curl</code>
Exploitability	Custom exploit, deep knowledge	Script available	<code>sqlmap</code> / trivial
Affected users	One tenant, one endpoint	All tenants, one flow	All tenants, all flows
Discoverability	Hidden internal endpoint	Authenticated enumeration	Public endpoint, indexed

DREAD = (D+R+E+A+D)/5 . 8–10 = critical, 5–7 = high, 3–4 = medium, <3 = low. DREAD is subjective — use it to *order* the backlog, not to compute a precise risk dollar.

CVSS (for dependency and vulnerability prioritization — not for design flaws)

CVSS 3.1/4.0 scores *known vulnerabilities* (CVEs) on exploitability + impact + scope. Use it for SCA findings (Ch 9, Companion Book 2) — *which CVE to patch first* — not for design threats like missing authz,

which have no CVE. A threat model that scores everything with CVSS is misapplying the tool.

Worked scoring — the order flow

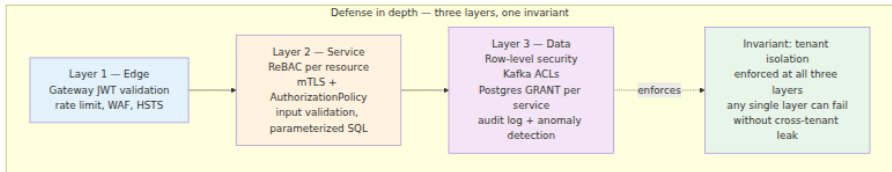
Threat	Severity	Likelihood	Priority	Fix
Webhook HMAC bypass (forge payment)	High	High (public endpoint, well-known pattern)	Critical	HMAC verify + replay cache — ship before launch
BOLA GET /v1/orders/{id}	High	High (IDOR is top API risk — Ch 7, OWASP API1)	Critical	ReBAC check — blocking launch
Kafka topic world-readable	High	Medium (requires internal access, but flat network)	High	ACLs + SASL scope — next sprint
SQL injection in CreateOrder	High	Low (parameterized + SAST, but verify)	High	SAST taint gate in CI — this sprint
DoS — no rate limit on POST /v1/orders	Medium	High	High	Gateway token bucket — this sprint
Verbose error leaks stack trace	Low	Medium	Low	Generic error envelope — backlog

Secure design principles — the constraints

Threats enumerate *what* is missing; principles constrain *how* the design must be shaped. Apply them as invariants in design review.

Principle	Means in this system	Violation
Least privilege	Each service's Kafka ACL, DB grant, and IAM role allows only the topics/tables/APIs it needs	orders can SELECT * on payments table because it shares a DB user

Principle	Means in this system	Violation
Defense in depth	Gateway JWT validation <i>and</i> service-level ReBAC <i>and</i> DB row-level security — any one can fail without full bypass	Only gateway checks tenant — service trusts <code>X-Tenant-ID</code> header
Fail-closed / secure default	New endpoint denies by default; new topic denies produce/consume; new feature flag is off	New <code>POST /v1/admin/export</code> ships without authz because review missed it
Complete mediation	Every access to <code>order</code> checks <code>can_read(user, order)</code> — no cached "already authorized" bypass	<code>GET /v1/orders/{id}</code> checks on first page but <code>GET /v1/orders?cursor=</code> skips check
Separation of duties	Deploying <code>payments</code> and approving its ACL change are different identities/approvals	One engineer can push <code>payments</code> and widen its Kafka ACL in the same PR
Economy of mechanism	One authz path (ReBAC via Zanzibar — Ch 7) for all resource checks, not three ad-hoc <code>if</code> branches	<code>orders</code> checks tenant in handler, <code>payments</code> checks in middleware, <code>admin</code> checks in frontend
Zero trust (Ch 10)	Every hop authenticates with mTLS + SPIFFE ID and authorizes by that identity — network location is not a credential	Service trusts caller because it is inside the VPC
Minimize attack surface	No <code>debug</code> / <code>pprof</code> / <code>actuator</code> in prod, no wildcard CORS, no <code>SELECT *</code> , no overly broad IAM <code>*</code>	<code>GET /debug/vars</code> exposes env vars including secrets



The threat model as a living document

Template — one page per service or flow

Threat model — Order placement flow (orders + payments)

Owner: @backend-security — Review: quarterly or on material change
DFD: ./threat-models/orders-dfd.mmd — STRIDE table: ./threat-models/orders-stride.csv

Assets

- PII: user email, shipping address (orders DB, Kafka topic, logs)
- Money: order total, charge idempotency (payments → PSP)
- Tenant isolation: orders must never cross tenant boundary

Trust boundaries

- Internet → Gateway (unauthenticated → JWT)
- Gateway → Orders/Payments (mTLS + SPIFFE ID)
- Orders → Postgres/Kafka (SASL + ACL + GRANT)

Top threats (STRIDE, scored)

#	Threat	STRIDE	Severity	Likelihood	Priority	Mitigation	Owner	Status
T-01	Webhook HMAC bypass	S	High	High	Critical	HMAC verify + jti replay cache	@payments	Done
T-02	BOLA GET /orders/{id}	I/E	High	High	Critical	ReBAC can_read(user, order)	@orders	Done
...	...	...	...	...	...	...	...	...

Attack tree

- Root: forge payment — see ./threat-models/orders-attack-tree.mmd

Residual risk (accepted)

- R-01: Kafka delayed retry breaks ordering on replay — accepted, downstream idempotent (Vol 6 Ch 9 inbox)

Verification

- SAST taint (Semgrep/CodeQL) in CI — blocks on new SQLi/XSS — gate: .github/workflows/sast.yml
- DAST authenticated crawl (ZAP) nightly on staging — gate: staging-dast
- Fuzz: FuzzParseWebhookURL (go test -fuzz) — nightly 30m
- Authz matrix test: 12 BOLA/BFLA cases — gate: authz-matrix-test

Changelog

- 2026-08-15: Added PSP webhook flow (T-01), re-scored after HMAC fix
- 2026-08-20: Admin UI added (T-10/11), ReBAC required — design doc #48

2

Where the model lives in the engineering loop



- **Design doc gate:** No RFC merges without a threat-model section (DFD + top threats + mitigations) for any new service, external integration, or authz change. The reviewer checks the STRIDE table for empty cells, not for prose quality.
- **Code review:** Security requirements from the model are *testable* — HMAC verification exists and is tested with a forged callback, ReBAC check exists on every `GetOrder` path and is covered by 12 BOLA cases. Review verifies the requirement and the test, not just the code.
- **CI:** SAST taint (SQLi/XSS), SCA (known CVEs), secrets (gitleaks/TruffleHog), and authz-matrix tests are blocking gates (Ch 9). A new endpoint without an authz test fails CI.
- **Prod + IR:** Every incident or near-miss updates the model — a new ATT&CK technique, a missed trust boundary, a DLQ poison that was a schema threat not in the table. The model is the postmortem's *prevention* section.
- **Refresh cadence:** Quarterly for stable services, on every material change (new external dependency, new data store, new auth flow, new trust domain) otherwise. A model that is not refreshed is a model that is wrong.

Choosing what not to fix

Not every threat is fixed. The model makes *acceptance* explicit:

Decision	Means	Example
Mitigate	Ship a control before or soon after launch	HMAC on webhook, ReBAC on <code>GetOrder</code> — critical, blocks launch
Transfer	Move risk to a party better placed to handle it	PSP handles PCI DSS scope — we tokenize, never store PAN

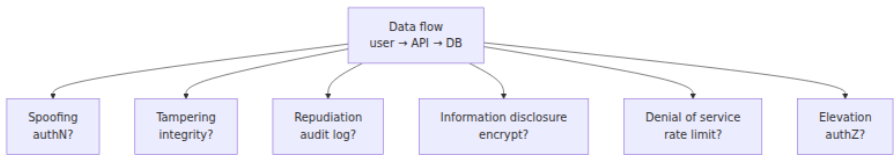
Decision	Means	Example
Accept	Document residual risk, monitor, revisit	Kafka replay breaks ordering — accepted because downstream is idempotent; logged as R-01
Avoid	Remove the feature or integration	Drop <code>yaml.load</code> webhook config — use <code>json</code> + schema; no deserialization risk to mitigate

Acceptance without a written residual-risk entry and a compensating detection (alert on DLQ, anomaly on authz deny rate) is not acceptance — it is neglect. Every accepted risk has an owner, a review date, and a detection.

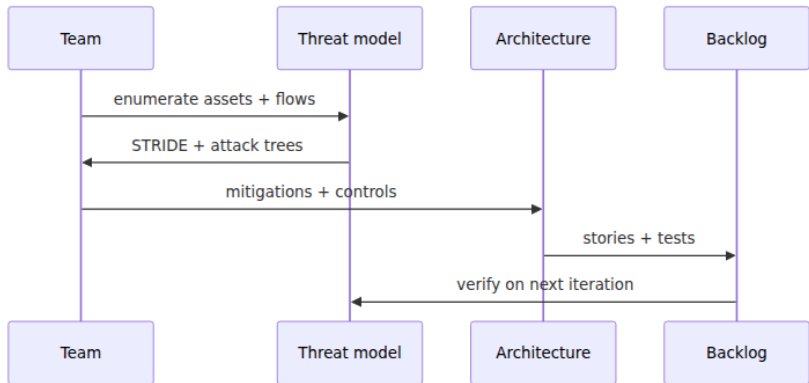
Distributed-systems lens

- **Threat models must compose.** A fleet of 40 services cannot have one monolithic model. Own a per-flow model (order placement, payment callback, admin read) and a per-service model, then compose trust boundaries: the gateway's *outbound* mTLS guarantee is the orders service's *inbound* assumption. Inconsistency between the two is a finding.
- **Zero trust is the trust-boundary discipline at scale.** In a flat network, the DFD has one trust boundary (Internet → VPC) and lateral movement is invisible. With mTLS + SPIFFE + `AuthorizationPolicy` (Ch 10), every service-to-service flow is a boundary — the DFD and the `AuthorizationPolicy` are the same artifact, and a missing policy is a missing boundary.
- **The DFD is the contract between teams.** When the `orders` team adds a new Kafka topic that `analytics` consumes, the DFD change is the notification that `analytics` now has PII and needs ACLs, retention, and audit. Without the DFD as a shared artifact, the data flow is discovered during an incident.
- **Verification must be as distributed as the system.** SAST/SCA/ secrets gates run per repo, authz-matrix tests run per service, DAST crawls per environment, and audit-log anomaly detection runs centrally. The threat model maps each mitigation to *where* it is verified — a mitigation with no gate is a mitigation that will regress.

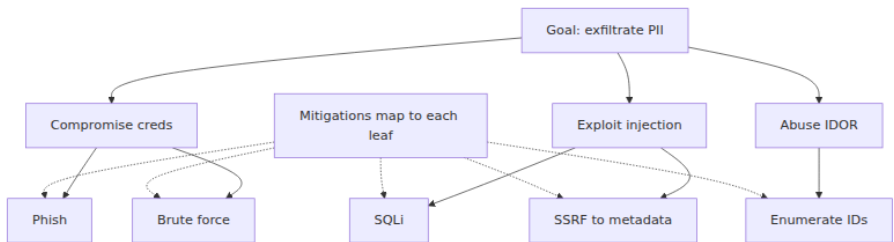
STRIDE per Component



Threat Model Iteration



Attack Tree Example



Key takeaways

- Threat modeling is the design-time discipline that turns architecture (DFD + trust boundaries) into prioritized, testable security requirements — it precedes and directs controls, not the reverse. Its output is tickets and tests, not just diagrams.

- The DFD (external entity, process, data store, data flow, trust boundary) is the modeling language; every flow that crosses a boundary is attack surface and every unlabeled flow is a finding.
- STRIDE-per-element makes enumeration systematic: check each element against its applicable STRIDE categories and record one row per threat with a mitigation mapped to a control in Ch 1-10. Empty cells are conscious, not accidental.
- Attack trees (AND/OR composition to a goal) and ATT&CK-mapped kill chains show how steps compose into paths; the mitigations that cut the most branches are the highest leverage.
- Score with qualitative severity $\times$ likelihood (default) or DREAD for roadmaps; reserve CVSS for known CVEs (SCA), not design flaws. Severity is business impact — tenant isolation break outranks RCE in dev.
- Secure design principles (least privilege, defense in depth, fail-closed, complete mediation, separation of duties, economy of mechanism, zero trust) are invariants enforced at every layer — the same invariant (tenant isolation) at edge, service, and data.
- The model lives in the RFC, is reviewed as code, gates CI (SAST/SCA/secrets/authz tests), and is refreshed on every material architecture change and every incident — with explicit, owned residual-risk entries for accepted threats.

Further reading

- **Method:** Shostack *Threat Modeling: Designing for Security* (2014) — DFD, STRIDE-per-element, and the four questions — the most practical single book. OWASP *Threat Modeling Cheat Sheet* (<https://cheatsheetseries.owasp.org/IndexTopTen.html>) and OWASP *Threat Modeling Playbook* — concise, actionable. Microsoft SDL *Threat Modeling Tool 7.x* (<https://aka.ms/threatmodelingtool>) — DFD + STRIDE automation.
- **STRIDE & DFD:** Microsoft *STRIDE fundamentals* (<https://learn.microsoft.com/en-us/security/engineering/threat-modeling>) — per-element applicability and examples. Seifert & Shostack *STRIDE-per-element* — the matrix that makes enumeration finite.
- **Attack trees & ATT&CK:** Schneier *Attack Trees* (Dr. Dobb's, 1999) — AND/OR composition. MITRE ATT&CK (<https://attack.mitre.org/>)

— techniques T1190, T1059, T1552, T1021, T1048 as they map to backend flows. Lockheed Martin *Cyber Kill Chain* and Mandiant *Attack Lifecycle*.

- **Risk scoring:** OWASP *Risk Rating Methodology* (<https://owasp.org/www-project-risk-rating-methodology/>) — likelihood × impact with business factors. FIRST CVSS 3.1/4.0 spec (<https://www.first.org/cvss/>) — when to use CVSS and when not to. NIST SP 800-30 — risk assessment.
- **Secure design principles:** Saltzer & Schroeder *The Protection of Information in Computer Systems* (1975) — least privilege, complete mediation, economy of mechanism — still the best source. NIST SP 800-160 Vol 1 — systems security engineering. Ch 1-10 of this volume for each control mapped to STRIDE.
- **Operationalizing:** OWASP SAMM 2.0 (<https://owaspsamm.org/>) — threat modeling maturity. Snyk / Semgrep / CodeQL docs — SAST taint gates that verify the model in CI (see Ch 9). Google *Building Secure and Reliable Systems* — Ch 13 (threat modeling at scale).